UsdOptixViewer

A Literate Program

Pablo Figueroa

Written in the style of Donald E. Knuth's *Literate Programming*.
The source files of the program are tangled from this document; `lp.py check`
verifies that the tangled output is identical to the files in the repository.

Contents

Introduction **1**
3. How to read the code . . . 1
4. The parts of the program . . . 1
5. A word on correctness . . . 2

The build system **3**
15. Device code . . . 5
16. The executable . . . 5
18. Embedding a file . . . 6

Shared parameters **7**
27. Meshes . . . 9
28. The launch parameters . . . 10

The scene **11**
34. Meshes for the renderer . . . 11
39. The implementation file . . . 13
41. Opening a stage . . . 14
46. Extracting the geometry . . . 16
49. From prims to triangles . . . 17
62. Materials . . . 21
86. Textures . . . 29
90. Materials per face . . . 30
92. Corners and vertices . . . 31
101. World space . . . 34
106. Cameras . . . 35

The renderer **38**
110. The interface . . . 38
121. The implementation . . . 41
127. Mipmaps . . . 43
131. Device buffers . . . 45
135. Life and death of the renderer . . . 46
140. The pipeline . . . 48
148. The shader binding table . . . 50
153. Loading a scene . . . 51
169. Textures . . . 55
175. The image and the camera . . . 57
179. Rendering . . . 58

The device programs **61**
189. Random numbers . . . 62
192. The payload . . . 63
195. The hit triangle . . . 63
199. Texture sampling with ray differentials . . . 64
206. Normal mapping . . . 67
209. The GGX microfacet model . . . 68
213. Ray generation . . . 69
218. Alpha cutouts . . . 70
219. Shading . . . 71
230. The directional light . . . 74
234. Glossy reflections . . . 75

The application **78**
242. The main program . . . 78
243. The application class . . . 78
253. The implementation . . . 81
256. Creation and destruction . . . 83
264. Opening files . . . 86
267. The main loop . . . 87
269. The layout . . . 88
273. Menus . . . 89
279. The hierarchy . . . 91
286. The inspector . . . 94
298. The camera . . . 99
310. The viewport . . . 104
322. The status bar and the popups . . . 108
324. The XR output . . . 109
331. Decoding images . . . 112

XR output **114**
333. The CloudXR service . . . 114
349. The XR output: interface . . . 119
359. The XR output: setting up . . . 122
373. Vulkan for the runtime . . . 127
382. Memory shared between Vulkan and CUDA . . . 130
388. The session . . . 132
397. Frames . . . 135
405. Eye cameras . . . 138

Names of the sections **141**

Introduction

1. UsdOptixViewer is a small, complete viewer for scenes described in Pixar's Universal Scene Description (USD). It opens a stage, shows its prims as a tree, lets the reader inspect every attribute, and draws the geometry with a ray tracer that runs on NVIDIA's OptiX engine. It can also send the view in stereo to a headset through OpenXR, and in particular through NVIDIA CloudXR, which streams it over the network. The program was written to exercise, in one place, a set of tools installed on a workstation: CUDA and OptiX for ray tracing, SDL3 for the window and input, Dear ImGui for the user interface, OpenUSD for reading scenes, meshoptimizer for preparing meshes, and OpenXR for the headset.

This document *is* the program. Following Donald Knuth's *Literate Programming* (1984), the code is presented in the order in which it is best understood by a human reader, not in the order a compiler wants it. Each piece of code lives in a numbered *section*, together with the prose that explains it. A tool, `lp.py`, *tangles* the sections back into the sixteen source files of the program, and *weaves* them into the book you are reading.

2. The rendering is progressive. While the reader does not touch anything, the viewer keeps adding one jittered sample per pixel and averages them, up to 256 samples. Materials follow the UsdPreviewSurface specification: diffuse colors and textures, roughness and metallic values or maps, tangent-space normal maps, alpha cutouts and per-face materials. Textures are filtered with mipmaps whose level is chosen by ray differentials, and glossy reflections are sampled from the GGX distribution of visible normals. All of these terms are explained where they appear.

3. How to read the code. A *chunk* of code has a name in angle brackets, followed by the number of the section where it is first defined, as in ⟨Install the viewer 17⟩; the ≡ sign introduces its definition, and +≡ marks a continuation of a chunk defined earlier. When a chunk name appears inside another chunk, the code it names is to be inserted at that point, indented like the reference. After each definition, a small note tells where the chunk is used. A chunk whose name is a file name, such as `UsdOptixViewer/src/main.cpp`, is a *root*: its expansion is written to that file.

Code is typeset in the manner of Knuth's CWEB: reserved words in **bold**, identifiers in *italic*, strings in `typewriter` type and comments in roman. Some operators are shown with their mathematical symbols: ≠ for `!=`, ≤ and ≥ for `<=` and `>=`, ≡ for `==`, ∧ and ∨ for `&&` and `||`, ¬ for `!`, and → for `->`. Long lines are broken and continued with a deeper indentation.

4. The parts of the program. The book follows the path of the data, from the build system to the pixels on the screen:

- *The build system* compiles the C++ code, turns the OptiX programs into OptiX-IR and embeds them in the executable.
- *Shared parameters* define the one structure that the CPU fills in and the GPU reads, `LaunchParams.h`.
- *The scene* reads a USD stage, which serves directly as the scene graph, resolves materials and textures, and turns every renderable prim into one indexed triangle mesh in the prim's local space, welded with meshoptimizer.
- *The renderer* creates the OptiX pipeline, builds one geometry acceleration structure per mesh and an instance acceleration structure that places them in the world, uploads the textures with their mipmaps, and launches the ray tracing.
- *The device programs* run on the GPU: ray generation, shading, shadows, reflections and alpha cutouts.
- *The application* ties everything together with SDL3 and Dear ImGui: the hierarchy, the inspector, the viewport and the camera, and drives the XR output.
- *The XR output* talks to an OpenXR runtime through Vulkan, shares the eye images between Vulkan and CUDA, and can run the CloudXR runtime inside the program.

The third-party image decoder `stb_image.h`, about eight thousand lines by Sean Barrett, is used as it comes and is not part of this book; nor is NVIDIA's header `cxrServiceAPI.h`, which declares the CloudXR service.

5. A word on correctness. Literate programs are easy to let drift away from the code that actually runs. To prevent that, the command `python3 lp.py check` tangles this book in memory and compares every root chunk, byte for byte, with the corresponding file in the repository; it also fails if any source file of the program has no root chunk. The book and the program therefore always agree, and every line of the program appears somewhere in these pages.

The build system

6. The program is built with CMake. The project file has five parts, which we shall meet in turn.

⟨`UsdOptixViewer/CMakeLists.txt` 6⟩ ≡
⟨Declare the project and the language standard 7⟩
⟨Locate the visualization tools 8⟩
⟨Compile the device programs to OptiX-IR and embed them 15⟩
⟨Build Dear ImGui and the viewer executable 16⟩
⟨Install the viewer 17⟩
This code is written to file `UsdOptixViewer/CMakeLists.txt`.

7. We need CMake 3.24 or newer and C++20, the language version the code is written in (it uses, for instance, the *contains* member of the standard containers). A build without an explicit type is an optimized *Release* build: ray tracing a scene in a debug build is no fun.

⟨Declare the project and the language standard 7⟩ ≡
```
cmake_minimum_required(VERSION 3.24)
project(UsdOptixViewer VERSION 0.1.0 LANGUAGES C CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
if(NOT CMAKE_BUILD_TYPE)
  set(CMAKE_BUILD_TYPE Release)
endif()
```
This code is used in section 6.

8.
Finding the tools. The tools are installed for all users under `/opt/viztools`, with CUDA in `/usr/local/cuda-13.2`. Every location can be overridden from the command line or the environment; the login script `/etc/profile.d/viztools.sh` sets the environment variables.

⟨Locate the visualization tools 8⟩ ≡
```
# ------------------------------------------------------------------------
# Dependencies, installed for all users in /opt/viztools (and CUDA in
# /usr/local/cuda-13.2). Locations can be changed through CMAKE_PREFIX_PATH or
# the variables / environment variables below (see /etc/profile.d/viztools.sh).
# ------------------------------------------------------------------------
```
⟨Choose the root of the installed tools and of CUDA 9⟩
⟨Find the OptiX headers 10⟩
⟨Find the Dear ImGui sources 11⟩
⟨Choose the default CloudXR SDK 12⟩
⟨Find SDL3, OpenUSD, meshoptimizer and CUDA 13⟩
⟨Find the OpenXR loader and the Vulkan headers 14⟩
This code is used in section 6.

9. The variable *VIZ_PREFIX* records the root of the tools. If that directory does not exist, configuration stops at once with a clear message, rather than later with a cryptic one about some missing package. CUDA has its own rule: an explicit *CUDAToolkit_ROOT* wins, then the *CUDA_HOME* variable set by the login script, then the standard system directory.

⟨Choose the root of the installed tools and of CUDA 9⟩ ≡
```
set(VIZ_PREFIX "/opt/viztools" CACHE PATH "Root of the installed visualization tools")
if(NOT EXISTS "${VIZ_PREFIX}")
  message(FATAL_ERROR "Visualization tools not found in VIZ_PREFIX=${VIZ_PREFIX}")
endif()
if(NOT DEFINED CUDAToolkit_ROOT)
  if(DEFINED ENV{CUDA_HOME})
    set(CUDAToolkit_ROOT "$ENV{CUDA_HOME}")
```

```
  else()
    set(CUDAToolkit_ROOT "/usr/local/cuda-13.2")
  endif()
endif()
```

This code is used in section 8.

10. OptiX is a header-only SDK: the implementation ships inside the NVIDIA driver, and the headers only declare a table of function pointers that is filled in at run time. All we need, then, is the directory that contains `optix.h`.

⟨Find the OptiX headers 10⟩ ≡

```
set(OptiX_INSTALL_DIR "$ENV{OptiX_INSTALL_DIR}" CACHE PATH "OptiX SDK root (contains include/
      optix.h)")
if(NOT OptiX_INSTALL_DIR)
  set(OptiX_INSTALL_DIR "${VIZ_PREFIX}/optix-9.1.0" CACHE PATH "" FORCE)
endif()
find_path(OptiX_INCLUDE_DIR optix.h PATHS "${OptiX_INSTALL_DIR}/include" REQUIRED NO_DEFAULT_PATH)
```

This code is used in section 8.

11. Dear ImGui is distributed as source files to be compiled into each application, so we look for its directory rather than for a package.

⟨Find the Dear ImGui sources 11⟩ ≡

```
set(IMGUI_DIR "$ENV{IMGUI_DIR}" CACHE PATH "Dear ImGui source directory")
if(NOT IMGUI_DIR)
  set(IMGUI_DIR "${VIZ_PREFIX}/imgui-1.93.0" CACHE PATH "" FORCE)
endif()
if(NOT EXISTS "${IMGUI_DIR}/imgui.h")
  message(FATAL_ERROR "imgui.h not found in IMGUI_DIR=${IMGUI_DIR}")
endif()
```

This code is used in section 8.

12. The CloudXR runtime is not linked either: when the reader asks for it with `--cloudxr`, the program loads `libcloudxr.so` from a CloudXR SDK directory. That directory is given by the option `--cloudxr-sdk`, or else by the environment variable *CLOUDXR_SDK_DIR*; failing both, the program uses the directory recorded here at build time, which the executable receives as the definition *CLOUDXR_SDK_DEFAULT* (see ⟨Build Dear ImGui and the viewer executable 16⟩).

⟨Choose the default CloudXR SDK 12⟩ ≡

```
# Default for --cloudxr-sdk when CLOUDXR_SDK_DIR is not set at run time
set(CLOUDXR_SDK_DIR "${VIZ_PREFIX}/cloudxr-6.2.1" CACHE PATH "CloudXR SDK (libcloudxr.so, openxr_
      cloudxr.json)")
```

This code is used in section 8.

13. The remaining dependencies provide CMake package files, which *find_package* looks for under the directories we add to *CMAKE_PREFIX_PATH*. OpenUSD's package is called `pxr`, after Pixar. meshoptimizer, a library by Arseny Kapoulkine, welds the corners of each mesh into shared vertices and reorders them for the GPU; the part on the scene shows how.

⟨Find SDL3, OpenUSD, meshoptimizer and CUDA 13⟩ ≡

```
list(APPEND CMAKE_PREFIX_PATH "${VIZ_PREFIX}/sdl3" "${VIZ_PREFIX}/usd-26.11" "${VIZ_PREFIX}/
      meshoptimizer-1.3"
    "${VIZ_PREFIX}/openxr-1.1.58")

find_package(CUDAToolkit 12 REQUIRED)
find_package(SDL3 REQUIRED CONFIG)
find_package(pxr REQUIRED CONFIG)
find_package(meshoptimizer REQUIRED CONFIG)
```

This code is used in section 8.

14. The XR output needs two more packages: the OpenXR loader, which finds the active OpenXR runtime and forwards calls to it, and the Vulkan headers, because the runtime talks to the application in Vulkan terms. The loader is installed with the other tools as a static library, so the binary carries it and needs no

extra shared library. Of Vulkan only the headers are used at build time: the program opens the driver's `libvulkan.so.1` at run time (see the part on the XR output). Without a headset none of this is used.

⟨Find the OpenXR loader and the Vulkan headers 14⟩ ≡

```
# XR output: the OpenXR loader is linked statically. Only the Vulkan headers
# are used at build time; Vulkan itself is loaded at run time from the
# driver's libvulkan.so.1, and the CloudXR runtime from the CloudXR SDK.
find_package(OpenXR REQUIRED CONFIG)
find_package(Vulkan REQUIRED)
```

This code is used in section 8.

15. Device code. The programs that run on the GPU (described in the part on device programs) are written in CUDA C++, but OptiX does not take machine code. We ask `nvcc` for *OptiX-IR*, the intermediate representation that NVIDIA recommends as the input of OptiX modules. It is a binary form of the program that is independent of the GPU model; the OptiX compiler in the driver translates it for the actual GPU when the pipeline is created. The target is the virtual architecture `compute_75`, the oldest one supported by CUDA 13, so that the same executable runs on any recent GPU. The option `-lineinfo` keeps the correspondence between the code and the source lines, which debuggers and profilers such as Nsight Compute use to point at the line of `devicePrograms.cu` a cost belongs to; it does not slow the code down. The warning 20044 that we silence is `nvcc`'s remark that an **extern** declaration of the launch parameters is treated as a definition, which is exactly what OptiX expects.

The OptiX-IR file is then turned into a C++ source file by a small CMake script, ⟨`UsdOptixViewer/cmake/EmbedFile.cmake` 18⟩, and compiled into the executable. The viewer therefore needs no data files at run time.

⟨Compile the device programs to OptiX-IR and embed them 15⟩ ≡

```
# ------------------------------------------------------------------------
# OptiX device code: compiled to OptiX-IR with nvcc and embedded into the binary.
# ------------------------------------------------------------------------
set(OPTIX_ARCH "compute_75" CACHE STRING "Virtual architecture used for the OptiX-IR")
set(OPTIXIR_FILE "${CMAKE_CURRENT_BINARY_DIR}/devicePrograms.optixir")
set(OPTIXIR_CPP  "${CMAKE_CURRENT_BINARY_DIR}/devicePrograms_optixir.cpp")
add_custom_command(
  OUTPUT "${OPTIXIR_FILE}"
  COMMAND "${CUDAToolkit_NVCC_EXECUTABLE}" -optix-ir -arch=${OPTIX_ARCH} -std=c++17 --use_fast_
        math -lineinfo
         -diag-suppress 20044 -ccbin "${CMAKE_CXX_COMPILER}"
         -I "${OptiX_INCLUDE_DIR}" -I "${CMAKE_CURRENT_SOURCE_DIR}/src"
         "${CMAKE_CURRENT_SOURCE_DIR}/src/devicePrograms.cu" -o "${OPTIXIR_FILE}"
  DEPENDS src/devicePrograms.cu src/LaunchParams.h
  COMMENT "Compiling OptiX programs to OptiX-IR")
add_custom_command(
  OUTPUT "${OPTIXIR_CPP}"
  COMMAND "${CMAKE_COMMAND}" -DINPUT=${OPTIXIR_FILE} -DOUTPUT=${OPTIXIR_CPP} -DSYMBOL=g_
        devicePrograms
         -P "${CMAKE_CURRENT_SOURCE_DIR}/cmake/EmbedFile.cmake"
  DEPENDS "${OPTIXIR_FILE}" cmake/EmbedFile.cmake
  COMMENT "Embedding OptiX-IR")
```

This code is used in section 6.

16. The executable. Dear ImGui and its two back ends (SDL3 for input, SDL_Renderer for drawing) become a static library. The viewer itself consists of seven C++ files plus the generated OptiX-IR source. It links meshoptimizer and the parts of OpenUSD it uses: `usd` for stages, `usdGeom` for geometry, `usdShade` for materials, `ar` for resolving and opening assets, and the base libraries `sdf`, `tf`, `gf` and `vt`. It also links the OpenXR loader and uses the Vulkan headers. Two definitions apply to every file: *VK_NO_PROTOTYPES*, because Vulkan functions are looked up at run time rather than linked, and *XR_USE_GRAPHICS_API_VULKAN*, which makes `openxr_platform.h` declare the Vulkan binding. A third, *CLOUDXR_SDK_DEFAULT*, carries the default CloudXR SDK directory of ⟨Choose the default CloudXR SDK 12⟩ into the code as a string literal.

⟨Build Dear ImGui and the viewer executable 16⟩ ≡

```
# ------------------------------------------------------------------------
# Application
# ------------------------------------------------------------------------
add_library(imgui STATIC
  ${IMGUI_DIR}/imgui.cpp ${IMGUI_DIR}/imgui_draw.cpp ${IMGUI_DIR}/imgui_tables.cpp
  ${IMGUI_DIR}/imgui_widgets.cpp ${IMGUI_DIR}/imgui_demo.cpp
  ${IMGUI_DIR}/backends/imgui_impl_sdl3.cpp ${IMGUI_DIR}/backends/imgui_impl_sdlrenderer3.cpp)
target_include_directories(imgui PUBLIC ${IMGUI_DIR} ${IMGUI_DIR}/backends)
target_link_libraries(imgui PUBLIC SDL3::SDL3)

add_executable(UsdOptixViewer
  src/main.cpp src/App.cpp src/Scene.cpp src/OptixRenderer.cpp src/StbImage.cpp src/XrOutput.cpp
  src/CloudXrService.cpp "${OPTIXIR_CPP}")
target_include_directories(UsdOptixViewer PRIVATE src ${OptiX_INCLUDE_DIR})
target_link_libraries(UsdOptixViewer PRIVATE
  imgui SDL3::SDL3 meshoptimizer::meshoptimizer usd usdGeom usdShade ar sdf tf gf vt
  CUDA::cudart CUDA::cuda_driver OpenXR::openxr_loader Vulkan::Headers ${CMAKE_DL_LIBS})
target_compile_definitions(UsdOptixViewer PRIVATE VK_NO_PROTOTYPES XR_USE_GRAPHICS_API_VULKAN
  CLOUDXR_SDK_DEFAULT="${CLOUDXR_SDK_DIR}")
# USD headers trigger a few deprecation warnings under C++20
target_compile_options(UsdOptixViewer PRIVATE -Wno-deprecated -Wno-deprecated-declarations)
```

This code is used in section 6.

17. Installation copies the executable and the test scenes. The libraries it depends on stay where they are: the run-time search path (RPATH) of the installed binary points to them, so the program starts without any `LD_LIBRARY_PATH` settings.

⟨Install the viewer 17⟩ ≡

```
# ------------------------------------------------------------------------
# Installation: bin/UsdOptixViewer + sample scenes. RPATH points to the
# dependency libraries so the installed binary runs without LD_LIBRARY_PATH.
# ------------------------------------------------------------------------
include(GNUInstallDirs)
set_target_properties(UsdOptixViewer PROPERTIES
  INSTALL_RPATH_USE_LINK_PATH ON
  INSTALL_RPATH "\$ORIGIN/../${CMAKE_INSTALL_LIBDIR}")
install(TARGETS UsdOptixViewer RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR})
install(DIRECTORY assets/ DESTINATION ${CMAKE_INSTALL_DATADIR}/UsdOptixViewer/assets)
```

This code is used in section 6.

18. Embedding a file. CMake has no command to turn a file into a C++ array, but its scripting language is enough. We read the OptiX-IR as a hexadecimal string, turn every pair of hex digits into a byte literal, and write a source file that defines two symbols: the bytes followed by a terminating zero, and the size without it. The renderer declares both as **extern** and hands them to OptiX.

⟨`UsdOptixViewer/cmake/EmbedFile.cmake` 18⟩ ≡

```
# Usage: cmake -DINPUT=file -DOUTPUT=file.cpp -DSYMBOL=name -P EmbedFile.cmake
# Writes a C++ source defining `extern const unsigned char SYMBOL[]` (NUL terminated)
# and `extern const size_t SYMBOL_size` (without the terminator).
file(READ "${INPUT}" content HEX)
string(LENGTH "${content}" hexLength)
math(EXPR size "${hexLength} / 2")
string(REGEX REPLACE "([0-9a-f][0-9a-f])" "0x\\1," bytes "${content}")
file(WRITE "${OUTPUT}"
  "#include <cstddef>\n"
  "extern const unsigned char ${SYMBOL}[] = {${bytes}0x00};\n"
  "extern const size_t ${SYMBOL}_size = ${size};\n")
```

This code is written to file `UsdOptixViewer/cmake/EmbedFile.cmake`.

Shared parameters

19. The CPU and the GPU must agree on one thing above all: the layout of the data that the host hands to every OptiX launch. That data is described once, in `LaunchParams.h`, and the header is compiled twice: by the host C++ compiler for `OptixRenderer.cpp`, which fills the structures in, and by `nvcc` for `devicePrograms.cu`, which reads them. Because both sides see the very same declarations, a field added here cannot be forgotten on one side; it can only be left unset.

⟨`UsdOptixViewer/src/LaunchParams.h` 19⟩ ≡

⟨Opening of the shared parameters header 20⟩

⟨The background mesh id and the ray types 21⟩

⟨The camera frame 22⟩

⟨Scalar material inputs 23⟩

⟨Material data on the device 24⟩

⟨Mesh data on the device 27⟩

⟨The launch parameters 28⟩

This code is written to file `UsdOptixViewer/src/LaunchParams.h`.

20. The header includes only CUDA and OptiX type definitions, never their function declarations. From OptiX we take *OptixTraversableHandle*, from CUDA's texture types *cudaTextureObject_t*, and from CUDA's vector types *float3*, *float4*, *uchar4* and friends. These headers declare the same sizes and the same alignments for the host compiler and for `nvcc`, which is what makes a byte-for-byte copy from one side to the other legitimate.

⟨Opening of the shared parameters header 20⟩ ≡

```
// Data shared between the host (OptixRenderer.cpp) and the OptiX programs
// (devicePrograms.cu).
#pragma once

#include <optix_types.h>
#include <texture_types.h>
#include <vector_types.h>
```

This code is used in section 19.

21. When a camera ray hits nothing, the pixel belongs to no mesh. We mark such pixels with the largest unsigned integer, *kNoMesh*, which can never be a valid mesh index. The application reads these ids back to implement picking: a click on the background clears the selection.

The renderer traces two kinds of rays. *Radiance* rays ask "what colour arrives along this ray?"; *shadow* rays only ask "is the light visible from here?". OptiX selects the program that handles a miss, and the record that handles a hit, by an index into the shader binding table; the enumeration gives these indices names. *RAY_TYPE_COUNT* is the number of hit records, one per ray type, and the stride that every call of *optixTrace* passes for them.

⟨The background mesh id and the ray types 21⟩ ≡

```
constexpr unsigned int kNoMesh = 0xffffffffu;

enum RayType { RAY_TYPE_RADIANCE = 0, RAY_TYPE_SHADOW = 1, RAY_TYPE_COUNT };
```

This code is used in section 19.

22. The camera is usually a pinhole. Rather than storing a field of view and an aspect ratio, the host precomputes three vectors: w is the unit vector from the eye towards the centre of the image plane, while u and v point from that centre to the middle of the right and top edges of the image when the plane is at distance 1. A point of the image with normalized device coordinates $(x, y) \in [-1, 1]^2$ then lies in the direction $w + x\,u + y\,v$, and the ray generation program needs no trigonometry at all. If ϕ is the vertical field of view and a the aspect ratio, the host sets $|w| = 1$, $|v| = \tan(\phi/2)$ and $|u| = a\tan(\phi/2)$.

A USD camera may also be orthographic. Then all rays are parallel to w and the image point moves the origin instead: the ray leaves $e + x\,u + y\,v$, and $|u|$, $|v|$ are half the width and height of the view in scene units. USD cameras also carry a clipping range. The renderer honours it for camera rays, as a depth measured along w, so that a near plane can cut away the wall in front of an interior shot. The orbit camera uses $[0, 10^{16}]$.

The eyes of a headset have asymmetric frusta: the image plane is not centred on the viewing direction. Such a camera still fits the formula if w points at the centre of the window rather than straight ahead; it is then no longer a unit vector, which matters only to the clipping range, so XR eyes use none.

⟨The camera frame 22⟩ ≡
```
// Perspective: rays leave `eye` through w + u * x + v * y, for x, y in [-1, 1]
// (u, v span the image plane at distance 1). Orthographic: rays leave
// eye + u * x + v * y along w (u, v are half the view size in scene units).
// Camera rays only see hits between clipNear and clipFar along w.
struct Camera
{
    float3 eye;
    float3 u, v, w;   // right, up and forward (w normalized, except for XR eyes, which have no clipping)
    unsigned int orthographic;
    float clipNear, clipFar;
};
```
This code is used in section 19.

23. Many inputs of a UsdPreviewSurface material, such as roughness or metallic, are single numbers that may either be constant or come from one channel of a texture. A *ScalarParam* holds both possibilities. When *texture* is negative the value is *value*; otherwise it is channel *channel* (0 to 3 for red, green, blue and alpha) of the texture, mapped through the *scale* and *bias* that the UsdUVTexture node specifies for that channel. Keeping the channel number lets one texture carry several inputs, as in the common "packed" maps where green is roughness and blue is metallic.

⟨Scalar material inputs 23⟩ ≡
```
// A float input: `value`, or texture[channel] * scale + bias when texture >= 0.
struct ScalarParam
{
    float value;
    int   texture;
    int   channel;
    float scale, bias;
};
```
This code is used in section 19.

24. Materials. The scene part of the program reduces every UsdPreviewSurface it meets to one *MaterialData*, and each triangle refers to one of them by index. The comment summarizes how the closest-hit program interprets the fields; we shall see each formula again when it is used.

⟨Material data on the device 24⟩ ≡
```
// UsdPreviewSurface subset.
// diffuse = texture(st) * texScale + texBias, or `color` when untextured
// cutout = hits where opacityTexture(st)[opacityChannel] < opacityThreshold are ignored
// normal = normalTexture(st) * normalScale + normalBias (tangent space)
// specular = GGX; F0 from metallic + ior, or specColor in the specular workflow
struct MaterialData
{
    ⟨Texture remapping coefficients of a material 25⟩
    ⟨Colors, textures and inputs of a material 26⟩
};
```
This code is used in section 19.

25. The six vectors are the *scale* and *bias* inputs of the UsdUVTexture nodes that feed the diffuse color, the normal and the specular color. They are applied componentwise to every texel, $c = s \cdot t + b$. For a normal map the specification asks for $s = (2, 2, 2)$ and $b = (-1, -1, -1)$, which maps the stored range $[0, 1]$ to the vector range $[-1, 1]$; files that use the DirectX convention flip the green channel through these same coefficients.

These vectors open the structure for a reason of layout. A *float4* must be aligned on a 16-byte boundary, while a *float3*, an **int** or a **float** needs only 4 bytes. Placed at the head of the structure, which is itself 16-byte aligned, the six *float4* members fill its first 96 bytes with no gap between them.

⟨Texture remapping coefficients of a material 25⟩ ≡
```
float4 texScale;
```

```
float4 texBias;
float4 normalScale;
float4 normalBias;
float4 specScale;
float4 specBias;
```

This code is used in section 24.

26. Then come the remaining inputs. *color* is the diffuse color of an untextured material. *specColor* and *specTexture* are used only in the *specular workflow*, where *specularWorkflow* is nonzero and the reflectance at normal incidence is given directly; otherwise the *metallic workflow* derives it from *metallic* and the index of refraction *ior*. The opacity fields describe an alpha cutout: the *opacityChannel* of *opacityTexture* is compared with *opacityThreshold*.

Every one of these members needs only 4-byte alignment, so none of them forces the compiler to insert padding. Each *float3* is followed by an **int**, which completes a 16-byte line, and the two *ScalarParam* records of 20 bytes each are followed by six 4-byte fields, which bring the total to 192 bytes, a multiple of 16. No padding is needed at the end either, and the structure has the same size and the same layout on both sides of the bus. Texture references are indices into the array of CUDA texture objects of ⟨The scene as seen by the device 31⟩, with −1 meaning "no texture".

⟨Colors, textures and inputs of a material 26⟩ ≡

```
float3 color;
int    texture;  // index into LaunchParams::textures, -1 for none
float3 specColor;
int    specTexture;
ScalarParam roughness;
ScalarParam metallic;
int    opacityTexture;
int    opacityChannel;
float  opacityThreshold;
int    normalTexture;
int    specularWorkflow;
float  ior;
```

This code is used in section 24.

27. Meshes. The scene keeps every renderable prim as one indexed triangle mesh in the prim's own local space, and the renderer builds a geometry acceleration structure (GAS) for each of them. One instance acceleration structure (IAS) then places the meshes in the world: instance i carries the transform of mesh i and points to its GAS. Since the instance id that the renderer gives to instance i is i itself, the closest-hit program learns from *optixGetInstanceId* both which *MeshData* to read and which mesh id to report for picking.

A *MeshData* holds device pointers to the arrays of one mesh. Vertices are indexed: *indices* gives the three vertices of each triangle, and *positions*, *normals* and *uvs* are per vertex, so a vertex has one position, one shading normal and one pair of texture coordinates. Face-varying data, where the corner of one face needs a value different from the same point on the neighbouring face, is handled by the scene before it gets here: such corners simply become different vertices. A mesh without smooth normals has a null *normals* pointer and is shaded with the flat geometric normal; one without texture coordinates has a null *uvs*. *triMaterial* gives the material of each triangle, since USD geometry subsets may bind different materials to the faces of one mesh. All of these are in local space; the device programs carry hit points and normals to world space with the transforms of the instance.

⟨Mesh data on the device 27⟩ ≡

```
// One mesh, in its local space. Instance i of the IAS is mesh i, so
// optixGetInstanceId() is both the index into LaunchParams::meshes and the
// mesh id used for picking.
struct MeshData
{
    const float3*        positions;
    const float3*        normals;      // shading normal per vertex; null: flat
    const float2*        uvs;          // st per vertex; null: no texture coordinates
    const uint3*         indices;      // 3 vertices per triangle
    const unsigned int* triMaterial;   // material of each triangle
```

```
};
```

This code is used in section 19.

28. The launch parameters. Everything a launch needs is gathered in a single structure. The host copies it to a small device buffer before each launch, and OptiX makes it visible to all programs as the *__constant__* variable *params*. Constant memory is cached and broadcast to all threads of a warp, which suits data that every ray reads and no ray writes; the output buffers it points to, of course, live in ordinary global memory.

⟨The launch parameters 28⟩ ≡

```
struct LaunchParams
{
    ⟨Output buffers of a launch 29⟩
    ⟨Camera, light and rendering switches 30⟩
    ⟨The scene as seen by the device 31⟩
};
```

This code is used in section 19.

29. The output of a launch. Rendering is progressive: each launch traces one new sample per pixel. *accumBuffer* keeps, for every pixel, the running sum of the samples in its first three components and their number in the fourth, and *colorBuffer* receives their average, encoded for display. *frameIndex* counts the samples already taken; the host sets it back to zero whenever the camera, the scene or any option changes, and the next launch then overwrites the sums instead of adding to them. *meshIdBuffer* records which mesh each pixel shows, and *hitBuffer* the world-space point it shows, with a fourth component of 1 (0 for the background); the orbit camera turns around such points. Both are written only on frame 0, whose rays go through the pixel centres. *hitBuffer* may be null, for views that need no points.

⟨Output buffers of a launch 29⟩ ≡

```
// output: colorBuffer = accumBuffer / (frameIndex + 1); meshIdBuffer and hitBuffer are written on frame 0
uchar4*          colorBuffer;
float4*          accumBuffer;
unsigned int*    meshIdBuffer;   // mesh under each pixel (kNoMesh for background)
float4*          hitBuffer;      // world-space point under each pixel, w = 1 (w = 0 for background); may be null
unsigned int     width, height;
unsigned int     frameIndex;     // progressive sample index, reset to 0 after any change
```

This code is used in section 28.

30. Next come the camera, the single directional light, and the switches of the *View* menu, each an integer used as a boolean. Turning a switch off does not change the scene, only how it is drawn, which is handy for comparing the effect of a feature. *selectedMesh* is the mesh highlighted in orange, or *kNoMesh*.

⟨Camera, light and rendering switches 30⟩ ≡

```
Camera camera;
float3 lightDir;   // direction towards the light, normalized
unsigned int shadows;
unsigned int mipmaps;        // 0: always sample the finest texture level
unsigned int normalMaps;     // 0: ignore normal maps
unsigned int reflections;    // 0: reflect the sky only, without tracing rays
unsigned int selectedMesh;
```

This code is used in section 28.

31. The scene. *handle* is the traversable handle of the IAS, the one that *optixTrace* needs. *meshes* is the array of *MeshData* records, indexed by instance id, *materials* the array of materials that *triMaterial* refers to, and *textures* the CUDA texture objects that the materials refer to by index.

⟨The scene as seen by the device 31⟩ ≡

```
// scene: an IAS with one instance (and one GAS) per mesh
OptixTraversableHandle handle;
const MeshData*          meshes;
const MaterialData*      materials;
const cudaTextureObject_t* textures;
```

This code is used in section 28.

The scene

32. Everything the viewer knows about a USD file passes through one object of class *Scene*. The USD stage is itself the scene graph: the user interface asks it directly for the hierarchy, the names and the types of the prims. What the stage does not offer is the form of the data that a ray tracer wants, and producing that form is the scene's work. It has three jobs. First, it turns every visible piece of geometry into a compact indexed triangle mesh, kept in the prim's own coordinate system and placed in the world by a transform. Second, it resolves the materials of the scene: it follows UsdShade connections to the inputs of the UsdPreviewSurface shader, and loads the textures they name. Third, it collects the cameras of the stage, so that the viewer can look through them.

The declarations live in `Scene.h` and the code in `Scene.cpp`. The header is short enough to be read as a table of contents for this part.

⟨`UsdOptixViewer/src/Scene.h` 32⟩ ≡

⟨Headers needed by the scene declarations 33⟩

⟨Texture wrap modes and decoded textures 62⟩

⟨The ScalarInput material parameter 63⟩

⟨The Material record 64⟩

⟨A renderable mesh 34⟩

⟨A camera of the stage 106⟩

⟨The Scene class 35⟩

This code is written to file `UsdOptixViewer/src/Scene.h`.

33. The header needs the standard containers and two USD headers: the stage itself and the UsdShade material class, which appears in the signature of a private member function. The hash map is the index from prim paths to meshes.

⟨Headers needed by the scene declarations 33⟩ ≡

```
// A USD stage and what the renderer needs from it. The stage itself is the
// scene graph (hierarchy, names, types); the scene adds one indexed
// triangle mesh per renderable prim, its materials and textures, and the
// stage's cameras.
#pragma once

#include <array>
#include <map>
#include <string>
#include <tuple>
#include <unordered_map>
#include <vector>

#include <pxr/usd/usd/stage.h>
#include <pxr/usd/usdShade/material.h>
```

This code is used in section 32.

34. Meshes for the renderer. The renderer traces rays through a two-level acceleration structure. At the bottom, every mesh has its own *geometry acceleration structure* (GAS), built over its triangles in the mesh's own coordinates. At the top, one *instance acceleration structure* (IAS) holds an *instance* per mesh: a reference to its GAS, a 3×4 transform that places it in the world, and a 32-bit instance id. This is the shape in which OptiX expects a scene of many objects, and it fits USD well: every prim has a local-to-world transform, so its points can stay exactly as the file authors them, and the transform rides along in the instance. A moved object changes only its instance; its GAS, the expensive part, stays valid. The instance id is the index of the mesh, which the device programs use to find its attributes and which the renderer writes into its picking buffer, so that a click on the image leads back to the prim.

A *RenderMesh* is the record of one such mesh. Each vertex has a position and, when the prim has them, a shading normal and texture coordinates, all in the prim's local space. The triangles are given by *indices* and, in parallel, by the material of each triangle in *triMaterial*: when a ray hits triangle t of a mesh, OptiX reports the index t, and that is all the device needs to find the three vertices and the material. An empty

normals array means “shade flat”; an empty *uvs* array means that the prim has no texture coordinates. The remaining fields serve the inspector and the camera: the path of the prim, the material bound to the prim itself, the number of GeomSubsets that override it, how many triangle corners the prim had before identical ones were merged, and the bounds of the mesh in world space.

⟨A renderable mesh 34⟩ ≡
```
// One renderable prim (Mesh, Cube or Sphere) as an indexed triangle mesh in
// the prim's local space, placed in the world by `toWorld`. Vertices are the
// distinct corners of the source faces (position, shading normal and st),
// welded and reordered with meshoptimizer.
struct RenderMesh
{
    pxr::SdfPath path;
    std::array<float, 12> toWorld;            // row-major 3x4, acting on column vectors
    std::vector<float>    positions;          // xyz per vertex
    std::vector<float>    normals;            // xyz per vertex; empty for flat shading
    std::vector<float>    uvs;                // st per vertex; empty without texture coordinates
    std::vector<unsigned int> indices;        // 3 per triangle, counter-clockwise from outside
    std::vector<unsigned int> triMaterial;    // material of each triangle
    unsigned int material;                    // material bound to the prim itself
    unsigned int subsetMaterials;             // number of GeomSubsets with their own material
    size_t corners;                           // triangle corners before welding
    float boundsMin[3], boundsMax[3];         // world space

    size_t triangleCount() const { return indices.size() / 3; }
    size_t vertexCount() const { return positions.size() / 3; }
};
```
This code is used in section 32.

35. The *Scene* class is small from the outside. A scene is either empty or holds one open stage; *load* replaces whatever was there. The accessors give the user interface the stage, the meshes, materials, textures and cameras, and the list of warnings collected while reading the file. The number of prims and the total number of triangles are for the status bar. The up axis matters to the camera, which must know whether “up” is $+y$ or $+z$, and the bounds of the whole scene let it frame everything.

⟨The Scene class 35⟩ ≡
```
class Scene
{
public:
    ⟨Public interface of the scene 36⟩

private:
    ⟨Private helpers of the scene 38⟩

    ⟨Data members of the scene 37⟩
};
```
This code is used in section 32.

36. A prim is identified by its *SdfPath* everywhere in the program: the selection is a path, and the hierarchy panel walks the stage. Two lookups turn a path into the index of its mesh or of its camera, with -1 for a prim that has neither, so “is this prim drawn?” is simply “is *meshIndex* nonnegative?”.

⟨Public interface of the scene 36⟩ ≡
```
bool load(const std::string& path, std::string& error);
void clear();

bool loaded() const { return static_cast<bool>(m_stage); }
const std::string& filePath() const { return m_filePath; }
const pxr::UsdStageRefPtr& stage() const { return m_stage; }
size_t primCount() const { return m_primCount; }
// The mesh id of a mesh is its index in meshes().
const std::vector<RenderMesh>& meshes() const { return m_meshes; }
const std::vector<Material>& materials() const { return m_materials; }
const std::vector<Texture>& textures() const { return m_textures; }
const std::vector<SceneCamera>& cameras() const { return m_cameras; }
const std::vector<std::string>& warnings() const { return m_warnings; }
```

```
bool zUp() const { return m_zUp; }
const float* boundsMin() const { return m_boundsMin; }
const float* boundsMax() const { return m_boundsMax; }
size_t triangleCount() const;

// -1 when the prim has no mesh or is not a camera
int meshIndex(const pxr::SdfPath& path) const;
int cameraIndex(const pxr::SdfPath& path) const;
```

This code is used in section 35.

37. Besides the stage and the vectors that the accessors expose, the scene keeps the time at which the stage is sampled, the index from paths to meshes, and two caches that we shall explain with the materials: a material is converted only once however many prims bind it, and a texture is decoded only once however many materials read it. Materials and textures refer to each other, and meshes to materials, by their indices in these vectors.

⟨Data members of the scene 37⟩ ≡

```
std::string            m_filePath;
pxr::UsdStageRefPtr    m_stage;
pxr::UsdTimeCode       m_time;
size_t                 m_primCount = 0;
std::vector<RenderMesh>   m_meshes;
std::vector<Material>     m_materials;
std::vector<Texture>      m_textures;
std::vector<SceneCamera>  m_cameras;
std::vector<std::string>  m_warnings;
std::unordered_map<pxr::SdfPath, unsigned int, pxr::SdfPath::Hash> m_meshByPath;
std::map<pxr::SdfPath, unsigned int> m_materialCache;
std::map<std::tuple<std::string, bool, int, int>, int> m_textureCache;
float                  m_boundsMin[3] = {0, 0, 0};
float                  m_boundsMax[3] = {0, 0, 0};
bool                   m_zUp = false;
```

This code is used in section 35.

38. The private member functions do the real work, and each has its own sections below.

⟨Private helpers of the scene 38⟩ ≡

```
void extractGeometry();
void extractCameras();
unsigned int materialFor(const pxr::UsdShadeMaterial& material);
unsigned int displayColorMaterial(const pxr::UsdPrim& prim);
// srgbByDefault: color space when sourceColorSpace is neither "raw" nor "sRGB"
int loadTexture(const pxr::UsdShadeShader& shader, bool srgbByDefault = true);
ScalarInput scalarInput(const pxr::UsdShadeShader& surface, const char* name, float fallback);
void warn(const std::string& message);
```

This code is used in section 35.

39. The implementation file. `Scene.cpp` begins with a number of helpers that know nothing about the *Scene* class: they turn USD geometry into triangles, weld the triangles into indexed meshes and read values out of shading networks. They live in an anonymous namespace. The member functions follow.

⟨`UsdOptixViewer/src/Scene.cpp` 39⟩ ≡

```
⟨Headers needed by the scene implementation 40⟩

namespace {

⟨Local triangle soup of one prim 50⟩

⟨Convert a UsdGeomMesh into a local triangle soup 51⟩

⟨Close the bookkeeping of a generated primitive 59⟩

⟨Generate the triangles of a cube 60⟩

⟨Generate the triangles of a sphere 61⟩

⟨Weld the corners of a prim into an indexed mesh 94⟩
```

⟨Read the value of a shader input 68⟩

⟨Find the shader that drives an input 69⟩

⟨Identify a shader and its wrap modes 70⟩

} // namespace

⟨Open a stage and build the scene 41⟩

⟨Forget the current stage 42⟩

⟨Collect a warning once 43⟩

⟨Count the triangles of the scene 44⟩

⟨Find the mesh or the camera of a prim 45⟩

⟨Material for a prim without a bound material 71⟩

⟨Convert a UsdShade material into a Material record 72⟩

⟨Read a float input that may come from a texture 85⟩

⟨Load a texture once 86⟩

⟨Extract the renderable meshes of the stage 46⟩

⟨Collect the cameras of the stage 107⟩

This code is written to file `UsdOptixViewer/src/Scene.cpp`.

40. Besides the USD schemas for geometry, shading and cameras, we need the prim range that walks the stage, the asset resolver (to read textures, even from inside `.usdz` packages), *meshoptimizer* (to weld and reorder triangles) and the image decoder. <*numeric*> provides *std*::*iota*. *PXR_NAMESPACE_USING_DIRECTIVE* lets us write *UsdPrim* instead of *pxr*::*UsdPrim* in this file.

⟨Headers needed by the scene implementation 40⟩ ≡

```
#include "Scene.h"

#include <algorithm>
#include <cmath>
#include <numeric>

#include <meshoptimizer.h>

#include <pxr/base/gf/camera.h>
#include <pxr/base/gf/frustum.h>
#include <pxr/base/gf/matrix4d.h>
#include <pxr/usd/ar/asset.h>
#include <pxr/usd/ar/resolver.h>
#include <pxr/usd/usd/primRange.h>
#include <pxr/usd/usdGeom/camera.h>
#include <pxr/usd/usdGeom/cube.h>
#include <pxr/usd/usdGeom/mesh.h>
#include <pxr/usd/usdGeom/metrics.h>
#include <pxr/usd/usdGeom/primvarsAPI.h>
#include <pxr/usd/usdGeom/sphere.h>
#include <pxr/usd/usdGeom/subset.h>
#include <pxr/usd/usdGeom/tokens.h>
#include <pxr/usd/usdGeom/xformCache.h>
#include <pxr/usd/usdShade/materialBindingAPI.h>
#include <pxr/usd/usdShade/utils.h>

#include "third_party/stb_image.h"

PXR_NAMESPACE_USING_DIRECTIVE
```

This code is used in section 39.

41. Opening a stage. Loading starts from a clean slate. *UsdStage*::*Open* composes the file with everything it references; if that fails, the error message is returned to the caller, who shows it in a dialog.

Two global properties of the stage are recorded. The up axis is part of the stage metadata and defaults to y. The time is the start of the authored time range if there is one, and USD's *default* time otherwise: the viewer shows a single moment, and for an animated file the first frame is the natural choice. Then we count the prims, extract the geometry and collect the cameras.

The prims are walked with a *UsdPrimRange*, which visits the stage depth first, in the order of the file, skipping the invisible pseudo-root. The predicate *UsdTraverseInstanceProxies* deserves a word. USD shares the contents of *instanceable* prims: all instances of a model point to a single hidden *prototype*, and by default the children of an instance are not visited at all. With this predicate, the walk descends into every instance and yields *instance proxies*, read-only stand-ins for the prototype's prims at the instance's location. Each instance therefore gets its own row in the hierarchy panel and its own mesh, with its own copy of the triangles. The two-level acceleration structure could let all instances of a prototype share one GAS; the viewer keeps one mesh per prim, which is simpler, and costs memory only in scenes with very many instances. The hierarchy panel walks the stage with the same predicate, so the prim count and the tree agree.

⟨Open a stage and build the scene 41⟩ ≡

```
bool Scene::load(const std::string& path, std::string& error)
{
    clear();
    UsdStageRefPtr stage = UsdStage::Open(path);
    if (¬stage)
    {
        error = "Could not open USD stage: " + path;
        return false;
    }
    m_stage = stage;
    m_filePath = path;
    m_zUp = UsdGeomGetStageUpAxis(stage) ≡ UsdGeomTokens→z;
    m_time = stage→HasAuthoredTimeCodeRange() ? UsdTimeCode(stage→GetStartTimeCode()) :
            UsdTimeCode::Default();
    const UsdPrimRange prims = UsdPrimRange::Stage(stage, UsdTraverseInstanceProxies());
    m_primCount = size_t(std::distance(prims.begin(), prims.end()));
    extractGeometry();
    extractCameras();
    return true;
}
```

This code is used in section 39.

42. Clearing forgets the meshes, materials, textures and cameras, empties the index and both caches, and resets the bounds to the zero box. Releasing the stage pointer closes the stage, unless somebody else still holds it.

⟨Forget the current stage 42⟩ ≡

```
void Scene::clear()
{
    m_meshes.clear();
    m_meshByPath.clear();
    m_materials.clear();
    m_textures.clear();
    m_cameras.clear();
    m_warnings.clear();
    m_materialCache.clear();
    m_textureCache.clear();
    m_primCount = 0;
    std::fill_n(m_boundsMin, 3, 0.0f);
    std::fill_n(m_boundsMax, 3, 0.0f);
    m_stage = nullptr;
    m_filePath.clear();
}
```

This code is used in section 39.

43. Problems found while reading a file are not errors: an unsupported shader or a missing texture should not prevent us from showing the rest of the scene. They are collected as warnings, which the inspector lists. A large scene can produce the same complaint for hundreds of prims, so each distinct message is kept only once.

⟨Collect a warning once 43⟩ ≡

```
void Scene::warn(const std::string& message)
```

```
{
    if (std::find(m_warnings.begin(), m_warnings.end(), message) ≡ m_warnings.end())
        m_warnings.push_back(message);
}
```

This code is used in section 39.

44. The total number of triangles is summed over the meshes when asked; the status bar asks once a frame, and a scene has far fewer meshes than triangles.

⟨Count the triangles of the scene 44⟩ ≡

```
size_t Scene::triangleCount() const
{
    size_t count = 0;
    for (const RenderMesh& mesh : m_meshes)
        count += mesh.triangleCount();
    return count;
}
```

This code is used in section 39.

45. Meshes are looked up in a hash map filled as they are created, because the user interface asks about the selected prim every frame and a large scene has tens of thousands of meshes. Cameras are few, so a linear search by path is enough.

⟨Find the mesh or the camera of a prim 45⟩ ≡

```
int Scene::meshIndex(const SdfPath& path) const
{
    const auto it = m_meshByPath.find(path);
    return it ≡ m_meshByPath.end() ? -1 : int(it→second);
}
int Scene::cameraIndex(const SdfPath& path) const
{
    for (size_t i = 0; i < m_cameras.size(); ++i)
        if (m_cameras[i].path ≡ path.GetString())
            return int(i);
    return -1;
}
```

This code is used in section 39.

46. Extracting the geometry. Extraction is one pass over the prims. For each prim that should be drawn we produce its triangles in local space, decide the material of each face, read its texture coordinates, gather the attributes of every triangle corner, weld the corners into an indexed mesh, and record the transform that places it in the world. Each of these steps gets a section of its own.

⟨Extract the renderable meshes of the stage 46⟩ ≡

```
// ---- geometry ----------------------------------------------------------
void Scene::extractGeometry()
{
    ⟨Prepare the extraction of the geometry 47⟩
    for (const UsdPrim& p : UsdPrimRange::Stage(m_stage, UsdTraverseInstanceProxies()))
    {
        ⟨Skip prims that are not visible renderable geometry 48⟩
        ⟨Triangulate a prim in its local space 49⟩
        ⟨Assign a material to every face of a prim 90⟩
        ⟨Read the texture coordinates of a prim 91⟩
        ⟨Gather the corners of a prim's triangles 92⟩
        ⟨Build the indexed mesh of a prim 93⟩
        ⟨Pass the transform of a mesh to the renderer 101⟩
        ⟨Compute the world bounds of a mesh 102⟩
        ⟨Warn about textured prims without texture coordinates 103⟩
```

⟨Register the mesh of a prim 104⟩
}
⟨Record the bounds of the scene 105⟩
}

This code is used in section 39.

47. A *UsdGeomXformCache* computes local-to-world matrices and remembers the matrices of ancestors, so that siblings deep in the tree do not recompute the same chain of transforms. The bounds of the scene start empty, at $+\infty$ and $-\infty$.

⟨Prepare the extraction of the geometry 47⟩ ≡
UsdGeomXformCache xformCache(*m_time*);
float *lo*[3] = {*INFINITY*, *INFINITY*, *INFINITY*}, *hi*[3] = {−*INFINITY*, −*INFINITY*, −*INFINITY*};

This code is used in section 46.

48. Only *gprims*, the geometric primitives of UsdGeom, can be drawn. Visibility is inherited, so we ask USD to *compute* it rather than read the prim's own attribute: a mesh under an invisible transform is invisible. The *purpose* works the same way; we draw what a final render would draw (purpose **default** or *render*) and skip the *proxy* and *guide* geometry that applications use for previews and manipulators.

⟨Skip prims that are not visible renderable geometry 48⟩ ≡
if (¬*p.IsA*<*UsdGeomGprim*>())
continue;
const *UsdGeomImageable imageable*(*p*);
const *TfToken purpose* = *imageable.ComputePurpose*();
if (*imageable.ComputeVisibility*(*m_time*) ≡ *UsdGeomTokens*→*invisible* ∨
(*purpose* ≠ *UsdGeomTokens*→*default_* ∧ *purpose* ≠ *UsdGeomTokens*→*render*))
continue;

This code is used in section 46.

49. From prims to triangles. Three kinds of gprims are supported: polygonal meshes, and the implicit cube and sphere. Each is converted into a *LocalMesh*, a triangle soup in the prim's own coordinate system. The other gprims (cylinders, cones, capsules, curves, points) produce no triangles and are skipped. The default sizes are USD's: a cube of edge 2 and a sphere of radius 1.

⟨Triangulate a prim in its local space 49⟩ ≡
LocalMesh local;
if (*p.IsA*<*UsdGeomMesh*>())
meshFromUsdMesh(*UsdGeomMesh*(*p*), *m_time*, *local*);
else if (*p.IsA*<*UsdGeomCube*>())
{
double *size* = 2.0;
UsdGeomCube(*p*).*GetSizeAttr*().*Get*(&*size*, *m_time*);
meshFromCube(*size*, *local*);
}
else if (*p.IsA*<*UsdGeomSphere*>())
{
double *radius* = 1.0;
UsdGeomSphere(*p*).*GetRadiusAttr*().*Get*(&*radius*, *m_time*);
meshFromSphere(*radius*, *local*);
}
if (*local.indices.empty*())
continue;

This code is used in section 46.

50. A triangle soup must remember more than three point indices per triangle. USD attaches data to meshes at different rates, called *interpolations*: once per point (*vertex* or *varying*), once per face (*uniform*), once per corner of each face (*faceVarying*), or once for the whole mesh (*constant*). To fetch the right value for each corner of each triangle later, we keep, besides the point index, the *face-vertex* index of the corner (its position in USD's *faceVertexIndices* array) and the index of the face the triangle came from. The face index also serves GeomSubsets, which assign materials to sets of faces. The shading normals, when there are any, are resolved per corner right away.

⟨Local triangle soup of one prim 50⟩ ≡
```
// Local-space triangle soup produced for one prim.
struct LocalMesh
{
    std::vector<GfVec3f> points;
    std::vector<unsigned int> indices;      // 3 point indices per triangle
    std::vector<unsigned int> cornerFv;     // 3 face-vertex indices per triangle
    std::vector<unsigned int> faceOfTri;    // source face of each triangle
    std::vector<GfVec3f> cornerNormals;     // 3 shading normals per triangle, empty = flat
    size_t faceCount = 1;
};
```
This code is used in section 39.

51. A *UsdGeomMesh* describes polygons by three arrays: the points, the number of vertices of each face, and the concatenated vertex indices of all faces. Converting it takes four steps.

⟨Convert a UsdGeomMesh into a local triangle soup 51⟩ ≡
```
bool meshFromUsdMesh(const UsdGeomMesh& mesh, UsdTimeCode time, LocalMesh& out)
{
    ⟨Read the points and the face topology of a mesh 52⟩
    ⟨Fan-triangulate the faces of a mesh 53⟩
    ⟨Rewind the triangles of a left-handed mesh 54⟩
    ⟨Compute the shading normals of each triangle corner 55⟩
    return true;
}
```
This code is used in section 39.

52. A mesh lacking any of the three arrays has nothing to draw.

⟨Read the points and the face topology of a mesh 52⟩ ≡
```
VtArray<GfVec3f> points;
VtArray<int> counts, faceIndices;
if (¬mesh.GetPointsAttr().Get(&points, time) ∨ ¬mesh.GetFaceVertexCountsAttr().Get(&counts, time) ∨
    ¬mesh.GetFaceVertexIndicesAttr().Get(&faceIndices, time))
    return false;
```
This code is used in section 51.

53. A face with n vertices $v_0, \ldots, v_{n-1}$ becomes the $n-2$ triangles (v_0, v_k, v_{k+1}) for $1 \le k \le n-2$, a *fan* around the first vertex. This is exact for triangles and for convex polygons, which is what real assets contain almost exclusively; a concave polygon may come out wrong, and we accept that. The variable *offset* is the face-vertex index of the face's first corner, so *c0*, *c1* and *c2* are the face-vertex indices of the triangle's corners.

Files are not always well formed, and a bad index would make the GPU read outside its buffers. A negative or excessive vertex count ends the conversion; a triangle that names a nonexistent point is dropped.

⟨Fan-triangulate the faces of a mesh 53⟩ ≡
```
out.points.assign(points.begin(), points.end());
out.faceCount = counts.size();
size_t offset = 0;
for (size_t face = 0; face < counts.size(); ++face)
{
    const int count = counts[face];
    if (count < 0 ∨ offset + count > faceIndices.size())
        break;
    for (int k = 1; k + 1 < count; ++k)   // fan triangulation
    {
        const size_t c0 = offset, c1 = offset + k, c2 = offset + k + 1;
        const int i0 = faceIndices[c0], i1 = faceIndices[c1], i2 = faceIndices[c2];
        if (i0 < 0 ∨ i1 < 0 ∨ i2 < 0 ∨ size_t(std::max({i0, i1, i2})) ≥ points.size())
            continue;
        out.indices.insert(out.indices.end(), {unsigned(i0), unsigned(i1), unsigned(i2)});
        out.cornerFv.insert(out.cornerFv.end(), {unsigned(c0), unsigned(c1), unsigned(c2)});
        out.faceOfTri.push_back(unsigned(face));
```

```
    }
    offset += count;
}
```

This code is used in section 51.

54. USD meshes are right-handed by default: seen from the outside, the vertices of a face go counter-clockwise, and the cross product of two edges, $(p_1 - p_0) \times (p_2 - p_0)$, points outward. A mesh may declare itself *leftHanded*, with the opposite winding. The renderer relies on the cross product to know which side of a triangle faces out (it needs this for the normal-map frame and to orient shading normals), so we normalize the winding here, once: swapping two corners of each triangle reverses it. The face-vertex indices are swapped along, so that each corner keeps its own texture coordinates and normals.

```
⟨Rewind the triangles of a left-handed mesh 54⟩ ≡
    // Make every triangle counter-clockwise when seen from outside, so the
    // renderer can derive the outward normal (and the normal-map frame) from it.
    TfToken orientation;
    if (mesh.GetOrientationAttr().Get(&orientation) ∧ orientation ≡ UsdGeomTokens→leftHanded)
        for (size_t t = 0; t < out.indices.size(); t += 3)
        {
            std::swap(out.indices[t + 1], out.indices[t + 2]);
            std::swap(out.cornerFv[t + 1], out.cornerFv[t + 2]);
        }
```

This code is used in section 51.

55. Shading normals. A glossy highlight on a sphere made of flat facets shows every facet, so we interpolate *shading normals* across triangles when the mesh is meant to look smooth. The rules follow what Pixar's Hydra renderers do. Normals authored in the file win. Otherwise the subdivision scheme decides: USD's default scheme is Catmull–Clark, meaning "this cage describes a smooth surface", and we approximate that surface by smoothing the normals of the unrefined cage; a mesh with *subdivisionScheme* = `"none"` is a true polygonal mesh and is shaded flat. (The test scenes of this repository mark their boxes and walls *none* for exactly this reason: with the default scheme a cube shades like a rounded blob, which is correct USD behavior.)

```
⟨Compute the shading normals of each triangle corner 55⟩ ≡
    // Shading normals: authored primvars:normals or normals, otherwise smooth
    // vertex normals for subdivision surfaces (as Hydra does) and flat for
    // polygonal meshes.
    ⟨Find the authored normals of a mesh 56⟩
    ⟨Compute smooth normals for a subdivision surface 57⟩
    ⟨Pick the shading normal of each triangle corner 58⟩
```

This code is used in section 51.

56. Normals may be authored in two places. The primvar `primvars:normals` takes precedence over the older *normals* attribute, as the UsdGeom documentation prescribes. *ComputeFlattened* expands an *indexed* primvar (values plus an index array) into one value per element, so we never deal with the indices ourselves.

```
⟨Find the authored normals of a mesh 56⟩ ≡
    VtArray<GfVec3f> normals;
    TfToken interpolation;
    if (const UsdGeomPrimvar pv = UsdGeomPrimvarsAPI(mesh).GetPrimvar(UsdGeomTokens→normals);
        pv ∧ pv.ComputeFlattened(&normals, time))
        interpolation = pv.GetInterpolation();
    else if (mesh.GetNormalsAttr().Get(&normals, time))
        interpolation = mesh.GetNormalsInterpolation();
```

This code is used in section 55.

57. Smooth normals are accumulated per point. The cross product of two edges of a triangle has length twice its area, so summing the unnormalized cross products weights each triangle by its area: large faces dominate, and a sliver triangle does not tilt the normal. Since the triangles were rewound in the previous step, all the cross products point outward. The sums are normalized later, when the corners of the triangles are gathered.

⟨Compute smooth normals for a subdivision surface 57⟩ ≡

```
TfToken scheme;
mesh.GetSubdivisionSchemeAttr().Get(&scheme);
if (normals.empty() ∧ scheme ≠ UsdGeomTokens→none)
{
    normals.assign(points.size(), GfVec3f(0.0f));
    for (size_t t = 0; t < out.indices.size(); t += 3)
    {
        const unsigned i0 = out.indices[t], i1 = out.indices[t + 1], i2 = out.indices[t + 2];
        const GfVec3f n = GfCross(points[i1] − points[i0], points[i2] − points[i0]);  // area weighted
        normals[i0] += n;
        normals[i1] += n;
        normals[i2] += n;
    }
    interpolation = UsdGeomTokens→vertex;
}
```

This code is used in section 55.

58. Finally each corner looks up its normal according to the interpolation: by face-vertex index, by point index, or by face. A *constant* normal falls through all the tests and uses element 0. An index beyond the array, which only a malformed file produces, gives the zero vector; where all three corners of a triangle carry it, the renderer falls back to the triangle's flat normal. When there are no normals at all, *cornerNormals* stays empty and the whole mesh is shaded flat.

⟨Pick the shading normal of each triangle corner 58⟩ ≡

```
if (¬normals.empty())
    for (size_t t = 0; t < out.indices.size(); t += 3)
        for (size_t c = 0; c < 3; ++c)
        {
            size_t i = 0;
            if (interpolation ≡ UsdGeomTokens→faceVarying)
                i = out.cornerFv[t + c];
            else if (interpolation ≡ UsdGeomTokens→vertex ∨ interpolation ≡ UsdGeomTokens→varying)
                i = out.indices[t + c];
            else if (interpolation ≡ UsdGeomTokens→uniform)
                i = out.faceOfTri[t / 3];
            out.cornerNormals.push_back(i < normals.size() ? normals[i] : GfVec3f(0.0f));
        }
```

This code is used in section 55.

59. Implicit primitives. The cube and the sphere have no faces in USD's sense, so after generating their triangles we fill in the bookkeeping: each corner's "face-vertex" index is simply its point index, and the whole primitive counts as a single face, number 0. (These primitives carry no texture coordinates, so the choice only has to be consistent.)

⟨Close the bookkeeping of a generated primitive 59⟩ ≡

```
void finishPrimitive(LocalMesh& out)
{
    out.cornerFv = out.indices;
    out.faceOfTri.assign(out.indices.size() / 3, 0u);
}
```

This code is used in section 39.

60. The eight corners of a cube centered at the origin are numbered by three bits: bit 0 selects $\pm x$, bit 1 $\pm y$ and bit 2 $\pm z$. The twelve triangles, two per side, are listed so that each is counter-clockwise when seen from outside; for instance $(0, 2, 1)$ lies on the $-z$ side, and $(p_2 - p_0) \times (p_1 - p_0)$ is a positive multiple of $(0, 1, 0) \times (1, 0, 0) = (0, 0, -1)$, the outward direction. A cube has no shading normals: it is meant to be flat.

⟨Generate the triangles of a cube 60⟩ ≡

```
void meshFromCube(double size, LocalMesh& out)
{
    const float h = float(size) * 0.5f;
    for (int i = 0; i < 8; ++i)
```

```
        out.points.emplace_back(i & 1 ? h : −h, i & 2 ? h : −h, i & 4 ? h : −h);
    out.indices = {0, 2, 1, 1, 2, 3, 4, 5, 6, 5, 7, 6, 0, 1, 4, 1, 5, 4,
                   2, 6, 3, 3, 6, 7, 0, 4, 2, 2, 4, 6, 1, 3, 5, 3, 7, 5};
    finishPrimitive(out);
}
```

This code is used in section 39.

61. The sphere is a latitude–longitude grid: 24 stacks from the north pole ($\phi = 0$, on the $+y$ axis) to the south pole ($\phi = \pi$), and 48 slices around. A point is

$$r\,(\sin\phi\cos\theta,\ \cos\phi,\ \sin\phi\sin\theta).$$

Each grid cell, with corners a and $a+1$ on one stack and b and $b+1$ on the next, becomes two triangles. The order $(a, a+1, b)$ is the outward one: near the equator at $\theta = 0$, the edge from a to $a+1$ runs toward $+z$ and the edge from a to b runs toward $-y$, and $(0,0,1) \times (0,-1,0) = (1,0,0)$ points away from the center. The winding matters: shading normals must agree with it.

On a sphere centered at the origin the normal at a point is the point itself, scaled; we store the points as normals and let the gathering of the corners normalize them. Every corner thus gets an exact, smooth normal. The grid repeats points: the seam at $\theta = 0$ and $\theta = 2\pi$ has two copies of each point, and each pole has 49. Welding, below, merges them back.

⟨Generate the triangles of a sphere 61⟩ ≡

```
void meshFromSphere(double radius, LocalMesh& out)
{
    const int slices = 48, stacks = 24;
    for (int j = 0; j ≤ stacks; ++j)
    {
        const double phi = M_PI * j / stacks;
        for (int i = 0; i ≤ slices; ++i)
        {
            const double theta = 2.0 * M_PI * i / slices;
            out.points.emplace_back(float(radius * std::sin(phi) * std::cos(theta)), float(radius * std::cos(phi)),
                                    float(radius * std::sin(phi) * std::sin(theta)));
        }
    }
    for (int j = 0; j < stacks; ++j)
        for (int i = 0; i < slices; ++i)
        {
            const unsigned a = j * (slices + 1) + i, b = a + slices + 1;
            out.indices.insert(out.indices.end(), {a, a + 1, b, a + 1, b + 1, b});   // counter-clockwise from outside
        }
    finishPrimitive(out);
    for (unsigned idx : out.indices)
        out.cornerNormals.push_back(out.points[idx]);   // normalized later
}
```

This code is used in section 39.

62. Materials. The viewer understands a subset of *UsdPreviewSurface*, the simple physically based material that every USD tool can read. A material in USD is a small graph: a *Material* prim whose surface output is connected to a *UsdPreviewSurface* shader, whose inputs are either values or connections to other shaders, typically *UsdUVTexture* nodes, which in turn read their texture coordinates from *UsdPrimvarReader_float2* nodes. We condense such a graph into a flat *Material* record that the GPU can read without following any pointers.

The first kind of record field is used for decoded images. A *Texture* holds eight-bit RGBA pixels, whether they should be converted from sRGB to linear values when sampled, and what happens outside the unit square of texture coordinates in each direction. USD's *UsdUVTexture* offers repeat, mirror, clamp and black, which map directly to CUDA's address modes.

⟨Texture wrap modes and decoded textures 62⟩ ≡

```
// ---- render data ----------------------------------------------------------
enum class TextureWrap { Repeat, Mirror, Clamp, Black };

struct Texture
```

```
{
    std::string path;  // resolved path (may point inside a .usdz)
    int width = 0, height = 0;
    std::vector<unsigned char> rgba;
    bool srgb = true;
    TextureWrap wrapS = TextureWrap::Repeat, wrapT = TextureWrap::Repeat;
};
```

This code is used in section 32.

63. Several inputs of UsdPreviewSurface, such as roughness and metallic, are single numbers that may instead come from one channel of a texture. Packing several such maps into the channels of one image is common practice, so we record which channel to read, and that channel's scale and bias from the *UsdUVTexture* node, which maps the texel t to $t \cdot scale + bias$.

⟨The ScalarInput material parameter 63⟩ ≡

```
// A float material input: a constant, or one channel of a texture
// (texture[channel] * scale + bias).
struct ScalarInput
{
    float value = 0.0f;
    int texture = -1;
    int channel = 0;
    float scale = 1.0f, bias = 0.0f;
};
```

This code is used in section 32.

64. The *Material* record. Texture fields are indices into the scene's *textures*, with -1 for "none".

⟨The Material record 64⟩ ≡

```
// Simplified UsdPreviewSurface: diffuse color, optionally from a texture
// (texture * scale + bias), tangent-space normal maps, alpha cutouts
// (opacity + opacityThreshold) and GGX specular in the metallic or the
// specular workflow.
struct Material
{
    ⟨Diffuse, cutout and normal-map fields of a material 65⟩
    ⟨Specular fields of a material 66⟩
    ⟨Test whether a material reads any texture 67⟩
};
```

This code is used in section 32.

65. The name is for the inspector. The diffuse color defaults to a light gray, which is what a prim without any material or display color looks like. The texture coordinates are read from the primvar named by *uvSet*, almost always *st*. A cutout discards hits where one channel of a texture, usually alpha, falls below a threshold. The defaults of the normal-map remapping, $2t - 1$, turn the usual eight-bit encoding of a unit vector, with components in $[0, 1]$, back into components in $[-1, 1]$.

⟨Diffuse, cutout and normal-map fields of a material 65⟩ ≡

```
std::string name;  // material prim path, or "displayColor"
std::array<float, 3> color = {0.7f, 0.7f, 0.7f};
int texture = -1;
std::array<float, 4> scale = {1, 1, 1, 1};
std::array<float, 4> bias = {0, 0, 0, 0};
std::string uvSet = "st";  // primvar read by the texture
// alpha cutout: hits where opacityTexture[opacityChannel] < opacityThreshold are ignored
int opacityTexture = -1;
int opacityChannel = 3;
float opacityThreshold = 0.0f;
// tangent-space normal = normalTexture * normalScale + normalBias
int normalTexture = -1;
std::array<float, 4> normalScale = {2, 2, 2, 1};
std::array<float, 4> normalBias = {-1, -1, -1, 0};
```

This code is used in section 64.

66. UsdPreviewSurface has two ways of describing the specular reflection. In the default *metallic workflow*, a metallic factor blends between a dielectric, whose reflectance at normal incidence follows from the index of refraction, and a metal, which reflects with its diffuse color. In the *specular workflow* the reflectance at normal incidence is given directly as a color. The defaults are those of the specification: roughness 0.5, metallic 0, index of refraction 1.5.

```
⟨Specular fields of a material 66⟩ ≡
    ScalarInput roughness{0.5f};
    ScalarInput metallic{0.0f};
    float ior = 1.5f;
    // useSpecularWorkflow: specularColor replaces metallic as the reflectance at normal incidence
    bool specularWorkflow = false;
    std::array<float, 3> specularColor = {0, 0, 0};
    int specularTexture = -1;
    std::array<float, 4> specularScale = {1, 1, 1, 1};
    std::array<float, 4> specularBias = {0, 0, 0, 0};
```

This code is used in section 64.

67. A material that reads any texture needs texture coordinates. This question is asked when choosing which primvar to read and when deciding whether to warn about missing coordinates.

```
⟨Test whether a material reads any texture 67⟩ ≡
    bool usesTextures() const
    {
        return texture ≥ 0 ∨ opacityTexture ≥ 0 ∨ normalTexture ≥ 0 ∨ roughness.texture ≥ 0 ∨
                metallic.texture ≥ 0 ∨ specularTexture ≥ 0;
    }
```

This code is used in section 64.

68. Reading shading networks. UsdShade connections can be indirect: a shader input may be connected to an input of the enclosing *Material* (its *interface*), which may in turn be connected to something else, possibly through node graphs. *GetValueProducingAttributes* follows such chains to their ends and returns the attributes that actually produce the value: either outputs of shaders, or inputs that hold a value. We want a plain value, so we accept only the latter. This is how a texture whose file name is published on the material's interface, as in our test scene, still finds its file.

The function is a template because inputs come in many types: colors, floats, integers, tokens, asset paths. When the input exists but holds a value of another type, *Get* fails and so do we, and the caller keeps its default.

```
⟨Read the value of a shader input 68⟩ ≡
    // Value of a shader input, following connections to material interface
    // inputs. Returns false when the input is unset or driven by another shader.
    template <typename T>
    bool inputValue(const UsdShadeShader& shader, const char* name, T* value)
    {
        const UsdShadeInput input = shader.GetInput(TfToken(name));
        if (¬input)
            return false;
        for (const UsdAttribute& attr : input.GetValueProducingAttributes())
            if (UsdShadeUtils::GetType(attr.GetName()) ≡ UsdShadeAttributeType::Input ∧ attr.Get(value))
                return true;
        return false;
    }
```

This code is used in section 39.

69. The other question to ask of an input is "which shader drives it?". With *shaderOutputsOnly* set, the same function returns only shader outputs; we take the first. Callers sometimes need the output itself, because its name (*outputs:rgb*, *outputs:r*, …) says which channels are used, and sometimes only the shader.

```
⟨Find the shader that drives an input 69⟩ ≡
    // Shader output that drives the given input (through node graphs), if any.
```

```
UsdAttribute sourceOutput(const UsdShadeInput& input)
{
    if (input)
        for (const UsdAttribute& attr : input.GetValueProducingAttributes(/*shaderOutputsOnly=*/true))
            return attr;
    return UsdAttribute();
}
UsdShadeShader sourceShader(const UsdShadeInput& input)
{
    const UsdAttribute output = sourceOutput(input);
    return output ? UsdShadeShader(output.GetPrim()) : UsdShadeShader();
}
```

This code is used in section 39.

70. A shader's kind is its *info:id*, such as *UsdUVTexture*. The wrap mode *useMetadata*, USD's default, asks to use whatever the image file says; the formats we decode carry no such metadata, so it is treated as *repeat*, like anything unrecognized.

⟨Identify a shader and its wrap modes 70⟩ ≡

```
TfToken shaderId(const UsdShadeShader& shader)
{
    TfToken id;
    shader.GetShaderId(&id);
    return id;
}
TextureWrap wrapMode(const UsdShadeShader& shader, const char* name)
{
    TfToken mode;
    inputValue(shader, name, &mode);
    if (mode ≡ "mirror")
        return TextureWrap::Mirror;
    if (mode ≡ "clamp")
        return TextureWrap::Clamp;
    if (mode ≡ "black")
        return TextureWrap::Black;
    return TextureWrap::Repeat;   // "repeat" and "useMetadata"
}
```

This code is used in section 39.

71. A prim without a bound material is drawn with its *display color*, a primvar that every gprim may carry, and with all other properties at their defaults. Display colors differ from prim to prim, so these materials are not cached; each such prim gets its own record. A display color may vary over the surface; we use its first value only.

⟨Material for a prim without a bound material 71⟩ ≡

```
// ---- materials and textures ------------------------------------------------
unsigned int Scene::displayColorMaterial(const UsdPrim& prim)
{
    Material m;
    m.name = "displayColor";
    VtArray<GfVec3f> colors;
    if (UsdGeomGprim(prim).GetDisplayColorPrimvar().ComputeFlattened(&colors, m_time) ∧ ¬colors.empty())
        m.color = {colors[0][0], colors[0][1], colors[0][2]};
    m_materials.push_back(m);
    return unsigned(m_materials.size() − 1);
}
```

This code is used in section 39.

72. Converting a UsdShade material proceeds input by input. *ComputeSurfaceSource* finds the shader connected to the material's surface output. A material without one keeps the defaults; one with a surface shader has its inputs read one group at a time.

⟨Convert a UsdShade material into a Material record 72⟩ ≡
```
unsigned int Scene::materialFor(const UsdShadeMaterial& material)
{
    ⟨Return the cached Material of a UsdShade material 73⟩
    ⟨Start a Material with the UsdPreviewSurface defaults 74⟩
    if (const UsdShadeShader surface = material.ComputeSurfaceSource())
    {
        ⟨Read the diffuse color of a surface 76⟩
        ⟨Read the diffuse texture of a surface 77⟩
        ⟨Read an alpha cutout of a surface 80⟩
        ⟨Read the specular parameters of a surface 81⟩
        ⟨Read the normal map of a surface 82⟩
    }
    ⟨Store a new Material in the cache 75⟩
}
```
This code is used in section 39.

73. Many prims usually share a material, so the conversion is cached by the path of the material prim.

⟨Return the cached Material of a UsdShade material 73⟩ ≡
```
const SdfPath path = material.GetPath();
if (auto it = m_materialCache.find(path); it ≠ m_materialCache.end())
    return it→second;
```
This code is used in section 72.

74. A material that does bind but specifies nothing is a UsdPreviewSurface with all inputs at their defaults, and the default diffuse color of the specification is a dark gray, 0.18.

⟨Start a Material with the UsdPreviewSurface defaults 74⟩ ≡
```
Material m;
m.name = path.GetString();
m.color = {0.18f, 0.18f, 0.18f};   // UsdPreviewSurface default
```
This code is used in section 72.

75.

⟨Store a new Material in the cache 75⟩ ≡
```
m_materials.push_back(m);
const unsigned index = unsigned(m_materials.size() − 1);
m_materialCache[path] = index;
return index;
```
This code is used in section 72.

76. The diffuse color. A plain value, possibly published on the material's interface, is the easy case. But an input can carry both a value and a connection: many exporters write the color that a texture would average to next to the connection to the texture. *inputValue* does not return that value, because the connection wins, so we read the attribute's own value separately. It becomes the color used if the texture cannot be loaded, so a textured model whose image is missing still looks plausible.

⟨Read the diffuse color of a surface 76⟩ ≡
```
GfVec3f c;
if (inputValue(surface, "diffuseColor", &c))
    m.color = {c[0], c[1], c[2]};
else if (const UsdShadeInput in = surface.GetInput(TfToken("diffuseColor")); in ∧ in.GetAttr().Get(&c))
    m.color = {c[0], c[1], c[2]};   // authored default beside a connection
```
This code is used in section 72.

77. If *diffuseColor* is driven by a *UsdUVTexture*, the record points to the decoded image, and the texture's coordinates tell us which primvar to read. Any other kind of shader (a procedural, or a MaterialX node) is beyond us, and we say so.

⟨Read the diffuse texture of a surface 77⟩ ≡
```
const UsdShadeShader texture = sourceShader(surface.GetInput(TfToken("diffuseColor")));
if (texture ∧ shaderId(texture) ≡ "UsdUVTexture")
{
    ⟨Read the scale, bias and fallback of the diffuse texture 78⟩
    ⟨Find the primvar that holds the diffuse texture coordinates 79⟩
}
else if (texture)
    warn("Unsupported shader driving diffuseColor: " + shaderId(texture).GetString() + " (" + m.name +
        ")");
```
This code is used in section 72.

78. The texture node's own *fallback* value, meant for when the image is missing, takes precedence over the color found above. Diffuse textures are decoded as sRGB unless the node says otherwise, which is what nearly every color image is.

⟨Read the scale, bias and fallback of the diffuse texture 78⟩ ≡
```
GfVec4f v;
if (inputValue(texture, "scale", &v))
    m.scale = {v[0], v[1], v[2], v[3]};
if (inputValue(texture, "bias", &v))
    m.bias = {v[0], v[1], v[2], v[3]};
if (inputValue(texture, "fallback", &v))
    m.color = {v[0], v[1], v[2]};
m.texture = loadTexture(texture);
```
This code is used in section 77.

79. The *st* input of the texture is normally connected to a primvar reader, whose *varname* names the primvar. The specification says *varname* is a string, but older files, and some current exporters, write a token; we accept both. A *UsdTransform2d* node between the reader and the texture would scale, rotate or offset the coordinates; we do not implement it, and the coordinates are then read, untransformed, from the default primvar *st*.

⟨Find the primvar that holds the diffuse texture coordinates 79⟩ ≡
```
// texture coordinates come from a UsdPrimvarReader_float2
const UsdShadeShader reader = sourceShader(texture.GetInput(TfToken("st")));
VtValue varname;
if (reader ∧ shaderId(reader) ≡ "UsdTransform2d")
    warn("UsdTransform2d texture transforms are ignored (" + m.name + ")");
else if (reader ∧ inputValue(reader, "varname", &varname))
{
    if (varname.IsHolding<TfToken>())
        m.uvSet = varname.UncheckedGet<TfToken>().GetString();
    else if (varname.IsHolding<std::string>())
        m.uvSet = varname.UncheckedGet<std::string>();
}
```
This code is used in section 77.

80. Alpha cutouts. UsdPreviewSurface treats opacity in two ways. With an *opacityThreshold* greater than zero, a point is either fully present (opacity at or above the threshold) or fully absent: this is how leaves, fences and grass blades are cut out of simple quads. With a zero threshold, opacity means partial transparency, which a ray tracer could do but this one does not; such materials are drawn opaque, with a warning. Only an opacity that comes from a texture can cut anything out, so a constant opacity is ignored. The channel is named by the texture output the input is connected to: *outputs:a* for alpha, or one of the color channels.

⟨Read an alpha cutout of a surface 80⟩ ≡
```
// alpha cutout from a texture channel (translucency is not supported)
float threshold = 0.0f;
inputValue(surface, "opacityThreshold", &threshold);
const UsdAttribute opacity = sourceOutput(surface.GetInput(TfToken("opacity")));
if (opacity ∧ shaderId(UsdShadeShader(opacity.GetPrim())) ≡ "UsdUVTexture")
{
    if (threshold > 0.0f)
```

```
    {
        const std::string channel = UsdShadeUtils::GetBaseNameAndType(opacity.GetName()).first.GetString();
        m.opacityTexture = loadTexture(UsdShadeShader(opacity.GetPrim()));
        m.opacityChannel = channel == "r" ? 0 : channel == "g" ? 1 : channel == "b" ? 2 : 3;
        m.opacityThreshold = threshold;
    }
    else
        warn("Translucent materials are rendered opaque (" + m.name + ")");
}
```

This code is used in section 72.

81. The specular parameters. Roughness and metallic are read by ⟨Read a float input that may come from a texture 85⟩, the index of refraction is a plain number, and *useSpecularWorkflow* is an integer flag. In the specular workflow the *specularColor* may be a constant or a color texture, with the texture's scale and bias.

```
⟨Read the specular parameters of a surface 81⟩ ≡
    // specular: metallic or specular workflow
    m.roughness = scalarInput(surface, "roughness", 0.5f);
    m.metallic = scalarInput(surface, "metallic", 0.0f);
    inputValue(surface, "ior", &m.ior);
    int useSpecularWorkflow = 0;
    inputValue(surface, "useSpecularWorkflow", &useSpecularWorkflow);
    m.specularWorkflow = useSpecularWorkflow ≠ 0;
    if (m.specularWorkflow)
    {
        GfVec3f c;
        if (inputValue(surface, "specularColor", &c))
            m.specularColor = {c[0], c[1], c[2]};
        const UsdShadeShader specular = sourceShader(surface.GetInput(TfToken("specularColor")));
        if (specular ∧ shaderId(specular) ≡ "UsdUVTexture")
        {
            m.specularTexture = loadTexture(specular);
            GfVec4f v;
            if (inputValue(specular, "scale", &v))
                m.specularScale = {v[0], v[1], v[2], v[3]};
            if (inputValue(specular, "bias", &v))
                m.specularBias = {v[0], v[1], v[2], v[3]};
        }
    }
```

This code is used in section 72.

82. Normal maps. A tangent-space normal map stores, in each texel, a direction relative to the surface: x along the direction of increasing s, y along increasing t, z out of the surface. The texel values are data, not colors, so the image is decoded *raw* unless the file insists on sRGB.

```
⟨Read the normal map of a surface 82⟩ ≡
    // tangent-space normal map
    const UsdShadeShader normal = sourceShader(surface.GetInput(TfToken("normal")));
    if (normal ∧ shaderId(normal) ≡ "UsdUVTexture")
    {
        m.normalTexture = loadTexture(normal, /*srgbByDefault=*/false);
        ⟨Decide how to decode the normal map 83⟩
        ⟨Find the primvar that holds the normal-map coordinates 84⟩
    }
    else if (normal)
        warn("Unsupported shader driving normal: " + shaderId(normal).GetString() + " (" + m.name + ")");
```

This code is used in section 72.

83. The specification asks files to say, through the texture's scale and bias, how texels become vectors: for eight-bit images, scale $(2, 2, 2, 1)$ and bias $(-1, -1, -1, 0)$. We follow what is authored exactly. This is not pedantry: normal maps made for DirectX have the y axis flipped, and well-made USD files compensate with scale $(2, -2, 2)$ and bias $(-1, 1, -1)$, as the McUsd sample asset does. If only one of the two is authored, the other keeps the *UsdUVTexture* default. If neither is authored, the file is not compliant (`usdchecker`

complains about it), but such files are common, so rather than read the vectors as nonsense we assume the usual remapping and warn.

```
⟨Decide how to decode the normal map 83⟩ ≡
    GfVec4f scale, bias;
    const bool hasScale = inputValue(normal, "scale", &scale), hasBias = inputValue(normal, "bias", &bias);
    if (hasScale ∨ hasBias)
    {
        // as authored; UsdUVTexture defaults are scale 1, bias 0
        m.normalScale = hasScale ? std::array<float, 4>{scale[0], scale[1], scale[2], scale[3]}
                                 : std::array<float, 4>{1, 1, 1, 1};
        m.normalBias = hasBias ? std::array<float, 4>{bias[0], bias[1], bias[2], bias[3]}
                               : std::array<float, 4>{0, 0, 0, 0};
    }
    else
        warn("Normal map without scale/bias, assuming the usual 2x-1 remap (" + m.name + ")");
```

This code is used in section 82.

84. A material has one *uvSet*. When it has a diffuse texture, that texture has already chosen it; otherwise the normal map's primvar reader decides.

```
⟨Find the primvar that holds the normal-map coordinates 84⟩ ≡
    // 'uvSet' is shared with the diffuse texture when there is one
    if (m.texture < 0)
        if (const UsdShadeShader reader = sourceShader(normal.GetInput(TfToken("st"))))
        {
            VtValue varname;
            if (inputValue(reader, "varname", &varname))
            {
                if (varname.IsHolding<TfToken>())
                    m.uvSet = varname.UncheckedGet<TfToken>().GetString();
                else if (varname.IsHolding<std::string>())
                    m.uvSet = varname.UncheckedGet<std::string>();
            }
        }
```

This code is used in section 82.

85. A float input such as roughness starts at its default, then takes the authored value, if any. If it is driven by a texture, we record which channel: the output *outputs:g* means green, and so on; *outputs:r* and anything else, such as *outputs:rgb*, mean the first channel. Like normal maps, roughness and metallic maps are data and are decoded raw by default. The texture's scale and bias are reduced to those of the chosen channel, and if the image cannot be loaded, the texture's *fallback* for that channel replaces the constant.

```
⟨Read a float input that may come from a texture 85⟩ ≡
    ScalarInput Scene::scalarInput(const UsdShadeShader& surface, const char* name, float fallback)
    {
        ScalarInput in;
        in.value = fallback;
        inputValue(surface, name, &in.value);
        const UsdAttribute output = sourceOutput(surface.GetInput(TfToken(name)));
        if (¬output)
            return in;
        const UsdShadeShader texture(output.GetPrim());
        if (shaderId(texture) ≠ "UsdUVTexture")
        {
            warn("Unsupported shader driving " + std::string(name) + ": " + shaderId(texture).GetString());
            return in;
        }
        const std::string channel = UsdShadeUtils::GetBaseNameAndType(output.GetName()).first.GetString();
        in.channel = channel ≡ "g" ? 1 : channel ≡ "b" ? 2 : channel ≡ "a" ? 3 : 0;
        in.texture = loadTexture(texture, /*srgbByDefault=*/false);
        GfVec4f v;
        if (inputValue(texture, "scale", &v))
            in.scale = v[in.channel];
```

```
    if (inputValue(texture, "bias", &v))
        in.bias = v[in.channel];
    if (in.texture < 0 ∧ inputValue(texture, "fallback", &v))
        in.value = v[in.channel];
    return in;
}
```

This code is used in section 39.

86. Textures. Loading a texture turns a *UsdUVTexture* node into an index into the scene's *textures*, or -1 with a warning if anything goes wrong.

```
⟨Load a texture once 86⟩ ≡
int Scene::loadTexture(const UsdShadeShader& shader, bool srgbByDefault)
{
    ⟨Resolve the file of a texture 87⟩
    ⟨Decide the color space and wrap modes, and consult the texture cache 88⟩
    ⟨Decode a texture through the asset resolver 89⟩
}
```

This code is used in section 39.

87. The file is an *asset path*. USD resolves asset paths when it composes the stage, relative to the layer that authored them, so *GetResolvedPath* already gives an absolute location. For a file inside a `.usdz` package, that location has the form `package.usdz[textures/image.png]`, which no ordinary file function can open; we shall see in a moment how it is read. UDIM textures, whose name contains the pattern `<UDIM>` and stands for a whole set of images, one per tile of texture space, are not supported.

```
⟨Resolve the file of a texture 87⟩ ≡
SdfAssetPath file;
if (¬inputValue(shader, "file", &file) ∨ file.GetAssetPath().empty())
{
    warn("Texture without a file: " + shader.GetPath().GetString());
    return -1;
}
if (file.GetAssetPath().find("<UDIM>") ≠ std::string::npos)
{
    warn("UDIM textures are not supported: " + file.GetAssetPath());
    return -1;
}
const std::string resolved = file.GetResolvedPath();
if (resolved.empty())
{
    warn("Texture not found: " + file.GetAssetPath());
    return -1;
}
```

This code is used in section 86.

88. The color space. *sourceColorSpace* may be *sRGB*, *raw* or **auto** (the default). An explicit choice is obeyed. For **auto** the decision should depend on the image; we let it depend on the use instead, which is the same thing in practice: colors are sRGB, data (normals, roughness, metallic) are raw. The caller says which.

The renderer turns each *Texture* into a CUDA texture object, in which both the sRGB conversion and the wrap modes are fixed. The same image read in two different ways is therefore two textures, and the cache key contains all four properties: resolved path, color space and both wrap modes.

```
⟨Decide the color space and wrap modes, and consult the texture cache 88⟩ ≡
TfToken colorSpace;
inputValue(shader, "sourceColorSpace", &colorSpace);
const bool srgb = colorSpace ≡ "sRGB" ∨ (colorSpace ≠ "raw" ∧ srgbByDefault);
const TextureWrap wrapS = wrapMode(shader, "wrapS"), wrapT = wrapMode(shader, "wrapT");
const auto key = std::make_tuple(resolved, srgb, int(wrapS), int(wrapT));
if (auto it = m_textureCache.find(key); it ≠ m_textureCache.end())
    return it→second;
```

This code is used in section 86.

89. The asset resolver, *Ar*, is USD's layer between asset paths and bytes. Asking it to open the resolved path works for plain files and for files inside packages alike, and gives us the whole file in memory, which is exactly what the decoder `stb_image` wants. We ask the decoder for four channels of eight bits whatever the file contains; grayscale and RGB images gain the missing channels, with an opaque alpha. Failures are remembered in the cache as well, so a missing or broken image is reported once, not once per material that reads it.

```
⟨Decode a texture through the asset resolver 89⟩ ≡
    // ArResolver also reads files packaged inside .usdz archives
    int index = -1;
    std::shared_ptr<ArAsset> asset = ArGetResolver().OpenAsset(ArResolvedPath(resolved));
    std::shared_ptr<const char> buffer = asset ? asset→GetBuffer() : nullptr;
    int w = 0, h = 0, channels = 0;
    unsigned char* pixels =
        buffer ? stbi_load_from_memory(reinterpret_cast<const stbi_uc*>(buffer.get()), int(asset→GetSize()), &w,
                &h,
                                        &channels, 4)
               : nullptr;
    if (pixels)
    {
        Texture t;
        t.path = resolved;
        t.width = w;
        t.height = h;
        t.rgba.assign(pixels, pixels + size_t(w) * h * 4);
        t.srgb = srgb;
        t.wrapS = wrapS;
        t.wrapT = wrapT;
        stbi_image_free(pixels);
        m_textures.push_back(std::move(t));
        index = int(m_textures.size() − 1);
    }
    else
        warn("Could not load texture " + resolved + (buffer ? std::string(": ") + stbi_failure_reason() : ""));
    m_textureCache[key] = index;
    return index;
```

This code is used in section 86.

90. Materials per face. Back in the extraction loop, each prim needs a material for each of its faces. The material bound to the prim is found by *ComputeBoundMaterial*, which applies USD's binding rules for us: bindings are inherited from ancestors, and a binding marked "stronger than descendants" overrides those below it. A prim with no material falls back to its display color.

A mesh may also be divided into *GeomSubsets*, named sets of faces, and the subsets of the family *materialBind* can carry their own material binding. Those faces get the subset's material instead; indices outside the mesh are ignored. The count of such subsets is shown in the inspector.

```
⟨Assign a material to every face of a prim 90⟩ ≡
    // materials: the prim's own binding, overridden per face by GeomSubsets
    UsdShadeMaterialBindingAPI binding(p);
    const UsdShadeMaterial primMaterial = binding.ComputeBoundMaterial();
    const unsigned baseMaterial = primMaterial ? materialFor(primMaterial) : displayColorMaterial(p);
    std::vector<unsigned> faceMaterial(local.faceCount, baseMaterial);
    unsigned subsetMaterials = 0;
    for (const UsdGeomSubset& subset : binding.GetMaterialBindSubsets())
    {
        const UsdShadeMaterial m = UsdShadeMaterialBindingAPI(subset.GetPrim()).ComputeBoundMaterial();
        VtIntArray faces;
        if (¬m ∨ ¬subset.GetIndicesAttr().Get(&faces, m_time))
            continue;
        const unsigned index = materialFor(m);
```

```
    for (int f : faces)
        if (f ≥ 0 ∧ size_t(f) < faceMaterial.size())
            faceMaterial[f] = index;
    ++subsetMaterials;
}
```

This code is used in section 46.

91. Texture coordinates. Each prim gets one set of texture coordinates per triangle corner, read from one primvar. Which primvar? The one the material asks for: the prim's own material, unless some face uses a textured material, in which case the first such material decides. Prims whose subsets read different primvars are not handled, but they are rare.

Primvars can be inherited: a primvar authored with *constant* interpolation on an ancestor applies to the whole subtree, which is why we use *FindPrimvarWithInheritance*. As with normals, *ComputeFlattened* resolves indexed primvars, and the lambda *cornerUV* picks the value of each corner by the same rules as ⟨Pick the shading normal of each triangle corner 58⟩. A missing value gives the coordinates $(0, 0)$.

```
⟨Read the texture coordinates of a prim 91⟩ ≡
    // texture coordinates, read from the primvar the material asks for
    std::string uvSet = m_materials[baseMaterial].uvSet;
    for (unsigned m : faceMaterial)
        if (m_materials[m].usesTextures())
        {
            uvSet = m_materials[m].uvSet;
            break;
        }
    VtArray<GfVec2f> uvs;
    TfToken interpolation;
    if (const UsdGeomPrimvar pv = UsdGeomPrimvarsAPI(p).FindPrimvarWithInheritance(TfToken(uvSet));
        pv ∧ pv.ComputeFlattened(&uvs, m_time))
        interpolation = pv.GetInterpolation();
    auto cornerUV = [&](unsigned tri, int corner) → GfVec2f {
        size_t i = 0;
        if (interpolation ≡ UsdGeomTokens→faceVarying)
            i = local.cornerFv[3 * tri + corner];
        else if (interpolation ≡ UsdGeomTokens→vertex ∨ interpolation ≡ UsdGeomTokens→varying)
            i = local.indices[3 * tri + corner];
        else if (interpolation ≡ UsdGeomTokens→uniform)
            i = local.faceOfTri[tri];
        return i < uvs.size() ? uvs[i] : GfVec2f(0.0f);
    };
```

This code is used in section 46.

92. Corners and vertices. A GPU vertex carries one value of every attribute: one position, one normal, one pair of texture coordinates. USD's points are not such vertices. A cube whose faces are each mapped to the whole texture has eight points, but the corner of three faces meets them with three different texture coordinates, and the same holds for the normals of a hard edge. The face-varying attributes make the *corner* of a face the natural unit, so we first spell out every corner of every triangle, with its position, its shading normal and its texture coordinates, all in local space, and leave it to the next step to find which corners are really the same vertex.

Normals are normalized here, which also takes care of the unnormalized sums of the smooth normals and of the sphere's points. Normalizing before welding matters: two corners whose normals point the same way but have different lengths would otherwise remain distinct vertices. A degenerate normal is left as it is. The material of each triangle is that of its source face. Only the attributes the prim has are gathered, so *normals* and *cornerUVs* may stay empty.

```
⟨Gather the corners of a prim's triangles 92⟩ ≡
    // every triangle corner with its attributes, in the prim's local space
    const unsigned triCount = unsigned(local.indices.size() / 3);
    std::vector<float> positions, normals, cornerUVs;
    std::vector<unsigned> triMaterial(triCount);
    for (unsigned t = 0; t < triCount; ++t)
```

```
{
    triMaterial[t] = faceMaterial[local.faceOfTri[t]];
    for (int c = 0; c < 3; ++c)
    {
        const GfVec3f& pt = local.points[local.indices[3 * t + c]];
        positions.insert(positions.end(), {pt[0], pt[1], pt[2]});
        if (¬local.cornerNormals.empty())
        {
            GfVec3f n = local.cornerNormals[3 * t + c];
            if (n.GetLength() > 1e-12f)
                n.Normalize();
            normals.insert(normals.end(), {n[0], n[1], n[2]});
        }
        if (¬uvs.empty())
        {
            const GfVec2f uv = cornerUV(t, c);
            cornerUVs.insert(cornerUVs.end(), {uv[0], uv[1]});
        }
    }
}
```

This code is used in section 46.

93. The mesh record is started with what the inspector shows, and the corners are handed to *weldCorners*, which fills in the vertices, the triangles and their materials.

⟨Build the indexed mesh of a prim 93⟩ ≡

```
RenderMesh mesh;
mesh.path = p.GetPath();
mesh.material = baseMaterial;
mesh.subsetMaterials = subsetMaterials;
weldCorners(std::move(positions), std::move(normals), std::move(cornerUVs), triMaterial, mesh);
```

This code is used in section 46.

94. Welding. Corners with the same position, the same normal and the same texture coordinates are the same vertex, and should be stored once: on a smooth mesh, a vertex is typically shared by six triangles, so the vertex arrays shrink to about a sixth of the corner arrays. Corners that differ in any attribute, like the three corners of a textured cube at one point, stay separate, which is exactly what the face-varying data require. The tool for this is *meshoptimizer*, a library of mesh-processing algorithms by Arseny Kapoulkine. Along the way it reorders the triangles and the vertices for memory locality, for reasons explained in the sections that follow. The arguments are taken by value, so that the caller can move its arrays in and the function can reorder them in place.

⟨Weld the corners of a prim into an indexed mesh 94⟩ ≡

```
// Turns per-corner attributes (3 corners per triangle) into an indexed mesh
// with meshoptimizer. Corners with the same position, normal and st become one
// vertex. The triangles are grouped by material and sorted for spatial
// locality within each group, and the vertices are stored in the order the
// triangles first use them, so a ray's hits touch nearby memory.
void weldCorners(std::vector<float> positions, std::vector<float> normals, std::vector<float> uvs,
                 const std::vector<unsigned>& triMaterial, RenderMesh& out)
{
    ⟨Find the distinct corners of a mesh 95⟩
    ⟨Reorder one attribute stream through a remap table 96⟩
    ⟨Replace the corners by the distinct vertices 97⟩

    ⟨Group the triangles by material 98⟩
    ⟨Sort the triangles of each material spatially 99⟩

    ⟨Store the vertices in the order the triangles first use them 100⟩
}
```

This code is used in section 39.

95. The corners form an unindexed vertex buffer, three per triangle, split into separate *streams*, one per attribute. Given the streams, *meshopt_generateVertexRemapMulti* compares corners bit for bit across all of

them, and fills *remap* with the new index of each corner: equal corners get the same index, and the indices count up from zero without gaps in the order of first appearance. It returns the number of distinct vertices. A null index buffer means "the corners are in order".

```
⟨Find the distinct corners of a mesh 95⟩ ≡
    const size_t corners = positions.size() / 3, triCount = corners / 3;
    std::vector<meshopt_Stream> streams = {{positions.data(), 3 * sizeof(float), 3 * sizeof(float)}};
    if (¬normals.empty())
        streams.push_back({normals.data(), 3 * sizeof(float), 3 * sizeof(float)});
    if (¬uvs.empty())
        streams.push_back({uvs.data(), 2 * sizeof(float), 2 * sizeof(float)});
    std::vector<unsigned> remap(corners);
    const size_t vertexCount =
        meshopt_generateVertexRemapMulti(remap.data(), nullptr, corners, corners, streams.data(), streams.size());
```

This code is used in section 94.

96. Every attribute array must follow every reordering of the vertices, so a small helper applies a remap table to one stream: it builds an array of *to* vertices from one of *from* vertices, each element made of *floats* floats, and swaps it in. An attribute the prim does not have is left empty.

```
⟨Reorder one attribute stream through a remap table 96⟩ ≡
    auto remapStream = [&](std::vector<float>& stream, size_t floats, size_t from, size_t to,
                           const std::vector<unsigned>& table) {
        if (stream.empty())
            return;
        std::vector<float> result(to * floats);
        meshopt_remapVertexBuffer(result.data(), stream.data(), from, floats * sizeof(float), table.data());
        stream.swap(result);
    };
```

This code is used in section 94.

97. The triangles become an index buffer, *welded*, whose entry for each corner is that corner's new vertex index, and the three attribute streams shrink to the distinct vertices.

```
⟨Replace the corners by the distinct vertices 97⟩ ≡
    std::vector<unsigned> welded(corners);
    meshopt_remapIndexBuffer(welded.data(), nullptr, corners, remap.data());
    remapStream(positions, 3, corners, vertexCount, remap);
    remapStream(normals, 3, corners, vertexCount, remap);
    remapStream(uvs, 2, corners, vertexCount, remap);
```

This code is used in section 94.

98. Reordering the triangles. The order of the triangles in a GAS is free: OptiX builds its own hierarchy over them anyway. But each triangle carries its material in *triMaterial*, indexed by the triangle's position, so whatever order we choose, the materials must move with the triangles. We first sort the triangles by material, with a stable sort so that the source order survives within each material, and copy the indices and the materials in that order together. Grouping by material makes the next step possible: a spatial sort of each group on its own leaves every triangle in its group, and *triMaterial* stays aligned with the triangle order.

```
⟨Group the triangles by material 98⟩ ≡
    std::vector<unsigned> order(triCount);
    std::iota(order.begin(), order.end(), 0u);
    std::stable_sort(order.begin(), order.end(),
                     [&](unsigned a, unsigned b) { return triMaterial[a] < triMaterial[b]; });
    std::vector<unsigned> sorted(corners);
    out.triMaterial.resize(triCount);
    for (size_t t = 0; t < triCount; ++t)
    {
        std::copy_n(&welded[3 * size_t(order[t])], 3, &sorted[3 * t]);
        out.triMaterial[t] = triMaterial[order[t]];
    }
```

This code is used in section 94.

99. Within each run of triangles with the same material, *meshopt_spatialSortTriangles* orders the triangles along a space-filling curve through their centroids, so that triangles close in space are close in the index buffer. A ray tracer reads memory in an order dictated by space: a ray, and the rays of its neighboring pixels, hit triangles that lie near each other. When those triangles are also near each other in memory, their indices, materials and vertices share cache lines, on the GPU and in the acceleration structure built over them. The sort works in place.

⟨Sort the triangles of each material spatially 99⟩ ≡
```
for (size_t begin = 0, end; begin < triCount; begin = end)
{
    for (end = begin + 1; end < triCount ∧ out.triMaterial[end] ≡ out.triMaterial[begin]; ++end)
        ;
    meshopt_spatialSortTriangles(&sorted[3 * begin], &sorted[3 * begin], 3 * (end − begin), positions.data(),
                                 vertexCount, 3 * sizeof(float));
}
```
This code is used in section 94.

100. The vertices still have the order in which their corners first appeared in the source, which does not match the sorted triangles. *meshopt_optimizeVertexFetchRemap* renumbers them in the order in which the sorted triangles first use them, so that walking the triangles walks the vertex arrays almost sequentially, and the vertices of neighboring triangles are neighbors in memory too. It returns the number of vertices used, which equals the number of distinct vertices, since every vertex comes from some corner. The final index buffer and the attribute streams are rewritten through this remap and moved into the mesh, together with the number of corners the mesh started with.

⟨Store the vertices in the order the triangles first use them 100⟩ ≡
```
std::vector<unsigned> fetch(vertexCount);
const size_t used = meshopt_optimizeVertexFetchRemap(fetch.data(), sorted.data(), corners, vertexCount);
out.indices.resize(corners);
meshopt_remapIndexBuffer(out.indices.data(), sorted.data(), corners, fetch.data());
remapStream(positions, 3, vertexCount, used, fetch);
remapStream(normals, 3, vertexCount, used, fetch);
remapStream(uvs, 2, vertexCount, used, fetch);
out.positions = std::move(positions);
out.normals = std::move(normals);
out.uvs = std::move(uvs);
out.corners = corners;
```
This code is used in section 94.

101. World space. The mesh itself stays in local space; what goes to the renderer is the local-to-world matrix M of the prim. The two libraries write matrices differently. USD's matrices act on row vectors: a point is transformed as $p' = pM$, so the translation is the last *row* of M. An OptiX instance transform is a 3×4 matrix stored row by row that acts on column vectors, $p' = Tp$, with the translation in the last *column*. Since $pM = (M^{\mathsf{T}} p^{\mathsf{T}})^{\mathsf{T}}$, the OptiX matrix is the transpose of USD's, without its last row (which, for an affine transform, is $(0, 0, 0, 1)$): $T_{rc} = M_{cr}$ for $r < 3$ and $c < 4$. The matrix is computed in double precision and stored as floats.

Normals are not transformed here. They cannot be transformed with M itself: under a non-uniform scale, a vector perpendicular to the surface would stop being perpendicular. The correct matrix is the inverse transpose, $(M^{-1})^{\mathsf{T}}$, applied without translation. It keeps the transformed normal perpendicular to every transformed tangent, because for a tangent u and a normal n with $u \cdot n = 0$ we have $(uM) \cdot \big(n(M^{-1})^{\mathsf{T}}\big) = u\,M M^{-1}\,n^{\mathsf{T}} = u \cdot n = 0$. The device programs apply it at each hit, with OptiX's *optixTransformNormalFromObjectToWorldSpace*, which derives it from the instance transform.

⟨Pass the transform of a mesh to the renderer 101⟩ ≡
```
// USD transforms row vectors (p * M); OptiX wants the transpose
const GfMatrix4d toWorld = xformCache.GetLocalToWorldTransform(p);
for (int r = 0; r < 3; ++r)
    for (int c = 0; c < 4; ++c)
        mesh.toWorld[4 * r + c] = float(toWorld[c][r]);
```
This code is used in section 46.

102. The camera needs to know where things are in the world: the bounds of the mesh frame the selected prim, and the bounds of the scene frame everything. The welded vertices are transformed in double precision, with *Transform*, which includes the translation, and the mesh's bounds grow to include them; then the scene's bounds grow to include the mesh's.

```
⟨Compute the world bounds of a mesh 102⟩ ≡
    std::fill_n(mesh.boundsMin, 3, INFINITY);
    std::fill_n(mesh.boundsMax, 3, −INFINITY);
    for (size_t v = 0; v < mesh.vertexCount(); ++v)
    {
        const GfVec3d w = toWorld.Transform(GfVec3d(mesh.positions[3 * v], mesh.positions[3 * v + 1],
                                                    mesh.positions[3 * v + 2]));
        for (int k = 0; k < 3; ++k)
        {
            mesh.boundsMin[k] = std::min(mesh.boundsMin[k], float(w[k]));
            mesh.boundsMax[k] = std::max(mesh.boundsMax[k], float(w[k]));
        }
    }
    for (int k = 0; k < 3; ++k)
    {
        lo[k] = std::min(lo[k], mesh.boundsMin[k]);
        hi[k] = std::max(hi[k], mesh.boundsMax[k]);
    }
```

This code is used in section 46.

103. A prim whose material reads textures but which has no texture coordinates would show one texel stretched over the whole surface. Better to say so.

```
⟨Warn about textured prims without texture coordinates 103⟩ ≡
    bool textured = false;
    for (unsigned m : faceMaterial)
        textured = textured ∨ m_materials[m].usesTextures();
    if (textured ∧ uvs.empty())
        warn("Textured prim without '" + uvSet + "' texture coordinates: " + p.GetPath().GetString());
```

This code is used in section 46.

104. The mesh receives the next mesh id, its index in *m_meshes*, and the index from paths learns it before the mesh is moved into the vector.

```
⟨Register the mesh of a prim 104⟩ ≡
    m_meshByPath[mesh.path] = unsigned(m_meshes.size());
    m_meshes.push_back(std::move(mesh));
```

This code is used in section 46.

105. When the loop is over, the bounds are stored, but only if some geometry was found; an empty scene keeps the zero box rather than an inverted infinite one.

```
⟨Record the bounds of the scene 105⟩ ≡
    if (¬m_meshes.empty())
        for (int k = 0; k < 3; ++k)
        {
            m_boundsMin[k] = lo[k];
            m_boundsMax[k] = hi[k];
        }
```

This code is used in section 46.

106. Cameras. The scene keeps a list of *SceneCamera* records, one per *UsdGeomCamera* prim. Like the geometry, each is evaluated once, at the scene's time, and unlike the geometry it is stored in world space, because the renderer reads it directly. A USD camera is described by its transform and by physical lens parameters: a focal length and the size of the film back (the *aperture*), in tenths of a scene unit for orthographic cameras. What the renderer wants is simpler: an eye point, a forward and an up direction, and the half size of the *window*, the part of the image plane that the camera sees. For a perspective camera the window lies at distance 1, so its half height is the tangent of half the vertical field of view; for an orthographic camera

it is measured in scene units. The viewport rarely has the shape of the camera's film back, so the window is fitted to it later, in the application. The path identifies the prim, and the vertical field of view is kept for the inspector.

⟨A camera of the stage 106⟩ ≡
```
// A UsdGeomCamera at the stage's start time, in world space. The window is
// the image plane at distance 1 (perspective) or the view size in scene
// units (orthographic); the viewer fits it to the viewport's aspect ratio.
struct SceneCamera
{
    std::string path;
    std::array<float, 3> eye = {0, 0, 0}, forward = {0, 0, -1}, up = {0, 1, 0};
    float halfWidth = 1.0f, halfHeight = 1.0f;
    bool orthographic = false;
    float clipNear = 0.0f, clipFar = 1e16f;
    float fovYDeg = 0.0f;   // vertical field of view of a perspective camera
};
```
This code is used in section 32.

107. USD already knows how to turn a camera prim into the quantities we need. The method *GetCamera* of the *UsdGeomCamera* schema evaluates the prim's attributes and its full transform into a *GfCamera*, and the camera's *GfFrustum* gives the eye position, the view and up directions, the window and the clipping range. The frustum also discards any scale in the transform, which USD requires: a camera under a scaled parent must still see with its own lens. An aperture offset moves the window off centre (a shifted lens); the renderer has no use for it, so it is reported rather than silently dropped.

The cameras are collected by a walk over the stage with the same *UsdPrimRange* as the geometry. The range visits the prims depth first, in the order of the file, so the camera menu lists the cameras as the file does.

⟨Collect the cameras of the stage 107⟩ ≡
```
// ---- cameras ------------------------------------------------------------
// Walks the hierarchy depth first, so cameras keep the order of the stage.
void Scene::extractCameras()
{
    for (const UsdPrim& p : UsdPrimRange::Stage(m_stage, UsdTraverseInstanceProxies()))
    {
        if (¬p.IsA<UsdGeomCamera>())
            continue;
        // GfFrustum drops any scale in the camera's transform, as USD requires
        const GfCamera camera = UsdGeomCamera(p).GetCamera(m_time);
        const GfFrustum frustum = camera.GetFrustum();
        const GfVec3d eye = frustum.GetPosition(), forward = frustum.ComputeViewDirection(),
                      up = frustum.ComputeUpVector();
        SceneCamera c;
        c.path = p.GetPath().GetString();
        for (int k = 0; k < 3; ++k)
        {
            c.eye[k] = float(eye[k]);
            c.forward[k] = float(forward[k]);
            c.up[k] = float(up[k]);
        }
        ⟨Read the window and the clipping range of a camera 108⟩
        m_cameras.push_back(c);
    }
}
```
This code is used in section 39.

108. *GfCamera::GetFrustum* already divides the aperture by the focal length for a perspective camera, and converts it to scene units for an orthographic one, so the window can be used as it is.

⟨Read the window and the clipping range of a camera 108⟩ ≡
```
const GfRange2d window = frustum.GetWindow();
c.halfWidth = float(0.5 * window.GetSize()[0]);
```

```
c.halfHeight = float(0.5 * window.GetSize()[1]);
c.orthographic = frustum.GetProjectionType() ≡ GfFrustum::Orthographic;
c.clipNear = float(frustum.GetNearFar().GetMin());
c.clipFar = float(frustum.GetNearFar().GetMax());
if (¬c.orthographic)
    c.fovYDeg = camera.GetFieldOfView(GfCamera::FOVVertical);
if (window.GetMidpoint() ≠ GfVec2d(0.0))
    warn("Camera aperture offsets are ignored: " + c.path);
```

This code is used in section 107.

The renderer

109. The renderer is the part of the program that talks to the GPU. It owns the CUDA stream, the OptiX device context and pipeline, the meshes and the acceleration structures built over them, the textures, and the buffers into which the device programs write their pixels. The rest of the program sees a small interface: hand it a *Scene*, a camera and a few switches, call *render*, and read back an RGBA image together with the identity of the mesh under every pixel.

Everything the GPU needs for one launch travels in a single structure, *LaunchParams*; its definition is in `LaunchParams.h`. The renderer keeps a host copy of it, *m_params*, updates its fields as the user changes things, and copies it to the device right before every launch. Most setters below therefore do no more than write one field.

110. The interface. The header declares the class *OptixRenderer*. It cannot be copied: it owns GPU resources that must be released exactly once.

⟨`UsdOptixViewer/src/OptixRenderer.h` 110⟩ ≡

```
// Ray traces a Scene with OptiX into a host-side RGBA image.
#pragma once

⟨Headers and declarations of the renderer interface 111⟩

class OptixRenderer
{
public:
    ⟨Renderer construction and initialization 112⟩

    ⟨Renderer scene and view settings 113⟩

    ⟨Renderer frame production and results 114⟩

private:
    ⟨The device buffer helper 115⟩

    ⟨The device buffers of one mesh 116⟩

    ⟨Private steps of the renderer 117⟩

    ⟨CUDA and OptiX handles of the renderer 118⟩

    ⟨Scene and output buffers of the renderer 119⟩

    ⟨Host-side results of the renderer 120⟩
};
```

This code is written to file `UsdOptixViewer/src/OptixRenderer.h`.

111. The header needs the CUDA types (*CUdeviceptr*, *CUstream*, *cudaMipmappedArray_t*), the OptiX handle types, and the shared parameter structure. The *Scene* is only passed by reference, so a forward declaration is enough and the header does not drag in OpenUSD.

⟨Headers and declarations of the renderer interface 111⟩ ≡

```
#include <array>
#include <string>
#include <vector>

#include <cuda.h>
#include <cuda_runtime.h>
#include <optix_types.h>

#include "LaunchParams.h"

class Scene;
```

This code is used in section 110.

112. Construction does nothing; the real work happens in *init*, which may fail (no GPU, an old driver, device programs that do not compile for this GPU) and reports the reason as text instead of throwing, so that the application can show it in a dialog and keep running without a viewport.

Normally the renderer runs on the first GPU. A headset's runtime may choose another one, and the eye images can only be shared with it if OptiX renders on that same GPU; *init* therefore accepts the GPU's

UUID, a 16-byte identifier that CUDA and Vulkan both report. *release* gives everything back, so that the application can call *init* again for another GPU.

⟨Renderer construction and initialization 112⟩ ≡
```
OptixRenderer() = default;
~OptixRenderer();
OptixRenderer(const OptixRenderer&) = delete;
OptixRenderer& operator=(const OptixRenderer&) = delete;

// Creates the CUDA/OptiX context and pipeline on the GPU with the given
// UUID (the first GPU if null). Returns false and sets `error` on failure.
bool init(std::string& error, const unsigned char* deviceUuid = nullptr);
// Frees all GPU resources; init() may then be called again.
void release();
const std::string& deviceName() const { return m_deviceName; }
const std::array<unsigned char, 16>& deviceUuid() const { return m_deviceUuid; }
```
This code is used in section 110.

113. The scene is replaced as a whole with *setScene*, which uploads its meshes, materials and textures and builds the acceleration structures over them. The view is described by the image size, the camera and the light. There are two ways to set the camera: *setCamera* takes the orbit camera's eye, target and field of view, while *setCameraView* takes the more general description of a USD camera, which may be orthographic and has a clipping range. The remaining switches map one-to-one onto fields of the launch parameters and are read by the device programs: shadows, mipmapped texture filtering, normal maps, traced reflections, and the mesh to highlight as selected.

⟨Renderer scene and view settings 113⟩ ≡
```
// Uploads the meshes, materials and textures and builds the acceleration structures.
void setScene(const Scene& scene);
void resize(unsigned int width, unsigned int height);
void setCamera(const float eye[3], const float target[3], const float up[3], float fovYDeg);
// halfWidth/halfHeight: the image plane at distance 1 (perspective) or half
// the view size in scene units (orthographic). Camera rays are clipped to
// [clipNear, clipFar] along `forward`.
void setCameraView(const float eye[3], const float forward[3], const float up[3], float halfWidth,
                   float halfHeight, bool orthographic, float clipNear, float clipFar);
void setLightDir(const float dir[3]);
void setShadows(bool on) { m_params.shadows = on ? 1u : 0u; }
void setMipmaps(bool on) { m_params.mipmaps = on ? 1u : 0u; }
void setNormalMaps(bool on) { m_params.normalMaps = on ? 1u : 0u; }
void setReflections(bool on) { m_params.reflections = on ? 1u : 0u; }
void setSelectedMesh(unsigned int meshId) { m_params.selectedMesh = meshId; }
```
This code is used in section 110.

114. Rendering is progressive. Each call of *render* traces one new sample per pixel and adds it to an accumulation buffer on the GPU; the image handed back is the running average. The counter *frameIndex* in the launch parameters is the number of samples accumulated so far, so resetting the accumulation is just setting it to zero. The caller must do that after any change that invalidates the picture, such as a camera move.

The results are an array of RGBA pixels, ready to be uploaded to an SDL texture, and a lookup *meshIdAt* used for picking with the mouse.

The eyes of a headset are rendered differently. *renderView* takes a complete camera and writes into a buffer that already lives on the GPU (it is shared with the XR runtime), and leaves the desktop image and its accumulation alone. A head never stays still long enough to accumulate over frames, so it takes a fixed number of samples per frame instead.

⟨Renderer frame production and results 114⟩ ≡
```
// Adds one progressive sample per pixel and copies the average to the
// host buffers. Call resetAccumulation() after changing anything.
void render();
void resetAccumulation() { m_params.frameIndex = 0; }
unsigned int samples() const { return m_params.frameIndex; }

unsigned int width() const { return m_params.width; }
```

```
unsigned int height() const { return m_params.height; }
const std::vector<uchar4>& pixels() const { return m_pixels; }
unsigned int meshIdAt(unsigned int x, unsigned int y) const;
// World-space surface point at a pixel of the last rendered view; false
// for the background or when no frame was rendered since the last reset.
bool hitPointAt(unsigned int x, unsigned int y, float point[3]) const;
float lastRenderMs() const { return m_lastRenderMs; }

// Renders one view (an XR eye) into a device buffer of width x height
// pixels, averaging `samples` jittered samples. The desktop image and
// its accumulation are left alone.
void renderView(const Camera& camera, unsigned int width, unsigned int height, uchar4* target,
                unsigned int samples);
```

This code is used in section 110.

115. A *DeviceBuffer* is a block of GPU memory together with its size. OptiX wants device addresses as *CUdeviceptr*, a plain integer, so that is how we store them. The helper is deliberately small: it can be resized, filled from host memory, and freed. It has no destructor, because the order in which GPU memory is released matters less than the fact that it is released before the CUDA context goes away; the renderer's *release* frees every buffer explicitly.

⟨The device buffer helper 115⟩ ≡

```
struct DeviceBuffer
{
    CUdeviceptr ptr = 0;
    size_t size = 0;
    void resize(size_t bytes);
    void upload(const void* data, size_t bytes);
    void free();
};
```

This code is used in section 110.

116. Every mesh of the scene keeps its own vertex data on the GPU: positions, shading normals and texture coordinates per vertex, three indices per triangle, and the material of each triangle. Next to them lives the mesh's geometry acceleration structure (*gas*) and the traversable handle by which OptiX refers to it. Optional data that the mesh lacks, such as normals for a flat-shaded mesh, is simply an empty buffer with a null pointer, and the device programs test for that null.

⟨The device buffers of one mesh 116⟩ ≡

```
// A mesh's vertex data and its GAS
struct MeshBuffers
{
    DeviceBuffer positions, normals, uvs, indices, triMaterial, gas;
    OptixTraversableHandle handle = 0;
    void free();
};
```

This code is used in section 110.

117. The private steps are the parts of initialization and scene loading that deserve their own functions. *buildAccel* builds an acceleration structure from one build input, and serves both the per-mesh structures and the one over the instances; *freeScene* releases everything *setScene* created.

⟨Private steps of the renderer 117⟩ ≡

```
void createPipeline();
void buildSbt();
OptixTraversableHandle buildAccel(const OptixBuildInput& input, DeviceBuffer& output);
void uploadTextures(const Scene& scene);
void freeTextures();
void freeScene();
```

This code is used in section 110.

118. The handles fall into three groups. First come CUDA and OptiX objects that live as long as the renderer: the CUDA context and stream, the OptiX context, the module compiled from the OptiX-IR, the

program groups and the pipeline linked from them, and the shader binding table with its three record buffers.

⟨CUDA and OptiX handles of the renderer 118⟩ ≡
```
std::string m_deviceName;
std::array<unsigned char, 16> m_deviceUuid{};
CUcontext m_cuContext = nullptr;
CUstream m_stream = nullptr;
OptixDeviceContext m_context = nullptr;
OptixModule m_module = nullptr;
OptixPipeline m_pipeline = nullptr;
OptixProgramGroup m_raygenPG = nullptr, m_missPG[RAY_TYPE_COUNT] = {}, m_hitPG = nullptr;
OptixShaderBindingTable m_sbt = {};
DeviceBuffer m_raygenRecord, m_missRecords, m_hitRecords;
```
This code is used in section 110.

119. Then come the per-scene and per-image buffers. *m_meshes* holds the buffers of every mesh, in the order of the scene's meshes, so that its index is the mesh id. The device programs reach those buffers through *m_meshRecords*, an array of *MeshData* records with one device pointer per array. *m_instances* holds the instances that place the meshes in the world, *m_ias* the acceleration structure built over them, and *m_materials* the materials of the whole scene. The image needs three buffers: the 8-bit colors shown on screen, the floating-point sums of the progressive samples, the mesh ids for picking, and the surface points for the orbit pivot. Finally, every texture is a mipmapped CUDA array plus a texture object, and the objects are gathered in a device table that the shaders index by number. *renderView* has its own scratch buffers for the sums, the mesh ids and the parameters, so that an eye never disturbs the desktop image.

⟨Scene and output buffers of the renderer 119⟩ ≡
```
std::vector<MeshBuffers> m_meshes;
DeviceBuffer m_meshRecords, m_instances, m_ias, m_materials;
DeviceBuffer m_colorBuffer, m_accumBuffer, m_meshIdBuffer, m_hitBuffer, m_paramsBuffer;
DeviceBuffer m_viewAccum, m_viewMeshIds, m_viewParams;   // scratch for renderView
std::vector<cudaMipmappedArray_t> m_textureArrays;
std::vector<cudaTextureObject_t> m_textureObjects;
DeviceBuffer m_textureTable;
LaunchParams m_params = {};
```
This code is used in section 110.

120.

⟨Host-side results of the renderer 120⟩ ≡
```
std::vector<uchar4> m_pixels;
std::vector<unsigned int> m_meshIds;
float m_lastRenderMs = 0.0f;
```
This code is used in section 110.

121. The implementation. The source file is organized as follows; the helpers in the anonymous namespace are private to it.

⟨`UsdOptixViewer/src/OptixRenderer.cpp` 121⟩ ≡
```
⟨Headers of the renderer implementation 122⟩
⟨Declare the embedded OptiX-IR 123⟩
namespace {
⟨Error checking of CUDA and OptiX calls 124⟩
⟨The shader binding table record type 125⟩
⟨Small vector helpers of the host 126⟩
⟨Compute the next mip level of a texture 127⟩
}  // namespace
⟨Resize a device buffer 131⟩
⟨Upload host data to a device buffer 132⟩
⟨Free a device buffer 133⟩
```

⟨Free the buffers of a mesh 134⟩

⟨Destroy the renderer 135⟩

⟨Initialize CUDA and OptiX 136⟩

⟨Create the OptiX pipeline 140⟩

⟨Build the shader binding table 148⟩

⟨Set the scene 153⟩

⟨Build an acceleration structure 165⟩

⟨Free the scene 168⟩

⟨Upload the textures 169⟩

⟨Free the textures 174⟩

⟨Resize the output image 175⟩

⟨Set the camera 176⟩

⟨Set a general camera view 177⟩

⟨Set the light direction 178⟩

⟨Render one progressive sample 179⟩

⟨Render one view into a device buffer 183⟩

⟨Look up the mesh under a pixel 184⟩

⟨Read back the surface point under a pixel 185⟩

This code is written to file `UsdOptixViewer/src/OptixRenderer.cpp`.

122. OptiX's host functions are not exported by any library. Instead, one header defines a global table of function pointers,

`optix_function_table_definition.h`,

and another, `optix_stubs.h`, defines inline functions with the documented names that call through that table; *optixInit* fills the table from the driver. The definition header must be included in exactly one translation unit of the program, and this is it.

⟨Headers of the renderer implementation 122⟩ ≡

```
#include "OptixRenderer.h"

#include <algorithm>
#include <array>
#include <chrono>
#include <cmath>
#include <cstring>
#include <stdexcept>

#include <cuda_runtime.h>
#include <optix.h>
#include <optix_function_table_definition.h>
#include <optix_stubs.h>

#include "Scene.h"
```

This code is used in section 121.

123. The device programs reach us as OptiX-IR, the intermediate form that `nvcc` produces for OptiX and that the build system compiles into the executable (see ⟨Compile the device programs to OptiX-IR and embed them 15⟩). OptiX-IR is binary, so the size is what delimits it; the array also ends with a zero byte, which the size excludes. Its bytes are **unsigned char**, because a plain **char** is signed here and cannot hold values above 127; *optixModuleCreate* takes them through a **const char***.

⟨Declare the embedded OptiX-IR 123⟩ ≡

```
extern const unsigned char g_devicePrograms[];
extern const size_t g_devicePrograms_size;
```

This code is used in section 121.

124. Errors. Every CUDA and OptiX call returns a status code. Rather than test each one by hand, we

pass it to an overloaded *check* that throws an exception carrying the name of the operation and the library's description of the error. The exceptions are caught in *init* and in the application's render loop, which turn them into a message for the user.

⟨Error checking of CUDA and OptiX calls 124⟩ ≡

```
void check(cudaError_t r, const char* what)
{
    if (r ≠ cudaSuccess)
        throw std::runtime_error(std::string(what) + ": " + cudaGetErrorString(r));
}
void check(OptixResult r, const char* what)
{
    if (r ≠ OPTIX_SUCCESS)
        throw std::runtime_error(std::string(what) + ": " + optixGetErrorName(r));
}
```

This code is used in section 121.

125. Shader binding table records. OptiX finds the program to run for a ray generation, a miss or a hit through the *shader binding table* (SBT), an array of records in device memory. Each record starts with an opaque header of *OPTIX_SBT_RECORD_HEADER_SIZE* bytes, which identifies a program group, and may continue with data of our own; records must be aligned to *OPTIX_SBT_RECORD_ALIGNMENT* bytes. Our programs find all their data in the launch parameters, so the payload type is the empty *Empty*, and every record is just an aligned header.

⟨The shader binding table record type 125⟩ ≡

```
template <typename T>
struct alignas(OPTIX_SBT_RECORD_ALIGNMENT) SbtRecord
{
    alignas(OPTIX_SBT_RECORD_ALIGNMENT) char header[OPTIX_SBT_RECORD_HEADER_SIZE];
    T data;
};
struct Empty {};
```

This code is used in section 121.

126. CUDA's *float3* is a plain structure without operators on the host side, and the camera code needs a handful of vector operations. These four functions are all it takes; *normalize* leaves a zero vector unchanged instead of dividing by zero.

⟨Small vector helpers of the host 126⟩ ≡

```
float3 make3(const float v[3]) { return make_float3(v[0], v[1], v[2]); }
float3 scale(float3 a, float s) { return make_float3(a.x * s, a.y * s, a.z * s); }
float3 cross(float3 a, float3 b) { return make_float3(a.y * b.z − a.z * b.y, a.z * b.x − a.x * b.z, a.x * b.y − a.y *
        b.x); }
float3 normalize(float3 a)
{
    const float len = std::sqrt(a.x * a.x + a.y * a.y + a.z * a.z);
    return len > 0.0f ? scale(a, 1.0f / len) : a;
}
```

This code is used in section 121.

127. Mipmaps. A texture seen from far away covers many texels per pixel. If we sampled only the full-resolution image, neighboring pixels would pick unrelated texels and the picture would shimmer with moiré patterns. The classical cure is a *mip chain*: a sequence of images, each half the size of the previous one, down to 1×1. The texture unit then reads from the level whose texels match the footprint of a pixel; the device programs estimate that footprint with ray differentials.

CUDA can store a mip chain but does not compute one, so we build the levels on the CPU. Each texel of level $k+1$ is the average of a 2×2 block of level k (a box filter). When a dimension is odd, the last row or column simply has no partner, and we repeat the edge texel; when it is already 1, it stays 1.

⟨Compute the next mip level of a texture 127⟩ ≡

```
// Next mip level with a 2x2 box filter (edge texels are repeated for odd
// sizes). sRGB color channels are averaged in linear space.
```

```
std::vector<unsigned char> downsample(const std::vector<unsigned char>& src, int w, int h, bool srgb)
{
    ⟨The sRGB to linear lookup table 128⟩
    const int nw = std::max(1, w / 2), nh = std::max(1, h / 2);
    std::vector<unsigned char> dst(size_t(nw) * nh * 4);
    for (int y = 0; y < nh; ++y)
        for (int x = 0; x < nw; ++x)
        {
            ⟨Gather the 2x2 block of source texels 129⟩
            for (int c = 0; c < 4; ++c)
            {
                ⟨Average one channel of the block 130⟩
            }
        }
    return dst;
}
```

This code is used in section 121.

128. Color textures are almost always stored in sRGB, a nonlinear encoding that spends more of the 256 levels on dark tones. Averaging sRGB values directly makes the smaller levels too dark: the average of black and white should be light that is half as bright, whose sRGB code is 188, not 128. We therefore convert to linear light, average, and convert back. The conversion from an 8-bit code $c/255$ to linear light is

$$\ell = \begin{cases} c/12.92, & \text{if } c \le 0.04045; \\ \big((c+0.055)/1.055\big)^{2.4}, & \text{otherwise,} \end{cases}$$

and since it has only 256 possible inputs we tabulate it once, the first time the function is called.

⟨The sRGB to linear lookup table 128⟩ ≡

```
static const auto toLinear = [] {
    std::array<float, 256> lut{};
    for (int i = 0; i < 256; ++i)
    {
        const float c = i / 255.0f;
        lut[i] = c ≤ 0.04045f ? c / 12.92f : std::pow((c + 0.055f) / 1.055f, 2.4f);
    }
    return lut;
}();
```

This code is used in section 127.

129. The four source texels of output texel (x, y) are those at $(2x, 2y)$ to $(2x+1, 2y+1)$, clamped to the image; pixels are four bytes, RGBA.

⟨Gather the 2x2 block of source texels 129⟩ ≡

```
const int x0 = std::min(2 * x, w − 1), x1 = std::min(2 * x + 1, w − 1);
const int y0 = std::min(2 * y, h − 1), y1 = std::min(2 * y + 1, h − 1);
const unsigned char* texels[4] = {&src[(size_t(y0) * w + x0) * 4], &src[(size_t(y0) * w + x1) * 4],
                                  &src[(size_t(y1) * w + x0) * 4], &src[(size_t(y1) * w + x1) * 4]};
unsigned char* out = &dst[(size_t(y) * nw + x) * 4];
```

This code is used in section 127.

130. The color channels of an sRGB texture go through linear space, and the result is encoded again with the inverse of the curve above,

$$c = \begin{cases} 12.92\,\ell, & \text{if } \ell \le 0.0031308; \\ 1.055\,\ell^{1/2.4} - 0.055, & \text{otherwise.} \end{cases}$$

The alpha channel, and every channel of a raw texture (normal maps, roughness, metallic), is linear already and is averaged as integers, with +2 for rounding to nearest.

⟨Average one channel of the block 130⟩ ≡

```
if (srgb ∧ c < 3)
{
```

```
        float sum = 0.0f;
        for (const unsigned char* t : texels)
            sum += toLinear[t[c]];
        const float l = sum * 0.25f;
        const float e = l ≤ 0.0031308f ? l * 12.92f : 1.055f * std::pow(l, 1.0f / 2.4f) − 0.055f;
        out[c] = (unsigned char)std::lround(std::clamp(e, 0.0f, 1.0f) * 255.0f);
    }
    else
        out[c] = (unsigned char)((texels[0][c] + texels[1][c] + texels[2][c] + texels[3][c] + 2) / 4);
```

This code is used in section 127.

131. Device buffers. Resizing to the current size is free; otherwise the old block is released and a new one allocated. A size of zero leaves the buffer empty, with a null pointer, which happens for a scene without textures or without triangles.

⟨Resize a device buffer 131⟩ ≡

```
// ---- DeviceBuffer ------------------------------------------------------
void OptixRenderer::DeviceBuffer::resize(size_t bytes)
{
    if (bytes ≡ size)
        return;
    free();
    if (bytes)
        check(cudaMalloc(reinterpret_cast<void**>(&ptr), bytes), "cudaMalloc");
    size = bytes;
}
```

This code is used in section 121.

132. Uploads are synchronous: they happen when a scene is loaded, not in the render loop, so simplicity wins over overlap.

⟨Upload host data to a device buffer 132⟩ ≡

```
void OptixRenderer::DeviceBuffer::upload(const void* data, size_t bytes)
{
    resize(bytes);
    if (bytes)
        check(cudaMemcpy(reinterpret_cast<void*>(ptr), data, bytes, cudaMemcpyHostToDevice),
              "cudaMemcpy");
}
```

This code is used in section 121.

133.

⟨Free a device buffer 133⟩ ≡

```
void OptixRenderer::DeviceBuffer::free()
{
    if (ptr)
        cudaFree(reinterpret_cast<void*>(ptr));
    ptr = 0;
    size = 0;
}
```

This code is used in section 121.

134. A mesh frees its buffers one by one, its acceleration structure among them, and forgets the handle that pointed into that structure.

⟨Free the buffers of a mesh 134⟩ ≡

```
void OptixRenderer::MeshBuffers::free()
{
    for (DeviceBuffer* b : {&positions, &normals, &uvs, &indices, &triMaterial, &gas})
        b→free();
    handle = 0;
}
```

This code is used in section 121.

135. Life and death of the renderer. The destructor only calls *release*, which does the work. If *init* never got as far as creating an OptiX context, there is nothing to release. Otherwise we wait for the GPU to finish any work in flight and release the resources in the reverse order of their creation: the scene and the buffers first, then the pipeline, the program groups and the module it was linked from, then the OptiX context, and last the CUDA stream. The CUDA primary context itself is reference-counted by the runtime and is released when the process exits. Because *release* may be followed by another *init*, it also resets every handle and the launch parameters, so that the renderer looks freshly constructed.

⟨Destroy the renderer 135⟩ ≡

```
// ---- OptixRenderer ------------------------------------------------------
OptixRenderer::~OptixRenderer()
{
    release();
}
void OptixRenderer::release()
{
    if (¬m_context)
        return;
    cudaDeviceSynchronize();
    freeScene();
    for (DeviceBuffer* b : {&m_raygenRecord, &m_missRecords, &m_hitRecords, &m_colorBuffer, &m_accumBuffer,
                            &m_meshIdBuffer, &m_hitBuffer, &m_paramsBuffer, &m_viewAccum, &m_
                                viewMeshIds, &m_viewParams})
        b→free();
    optixPipelineDestroy(m_pipeline);
    optixProgramGroupDestroy(m_raygenPG);
    for (OptixProgramGroup pg : m_missPG)
        optixProgramGroupDestroy(pg);
    optixProgramGroupDestroy(m_hitPG);
    optixModuleDestroy(m_module);
    optixDeviceContextDestroy(m_context);
    cudaStreamDestroy(m_stream);
    m_context = nullptr;
    m_module = nullptr;
    m_pipeline = nullptr;
    m_raygenPG = m_hitPG = nullptr;
    m_missPG[RAY_TYPE_RADIANCE] = m_missPG[RAY_TYPE_SHADOW] = nullptr;
    m_sbt = {};
    m_stream = nullptr;
    m_cuContext = nullptr;
    m_params = {};
    m_pixels.clear();
    m_meshIds.clear();
}
```

This code is used in section 121.

136. Initialization proceeds in three steps, any of which may throw; the exception's message becomes the error text.

⟨Initialize CUDA and OptiX 136⟩ ≡

```
bool OptixRenderer::init(std::string& error, const unsigned char* deviceUuid)
{
    try
    {
        ⟨Select the GPU and its primary context 137⟩
        ⟨Create the OptiX device context 138⟩
        ⟨Build the pipeline and set the default parameters 139⟩
    }
    catch (const std::exception& e)
    {
```

```
            error = e.what();
            return false;
        }
        return true;
    }
```

This code is used in section 121.

137. The workstation has two GPUs; one is plenty for an interactive viewer, so we use device 0, unless a UUID names another. CUDA and Vulkan number the GPUs independently, so the UUID is the only reliable way to find the GPU that the XR runtime picked: we compare it with the UUID of each CUDA device. The CUDA runtime creates a device's *primary context* lazily, on the first call that needs one; *cudaFree*(**nullptr**) is the customary way to force that creation without doing anything else. OptiX, however, is built on the lower-level driver API and wants a *CUcontext*, so we ask the driver for the context that is now current, which is that primary context. Runtime and driver calls then share the same context, and memory allocated with *cudaMalloc* is valid for OptiX. The stream orders all our GPU work: acceleration builds, launches and copies.

```
⟨Select the GPU and its primary context 137⟩ ≡
    int deviceCount = 0;
    check(cudaGetDeviceCount(&deviceCount), "cudaGetDeviceCount");
    if (deviceCount ≡ 0)
        throw std::runtime_error("no CUDA capable device found");
    int device = 0;
    cudaDeviceProp props{};
    if (deviceUuid)
    {
        // the XR runtime chose a GPU; CUDA numbers GPUs differently from Vulkan
        device = -1;
        for (int d = 0; d < deviceCount ∧ device < 0; ++d)
            if (cudaGetDeviceProperties(&props, d) ≡ cudaSuccess ∧
                std::memcmp(props.uuid.bytes, deviceUuid, sizeof(props.uuid.bytes)) ≡ 0)
                device = d;
        if (device < 0)
            throw std::runtime_error("no CUDA device matches the GPU of the XR runtime");
    }
    check(cudaSetDevice(device), "cudaSetDevice");
    check(cudaFree(nullptr), "cudaFree");   // creates the primary context
    check(cudaGetDeviceProperties(&props, device), "cudaGetDeviceProperties");
    m_deviceName = props.name;
    std::memcpy(m_deviceUuid.data(), props.uuid.bytes, m_deviceUuid.size());
    check(cudaStreamCreate(&m_stream), "cudaStreamCreate");
    if (cuCtxGetCurrent(&m_cuContext) ≠ CUDA_SUCCESS)
        throw std::runtime_error("cuCtxGetCurrent failed");
```

This code is used in section 136.

138. *optixInit* locates the OptiX implementation inside the installed driver and fills the function table; it fails if the driver is too old for the headers we compiled against (OptiX 9.1 needs driver R590 or newer). The device context is then created on top of the CUDA context, with default options.

```
⟨Create the OptiX device context 138⟩ ≡
    check(optixInit(), "optixInit");
    OptixDeviceContextOptions options = {};
    check(optixDeviceContextCreate(m_cuContext, &options, &m_context), "optixDeviceContextCreate");
```

This code is used in section 136.

139. With a context in hand we compile the pipeline and fill in the shader binding table, then give the launch parameters their initial values: nothing selected, every feature on, and a light coming from above, to the right and in front of the default camera. The device copy of the parameters needs space for one *LaunchParams*.

```
⟨Build the pipeline and set the default parameters 139⟩ ≡
    createPipeline();
    buildSbt();
    m_params.selectedMesh = kNoMesh;
    m_params.shadows = 1u;
```

```
m_params.mipmaps = 1u;
m_params.normalMaps = 1u;
m_params.reflections = 1u;
const float light[3] = {0.4f, 0.8f, 0.45f};
setLightDir(light);
m_paramsBuffer.resize(sizeof(LaunchParams));
```

This code is used in section 136.

140. The pipeline. An OptiX *pipeline* is the complete program run by a launch. It is assembled from *program groups*, which in turn refer to entry functions inside *modules*, which OptiX compiles from OptiX-IR. Our single module holds all the device programs.

```
⟨Create the OptiX pipeline 140⟩ ≡
    void OptixRenderer::createPipeline()
    {
        ⟨Module compile options 141⟩
        ⟨Pipeline compile options 142⟩
        ⟨Compile the embedded OptiX-IR into a module 143⟩
        ⟨Create the ray generation program group 144⟩
        ⟨Create the miss program groups 145⟩
        ⟨Create the hit group 146⟩
        ⟨Link the pipeline and set its stack size 147⟩
    }
```

This code is used in section 121.

141. The module is compiled with the default optimization level and the minimal debug level, which keeps line information for profilers without slowing the code down.

```
⟨Module compile options 141⟩ ≡
    OptixModuleCompileOptions moduleOptions = {};
    moduleOptions.maxRegisterCount = OPTIX_COMPILE_DEFAULT_MAX_REGISTER_COUNT;
    moduleOptions.optLevel = OPTIX_COMPILE_OPTIMIZATION_DEFAULT;
    moduleOptions.debugLevel = OPTIX_COMPILE_DEBUG_LEVEL_MINIMAL;
```

This code is used in section 140.

142. The pipeline compile options must be identical for every module of a pipeline, because they shape the calling convention between programs.

- The scene is a two-level structure: an instance acceleration structure (IAS) whose instances each point to the geometry acceleration structure (GAS) of one mesh. We promise OptiX exactly that, single-level instancing, so that it can drop support for deeper graphs and for motion.
- The *payload* is the set of 32-bit registers a ray carries between the program that traces it and the programs it triggers. Radiance rays use seven: the three color components as float bits, the id of the mesh that was hit, the recursion depth, the state of the random number generator, and the distance to the hit. Shadow rays use only the first.
- Triangles report two *attributes*, their barycentric coordinates.
- The launch parameters live in a constant-memory variable that the device code calls *params*; OptiX copies our parameter buffer into it.
- Only built-in triangles are used, which lets OptiX drop support for other primitive types.

```
⟨Pipeline compile options 142⟩ ≡
    OptixPipelineCompileOptions pipelineOptions = {};
    pipelineOptions.traversableGraphFlags = OPTIX_TRAVERSABLE_GRAPH_FLAG_ALLOW_SINGLE_LEVEL_
            INSTANCING;
    pipelineOptions.numPayloadValues = 7;
    pipelineOptions.numAttributeValues = 2;
    pipelineOptions.exceptionFlags = OPTIX_EXCEPTION_FLAG_NONE;
```

```
    pipelineOptions.pipelineLaunchParamsVariableName = "params";
    pipelineOptions.usesPrimitiveTypeFlags = OPTIX_PRIMITIVE_TYPE_FLAGS_TRIANGLE;
```

This code is used in section 140.

143. The OptiX-IR is compiled to machine code for the installed GPU here, at run time; that is what lets one executable run on any GPU that OptiX supports. OptiX writes warnings and errors to a log buffer; *logSize* goes in as the capacity and comes back as the length used, so it is reset before every call. We do not print the log: on failure the error code already says what went wrong, and the device programs were compiled and checked at build time.

```
⟨Compile the embedded OptiX-IR into a module 143⟩ ≡
    char log[2048];
    size_t logSize = sizeof(log);
    check(optixModuleCreate(m_context, &moduleOptions, &pipelineOptions,
                            reinterpret_cast<const char*>(g_devicePrograms), g_devicePrograms_size, log,
                                &logSize,
                            &m_module),
          "optixModuleCreate");
```

This code is used in section 140.

144. Program groups name their entry functions by string. OptiX requires the names to start with a prefix that tells the kind of program: `__raygen__`, `__miss__`, `__closesthit__` or `__anyhit__`. The ray generation program is launched once per pixel.

```
⟨Create the ray generation program group 144⟩ ≡
    OptixProgramGroupOptions pgOptions = {};
    OptixProgramGroupDesc desc = {};
    desc.kind = OPTIX_PROGRAM_GROUP_KIND_RAYGEN;
    desc.raygen.module = m_module;
    desc.raygen.entryFunctionName = "__raygen__renderFrame";
    logSize = sizeof(log);
    check(optixProgramGroupCreate(m_context, &desc, 1, &pgOptions, log, &logSize, &m_raygenPG), "raygen group");
```

This code is used in section 140.

145. There are two ray types, radiance and shadow, and each has its own miss program: a radiance ray that escapes returns the color of the sky, while a shadow ray that escapes reports that the light is visible.

```
⟨Create the miss program groups 145⟩ ≡
    const char* missNames[RAY_TYPE_COUNT] = {"__miss__radiance", "__miss__shadow"};
    for (int i = 0; i < RAY_TYPE_COUNT; ++i)
    {
        desc = {};
        desc.kind = OPTIX_PROGRAM_GROUP_KIND_MISS;
        desc.miss.module = m_module;
        desc.miss.entryFunctionName = missNames[i];
        logSize = sizeof(log);
        check(optixProgramGroupCreate(m_context, &desc, 1, &pgOptions, log, &logSize, &m_missPG[i]), "miss
                group");
    }
```

This code is used in section 140.

146. A *hit group* bundles the programs run when a ray meets a triangle. The closest-hit program shades the nearest intersection; the any-hit program runs for every candidate intersection along the ray and is where alpha cutouts discard the transparent parts of leaves and fences. A single hit group serves both ray types: shadow rays are traced with closest-hit disabled, so for them only the any-hit program matters, and that is exactly what a shadow test needs.

```
⟨Create the hit group 146⟩ ≡
    desc = {};
    desc.kind = OPTIX_PROGRAM_GROUP_KIND_HITGROUP;
    desc.hitgroup.moduleCH = m_module;
    desc.hitgroup.entryFunctionNameCH = "__closesthit__radiance";
    desc.hitgroup.moduleAH = m_module;
```

```
desc.hitgroup.entryFunctionNameAH = "__anyhit__cutout";
logSize = sizeof(log);
check(optixProgramGroupCreate(m_context, &desc, 1, &pgOptions, log, &logSize, &m_hitPG), "hit group");
```

This code is used in section 140.

147. Linking needs the maximum depth of nested *optixTrace* calls. A camera ray (depth 1) may hit a glossy surface whose closest-hit program traces a reflection ray (depth 2); where that one lands, the closest-hit program traces a shadow ray (depth 3). Nothing goes deeper: the reflected hit does not reflect again, and shadow rays do not shade.

Recursion lives on a stack whose size OptiX must know. In general one adds up the stack needs of all programs; OptiX 9 can do this for us from the call depths: three levels of tracing, no continuation or direct callables, and a traversable graph two levels deep, the IAS above the GASes.

⟨Link the pipeline and set its stack size 147⟩ ≡

```
OptixProgramGroup groups[] = {m_raygenPG, m_missPG[0], m_missPG[1], m_hitPG};
OptixPipelineLinkOptions linkOptions = {};
linkOptions.maxTraceDepth = 3;   // camera -> reflection -> shadow
logSize = sizeof(log);
check(optixPipelineCreate(m_context, &pipelineOptions, &linkOptions, groups, 4, log, &logSize, &m_pipeline),
      "optixPipelineCreate");
// traversable graph depth 2: IAS -> GAS
check(optixPipelineSetStackSizeFromCallDepths(m_pipeline, linkOptions.maxTraceDepth, 0, 0, 0, 2),
      "optixPipelineSetStackSizeFromCallDepths");
```

This code is used in section 140.

148. The shader binding table. When a ray hits geometry, OptiX picks the hit record at index

$$\text{instance offset} + \text{stride} \times \text{geometry index} + \text{ray offset},$$

where the instance offset is the *sbtOffset* of the instance that was hit, the geometry index numbers the build inputs of its GAS, and stride and ray offset are arguments of *optixTrace*; misses use the miss record named by another argument. The device programs pass the ray type as the miss index, a ray offset of 0 and a stride of *RAY_TYPE_COUNT*. Every instance has an offset of 0 and every GAS a single build input (index 0), so every ray lands on hit record 0: all meshes share one hit group, and the programs tell the meshes apart by the instance id. The second record exists only so that the table has one record per ray type, the layout that a stride of *RAY_TYPE_COUNT* presumes.

⟨Build the shader binding table 148⟩ ≡

```
void OptixRenderer::buildSbt()
{
    ⟨Pack the ray generation record 149⟩
    ⟨Pack the miss records 150⟩
    ⟨Pack the hit records 151⟩
    ⟨Point the shader binding table at the records 152⟩
}
```

This code is used in section 121.

149. Packing a record writes the header that identifies the program group; then the record is copied to the GPU.

⟨Pack the ray generation record 149⟩ ≡

```
SbtRecord<Empty> raygen;
check(optixSbtRecordPackHeader(m_raygenPG, &raygen), "pack raygen");
m_raygenRecord.upload(&raygen, sizeof(raygen));
```

This code is used in section 148.

150.

⟨Pack the miss records 150⟩ ≡

```
SbtRecord<Empty> miss[RAY_TYPE_COUNT];
for (int i = 0; i < RAY_TYPE_COUNT; ++i)
    check(optixSbtRecordPackHeader(m_missPG[i], &miss[i]), "pack miss");
```

m_missRecords.upload(*miss*, **sizeof**(*miss*));

This code is used in section 148.

151.

⟨Pack the hit records 151⟩ ≡

```
// Both ray types use the same record; the shadow ray disables closest-hit.
SbtRecord<Empty> hit[RAY_TYPE_COUNT];
for (int i = 0; i < RAY_TYPE_COUNT; ++i)
    check(optixSbtRecordPackHeader(m_hitPG, &hit[i]), "pack hit");
m_hitRecords.upload(hit, sizeof(hit));
```

This code is used in section 148.

152.

⟨Point the shader binding table at the records 152⟩ ≡

```
m_sbt.raygenRecord = m_raygenRecord.ptr;
m_sbt.missRecordBase = m_missRecords.ptr;
m_sbt.missRecordStrideInBytes = sizeof(SbtRecord<Empty>);
m_sbt.missRecordCount = RAY_TYPE_COUNT;
m_sbt.hitgroupRecordBase = m_hitRecords.ptr;
m_sbt.hitgroupRecordStrideInBytes = sizeof(SbtRecord<Empty>);
m_sbt.hitgroupRecordCount = RAY_TYPE_COUNT;
```

This code is used in section 148.

153. Loading a scene. *setScene* replaces everything that depends on the scene: the materials, the textures, the meshes and the acceleration structures. The materials and textures come first, because all meshes share them; then every mesh is uploaded and gets its own acceleration structure; last, one structure over all the meshes places them in the world.

Why one structure per mesh, rather than one over all the triangles of the scene in world space? A USD stage describes each mesh in its own local space and positions it with a transform, and keeping that split costs nothing on the GPU. The ray-tracing hardware applies the transform as it descends from the top structure into a mesh, so a mesh's vertices are uploaded exactly as the scene holds them; only the small top structure would have to be rebuilt to move an object. The instance also gives every hit a number for free, the *instance id*, which the device programs use both to find the mesh's data and as the mesh id for picking.

⟨Set the scene 153⟩ ≡

```
void OptixRenderer::setScene(const Scene& scene)
{
    ⟨Free the old scene and restart the accumulation 154⟩
    ⟨Pack the materials for the device 155⟩
    ⟨Upload the textures and publish the material tables 158⟩
    ⟨Upload every mesh and build its acceleration structure 159⟩
    ⟨Publish the mesh records 163⟩
    ⟨Build the instance acceleration structure 164⟩
}
```

This code is used in section 121.

154. The handle in the launch parameters points into the acceleration structures that *freeScene* is about to release, so it is cleared first. Should anything below throw, *render* then sees no scene rather than a dangling handle. The accumulated image belongs to the old scene and restarts.

⟨Free the old scene and restart the accumulation 154⟩ ≡

```
m_params.handle = 0;
m_params.frameIndex = 0;
freeScene();
```

This code is used in section 153.

155. Materials. The scene describes them with C++ types that are convenient on the host (*std*::*array*, **bool**, *ScalarInput*); the device wants the flat, CUDA-aligned *MaterialData* of `LaunchParams.h`. We convert field by field.

⟨Pack the materials for the device 155⟩ ≡
```
std::vector<MaterialData> materials;
for (const Material& m : scene.materials())
{
    ⟨Conversions from host material fields 156⟩
    ⟨Fill in one device material 157⟩
}
m_materials.upload(materials.data(), materials.size() * sizeof(MaterialData));
```
This code is used in section 153.

156. Three small lambdas turn the host types into their device counterparts: a four-component array into a *float4*, a three-component one into a *float3*, and a scalar input (a constant, or a channel of a texture with a scale and a bias) into a *ScalarParam*.

⟨Conversions from host material fields 156⟩ ≡
```
auto f4 = [](const std::array<float, 4>& v) { return make_float4(v[0], v[1], v[2], v[3]); };
auto f3 = [](const std::array<float, 3>& v) { return make_float3(v[0], v[1], v[2]); };
auto scalar = [](const ScalarInput& in) {
    return ScalarParam{in.value, in.texture, in.channel, in.scale, in.bias};
};
```
This code is used in section 155.

157. The fields follow the UsdPreviewSurface inputs: the diffuse texture's scale and bias, those of the normal map and the specular color, the constant colors, texture indices (−1 meaning none), roughness and metallic, the alpha-cutout channel and threshold, the workflow flag and the index of refraction. Texture indices refer to the table built by *uploadTextures*.

⟨Fill in one device material 157⟩ ≡
```
MaterialData d = {};
d.texScale = f4(m.scale);
d.texBias = f4(m.bias);
d.normalScale = f4(m.normalScale);
d.normalBias = f4(m.normalBias);
d.specScale = f4(m.specularScale);
d.specBias = f4(m.specularBias);
d.color = f3(m.color);
d.texture = m.texture;
d.specColor = f3(m.specularColor);
d.specTexture = m.specularTexture;
d.roughness = scalar(m.roughness);
d.metallic = scalar(m.metallic);
d.opacityTexture = m.opacityTexture;
d.opacityChannel = m.opacityChannel;
d.opacityThreshold = m.opacityThreshold;
d.normalTexture = m.normalTexture;
d.specularWorkflow = m.specularWorkflow ? 1 : 0;
d.ior = m.ior;
materials.push_back(d);
```
This code is used in section 155.

158. With the textures uploaded, the device addresses of the two shared tables are known and go into the launch parameters.

⟨Upload the textures and publish the material tables 158⟩ ≡
```
uploadTextures(scene);
m_params.materials = reinterpret_cast<const MaterialData*>(m_materials.ptr);
m_params.textures = reinterpret_cast<const cudaTextureObject_t*>(m_textureTable.ptr);
```
This code is used in section 153.

159. The meshes. For each mesh of the scene we upload its arrays, build its geometry acceleration structure (GAS), and describe the instance that places it in the world. Two host vectors collect the results: the *MeshData* records through which the device programs find the arrays, and the *OptixInstance*s for the top-level structure. Both are filled in the order of the scene's meshes, so that index i of either is mesh i.

⟨Upload every mesh and build its acceleration structure 159⟩ ≡
```
// one GAS per mesh, in the mesh's own space
std::vector<MeshData> records;
std::vector<OptixInstance> instances;
for (const RenderMesh& mesh : scene.meshes())
{
    ⟨Upload the vertex data of a mesh 160⟩
    ⟨Build the geometry acceleration structure of a mesh 161⟩
    ⟨Place the mesh in the world with an instance 162⟩
}
```
This code is used in section 153.

160. A *RenderMesh* holds its data in flat vectors of **float**s and **unsigned int**s; their device element types (*float3*, *float2*, *uint3*) have the same layout, so the vectors are copied as they are. The scene has already welded the corners of the faces into shared vertices, which is why one index can address the position, the normal and the texture coordinates alike. A mesh without normals or without texture coordinates uploads an empty vector, and its record holds a null pointer there.

⟨Upload the vertex data of a mesh 160⟩ ≡
```
MeshBuffers& b = m_meshes.emplace_back();
b.positions.upload(mesh.positions.data(), mesh.positions.size() * sizeof(float));
b.normals.upload(mesh.normals.data(), mesh.normals.size() * sizeof(float));
b.uvs.upload(mesh.uvs.data(), mesh.uvs.size() * sizeof(float));
b.indices.upload(mesh.indices.data(), mesh.indices.size() * sizeof(unsigned int));
b.triMaterial.upload(mesh.triMaterial.data(), mesh.triMaterial.size() * sizeof(unsigned int));
records.push_back({reinterpret_cast<const float3*>(b.positions.ptr),
                   reinterpret_cast<const float3*>(b.normals.ptr), reinterpret_cast<const
                       float2*>(b.uvs.ptr),
                   reinterpret_cast<const uint3*>(b.indices.ptr),
                   reinterpret_cast<const unsigned int*>(b.triMaterial.ptr)});
```
This code is used in section 159.

161. OptiX builds a bounding volume hierarchy over the triangles of the mesh, the *geometry acceleration structure*, which the hardware ray-tracing cores traverse. The mesh is one build input: an indexed triangle array of *float3* positions in the mesh's local space and *uint3* index triplets, with one SBT record.

The geometry flag deserves a word. The usual advice for opaque geometry is the flag

$$OPTIX_GEOMETRY_FLAG_DISABLE_ANYHIT,$$

which spares the traversal a call per candidate hit. Alpha cutouts need the any-hit program, though, so the flag is *OPTIX_GEOMETRY_FLAG_NONE*; for opaque materials the program returns at once.

⟨Build the geometry acceleration structure of a mesh 161⟩ ≡
```
// any-hit stays enabled for alpha cutouts; it returns early for opaque materials
const unsigned int flags[] = {OPTIX_GEOMETRY_FLAG_NONE};
OptixBuildInput input = {};
input.type = OPTIX_BUILD_INPUT_TYPE_TRIANGLES;
OptixBuildInputTriangleArray& tri = input.triangleArray;
tri.vertexFormat = OPTIX_VERTEX_FORMAT_FLOAT3;
tri.vertexStrideInBytes = sizeof(float3);
tri.numVertices = unsigned(mesh.vertexCount());
tri.vertexBuffers = &b.positions.ptr;
tri.indexFormat = OPTIX_INDICES_FORMAT_UNSIGNED_INT3;
tri.indexStrideInBytes = sizeof(uint3);
tri.numIndexTriplets = unsigned(mesh.triangleCount());
tri.indexBuffer = b.indices.ptr;
tri.flags = flags;
tri.numSbtRecords = 1;
b.handle = buildAccel(input, b.gas);
```
This code is used in section 159.

162. An instance couples a GAS with the affine transform that takes it from the mesh's space to the world. OptiX wants that transform as the first three rows of a 4×4 matrix, row-major, acting on column vectors;

the scene stores *toWorld* in exactly that form, so it is copied as it is.

The *instanceId* is a 28-bit number of our choosing that the device programs read back with *optixGetInstanceId*. We set it to the index of the mesh, which makes it at once the index into the *MeshData* records and the mesh id written into the picking buffer. All rays may hit every instance (the visibility mask is all ones), every instance uses the one hit record (offset 0), and none overrides the geometry flags.

⟨Place the mesh in the world with an instance 162⟩ ≡
```
OptixInstance instance = {};
std::copy(mesh.toWorld.begin(), mesh.toWorld.end(), instance.transform);
instance.instanceId = unsigned(instances.size());  // the mesh id
instance.visibilityMask = 255;
instance.sbtOffset = 0;
instance.flags = OPTIX_INSTANCE_FLAG_NONE;
instance.traversableHandle = b.handle;
instances.push_back(instance);
```
This code is used in section 159.

163. The records go to the device as one array, which the launch parameters point to. A scene without meshes stops here: the handle stays zero, and *render* then simply paints the background.

⟨Publish the mesh records 163⟩ ≡
```
m_meshRecords.upload(records.data(), records.size() * sizeof(MeshData));
m_params.meshes = reinterpret_cast<const MeshData*>(m_meshRecords.ptr);
if (instances.empty())
    return;
```
This code is used in section 153.

164. The instances themselves must also live in device memory, where the build reads them. Built over them, the *instance acceleration structure* (IAS) is a hierarchy of the instances' world-space bounding boxes; its handle is the one that every *optixTrace* starts from.

⟨Build the instance acceleration structure 164⟩ ≡
```
// the IAS places every mesh in the world
m_instances.upload(instances.data(), instances.size() * sizeof(OptixInstance));
OptixBuildInput input = {};
input.type = OPTIX_BUILD_INPUT_TYPE_INSTANCES;
input.instanceArray.instances = m_instances.ptr;
input.instanceArray.numInstances = unsigned(instances.size());
m_params.handle = buildAccel(input, m_ias);
```
This code is used in section 153.

165. Building an acceleration structure is the same for both levels. The scene is static once loaded, so we ask for the structure that traces fastest, at the price of a slower build, and do not request updates or compaction.

⟨Build an acceleration structure 165⟩ ≡
```
OptixTraversableHandle OptixRenderer::buildAccel(const OptixBuildInput& input, DeviceBuffer& output)
{
    ⟨Choose the acceleration structure build options 166⟩
    ⟨Run the build and wait for it 167⟩
}
```
This code is used in section 121.

166.

⟨Choose the acceleration structure build options 166⟩ ≡
```
OptixAccelBuildOptions accelOptions = {};
accelOptions.buildFlags = OPTIX_BUILD_FLAG_PREFER_FAST_TRACE;
accelOptions.operation = OPTIX_BUILD_OPERATION_BUILD;
```
This code is used in section 165.

167. OptiX first tells us how much memory the build needs: scratch space for the build itself and room for the result, which goes into the caller's buffer. The build runs on our stream and returns the *traversable handle*; we wait for it before releasing the scratch buffer. Waiting after every mesh serializes the builds, but

they happen once per scene, and it keeps only one scratch buffer alive at a time.

⟨Run the build and wait for it 167⟩ ≡
```
OptixAccelBufferSizes sizes = {};
check(optixAccelComputeMemoryUsage(m_context, &accelOptions, &input, 1, &sizes),
      "optixAccelComputeMemoryUsage");
DeviceBuffer temp;
temp.resize(sizes.tempSizeInBytes);
output.resize(sizes.outputSizeInBytes);
OptixTraversableHandle handle = 0;
check(optixAccelBuild(m_context, m_stream, &accelOptions, &input, 1, temp.ptr, temp.size, output.ptr, output.size,
                      &handle, nullptr, 0),
      "optixAccelBuild");
check(cudaStreamSynchronize(m_stream), "accel build");
temp.free();
return handle;
```
This code is used in section 165.

168. *freeScene* undoes *setScene*: the buffers of every mesh, the records, the instances and their structure, the materials and the textures.

⟨Free the scene 168⟩ ≡
```
void OptixRenderer::freeScene()
{
    for (MeshBuffers& b : m_meshes)
        b.free();
    m_meshes.clear();
    for (DeviceBuffer* b : {&m_meshRecords, &m_instances, &m_ias, &m_materials})
        b→free();
    freeTextures();
}
```
This code is used in section 121.

169. Textures. Every texture of the scene becomes a CUDA *mipmapped array* holding its full mip chain, and a *texture object*: a 64-bit handle that combines the array with a description of how to sample it. The handles go into a table on the device, indexed by the texture numbers stored in the materials.

⟨Upload the textures 169⟩ ≡
```
void OptixRenderer::uploadTextures(const Scene& scene)
{
    freeTextures();
    for (const Texture& t : scene.textures())
    {
        ⟨Allocate a mipmapped array for the texture 170⟩
        ⟨Upload every mip level 171⟩
        ⟨Describe how the texture is sampled 172⟩
        ⟨Create the texture object 173⟩
    }
    m_textureTable.upload(m_textureObjects.data(), m_textureObjects.size() * sizeof(cudaTextureObject_t));
}
```
This code is used in section 121.

170. A chain that halves down to 1×1 has $1 + \lfloor \log_2 \max(w, h) \rfloor$ levels. Texels are four unsigned bytes, as decoded by the scene.

⟨Allocate a mipmapped array for the texture 170⟩ ≡
```
// full mip chain, down to 1x1
const unsigned levels = 1u + unsigned(std::floor(std::log2(double(std::max(t.width, t.height)))));
const cudaChannelFormatDesc format = cudaCreateChannelDesc<uchar4>();
cudaMipmappedArray_t mipmaps = nullptr;
check(cudaMallocMipmappedArray(&mipmaps, &format, make_cudaExtent(size_t(t.width), size_t(t.height), 0),
                               levels),
      "cudaMallocMipmappedArray");
```

```
m_textureArrays.push_back(mipmaps);
```

This code is used in section 169.

171. Level 0 is the image itself. After uploading each level we compute the next one from it with *downsample*, which halves the dimensions the same way CUDA does, never going below 1.

```
⟨Upload every mip level 171⟩ ≡
    std::vector<unsigned char> level = t.rgba;
    int w = t.width, h = t.height;
    for (unsigned l = 0; l < levels; ++l)
    {
        cudaArray_t array = nullptr;
        check(cudaGetMipmappedArrayLevel(&array, mipmaps, l), "cudaGetMipmappedArrayLevel");
        check(cudaMemcpy2DToArray(array, 0, 0, level.data(), size_t(w) * 4, size_t(w) * 4, size_t(h),
                                  cudaMemcpyHostToDevice),
              "texture upload");
        if (l + 1 < levels)
        {
            level = downsample(level, w, h, t.srgb);
            w = std::max(1, w / 2);
            h = std::max(1, h / 2);
        }
    }
```

This code is used in section 169.

172. The sampling description is where the texture hardware earns its keep.

- The USD wrap modes map onto CUDA address modes: `repeat` to wrap, `mirror` to mirror, `clamp` to clamp, and `black` to border, whose border color is zero.
- Filtering is *trilinear*: bilinear within a level, and linear between the two levels nearest to the requested one.
- Up to eight anisotropic samples are taken along the longer axis of a pixel's footprint. This only works with explicit gradients, which is why the device programs sample with *tex2DGrad*; it keeps floors seen at grazing angles sharp instead of blurring them to the coarsest level the short axis would allow.
- Texels are read as normalized floats in $[0, 1]$, addressed with normalized coordinates.
- For sRGB textures the hardware converts each texel to linear light *before* filtering, which is the correct order and costs nothing.

```
⟨Describe how the texture is sampled 172⟩ ≡
    cudaResourceDesc resource = {};
    resource.resType = cudaResourceTypeMipmappedArray;
    resource.res.mipmap.mipmap = mipmaps;
    cudaTextureDesc desc = {};
    auto mode = [](TextureWrap w) {
        switch (w)
        {
        case TextureWrap::Mirror: return cudaAddressModeMirror;
        case TextureWrap::Clamp: return cudaAddressModeClamp;
        case TextureWrap::Black: return cudaAddressModeBorder;
        default: return cudaAddressModeWrap;
        }
    };
    desc.addressMode[0] = mode(t.wrapS);
    desc.addressMode[1] = mode(t.wrapT);
    desc.filterMode = cudaFilterModeLinear;
    desc.mipmapFilterMode = cudaFilterModeLinear;
    desc.maxMipmapLevelClamp = float(levels - 1);
    desc.maxAnisotropy = 8;
    desc.readMode = cudaReadModeNormalizedFloat;
    desc.normalizedCoords = 1;
```

```
desc.sRGB = t.srgb ? 1 : 0;   // the hardware converts sRGB texels to linear
```

This code is used in section 169.

173.

```
⟨Create the texture object 173⟩ ≡
    cudaTextureObject_t object = 0;
    check(cudaCreateTextureObject(&object, &resource, &desc, nullptr), "cudaCreateTextureObject");
    m_textureObjects.push_back(object);
```

This code is used in section 169.

174. Texture objects must be destroyed before the arrays they refer to.

```
⟨Free the textures 174⟩ ≡
    void OptixRenderer::freeTextures()
    {
        for (cudaTextureObject_t object : m_textureObjects)
            cudaDestroyTextureObject(object);
        for (cudaMipmappedArray_t mipmaps : m_textureArrays)
            cudaFreeMipmappedArray(mipmaps);
        m_textureObjects.clear();
        m_textureArrays.clear();
        m_textureTable.free();
    }
```

This code is used in section 121.

175. The image and the camera. The output buffers follow the size of the viewport. A minimized or collapsed viewport may report a zero size, which we round up to one pixel so that every buffer stays valid. Resizing restarts the accumulation, since the old sums belong to other pixels. The accumulation buffer holds one *float4* per pixel: the sum of the sample colors, and in the fourth component the number of samples.

```
⟨Resize the output image 175⟩ ≡
    void OptixRenderer::resize(unsigned int width, unsigned int height)
    {
        width = width ? width : 1;
        height = height ? height : 1;
        if (width ≡ m_params.width ∧ height ≡ m_params.height)
            return;
        m_params.width = width;
        m_params.height = height;
        m_colorBuffer.resize(size_t(width) * height * sizeof(uchar4));
        m_accumBuffer.resize(size_t(width) * height * sizeof(float4));
        m_params.accumBuffer = reinterpret_cast<float4*>(m_accumBuffer.ptr);
        m_params.frameIndex = 0;
        m_meshIdBuffer.resize(size_t(width) * height * sizeof(unsigned int));
        m_hitBuffer.resize(size_t(width) * height * sizeof(float4));
        m_params.colorBuffer = reinterpret_cast<uchar4*>(m_colorBuffer.ptr);
        m_params.meshIdBuffer = reinterpret_cast<unsigned int*>(m_meshIdBuffer.ptr);
        m_params.hitBuffer = reinterpret_cast<float4*>(m_hitBuffer.ptr);
        m_pixels.resize(size_t(width) * height);
        m_meshIds.resize(size_t(width) * height);
    }
```

This code is used in section 121.

176. A pinhole camera is described to the device by its eye point and three vectors. Let $\mathbf{w}$ be the unit vector from the eye towards the target, $\mathbf{u} = \mathbf{w} \times \mathbf{up}$ normalized (pointing right), and $\mathbf{v} = \mathbf{u} \times \mathbf{w}$ (pointing up). If ϕ is the vertical field of view and a the aspect ratio, the image plane at distance 1 spans $\pm\tan(\phi/2)$ vertically and $\pm a\tan(\phi/2)$ horizontally. Scaling $\mathbf{u}$ and $\mathbf{v}$ by those half-extents lets the ray generation program write the direction through normalized device coordinates $(x, y) \in [-1, 1]^2$ simply as $\mathbf{w} + x\,\mathbf{u} + y\,\mathbf{v}$. The aspect ratio is taken from the current image size, so the camera must be set after *resize*. The orbit camera sees everything in front of it, so its clipping range is $[0, 10^{16}]$.

⟨Set the camera 176⟩ ≡
```
void OptixRenderer::setCamera(const float eye[3], const float target[3], const float up[3], float fovYDeg)
{
    const float forward[3] = {target[0] − eye[0], target[1] − eye[1], target[2] − eye[2]};
    const float halfH = std::tan(fovYDeg * 0.5f * float(M_PI) / 180.0f);
    const float aspect = m_params.height ? float(m_params.width) / float(m_params.height) : 1.0f;
    setCameraView(eye, forward, up, halfH * aspect, halfH, false, 0.0f, 1e16f);
}
```
This code is used in section 121.

177. The general form builds the same frame from a forward direction and scales **u** and **v** by the half-extents it is given. Those already include the aspect ratio: for a USD camera the application decides how the camera's window is fitted to the viewport.

⟨Set a general camera view 177⟩ ≡
```
void OptixRenderer::setCameraView(const float eye[3], const float forward[3], const float up[3], float halfWidth,
                                  float halfHeight, bool orthographic, float clipNear, float clipFar)
{
    const float3 w = normalize(make3(forward));
    const float3 u = normalize(cross(w, make3(up)));
    const float3 v = cross(u, w);
    m_params.camera = {make3(eye), scale(u, halfWidth), scale(v, halfHeight), w, orthographic ? 1u : 0u,
                       clipNear, clipFar};
}
```
This code is used in section 121.

178. The light is directional; the device programs want the unit vector that points towards it.

⟨Set the light direction 178⟩ ≡
```
void OptixRenderer::setLightDir(const float dir[3])
{
    m_params.lightDir = normalize(make3(dir));
}
```
This code is used in section 121.

179. Rendering. Each call adds one sample per pixel. If there is no image or no scene there is nothing to trace: an empty scene is painted in a dark gray and reports no mesh anywhere.

⟨Render one progressive sample 179⟩ ≡
```
void OptixRenderer::render()
{
    ⟨Handle an empty image or scene 180⟩
    ⟨Launch one sample per pixel 181⟩
    ⟨Download the image and the picking ids 182⟩
}
```
This code is used in section 121.

180.

⟨Handle an empty image or scene 180⟩ ≡
```
if (¬m_params.width ∨ ¬m_params.height)
    return;
const auto start = std::chrono::steady_clock::now();
if (¬m_params.handle)
{
    // Nothing to trace: clear to the background color.
    std::fill(m_pixels.begin(), m_pixels.end(), make_uchar4(40, 42, 48, 255));
    std::fill(m_meshIds.begin(), m_meshIds.end(), kNoMesh);
    return;
}
```
This code is used in section 179.

181. The host copy of the launch parameters goes to the device, and the pipeline is launched over a two-

dimensional grid with one thread per pixel. Both operations are queued on the same stream, so the launch sees the new parameters; the frame index tells the ray generation program whether to start a new sum (frame 0) or add to the existing one.

```
⟨Launch one sample per pixel 181⟩ ≡
    check(cudaMemcpyAsync(reinterpret_cast<void*>(m_paramsBuffer.ptr), &m_params, sizeof(m_params),
                          cudaMemcpyHostToDevice, m_stream),
          "upload params");
    check(optixLaunch(m_pipeline, m_stream, m_paramsBuffer.ptr, sizeof(LaunchParams), &m_sbt, m_params.width,
                      m_params.height, 1),
          "optixLaunch");
```

This code is used in section 179.

182. The averaged image comes back after every sample, so the user sees it converge. The mesh ids are another story: the first sample shoots its rays through the pixel centers and is the only one that writes them, because later samples jitter their rays inside the pixel, and a jittered id would flicker along the edges of objects. We download them only after that first sample. Then we wait for the stream, count the sample, and record how long it took, for the status bar.

```
⟨Download the image and the picking ids 182⟩ ≡
    check(cudaMemcpyAsync(m_pixels.data(), reinterpret_cast<void*>(m_colorBuffer.ptr), m_colorBuffer.size,
                          cudaMemcpyDeviceToHost, m_stream),
          "download color");
    if (m_params.frameIndex ≡ 0)
        check(cudaMemcpyAsync(m_meshIds.data(), reinterpret_cast<void*>(m_meshIdBuffer.ptr), m_
                meshIdBuffer.size,
                              cudaMemcpyDeviceToHost, m_stream),
              "download ids");
    check(cudaStreamSynchronize(m_stream), "render");
    ++m_params.frameIndex;
    m_lastRenderMs = std::chrono::duration<float, std::milli>(std::chrono::steady_clock::now() − start).count();
```

This code is used in section 179.

183. An eye of the headset. The launch parameters are copied, so the desktop's camera, image and sample count stay as they are, and the copy is pointed at the eye's camera, its size, the target buffer, and the scratch buffers for the sums and the mesh ids. The ray generation program then works exactly as for the desktop: launch s adds sample s to the sums and writes the running average to the target, so after the last launch the target holds the average of all the samples. The first sample goes through the pixel centres and the others are jittered, which antialiases the edges. All launches use one device copy of the parameters; that is safe because the copy before each launch is queued on the same stream, after the previous launch.

An empty scene still owes the headset an image: the background color is written into the target with a 32-bit memset, whose word holds the bytes $(40, 42, 48, 255)$ in little-endian order. The function returns only when the GPU has finished, because the XR runtime reads the buffer next.

```
⟨Render one view into a device buffer 183⟩ ≡
    void OptixRenderer::renderView(const Camera& camera, unsigned int width, unsigned int height, uchar4*
            target,
                                   unsigned int samples)
    {
        const size_t pixels = size_t(width) * height;
        if (¬m_params.handle)
        {
            // nothing to trace: the background color (40, 42, 48) as one 32-bit word
            if (cuMemsetD32Async(CUdeviceptr(target), 0xff302a28u, pixels, m_stream) ≠ CUDA_SUCCESS)
                throw std::runtime_error("cuMemsetD32Async failed");
            check(cudaStreamSynchronize(m_stream), "renderView");
            return;
        }
        m_viewAccum.resize(pixels * sizeof(float4));
        m_viewMeshIds.resize(pixels * sizeof(unsigned int));
        m_viewParams.resize(sizeof(LaunchParams));
        LaunchParams params = m_params;
        params.camera = camera;
```

```
    params.width = width;
    params.height = height;
    params.colorBuffer = target;
    params.accumBuffer = reinterpret_cast<float4*>(m_viewAccum.ptr);
    params.meshIdBuffer = reinterpret_cast<unsigned int*>(m_viewMeshIds.ptr);
    params.hitBuffer = nullptr;
    for (unsigned int s = 0; s < std::max(samples, 1u); ++s)
    {
        params.frameIndex = s;
        check(cudaMemcpyAsync(reinterpret_cast<void*>(m_viewParams.ptr), &params, sizeof(params),
                              cudaMemcpyHostToDevice, m_stream),
              "upload view params");
        check(optixLaunch(m_pipeline, m_stream, m_viewParams.ptr, sizeof(LaunchParams), &m_sbt, width,
                          height, 1),
              "optixLaunch");
    }
    check(cudaStreamSynchronize(m_stream), "renderView");
}
```

This code is used in section 121.

184. Picking reads the host copy of the ids; coordinates outside the image hit nothing.

```
⟨Look up the mesh under a pixel 184⟩ ≡
unsigned int OptixRenderer::meshIdAt(unsigned int x, unsigned int y) const
{
    if (x ≥ m_params.width ∨ y ≥ m_params.height)
        return kNoMesh;
    return m_meshIds[size_t(y) * m_params.width + x];
}
```

This code is used in section 121.

185. The surface points, by contrast, stay on the GPU. They are needed only when the user moves the orbit pivot, a rare event, so copying 16 bytes per pixel back with every frame 0 would waste bandwidth during every drag. Instead we copy the one pixel asked for. The answer is valid only if a frame was rendered since the last reset (*frameIndex* is then above 0); before that the buffer still holds an older view. A CUDA error returns false rather than throwing, because the caller is in the middle of handling the mouse.

```
⟨Read back the surface point under a pixel 185⟩ ≡
// Reads one pixel of the hit buffer back on demand: it is only needed when
// the orbit pivot moves, so it is not downloaded with every frame.
bool OptixRenderer::hitPointAt(unsigned int x, unsigned int y, float point[3]) const
{
    if (x ≥ m_params.width ∨ y ≥ m_params.height ∨ ¬m_params.handle ∨ m_params.frameIndex ≡ 0)
        return false;
    float4 hit;
    if (cudaMemcpyAsync(&hit, reinterpret_cast<const float4*>(m_hitBuffer.ptr) + size_t(y) * m_params.width
            + x,
                        sizeof(hit), cudaMemcpyDeviceToHost, m_stream) ≠ cudaSuccess ∨
        cudaStreamSynchronize(m_stream) ≠ cudaSuccess)
        return false;
    point[0] = hit.x, point[1] = hit.y, point[2] = hit.z;
    return hit.w ≠ 0.0f;
}
```

This code is used in section 121.

The device programs

186. This part runs on the GPU. OptiX organizes a ray tracer as a handful of small programs that it calls at well-defined moments: a *ray generation* program runs once per pixel and shoots the first rays; when a ray finds its nearest intersection, the *closest-hit* program computes the color it carries back; when a ray escapes, a *miss* program does; and for every candidate intersection an optional *any-hit* program may decide that the hit does not count. Our shading model follows UsdPreviewSurface: Lambertian diffuse reflection plus GGX specular reflection, lit by one directional light with hard shadows and by one glossy reflection bounce, under a simple gradient sky.

The file is compiled to OptiX-IR by `nvcc` and embedded in the executable, as described in the build-system part; OptiX compiles it for the actual GPU when the renderer creates its module.

The scene the programs see has two levels. Every mesh keeps its triangles in its own local space, under a geometry acceleration structure, and an instance acceleration structure places the meshes in the world. Rays are traced in world space, so whenever a program needs the geometry of the hit it reads it in the mesh's space and transforms it to the world.

⟨`UsdOptixViewer/src/devicePrograms.cu` 186⟩ ≡

⟨Opening of the device programs file 187⟩

⟨Small vector algebra on the device 188⟩

⟨Random numbers on the device 189⟩

⟨Payload helpers and the sky 192⟩

⟨The triangle under the hit 195⟩

⟨Texture sampling with ray differentials 199⟩

⟨Reading scalar material inputs 205⟩

⟨Normal mapping in the tangent frame 206⟩

⟨The GGX microfacet model 209⟩

⟨The ray generation program 213⟩

⟨The miss programs 217⟩

⟨The any-hit program for alpha cutouts 218⟩

⟨The closest-hit program 219⟩

This code is written to file `UsdOptixViewer/src/devicePrograms.cu`.

187. The launch parameters of the shared-parameters part are declared here as a *__constant__* variable called *params*. The name matters: the host tells OptiX, when it compiles the module, that the launch parameters live in a variable of that name, and OptiX copies the host's buffer into it before each launch. The **extern** `"C"` keeps the name from being mangled. As noted in the build system, `nvcc` remarks that this **extern** declaration is treated as a definition, which is precisely what we want.

The light intensity looks arbitrary, and in a sense it is. We want a lit diffuse surface of color ρ to show the simple brightness $0.7\,\rho\,(n \cdot l)$, bright but well short of white. A Lambertian surface reflects $\rho/\pi \cdot E\,(n \cdot l)$ under irradiance E, so the irradiance is $E = 0.7\pi$, and the shading stays physically based.

⟨Opening of the device programs file 187⟩ ≡

```
// OptiX programs: progressive pinhole or orthographic camera, UsdPreviewSurface-style shading
// (Lambert diffuse + GGX specular) with a directional light, hard shadows,
// one glossy reflection bounce and a gradient sky.
#include <optix.h>

#include "LaunchParams.h"

extern "C" __constant__ LaunchParams params;

constexpr float kPi = 3.14159265358979f;
constexpr float kLightIntensity = 0.7f * kPi;   // a surface facing the light reflects 0.7 x its albedo
```

This code is used in section 186.

188. CUDA's vector types come without arithmetic. The CUDA samples ship a header of operators, but it is not part of the toolkit, so we write the few we need. Every function is *__forceinline__*: they are tiny, and

calls in OptiX programs are costly. *normalize* uses the fast reciprocal square root and refuses to divide by a length below 10^{-10}, so a degenerate vector produces a huge but finite result instead of a NaN that would poison the accumulated image forever. *mix* is linear interpolation, $\mathrm{mix}(a, b, t) = (1 - t)\, a + t\, b$.

⟨Small vector algebra on the device 188⟩ ≡
```
static __forceinline__ __device__ float3 operator+(float3 a, float3 b) { return make_float3(a.x + b.x, a.y + b.y, a.z
        + b.z); }
static __forceinline__ __device__ float3 operator−(float3 a, float3 b) { return make_float3(a.x − b.x, a.y − b.y, a.z
        − b.z); }
static __forceinline__ __device__ float3 operator*(float3 a, float s) { return make_float3(a.x * s, a.y * s, a.z * s); }
static __forceinline__ __device__ float3 operator*(float3 a, float3 b) { return make_float3(a.x * b.x, a.y * b.y, a.z *
        b.z); }
static __forceinline__ __device__ float dot(float3 a, float3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
static __forceinline__ __device__ float3 cross(float3 a, float3 b)
{
    return make_float3(a.y * b.z − a.z * b.y, a.z * b.x − a.x * b.z, a.x * b.y − a.y * b.x);
}
static __forceinline__ __device__ float3 normalize(float3 a) { return a * rsqrtf(fmaxf(dot(a, a), 1e-20f)); }
static __forceinline__ __device__ float3 mix(float3 a, float3 b, float t) { return a * (1.0f − t) + b * t; }
```
This code is used in section 186.

189. Random numbers. Progressive rendering needs random numbers that differ from pixel to pixel and from frame to frame, yet cost almost nothing. We use the usual two-stage recipe of the OptiX samples. First, the pixel index and the frame number are hashed together with four rounds of TEA, the Tiny Encryption Algorithm of Wheeler and Needham; four rounds are far fewer than encryption would require, but enough to make the seeds of neighbouring pixels and consecutive frames look unrelated.

⟨Random numbers on the device 189⟩ ≡
```
// ---- random numbers ------------------------------------------------------------
⟨The TEA hash 190⟩

⟨A linear congruential generator 191⟩
```
This code is used in section 186.

190. The constant 0x9e3779b9 is $2^{32}/\varphi$, with φ the golden ratio; the other four are TEA's key. All arithmetic is modulo 2^{32}, courtesy of unsigned overflow.

⟨The TEA hash 190⟩ ≡
```
static __forceinline__ __device__ unsigned int tea(unsigned int v0, unsigned int v1)
{
    unsigned int s0 = 0;
    for (int n = 0; n < 4; ++n)
    {
        s0 += 0x9e3779b9;
        v0 += ((v1 << 4) + 0xa341316c) ^ (v1 + s0) ^ ((v1 >> 5) + 0xc8013ea4);
        v1 += ((v0 << 4) + 0xad90777d) ^ (v0 + s0) ^ ((v0 >> 5) + 0x7e95761e);
    }
    return v0;
}
```
This code is used in section 189.

191. From the seed, a linear congruential generator with the constants of *Numerical Recipes* produces the numbers themselves. The low bits of an LCG are its weakest, but we keep 24 bits and divide by 2^{24}, which gives floats uniformly spaced in $[0, 1)$ that are exactly representable in single precision. The seed is passed by reference and advanced in place, so one seed can serve a whole path, even across calls to *optixTrace*.

⟨A linear congruential generator 191⟩ ≡
```
static __forceinline__ __device__ float rnd(unsigned int& seed)
{
    seed = 1664525u * seed + 1013904223u;
    return float(seed & 0x00ffffffu) / float(0x01000000u);
}
```
This code is used in section 189.

192. The payload. A ray carries a few 32-bit registers, the *payload*, that the programs it reaches can read and write. Radiance rays use seven: the red, green and blue components of the returned color, stored as the bit patterns of **float**s; the id of the mesh that was hit, for picking; the recursion depth of the ray, which tells primary rays from reflection rays; and the random seed, which travels with the ray so that a reflection continues the same random sequence instead of starting a correlated one. The seventh is the distance along the ray to the hit, which the closest-hit program sets and a miss leaves alone. Shadow rays use a single register that the shadow miss program sets to 1.

⟨Payload helpers and the sky 192⟩ ≡
```
// ---- payload ----------------------------------------------------------------
// Radiance rays: p0..p2 = rgb (float bits), p3 = mesh id, p4 = depth, p5 = rng seed,
// p6 = hit distance (float bits; left alone on a miss).
// Shadow rays: p0 = 1 when the light is visible.
static __forceinline__ __device__ void setColor(float3 c, unsigned int meshId)
{
    optixSetPayload_0(__float_as_uint(c.x));
    optixSetPayload_1(__float_as_uint(c.y));
    optixSetPayload_2(__float_as_uint(c.z));
    optixSetPayload_3(meshId);
}
```
⟨Recognize shadow rays 193⟩

⟨The sky 194⟩

This code is used in section 186.

193. Some code, such as texture sampling for the cutout test, runs for both kinds of rays and must know which one it serves. Shadow rays are the only rays traced with *OPTIX_RAY_FLAG_DISABLE_CLOSESTHIT*, so that flag identifies them. The distinction matters because a shadow ray has only one payload register: reading register 4 of a shadow ray would be an error.

⟨Recognize shadow rays 193⟩ ≡
```
static __forceinline__ __device__ bool isShadowRay()
{
    return optixGetRayFlags() & OPTIX_RAY_FLAG_DISABLE_CLOSESTHIT;
}
```
This code is used in section 192.

194. The sky is a vertical gradient, from a dark grey below the horizon to a greyish blue overhead, interpolated by the height d_y of the direction. It is the background of the image and, through reflections, the only environment lighting of the scene. It always uses the world's y axis, even for stages whose up axis is z.

⟨The sky 194⟩ ≡
```
static __forceinline__ __device__ float3 sky(float3 dir)
{
    return mix(make_float3(0.10f, 0.10f, 0.12f), make_float3(0.32f, 0.38f, 0.48f), 0.5f * (dir.y + 1.0f));
}
```
This code is used in section 192.

195. The hit triangle. Texture sampling, normal mapping and shading all need the triangle that was hit: its mesh, the indices of its three vertices, and the positions of its corners in world space. *hitTriangle* gathers them once, in a small record that the other functions receive by reference.

⟨The triangle under the hit 195⟩ ≡
```
// ---- the hit triangle -------------------------------------------------------
// The triangle under the current hit: its mesh (the instance), its vertex
// indices and its corners moved from the mesh's space to world space.
```
⟨The hit triangle record 196⟩

⟨Gather the hit triangle in world space 197⟩

⟨Read the texture coordinates of the corners 198⟩

This code is used in section 186.

196. *prim* is the index of the triangle within its mesh, *tri* its three vertex indices, and a, b, c its corners in

world space.

⟨The hit triangle record 196⟩ ≡
```
struct HitTriangle
{
    const MeshData* mesh;
    unsigned int prim;
    uint3 tri;
    float3 a, b, c;
};
```
This code is used in section 195.

197. The renderer gives every instance the index of its mesh as instance id, so *optixGetInstanceId* leads straight to the mesh's *MeshData*, and *optixGetPrimitiveIndex*, the triangle's index within the build input of its GAS, to the triangle's entry in the mesh's index array.

The positions are stored in the mesh's local space, but everything the programs compare them with (the ray, the camera, the light) is in world space. *optixTransformPointFromObjectToWorldSpace* applies the transform of the instance that was hit, the same one the traversal used, so the corners land exactly where the ray met the triangle. Transforming three points per hit is cheaper than it sounds, and it lets every mesh stay in device memory as the scene describes it.

⟨Gather the hit triangle in world space 197⟩ ≡
```
static __forceinline__ __device__ HitTriangle hitTriangle()
{
    HitTriangle h;
    h.mesh = &params.meshes[optixGetInstanceId()];
    h.prim = optixGetPrimitiveIndex();
    h.tri = h.mesh→indices[h.prim];
    h.a = optixTransformPointFromObjectToWorldSpace(h.mesh→positions[h.tri.x]);
    h.b = optixTransformPointFromObjectToWorldSpace(h.mesh→positions[h.tri.y]);
    h.c = optixTransformPointFromObjectToWorldSpace(h.mesh→positions[h.tri.z]);
    return h;
}
```
This code is used in section 195.

198. Texture coordinates need no transform: they live on the surface, not in space. A mesh without them has a null pointer in its record, and its corners all get $(0, 0)$; such a mesh has no business sampling textures, and the zeros merely keep the arithmetic below well defined.

⟨Read the texture coordinates of the corners 198⟩ ≡
```
static __forceinline__ __device__ void cornerUVs(const HitTriangle& h, float2 uv[3])
{
    const float2* uvs = h.mesh→uvs;
    uv[0] = uvs ? uvs[h.tri.x] : make_float2(0.0f, 0.0f);
    uv[1] = uvs ? uvs[h.tri.y] : make_float2(0.0f, 0.0f);
    uv[2] = uvs ? uvs[h.tri.z] : make_float2(0.0f, 0.0f);
}
```
This code is used in section 195.

199. Texture sampling with ray differentials. A texture seen from afar covers many texels within one pixel; sampling it at a single point then gives the shimmering moiré patterns of aliasing. GPUs avoid this with *mipmaps*, prefiltered copies of the texture at half, quarter, … resolution, and pick the level whose texels match the pixel's footprint. A rasterizer knows the footprint for free, from the differences between neighbouring pixels. A ray tracer must compute it, and we do so with *ray differentials*: we ask where the rays through the next pixel to the right and the next pixel below would meet the plane of the triangle, and how far apart in texture space those points are from the hit.

⟨Texture sampling with ray differentials 199⟩ ≡
```
// ---- textures ----------------------------------------------------------
// Samples a texture at the current hit, flipping USD's bottom-left st origin
// to the image's top-left row order. For camera rays the mip level comes from
// ray differentials: the rays through the neighbouring pixels are intersected
// with the triangle's plane, and the offsets are converted to st gradients.
```

```
// Shadow and reflection rays sample the finest level.
static __forceinline__ __device__ float4 sampleTexture(int texture, const HitTriangle& h)
{
    ⟨Interpolate the texture coordinates at the hit 200⟩
    float2 dx = make_float2(0.0f, 0.0f), dy = make_float2(0.0f, 0.0f);
    if (params.mipmaps ∧ ¬isShadowRay() ∧ optixGetPayload_4() ≡ 0)
    {
        ⟨Compute the st gradients of the pixel footprint 201⟩
    }
    return tex2DGrad<float4>(params.textures[texture], s, 1.0f − t, dx, dy);
}
```

This code is used in section 186.

200. OptiX reports a triangle hit with barycentric coordinates (β, γ): the hit point is $(1-\beta-\gamma)\,a+\beta\,b+\gamma\,c$, and any quantity stored at the corners is interpolated with the same weights. The texture coordinates (s, t) are interpolated from those of the three corners.

The last line of ⟨Texture sampling with ray differentials 199⟩ deserves a word. USD places the origin of texture space at the bottom-left corner of the image, with t growing upward, while images are stored, and CUDA addresses them, from the top row down. Sampling at $1 - t$ reconciles the two conventions. The function *tex2DGrad* takes the screen-space derivatives of the texture coordinates; from their lengths the hardware chooses the mipmap level, and from their ratio and directions the anisotropic filter. With zero derivatives it samples the finest level, which is what happens for shadow rays, reflection rays (depth > 0), and whenever the *Texture mipmaps* switch is off. The test of *isShadowRay* must come before the payload is read, because shadow rays have no fifth payload register.

```
⟨Interpolate the texture coordinates at the hit 200⟩ ≡
    const float2 bary = optixGetTriangleBarycentrics();
    float2 uv[3];
    cornerUVs(h, uv);
    const float w0 = 1.0f − bary.x − bary.y;
    const float s = w0 * uv[0].x + bary.x * uv[1].x + bary.y * uv[2].x;
    const float t = w0 * uv[0].y + bary.x * uv[1].y + bary.y * uv[2].y;
```

This code is used in section 199.

201. The computation of the gradients has three steps: gather the triangle and the hit point, build the directions of the neighbouring rays, and turn each of them into an offset in texture space.

```
⟨Compute the st gradients of the pixel footprint 201⟩ ≡
    ⟨Gather the triangle and the hit point 202⟩
    ⟨Build the rays through the neighbouring pixels 203⟩
    ⟨Turn a neighbouring ray into an st offset 204⟩
    dx = gradient(oX, dirX);
    dy = gradient(oY, dirY);
```

This code is used in section 199.

202. We need the triangle's edges $e_1 = b - a$ and $e_2 = c - a$, the normal $n = e_1 \times e_2$ of its plane (its length is irrelevant here), the ray origin o and the hit point $p = o + t_{\max}\, d$, where $t_{\max}$ is the distance along the ray at which the closest hit was found. All of them are in world space, the space of the camera from which the neighbouring rays are built: the corners come from *hitTriangle*, and the ray is asked for its world-space origin and direction.

```
⟨Gather the triangle and the hit point 202⟩ ≡
    const float3 e1 = h.b − h.a, e2 = h.c − h.a;
    const float3 n = cross(e1, e2);
    const float3 o = optixGetWorldRayOrigin();
    const float3 p = o + optixGetWorldRayDirection() * optixGetRayTmax();
```

This code is used in section 201.

203. The rays through the neighbouring pixels are built exactly as in the ray generation program, from normalized device coordinates. One pixel is $2/\mathit{width}$ wide in those coordinates, and $2/\mathit{height}$ high; the vertical step is subtracted because the y axis of the image points down. Multiplied by the camera vectors, these steps

become the vectors *du* and *dv* by which the neighbouring rays differ from this one.

For an orthographic camera all rays are parallel, so the neighbours start at $o + du$ and $o + dv$ and keep the direction **w**; this is exact, because o is the actual origin of the camera ray. For a perspective camera the neighbours share the origin o, and their directions are built from the pixel centre even when the actual sample was jittered: the footprint changes very little within a pixel. The directions need not be normalized, since only the point where the rays meet the plane matters.

⟨Build the rays through the neighbouring pixels 203⟩ ≡
```
// neighbouring rays: shifted origins (orthographic) or directions (perspective)
const Camera& cam = params.camera;
const float3 du = cam.u * (2.0f / params.width), dv = cam.v * (-2.0f / params.height);
float3 oX = o, oY = o, dirX, dirY;
if (cam.orthographic)
{
    oX = o + du;
    oY = o + dv;
    dirX = dirY = cam.w;
}
else
{
    const uint3 idx = optixGetLaunchIndex();
    const float nx = 2.0f * (idx.x + 0.5f) / params.width - 1.0f;
    const float ny = 1.0f - 2.0f * (idx.y + 0.5f) / params.height;
    dirX = cam.w + cam.u * nx + cam.v * ny + du;
    dirY = cam.w + cam.u * nx + cam.v * ny + dv;
}
```
This code is used in section 201.

204. Now the geometry. A ray $o' + \tau d'$ meets the plane through p with normal n where $(o' + \tau d' - p) \cdot n = 0$, that is, at

$$\tau = \frac{(p - o') \cdot n}{d' \cdot n}.$$

The offset $\Delta = o' + \tau d' - p$ lies in the plane of the triangle, so it can be written as $\Delta = \beta\, e_1 + \gamma\, e_2$. Taking dot products with e_1 and e_2 gives two linear equations,

$$\begin{pmatrix} e_1 \cdot e_1 & e_1 \cdot e_2 \\ e_1 \cdot e_2 & e_2 \cdot e_2 \end{pmatrix} \begin{pmatrix} \beta \\ \gamma \end{pmatrix} = \begin{pmatrix} \Delta \cdot e_1 \\ \Delta \cdot e_2 \end{pmatrix},$$

which Cramer's rule solves with the determinant $d_{00} d_{11} - d_{01}^2$, computed once for both rays. Since texture coordinates vary linearly over the triangle, the offset in texture space is $\beta\,(uv_1 - uv_0) + \gamma\,(uv_2 - uv_0)$; its t component changes sign because we sample at $1 - t$. A ray parallel to the plane, or a degenerate triangle, gives no usable answer, and we fall back to the finest level.

⟨Turn a neighbouring ray into an st offset 204⟩ ≡
```
const float d00 = dot(e1, e1), d01 = dot(e1, e2), d11 = dot(e2, e2);
const float denom = d00 * d11 - d01 * d01;
auto gradient = [&](float3 origin, float3 dir) {
    const float dn = dot(dir, n);
    if (fabsf(dn) < 1e-20f ∨ fabsf(denom) < 1e-30f)
        return make_float2(0.0f, 0.0f);
    const float3 offset = origin + dir * (dot(p - origin, n) / dn) - p;
    const float d20 = dot(offset, e1), d21 = dot(offset, e2);
    const float b1 = (d11 * d20 - d01 * d21) / denom, b2 = (d00 * d21 - d01 * d20) / denom;
    return make_float2(b1 * (uv[1].x - uv[0].x) + b2 * (uv[2].x - uv[0].x),
                       -(b1 * (uv[1].y - uv[0].y) + b2 * (uv[2].y - uv[0].y)));   // t is flipped
};
```
This code is used in section 201.

205. Scalar inputs. A *ScalarParam* is either a constant or one channel of a texture, remapped by that channel's scale and bias. *channel* picks a component of a *float4* by number; the conditional chain compiles to a few selects, without any indexing into local memory.

⟨Reading scalar material inputs 205⟩ ≡
```
static __forceinline__ __device__ float channel(float4 v, int c)
```

```
{
    return c ≡ 0 ? v.x : c ≡ 1 ? v.y : c ≡ 2 ? v.z : v.w;
}
static __forceinline__ __device__ float evalScalar(const ScalarParam& p, const HitTriangle& h)
{
    if (p.texture < 0)
        return p.value;
    return channel(sampleTexture(p.texture, h), p.channel) * p.scale + p.bias;
}
```

This code is used in section 186.

206. Normal mapping. A normal map stores, in each texel, a direction expressed in a frame attached to the surface: the first axis follows increasing s (the *tangent*), the second follows increasing t (the *bitangent*, pointing to the top of the image), and the third is the surface normal. To use it we must know that frame in world space.

⟨Normal mapping in the tangent frame 206⟩ ≡

```
// Shading normal from a tangent-space normal map. The tangent frame comes from
// the triangle's positions and st (tangent along +s, bitangent along +t, the
// image's "up"), orthonormalized around the base normal `n`.
static __forceinline__ __device__ float3 normalMapped(const MaterialData& mat, const HitTriangle& h, float3 n)
{
    ⟨Build the tangent frame from positions and texture coordinates 207⟩
    ⟨Decode the normal map and leave tangent space 208⟩
}
```

This code is used in section 186.

207. Over a triangle, position is a linear function of the texture coordinates. If T and B are the derivatives of position with respect to s and t, the edges satisfy $e_1 = \Delta s_1\, T + \Delta t_1\, B$ and $e_2 = \Delta s_2\, T + \Delta t_2\, B$, where the Δs are the differences of texture coordinates along the edges. Solving this 2×2 system gives

$$T = \frac{e_1\,\Delta t_2 - e_2\,\Delta t_1}{\delta}, \qquad B = \frac{e_2\,\Delta s_1 - e_1\,\Delta s_2}{\delta}, \qquad \delta = \Delta s_1\,\Delta t_2 - \Delta s_2\,\Delta t_1.$$

When δ vanishes the texture is degenerate on this triangle and we keep the normal we were given. Otherwise we make the frame orthonormal around the base normal n (which may be a smooth, interpolated normal rather than the flat one): the tangent loses its component along n and is normalized, and the bitangent becomes $n \times t$. That cross product fixes the handedness, so we compare it with the true B and flip it when the texture is mirrored on this triangle; without that test the bumps of a mirrored texture would appear lit from the wrong side. The edges are those of the world-space corners, so the frame comes out in world space, like the normal n it is built around.

⟨Build the tangent frame from positions and texture coordinates 207⟩ ≡

```
const float3 e1 = h.b − h.a, e2 = h.c − h.a;
float2 uv[3];
cornerUVs(h, uv);
const float du1 = uv[1].x − uv[0].x, dv1 = uv[1].y − uv[0].y;
const float du2 = uv[2].x − uv[0].x, dv2 = uv[2].y − uv[0].y;
const float det = du1 * dv2 − du2 * dv1;
if (fabsf(det) < 1e-12f)
    return n;   // degenerate st: no usable frame
const float3 tangent = (e1 * dv2 − e2 * dv1) * (1.0f / det);
const float3 bitangent = (e2 * du1 − e1 * du2) * (1.0f / det);
const float3 t = normalize(tangent − n * dot(n, tangent));
float3 b = cross(n, t);
if (dot(b, bitangent) < 0.0f)
    b = b * -1.0f;   // mirrored st
```

This code is used in section 206.

208. The texel is sampled like any other texture, mipmaps included, and decoded with the scale and bias authored in the file, usually $2x - 1$. The decoded vector is expressed in the frame (t, b, n) and normalized: filtering averages neighbouring normals, which shortens them.

⟨Decode the normal map and leave tangent space 208⟩ ≡
```
const float4 texel = sampleTexture(mat.normalTexture, h);
const float3 local = make_float3(texel.x * mat.normalScale.x + mat.normalBias.x,
                                 texel.y * mat.normalScale.y + mat.normalBias.y,
                                 texel.z * mat.normalScale.z + mat.normalBias.z);
return normalize(t * local.x + b * local.y + n * local.z);
```
This code is used in section 206.

209. The GGX microfacet model. A glossy surface is modelled as a myriad of tiny mirrors, the *microfacets*, whose normals scatter around the surface normal n. Light from direction l reaches the viewer in direction v only through facets whose normal is the half vector $h = (v + l)/|v + l|$, and the specular part of the reflectance (the BRDF) is

$$f_s(l, v) = \frac{D(h)\, G(l, v)\, F(v \cdot h)}{4\,(n \cdot l)\,(n \cdot v)}.$$

D is the density of facets oriented along h, G the fraction of them that are neither shadowed nor masked by others, and F the Fresnel reflectance of one facet. We use the GGX (Trowbridge–Reitz) distribution with the Smith masking function, as UsdPreviewSurface does. Its width α is the square of the authored roughness, which makes the roughness slider perceptually even. All three functions take α^2, called *a2*.

⟨The GGX microfacet model 209⟩ ≡
```
// ---- GGX microfacet model ----------------------------------------------------
```
⟨The GGX normal distribution 210⟩

⟨The Smith masking function 211⟩

⟨Schlick's Fresnel approximation 212⟩

This code is used in section 186.

210. The GGX distribution is

$$D(h) = \frac{\alpha^2}{\pi\left((n \cdot h)^2(\alpha^2 - 1) + 1\right)^2}.$$

For small α it is a sharp peak around $h = n$; for $\alpha = 1$ it is constant, $1/\pi$, over the hemisphere.

⟨The GGX normal distribution 210⟩ ≡
```
static __forceinline__ __device__ float ggxD(float nDotH, float a2)
{
    const float d = nDotH * nDotH * (a2 - 1.0f) + 1.0f;
    return a2 / (kPi * d * d);
}
```
This code is used in section 209.

211. Smith's masking function for GGX, for a direction x making the cosine $n \cdot x$ with the normal, is

$$G_1(x) = \frac{2\,(n \cdot x)}{n \cdot x + \sqrt{\alpha^2 + (1 - \alpha^2)(n \cdot x)^2}}.$$

It is 1 for directions along the normal and falls to 0 at grazing angles. We use the separable form $G(l, v) = G_1(l)\, G_1(v)$.

⟨The Smith masking function 211⟩ ≡
```
static __forceinline__ __device__ float smithG1(float nDotX, float a2)
{
    return 2.0f * nDotX / (nDotX + sqrtf(a2 + (1.0f - a2) * nDotX * nDotX));
}
```
This code is used in section 209.

212. Schlick's approximation of the Fresnel equations,

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5,$$

interpolates from the reflectance at normal incidence, F_0, to total reflection at grazing incidence. F_0 is a color, because metals tint their reflections. The cosine is clamped to $[0, 1]$ so that rounding errors cannot push the result outside $[F_0, 1]$.

⟨Schlick's Fresnel approximation 212⟩ ≡

```
static __forceinline__ __device__ float3 fresnel(float3 f0, float cosTheta)
{
    const float m = 1.0f − fminf(fmaxf(cosTheta, 0.0f), 1.0f);
    const float m5 = m * m * m * m * m;
    return f0 + (make_float3(1.0f, 1.0f, 1.0f) − f0) * m5;
}
```

This code is used in section 209.

213. Ray generation. OptiX launches one thread per pixel, and each runs the ray generation program. Its name, like the names of all programs, begins with a prefix that tells OptiX its role (*__raygen__*, *__miss__*, *__closesthit__*, *__anyhit__*); the rest is the name by which the host refers to it when it creates the program groups.

⟨The ray generation program 213⟩ ≡

```
// ---- programs ----------------------------------------------------------------
extern "C" __global__ void __raygen__renderFrame()
{
    ⟨Seed the random generator for this pixel and frame 214⟩
    ⟨Shoot the camera ray through a jittered point of the pixel 215⟩
    ⟨Accumulate the sample and write the pixel 216⟩
}
```

This code is used in section 186.

214. Each pixel and each frame get their own seed.

⟨Seed the random generator for this pixel and frame 214⟩ ≡

```
const uint3 idx = optixGetLaunchIndex();
const unsigned int pixel = idx.y * params.width + idx.x;
unsigned int seed = tea(pixel, params.frameIndex);
```

This code is used in section 213.

215. Averaging samples taken at random points of the pixel is what removes the jagged edges of the image; it is antialiasing for free. The first frame, however, aims at the exact centre of the pixel: its mesh ids serve for picking, and its image, shown while the reader drags the camera, should be stable rather than noisy. The point in the pixel is converted to normalized device coordinates, with y flipped because image rows go down, and then to a ray with the camera vectors of the shared parameters: a direction through the image plane for a pinhole camera, or a shifted origin for an orthographic one.

The camera's clipping range is a depth, measured along **w**, while *optixTrace* limits the distance along the ray. A ray of unit direction d advances $d \cdot \mathbf{w}$ in depth per unit of distance, so the near and far depths are divided by that factor; it is 1 for the orthographic camera and at the centre of the pinhole image. The seven payload variables follow the layout of ⟨Payload helpers and the sky 192⟩; *depth* starts at zero, which marks a primary ray.

⟨Shoot the camera ray through a jittered point of the pixel 215⟩ ≡

```
// frame 0 goes through the pixel center (also used for picking), later frames jitter
const float jx = params.frameIndex ≡ 0 ? 0.5f : rnd(seed);
const float jy = params.frameIndex ≡ 0 ? 0.5f : rnd(seed);
const float2 ndc = make_float2(2.0f * (idx.x + jx) / params.width − 1.0f,
                               1.0f − 2.0f * (idx.y + jy) / params.height);
const Camera& cam = params.camera;
float3 origin = cam.eye, dir;
if (cam.orthographic)
{
    origin = cam.eye + cam.u * ndc.x + cam.v * ndc.y;
    dir = cam.w;
}
else
    dir = normalize(cam.w + cam.u * ndc.x + cam.v * ndc.y);
const float depthPerT = dot(dir, cam.w);   // the clipping range is a depth along w, not a ray distance
unsigned int r, g, b, meshId, depth = 0, hitT = 0;
```

```
optixTrace(params.handle, origin, dir, cam.clipNear / depthPerT, cam.clipFar / depthPerT, 0.0f,
        OptixVisibilityMask(255), OPTIX_RAY_FLAG_NONE, RAY_TYPE_RADIANCE, RAY_TYPE_
            COUNT, RAY_TYPE_RADIANCE,
        r, g, b, meshId, depth, seed, hitT);
```

This code is used in section 213.

216. The new sample is added to the sum kept for the pixel, together with a count in the fourth component; on frame 0 the sum is simply replaced, which is how a reset takes effect. The displayed color is the average, $\bar{c} = \sum c_i / N$, whose noise falls as $1/\sqrt{N}$.

The conversion to 8 bits is a deliberate approximation. Displays expect sRGB, whose transfer curve is close to a power of 1/2.2; we use 1/2, a square root, which is cheaper and slightly brightens the midtones, a look that suits an interactive viewer. Values above 1 are clamped by *__saturatef*. Finally, the mesh id is stored on frame 0 only, the frame the host reads back for picking, together with the point that was hit, $\mathbf{o} + t\,\mathbf{d}$, from the distance t in the payload. A miss leaves t at 0 and marks the point as background in its fourth component.

```
⟨Accumulate the sample and write the pixel 216⟩ ≡
    const float4 sample = make_float4(__uint_as_float(r), __uint_as_float(g), __uint_as_float(b), 1.0f);
    float4 sum = sample;
    if (params.frameIndex > 0)
    {
        const float4 prev = params.accumBuffer[pixel];
        sum = make_float4(prev.x + sample.x, prev.y + sample.y, prev.z + sample.z, prev.w + 1.0f);
    }
    params.accumBuffer[pixel] = sum;
    const float inv = 1.0f / sum.w;
    // approximate sRGB encoding
    params.colorBuffer[pixel] = make_uchar4((unsigned char)(255.0f * __saturatef(sqrtf(sum.x * inv))),
                                            (unsigned char)(255.0f * __saturatef(sqrtf(sum.y * inv))),
                                            (unsigned char)(255.0f * __saturatef(sqrtf(sum.z * inv))), 255);
    if (params.frameIndex ≡ 0)
    {
        params.meshIdBuffer[pixel] = meshId;
        // the orbit camera pivots on these points
        if (params.hitBuffer)
        {
            const float3 p = origin + dir * __uint_as_float(hitT);
            params.hitBuffer[pixel] = make_float4(p.x, p.y, p.z, meshId ≡ kNoMesh ? 0.0f : 1.0f);
        }
    }
```

This code is used in section 213.

217. The miss programs. A radiance ray that escapes returns the sky in its direction and no mesh. A shadow ray that escapes has reached the light; its caller initializes the payload to 0, so the light counts as blocked unless this program runs.

```
⟨The miss programs 217⟩ ≡
    extern "C" __global__ void __miss__radiance()
    {
        setColor(sky(optixGetWorldRayDirection()), kNoMesh);
    }
    extern "C" __global__ void __miss__shadow()
    {
        optixSetPayload_0(1u);   // light is visible
    }
```

This code is used in section 186.

218. Alpha cutouts. Leaves, fences and rails are often drawn as simple quads whose texture is transparent outside the shape. UsdPreviewSurface describes them with an opacity, usually the alpha channel of the diffuse texture, and an *opacityThreshold*: where the opacity falls below the threshold, the surface is not there at all. The any-hit program runs for every candidate intersection, before OptiX decides which is closest, and

calls *optixIgnoreIntersection* for the transparent ones, so that the ray goes on as if the triangle did not exist.

Because the same hit-group record serves both ray types, the test applies to shadow rays too: the shadow of a fern is the shape of its leaves, not of its quads. For materials without a cutout the program returns at once; the cost of calling it at all is the price of not splitting the geometry into opaque and cutout parts. The material is looked up from the instance's mesh and the triangle's index before anything else, so that an opaque hit returns without transforming any corners; only a cutout needs the whole hit triangle, for its texture coordinates and its mipmap footprint.

⟨The any-hit program for alpha cutouts 218⟩ ≡

```
// Alpha cutouts (UsdPreviewSurface opacity + opacityThreshold), for camera,
// reflection and shadow rays.
extern "C" __global__ void __anyhit__cutout()
{
    const MeshData& mesh = params.meshes[optixGetInstanceId()];
    const MaterialData& mat = params.materials[mesh.triMaterial[optixGetPrimitiveIndex()]];
    if (mat.opacityTexture < 0)
        return;
    const float4 texel = sampleTexture(mat.opacityTexture, hitTriangle());
    if (channel(texel, mat.opacityChannel) < mat.opacityThreshold)
        optixIgnoreIntersection();
}
```

This code is used in section 186.

219. Shading. The closest-hit program is the heart of the renderer. It finds the normals at the hit, evaluates the material, and adds up three contributions: an ambient fill, the directional light, and a glossy reflection. It then hands the color, the mesh id and the advanced random seed back to the ray's caller.

⟨The closest-hit program 219⟩ ≡

```
extern "C" __global__ void __closesthit__radiance()
{
    ⟨Fetch the hit triangle and its material 220⟩
    ⟨Compute the geometric and shading normals 221⟩
    ⟨Evaluate the material at the hit 225⟩
    ⟨Add the ambient fill 229⟩
    ⟨Add the directional light 230⟩
    ⟨Add a glossy reflection 234⟩
    ⟨Return the color and the seed to the caller 240⟩
}
```

This code is used in section 186.

220. The payload tells us how deep in the recursion we are and gives us the seed. *hitTriangle* supplies the mesh, the triangle and its world-space corners; the triangle's material comes from the mesh's per-triangle array. The instance id is also the mesh id, which the ray generation program stores for picking and which we compare with the selection.

⟨Fetch the hit triangle and its material 220⟩ ≡

```
const unsigned int depth = optixGetPayload_4();
unsigned int seed = optixGetPayload_5();
const HitTriangle hit = hitTriangle();
const float3 rayDir = optixGetWorldRayDirection();
const unsigned int meshId = optixGetInstanceId();
const MaterialData& mat = params.materials[hit.mesh→triMaterial[hit.prim]];
```

This code is used in section 219.

221. Normals. We keep two of them. The *geometric* normal n is the true normal of the flat triangle; it decides which side of the surface we are on, where secondary rays start, and whether they would leave the surface. The *shading* normal ns is the one used for lighting; it may come from smooth or authored normals and be perturbed by a normal map, so that a mesh of flat triangles looks curved and bumpy.

⟨Compute the geometric and shading normals 221⟩ ≡
```
// geometric normal: outward for counter-clockwise triangles (see Scene.cpp)
float3 n = normalize(cross(hit.b − hit.a, hit.c − hit.a));
⟨Interpolate the shading normal from the corner normals 222⟩
⟨Perturb the shading normal with the normal map 223⟩
⟨Turn both normals towards the viewer 224⟩
```
This code is used in section 219.

222. The scene code orders the corners of every triangle counter-clockwise as seen from outside, so $(b-a)\times(c-a)$ points outward. Since the corners are already in world space, so is this normal.

When the mesh has shading normals, the three of the triangle's vertices are interpolated with the barycentric coordinates, in the mesh's own space, and the result is moved to world space. A normal cannot be moved like a point or a direction, though. If the instance transform has linear part M, a tangent t of the surface becomes Mt, and the transformed normal n' must stay perpendicular to it: $n' \cdot Mt = 0$ whenever $n \cdot t = 0$. That holds for $n' = (M^{-1})^{\mathsf{T}} n$, since then $n' \cdot Mt = n^{\mathsf{T}} M^{-1} Mt = n \cdot t$. For rotations and uniform scales the inverse transpose is proportional to M itself, but a scene full of squashed spheres and stretched boxes would be shaded wrongly with M. *optixTransformNormalFromObjectToWorldSpace* applies the inverse transpose, taking the inverse that OptiX keeps for every instance, and the result is normalized, since neither interpolation nor transformation keeps unit length.

A zero interpolated normal carries no direction, and the flat normal is kept. Should the shading normals disagree with the winding, as happens with meshes whose authored normals were made for the opposite orientation, the authored normals win and the geometric normal is turned around.

⟨Interpolate the shading normal from the corner normals 222⟩ ≡
```
// interpolated shading normal (smooth/authored normals); they win if the winding disagrees
float3 ns = n;
if (const float3* cn = hit.mesh→normals)
{
    const float2 bary = optixGetTriangleBarycentrics();
    const float3 local = cn[hit.tri.x] * (1.0f − bary.x − bary.y) + cn[hit.tri.y] * bary.x + cn[hit.tri.z] * bary.y;
    if (dot(local, local) > 0.0f)
    {
        ns = normalize(optixTransformNormalFromObjectToWorldSpace(local));
        if (dot(ns, n) < 0.0f)
            n = n * -1.0f;
    }
}
```
This code is used in section 221.

223. The normal map, when present and enabled, bends the shading normal; the tangent frame is built around the smooth normal, not the flat one, so that the bumps follow the curvature of the surface.

⟨Perturb the shading normal with the normal map 223⟩ ≡
```
if (params.normalMaps ∧ mat.normalTexture ≥ 0)
    ns = normalMapped(mat, hit, ns);
```
This code is used in section 221.

224. All surfaces are two-sided: a ray that hits the back of a triangle sees it lit as if it were the front. So when the geometric normal points away from the viewer, both normals are flipped.

Even then the shading normal may face away from the viewer: an interpolated or normal-mapped normal can tilt beyond the silhouette, which would give negative cosines and black, or NaN, pixels. We nudge it towards the view direction $v = -d$ just enough to make $n_s \cdot v$ at least 10^{-3}, and keep $n_s \cdot v$, which several formulas divide by, away from zero.

⟨Turn both normals towards the viewer 224⟩ ≡
```
if (dot(n, rayDir) > 0.0f)
{
    n = n * -1.0f;   // two-sided
    ns = ns * -1.0f;
}
// keep the shading normal facing the viewer
const float3 v = rayDir * -1.0f;
```

```
const float facing = dot(ns, v);
if (facing < 1e-3f)
    ns = normalize(ns + v * (1e-3f − facing));
const float nDotV = fmaxf(dot(ns, v), 1e-4f);
```

This code is used in section 221.

225. The material. We need three things from it: the diffuse color, the roughness, and the specular reflectance at normal incidence, F_0.

```
⟨Evaluate the material at the hit 225⟩ ≡
    // material
    ⟨Look up the base color 226⟩
    ⟨Derive the roughness and the reflectance at normal incidence 227⟩
    ⟨Split the energy between specular and diffuse reflection 228⟩
```

This code is used in section 219.

226. The base color, or *albedo*, is the texture remapped by its scale and bias, or the constant color. When the mesh is the current selection, the albedo is blended 60% towards orange; since the metallic workflow derives the reflections of metals from the albedo, a selected metal turns orange too.

```
⟨Look up the base color 226⟩ ≡
    float3 albedo = mat.color;
    if (mat.texture ≥ 0)
    {
        const float4 texel = sampleTexture(mat.texture, hit);
        albedo = make_float3(texel.x * mat.texScale.x + mat.texBias.x, texel.y * mat.texScale.y + mat.texBias.y,
                             texel.z * mat.texScale.z + mat.texBias.z);
    }
    if (meshId ≡ params.selectedMesh)
        albedo = mix(albedo, make_float3(1.0f, 0.55f, 0.1f), 0.6f);
```

This code is used in section 225.

227. UsdPreviewSurface has two ways to describe specular reflection. In the *specular workflow*, F_0 is given directly as *specularColor*, constant or textured, and the diffuse color is the albedo. In the more common *metallic workflow*, a dielectric such as plastic reflects $F_0 = \big((\eta-1)/(\eta+1)\big)^2$, about 4% for the default index of refraction $\eta = 1.5$, without tint, while a metal reflects with the color of its albedo and has no diffuse reflection at all. A *metallic* value between 0 and 1 blends the two. Roughness and metallic are clamped to $[0, 1]$, since a texture's scale and bias could carry them outside.

```
⟨Derive the roughness and the reflectance at normal incidence 227⟩ ≡
    const float roughness = __saturatef(evalScalar(mat.roughness, hit));
    float3 f0, diffuse;
    if (mat.specularWorkflow)
    {
        f0 = mat.specColor;
        if (mat.specTexture ≥ 0)
        {
            const float4 texel = sampleTexture(mat.specTexture, hit);
            f0 = make_float3(texel.x * mat.specScale.x + mat.specBias.x, texel.y * mat.specScale.y + mat.specBias.y,
                             texel.z * mat.specScale.z + mat.specBias.z);
        }
        diffuse = albedo;
    }
    else
    {
        const float metallic = __saturatef(evalScalar(mat.metallic, hit));
        const float r0 = (mat.ior − 1.0f) / (mat.ior + 1.0f);
        f0 = mix(make_float3(r0 * r0, r0 * r0, r0 * r0), albedo, metallic);
        diffuse = albedo * (1.0f − metallic);
    }
```

This code is used in section 225.

228. Light that is reflected specularly cannot also enter the surface and be reflected diffusely. We account

for that, simply, by weighting the diffuse term with $k_d = 1 - F(n \cdot v)$, the fraction not reflected at the viewing angle. This is the reason glossy objects show less of their color at grazing angles.

⟨Split the energy between specular and diffuse reflection 228⟩ ≡
```
const float3 fv = fresnel(f0, nDotV);
const float3 kd = make_float3(1.0f − fv.x, 1.0f − fv.y, 1.0f − fv.z);
```
This code is used in section 225.

229. The viewer does not compute indirect diffuse lighting, light that bounces off one surface onto another. Without it, the faces turned away from the light would be black. A constant ambient term of 0.15 and a "headlight" term that grows with $n \cdot v$ keep them readable and give a sense of shape.

⟨Add the ambient fill 229⟩ ≡
```
// ambient + headlight fill on the diffuse part (no indirect diffuse)
float3 color = diffuse * kd * (0.15f + 0.25f * nDotV);
```
This code is used in section 219.

230. The directional light. A directional light, like the sun, lights every point from the same direction l. The surface receives it only if l is above both the shading and the geometric horizons: with a bent shading normal alone, light could leak onto faces that actually look away from it.

⟨Add the directional light 230⟩ ≡
```
// directional light: Lambert + GGX
⟨Find the hit point and the offset for secondary rays 231⟩
const float3 l = params.lightDir;
const float nDotL = dot(ns, l);
if (nDotL > 0.0f ∧ dot(n, l) > 0.0f)
{
    ⟨Trace a shadow ray towards the light 232⟩
    if (visible)
    {
        ⟨Add the diffuse and specular reflection of the light 233⟩
    }
}
```
This code is used in section 219.

231. Secondary rays start at the hit point, but not exactly there: rounding errors would let them hit the very triangle they leave, speckling the surface with false shadows ("shadow acne"). We move the origin a little along the geometric normal, on the side the ray came from. Floating-point error grows with distance, so the offset does too, from 10^{-3} near the origin by a thousandth of the distance travelled.

⟨Find the hit point and the offset for secondary rays 231⟩ ≡
```
const float3 hitPos = optixGetWorldRayOrigin() + rayDir * optixGetRayTmax();
const float3 offset = n * (1e-3f * (1.0f + fabsf(optixGetRayTmax()) * 1e-3f));
```
This code is used in section 230.

232. The shadow ray only needs to know whether *anything* blocks the light, so it stops at the first hit it finds and runs no closest-hit program; if it escapes, the shadow miss program sets *visible*. It uses the radiance hit-group record, whose any-hit program applies the alpha cutouts, and the shadow miss program. With shadows switched off, the light is always visible.

⟨Trace a shadow ray towards the light 232⟩ ≡
```
unsigned int visible = 1u;
if (params.shadows)
{
    visible = 0u;
    optixTrace(params.handle, hitPos + offset, l, 0.0f, 1e16f, 0.0f, OptixVisibilityMask(255),
               OPTIX_RAY_FLAG_TERMINATE_ON_FIRST_HIT | OPTIX_RAY_FLAG_DISABLE_
                   CLOSESTHIT,
               RAY_TYPE_RADIANCE, RAY_TYPE_COUNT, RAY_TYPE_SHADOW, visible);
}
```
This code is used in section 230.

233. The reflected light is the BRDF times the irradiance $E\,(n \cdot l)$:

$$L = \Big(k_d\,\frac{\rho_d}{\pi} + \frac{D\,G\,F}{4\,(n \cdot l)(n \cdot v)}\Big)E\,(n \cdot l) = E\Big(k_d\,\rho_d\,\frac{n \cdot l}{\pi} + F\,\frac{D\,G}{4\,(n \cdot v)}\Big),$$

and the factor $n \cdot l$ cancels in the specular term, which *spec* holds. One detail needs care. A directional light is a point on the sphere of directions, and the highlight of such a light on a perfect mirror is an infinitely bright point of zero size: either invisible or a single white pixel. We therefore never let the GGX width for this light fall below $\alpha = 0.01$, which turns the highlight into a small, finite spot.

⟨Add the diffuse and specular reflection of the light 233⟩ ≡
```
const float3 h = normalize(v + l);
const float aLight = fmaxf(roughness * roughness, 0.01f);   // a delta light needs a minimum width
const float a2 = aLight * aLight;
const float3 f = fresnel(f0, dot(v, h));
const float spec = ggxD(fmaxf(dot(ns, h), 0.0f), a2) * smithG1(nDotV, a2) * smithG1(nDotL, a2) /
                   (4.0f * nDotV);   // BRDF * nDotL
color = color + (diffuse * kd * (nDotL / kPi) + f * spec) * kLightIntensity;
```
This code is used in section 230.

234. Glossy reflections. Light also arrives from every other direction: from the sky, and from the other objects of the scene. A smooth surface reflects it like a mirror and a rough one blurs it. Computing that blur exactly would mean integrating the BRDF against all incoming light; instead each sample of each pixel traces *one* reflection ray in a random direction, chosen in proportion to the glossy lobe, and the progressive averaging of the ray generation program does the integration over the frames.

The choice of the random direction matters a great deal. Drawing microfacet normals from D alone would be correct on average, but rough metals would glitter with noise even after 256 samples, because the weight of each sample, $F\,G\,(v \cdot h)/\big((n \cdot h)(n \cdot v)\big)$, becomes very large at grazing angles. We therefore sample the distribution of *visible* normals, the facets that the viewer can actually see, following Eric Heitz's "Sampling the GGX Distribution of Visible Normals" (2018).

⟨Add a glossy reflection 234⟩ ≡
```
// glossy reflection: one direction per sample, accumulated over frames.
// Visible-normal sampling of GGX (Heitz 2018): the weight is F * G1(r) <= 1,
// which keeps rough and grazing reflections free of fireflies.
{
    ⟨Express the view direction in a frame around the shading normal 235⟩
    ⟨Sample a visible normal of the GGX distribution 236⟩
    ⟨Reflect the view direction about the sampled normal 237⟩
    if (vDotH > 0.0f ∧ nDotR > 0.0f ∧ dot(n, r) > 0.0f)
    {
        ⟨Weight the reflected light and fetch it 238⟩
    }
}
```
This code is used in section 219.

235. The sampling works in a local frame whose z axis is the shading normal. Any two unit vectors perpendicular to n_s and to each other will do; we take the cross product with the z axis of the world, or with the x axis when n_s is nearly parallel to z. A minimum width $\alpha = 10^{-3}$ keeps the formulas finite for perfect mirrors.

⟨Express the view direction in a frame around the shading normal 235⟩ ≡
```
const float alpha = fmaxf(roughness * roughness, 1e-3f), a2 = alpha * alpha;
const float3 up = fabsf(ns.z) < 0.999f ? make_float3(0.0f, 0.0f, 1.0f) : make_float3(1.0f, 0.0f, 0.0f);
const float3 tx = normalize(cross(up, ns)), ty = cross(ns, tx);
const float3 ve = make_float3(dot(v, tx), dot(v, ty), dot(v, ns));   // view in tangent space
```
This code is used in section 234.

236. Heitz's method rests on a geometric observation. Stretch the GGX surface by $1/\alpha$ horizontally and its microfacets become those of a hemisphere; the visible facets of a hemisphere are distributed like its projected area as seen from the viewer, which is easy to sample. So we stretch the view vector into v_h, build an orthonormal basis (t_1, t_2, v_h), pick a uniform point (p_1, p_2) on the unit disk in polar coordinates, and squeeze

its second coordinate towards the visible half of the disk with the factor $s = (1 + v_{h,z})/2$; the point is then lifted onto the hemisphere and the resulting normal unstretched by α. The final line takes it back from the local frame to the world.

⟨Sample a visible normal of the GGX distribution 236⟩ ≡
```
const float3 vh = normalize(make_float3(alpha * ve.x, alpha * ve.y, ve.z));
const float lenSq = vh.x * vh.x + vh.y * vh.y;
const float3 t1 = lenSq > 0.0f ? make_float3(−vh.y, vh.x, 0.0f) * rsqrtf(lenSq) : make_float3(1.0f, 0.0f, 0.0f);
const float3 t2 = cross(vh, t1);
const float rad = sqrtf(rnd(seed)), phi = 2.0f * kPi * rnd(seed);
const float p1 = rad * cosf(phi), s = 0.5f * (1.0f + vh.z);
const float p2 = (1.0f − s) * sqrtf(fmaxf(0.0f, 1.0f − p1 * p1)) + s * rad * sinf(phi);
const float3 nh = t1 * p1 + t2 * p2 + vh * sqrtf(fmaxf(0.0f, 1.0f − p1 * p1 − p2 * p2));
const float3 hl = normalize(make_float3(alpha * nh.x, alpha * nh.y, fmaxf(nh.z, 0.0f)));
const float3 h = tx * hl.x + ty * hl.y + ns * hl.z;
```
This code is used in section 234.

237. The reflection of v about the facet normal h is $r = 2\,(v \cdot h)\,h - v$.

⟨Reflect the view direction about the sampled normal 237⟩ ≡
```
const float vDotH = dot(v, h);
const float3 r = h * (2.0f * vDotH) − v;
const float nDotR = dot(ns, r);
```
This code is used in section 234.

238. A sample is used only if the facet faces the viewer and the reflected ray leaves the surface, above both the shading and the geometric horizons; otherwise the light it would carry is simply lost, as the masking of real microfacets loses it.

Then comes the payoff of visible-normal sampling. The probability density of r is $D_v(h)/\big(4\,(v \cdot h)\big)$, where $D_v(h) = G_1(v)\,(v \cdot h)\,D(h)/(n \cdot v)$ is the density of visible normals. The Monte Carlo weight of the sample, BRDF times cosine over density, is

$$\frac{D\,G_1(v)\,G_1(r)\,F}{4\,(n \cdot v)(n \cdot r)}\,(n \cdot r) \cdot \frac{4\,(v \cdot h)\,(n \cdot v)}{G_1(v)\,(v \cdot h)\,D} = F\,G_1(r),$$

which never exceeds 1. Both D and $G_1(v)$ have cancelled.

⟨Weight the reflected light and fetch it 238⟩ ≡
```
const float3 weight = fresnel(f0, vDotH) * smithG1(nDotR, a2);
float3 incoming;
if (params.reflections ∧ depth ≡ 0)
{
    ⟨Trace the reflection ray and clamp fireflies 239⟩
}
else
    incoming = sky(r);
color = color + weight * incoming;
```
This code is used in section 234.

239. Only primary rays spawn a reflection ray; the reflection rays themselves, at depth 1, take the sky in the reflected direction instead, as does every hit when the *Reflections* switch is off. This bounds the recursion: a camera ray, a reflection ray, and the shadow ray traced from the reflection hit, three levels in all, which is the maximum trace depth the host declares for the pipeline. The reflection ray carries the seed, and receives it back advanced.

Visible-normal sampling leaves one problem: rare, very bright pixels, the “fireflies”. They appear when a rough reflection happens to land on the highlight of the light on another, smooth surface. That highlight is thousands of times brighter than its surroundings, so one such sample in 256 still shows as a white dot. Any value above 2 is displayed as white anyway, so we clamp the brightest component of the reflected light to 2, scaling the whole color to preserve its hue. The image loses nothing visible and the fireflies disappear.

⟨Trace the reflection ray and clamp fireflies 239⟩ ≡
```
unsigned int rr, rg, rb, rMesh, rDepth = depth + 1, rT;
optixTrace(params.handle, hitPos + offset, r, 0.0f, 1e16f, 0.0f, OptixVisibilityMask(255),
```

```
            OPTIX_RAY_FLAG_NONE, RAY_TYPE_RADIANCE, RAY_TYPE_COUNT, RAY_TYPE_
                RADIANCE,
            rr, rg, rb, rMesh, rDepth, seed, rT);
    incoming = make_float3(__uint_as_float(rr), __uint_as_float(rg), __uint_as_float(rb));
    // Firefly clamp: a rough reflection that lands on another surface's sharp
    // highlight of the (delta) light returns huge, rare values. Anything above
    // 2 already displays as white, so clamping there only removes the spikes.
    const float peak = fmaxf(incoming.x, fmaxf(incoming.y, incoming.z));
    if (peak > 2.0f)
        incoming = incoming * (2.0f / peak);
```

This code is used in section 238.

240. Finally the program writes the advanced seed back into the payload, so that a reflection's caller continues the random sequence, and returns the color with the id of the mesh that was hit and the hit distance, which *optixGetRayTmax* reports in a closest-hit program.

⟨Return the color and the seed to the caller 240⟩ ≡

```
    optixSetPayload_5(seed);
    optixSetPayload_6(__float_as_uint(optixGetRayTmax()));
    setColor(color, meshId);
```

This code is used in section 219.

The application

241. We have a scene that knows how to read a USD stage and a renderer that knows how to turn triangles into pixels. What remains is the program that a person actually sees: a window with a menu bar, a tree of prims on the left, an inspector on the right, the ray-traced image in the middle and a status line at the bottom. This part builds that program with SDL3, which gives us a window, input events and a simple 2D renderer, and Dear ImGui, an "immediate mode" user interface library in which the whole interface is described anew in every frame by ordinary function calls.

The immediate-mode style shapes the code that follows. There are no widget objects to create, connect and destroy; instead, each frame the application asks ImGui to draw, say, a menu item, and ImGui answers whether it was clicked. The state of the interface therefore lives in a handful of plain member variables of one class, *App*, and every panel is a member function that reads them and changes them.

242. The main program. The entry point is as short as it can be: it creates the application, initializes it with the command-line arguments, and runs it until the window is closed. Everything interesting happens inside *App*.

⟨`UsdOptixViewer/src/main.cpp` 242⟩ ≡

```
// USD OptiX Viewer: loads a USD stage, shows its prim hierarchy and
// properties, and ray traces its geometry with OptiX.
//
// UsdOptixViewer [file.usd]
#include "App.h"

int main(int argc, char** argv)
{
    App app;
    if (¬app.init(argc, argv))
        return 1;
    app.run();
    return 0;
}
```

This code is written to file `UsdOptixViewer/src/main.cpp`.

243. The application class. The header `App.h` declares the class. Its public face is tiny: initialization, the main loop, and one function, *requestOpen*, that other threads may call. All the rest is private, and falls into two groups: the functions that draw the parts of the interface or act on the scene, and the state they share.

⟨`UsdOptixViewer/src/App.h` 243⟩ ≡

```
// Application: SDL3 window, Dear ImGui panels (hierarchy, inspector,
// viewport) and the OptiX renderer.
#pragma once

#include <array>
#include <mutex>
#include <string>
#include <unordered_set>
#include <vector>

#include <SDL3/SDL.h>

#include "CloudXrService.h"
#include "OptixRenderer.h"
#include "Scene.h"
#include "XrOutput.h"

class App
{
public:
    ⟨Public interface of the application 244⟩

private:
    ⟨Private member functions of the application 245⟩
```

⟨Window and SDL objects of the application 246⟩

⟨Scene and renderer of the application 247⟩

⟨Camera and display options of the application 248⟩

⟨Selection and hierarchy filter state 249⟩

⟨Text input and dialog state 250⟩

⟨File requests from other threads 251⟩

⟨XR output state of the application 252⟩

};

This code is written to file `UsdOptixViewer/src/App.h`.

244. The destructor releases ImGui and SDL in the reverse order of their creation. The file dialog of SDL may run its callback on a thread of its own, which is why *requestOpen* is public and documented as thread-safe.

⟨Public interface of the application 244⟩ ≡

```
~App();
bool init(int argc, char** argv);
void run();

// Called from the SDL file dialog callback (possibly on another thread).
void requestOpen(const std::string& path);
```

This code is used in section 243.

245. The private functions follow the layout of the screen: one function per panel, plus a few actions on the camera and the selection. Each is presented below, in the section that deals with its panel.

⟨Private member functions of the application 245⟩ ≡

```
void loadFile(const std::string& path);
void processEvent(const SDL_Event& event);
void drawUi();
void drawMenuBar();
void drawHierarchy();
void drawHierarchyNode(const pxr::UsdPrim& prim);
void drawInspector();
void drawViewport();
void drawStatusBar();
void drawPopups();

void select(const pxr::SdfPath& path, bool reveal);
void frameBounds(const float lo[3], const float hi[3]);
void frameSelection();
void updateFilter();
void updateCamera();
void orbitOffset(float offset[3], float up[3]) const;
void viewFrame(float eye[3], float forward[3]) const;
void lookThrough(int camera);
void orbitFromUsdCamera();
void orbitAround(const float target[3], const float offset[3], float distance);
void pivotAt(unsigned int x, unsigned int y);
void fittedWindow(const SceneCamera& camera, float& halfWidth, float& halfHeight) const;
void openFileDialog();

void startXr(bool cloudXr, std::string sdkDir, const std::string& profile);
void updateXr();
void renderXrEyes(std::string& error);
bool useRendererOn(const std::array<unsigned char, 16>& uuid, std::string& error);
void drawXrMirror(float x, float y, float width, float height);
std::string xrStatus() const;
```

This code is used in section 243.

246. SDL gives us three objects: the window, a 2D renderer attached to it, and a texture that holds the latest ray-traced image. The flag *m_running* is cleared when the user asks to quit.

⟨Window and SDL objects of the application 246⟩ ≡
```
SDL_Window* m_window = nullptr;
SDL_Renderer* m_sdlRenderer = nullptr;
SDL_Texture* m_texture = nullptr;
bool m_running = true;
```
This code is used in section 243.

247. The scene and the OptiX renderer are plain members. *m_rendererOk* stays false if OptiX could not be initialized, for instance on a machine without a suitable GPU; the rest of the interface still works, so the program can at least be used to inspect a stage. *m_needsRender* is the one flag through which every part of the interface says "something that affects the image has changed".

⟨Scene and renderer of the application 247⟩ ≡
```
Scene m_scene;
OptixRenderer m_renderer;
bool m_rendererOk = false;
bool m_needsRender = true;
```
This code is used in section 243.

248. The camera is an *orbit* camera: it always looks at a target point, from a given distance, along a direction given by two angles, the yaw around the up axis and the pitch above the horizon. The field of view is vertical and measured in degrees. Alternatively, the view may come from one of the stage's own cameras; *m_camera* then holds its index, and the orbit parameters are ignored until the user moves the view. The light direction and the four switches of the View menu complete the parameters of the image.

⟨Camera and display options of the application 248⟩ ≡
```
// orbit camera
float m_target[3] = {0, 0, 0};
float m_distance = 5.0f, m_yaw = 0.6f, m_pitch = 0.4f, m_fovY = 45.0f;
int m_camera = -1;  // USD camera we look through (index into Scene::cameras()), -1 for the orbit camera
float m_light[3] = {0.4f, 0.8f, 0.45f};
bool m_shadows = true;
bool m_mipmaps = true;
bool m_normalMaps = true;
bool m_reflections = true;
```
This code is used in section 243.

249. The selection is the USD path of a prim, and the empty path means that nothing is selected. A path is the natural name for a prim: it is what the stage hands out, what the command line and the inspector show, and it stays meaningful whatever the scene and the renderer derive from the stage. When a prim is selected by clicking on the image, the tree must open all its ancestors so that it becomes visible; *m_reveal* holds their paths until the tree has been drawn once. The filter of the tree keeps the text being typed, the text for which the filter was last computed, and the resulting set of paths to show. Both sets are hash sets of paths, with the hash function that *SdfPath* provides.

⟨Selection and hierarchy filter state 249⟩ ≡
```
using PathSet = std::unordered_set<pxr::SdfPath, pxr::SdfPath::Hash>;
pxr::SdfPath m_selected;  // empty: nothing selected
bool m_revealSelection = false;
PathSet m_reveal;  // ancestors to open in the tree
char m_filter[128] = "";
std::string m_appliedFilter;
PathSet m_visibleByFilter;
```
This code is used in section 243.

250. ImGui's text fields edit fixed character buffers in place, hence the **char** arrays. *m_error* holds a message to be shown in a modal popup; an empty string means there is none.

⟨Text input and dialog state 250⟩ ≡
```
char m_pathInput[1024] = "";
std::string m_error;
bool m_showAbout = false;
```
This code is used in section 243.

251. A path to open, posted by the file dialog or by a dropped file, waits here until the main loop picks it up. The mutex protects it against the dialog's thread.

⟨File requests from other threads 251⟩ ≡
```
std::mutex m_pendingMutex;
std::string m_pendingOpen;
```
This code is used in section 243.

252. The XR output has its own state. *m_cloudXr* runs the CloudXR runtime when it lives inside this process, and *m_xr* is the OpenXR side; members are destroyed in the reverse order of their declaration, so the session ends before the service that serves it stops. The other fields are the user's settings for the headset view, the time the last headset frame took, and the texture and pixels of the optional mirror of the left eye in the viewport.

⟨XR output state of the application 252⟩ ≡
```
// XR output. The CloudXR service (if it runs in this process) is declared
// first, so that the XR session ends before the service stops.
CloudXrService m_cloudXr;
XrOutput m_xr;
bool m_xrEnabled = false;
bool m_xrPaced = false;          // xrWaitFrame paces the loop instead of vsync
bool m_xrClient = false;         // a CloudXR client is connected
float m_xrResolution = 1.0f;     // fraction of the headset's recommended eye resolution
float m_xrWorldScale = 1.0f;     // > 1 makes the user larger than the scene's units say
int m_xrSamples = 1;             // samples per pixel and frame
float m_xrFrameMs = 0.0f;
bool m_xrMirror = false;         // show the left eye in the viewport
SDL_Texture* m_mirrorTexture = nullptr;
int m_mirrorWidth = 0, m_mirrorHeight = 0;
std::vector<uchar4> m_mirrorPixels;
```
This code is used in section 243.

253. The implementation. The file `App.cpp` contains, after its headers and two helper functions, the member functions in the order of the life of the program: creation, the main loop, and then the panels from left to right.

⟨`UsdOptixViewer/src/App.cpp` 253⟩ ≡
```
⟨Headers of the application 254⟩

PXR_NAMESPACE_USING_DIRECTIVE

namespace {

⟨Constants of the application 255⟩

⟨Format an attribute value for the inspector 296⟩

⟨Search a name ignoring case 281⟩

}  // namespace

⟨Destroy the application 263⟩

⟨Initialize the application 256⟩

⟨Request a file to be opened 264⟩

⟨Load a USD file into the scene and the renderer 265⟩

⟨The main loop of the application 267⟩

⟨Handle an SDL event 268⟩

// ---- layout ------------------------------------------------------------
⟨Lay out the panels and handle shortcuts 269⟩

⟨Draw the menu bar 273⟩

⟨Show the file dialog 266⟩

// ---- hierarchy ---------------------------------------------------------
⟨Recompute the hierarchy filter 280⟩

⟨Draw the hierarchy panel 279⟩
```

⟨Draw one node of the hierarchy 282⟩

⟨Select a prim 285⟩

```
// ---- inspector ------------------------------------------------------------
```
⟨Draw the inspector 286⟩

```
// ---- viewport -------------------------------------------------------------
```
⟨Send the camera and the options to the renderer 298⟩

⟨Compute the orbit offset 301⟩

⟨Find the eye and direction of the desktop view 302⟩

⟨Fit a USD camera's window to the viewport 303⟩

⟨Look through a USD camera 304⟩

⟨Hand the view of a USD camera to the orbit camera 305⟩

⟨Orbit a target from a direction and distance 306⟩

⟨Orbit the surface point under a pixel 307⟩

⟨Frame a bounding box 308⟩

⟨Frame the selection 309⟩

⟨Draw the viewport 310⟩

⟨Draw the status bar 322⟩

⟨Draw the popups 323⟩

⟨The XR output of the application 324⟩

This code is written to file `UsdOptixViewer/src/App.cpp`.

254. Besides ImGui and its two back ends, the implementation needs a few pieces of OpenUSD for the inspector and the hierarchy (prim ranges walk the whole stage for the filter), and `optix.h` only for the version number that the status bar and the About box display. The macro *PXR_NAMESPACE_USING_DIRECTIVE* brings OpenUSD's versioned namespace into scope, as everywhere else in the program.

⟨Headers of the application 254⟩ ≡
```
#include "App.h"

#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdio>
#include <cstdlib>

#include <imgui.h>
#include <imgui_impl_sdl3.h>
#include <imgui_impl_sdlrenderer3.h>
#include <pxr/base/tf/stringUtils.h>
#include <pxr/usd/usd/attribute.h>
#include <pxr/usd/usd/modelAPI.h>
#include <pxr/usd/usd/primRange.h>
#include <pxr/usd/usd/relationship.h>
#include <pxr/usd/usdGeom/metrics.h>
#include <optix.h>
```
This code is used in section 253.

255. Two constants govern the whole interface. The renderer is progressive: it adds one sample per pixel and frame and averages them. After *kMaxSamples* samples the image has converged well enough, antialiased edges and glossy reflections included, and we stop spending the GPU's time on it. At the display's refresh rate of 60 frames per second, 256 samples take a little over four seconds.

The panels are not meant to be moved, resized or folded by the user; their places are computed every frame from the size of the window (⟨Lay out the panels and handle shortcuts 269⟩). *kPanelFlags* says so, and also asks ImGui not to save their positions in a settings file.

⟨Constants of the application 255⟩ ≡
```
constexpr unsigned int kMaxSamples = 256;   // progressive samples per pixel

constexpr ImGuiWindowFlags kPanelFlags =
```

ImGuiWindowFlags_NoMove | *ImGuiWindowFlags_NoResize* | *ImGuiWindowFlags_NoCollapse* |
ImGuiWindowFlags_NoSavedSettings;

This code is used in section 253.

256. Creation and destruction. Initialization proceeds in four steps: the window, the user interface library, the command line, and the ray tracer (after the XR output, if the command line asks for one), and finally the file, prim and camera that the command line names. A failure of SDL is fatal, but a failure of OptiX is not; it is reported in a popup and the viewport stays empty.

⟨Initialize the application 256⟩ ≡
```
bool App::init(int argc, char** argv)
{
    ⟨Create the window and the SDL renderer 257⟩
    ⟨Set up Dear ImGui 258⟩
    ⟨Parse the command line 259⟩
    ⟨Start the XR output and the OptiX renderer 260⟩
    ⟨Open the requested file and select the requested prim 261⟩
    ⟨Look through the requested camera 262⟩
    return true;
}
```
This code is used in section 253.

257. On a high-density display, a window of 1600 by 960 *logical* units may cover twice as many physical pixels. SDL reports the ratio the desktop uses for its own text as the display's *content scale*; we size the window by it so that it occupies the same fraction of the screen everywhere, and ask for a framebuffer at full pixel density with *SDL_WINDOW_HIGH_PIXEL_DENSITY*.

For drawing we use SDL's own 2D renderer rather than OpenGL. The interface is just textured triangles, which *SDL_Renderer* draws with whatever back end the platform offers, and our one large image—the ray-traced view—arrives as an ordinary array of pixels in main memory (the renderer copies it there anyway, because it also reads back the mesh identifiers used for picking). A streaming SDL texture is the simplest home for such an array; sharing a buffer between CUDA and OpenGL would avoid one copy per frame, but would tie the program to one graphics API for a saving that does not matter at these resolutions. Vertical synchronization paces the main loop at the display's refresh rate, which also paces the progressive rendering.

⟨Create the window and the SDL renderer 257⟩ ≡
```
if (¬SDL_Init(SDL_INIT_VIDEO))
{
    SDL_Log("SDL_Init failed: %s", SDL_GetError());
    return false;
}
const float scale = SDL_GetDisplayContentScale(SDL_GetPrimaryDisplay());
m_window = SDL_CreateWindow("USD OptiX Viewer", int(1600 * scale), int(960 * scale),
                            SDL_WINDOW_RESIZABLE | SDL_WINDOW_HIGH_PIXEL_DENSITY);
if (¬m_window)
{
    SDL_Log("SDL_CreateWindow failed: %s", SDL_GetError());
    return false;
}
m_sdlRenderer = SDL_CreateRenderer(m_window, nullptr);
if (¬m_sdlRenderer)
{
    SDL_Log("SDL_CreateRenderer failed: %s", SDL_GetError());
    return false;
}
SDL_SetRenderVSync(m_sdlRenderer, 1);
```
This code is used in section 256.

258. ImGui is set up as in its SDL3 example, with three changes. Keyboard navigation is enabled; the settings file `imgui.ini` is disabled (*IniFilename* set to null), since our panels have fixed places and there is nothing worth remembering; and window corners are square, because the panels tile the window. The same

content scale enlarges all sizes and fonts, so that the interface is as legible on a high-density screen as on an ordinary one. The two back ends connect ImGui to SDL's events and to SDL's renderer.

⟨Set up Dear ImGui 258⟩ ≡
```
IMGUI_CHECKVERSION();
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;
io.IniFilename = nullptr;
ImGui::StyleColorsDark();
ImGuiStyle& style = ImGui::GetStyle();
style.ScaleAllSizes(scale);
style.FontScaleDpi = scale;
style.WindowRounding = 0.0f;
ImGui_ImplSDL3_InitForSDLRenderer(m_window, m_sdlRenderer);
ImGui_ImplSDLRenderer3_Init(m_sdlRenderer);
```
This code is used in section 256.

259. The command line names at most one file, and may add options. Three of them switch off a feature of the renderer from the start, exactly as the corresponding items of the View menu do; they make it easy to compare images with and without mipmaps, normal maps or traced reflections.

The fourth, `--select`, selects a prim by its path and frames the camera on it. It exists mostly for testing: synthetic mouse clicks sent to the window by `xdotool` do not reach SDL on the development machine's display, so the way to check the selection, the tree and the inspector from a script is to select a prim from the command line and take a screenshot. For the same reason `--camera` looks through a USD camera given by its path, as the Camera menu does.

The remaining options concern the XR output, described at the end of this part. `--xr` sends the view to the active OpenXR runtime, and `--cloudxr` first starts NVIDIA's CloudXR runtime inside the program, from the SDK directory that `--cloudxr-sdk` names, with the device profile of `--cloudxr-profile`. The last three set the samples per pixel and frame, the eye resolution as a fraction of the headset's recommendation, and the mirror of the left eye; the numbers are clamped to sensible ranges.

⟨Parse the command line 259⟩ ≡
```
// UsdOptixViewer [file.usd] [--select /Prim/Path] [--camera /Camera/Path] [--no-mipmaps] [--no-normal-maps]
// [--no-reflections] [--xr] [--cloudxr] [--cloudxr-sdk DIR] [--cloudxr-profile NAME]
// [--xr-samples N] [--xr-resolution F] [--xr-mirror]
std::string file, selectPath, cameraPath, cloudXrSdk, cloudXrProfile;
bool xr = false, cloudXr = false;
for (int i = 1; i < argc; ++i)
{
    const std::string arg = argv[i];
    if (arg ≡ "--select" ∧ i + 1 < argc)
        selectPath = argv[++i];
    else if (arg ≡ "--camera" ∧ i + 1 < argc)
        cameraPath = argv[++i];
    else if (arg ≡ "--xr")
        xr = true;
    else if (arg ≡ "--cloudxr")
        cloudXr = true;
    else if (arg ≡ "--cloudxr-sdk" ∧ i + 1 < argc)
        cloudXrSdk = argv[++i];
    else if (arg ≡ "--cloudxr-profile" ∧ i + 1 < argc)
        cloudXrProfile = argv[++i];
    else if (arg ≡ "--xr-samples" ∧ i + 1 < argc)
        m_xrSamples = std::clamp(std::atoi(argv[++i]), 1, 64);
    else if (arg ≡ "--xr-resolution" ∧ i + 1 < argc)
        m_xrResolution = std::clamp(float(std::atof(argv[++i])), 0.1f, 2.0f);
    else if (arg ≡ "--xr-mirror")
        m_xrMirror = true;
    else if (arg ≡ "--no-mipmaps")
        m_mipmaps = false;
    else if (arg ≡ "--no-normal-maps")
        m_normalMaps = false;
```

```
    else if (arg ≡ "--no-reflections")
        m_reflections = false;
    else
        file = arg;
}
```

This code is used in section 256.

260. The XR runtime, if one is wanted, starts before the renderer: the CloudXR service must run before any OpenXR call, and the renderer may later have to move to the GPU that the runtime picks (see ⟨Follow the XR session 326⟩). If OptiX cannot start, the message goes into *m_error*, which the next frame shows in a popup (⟨Draw the popups 323⟩).

```
⟨Start the XR output and the OptiX renderer 260⟩ ≡
    // The XR runtime must exist before the renderer starts, because it may
    // choose another GPU (see updateXr).
    if (xr ∨ cloudXr)
        startXr(cloudXr, cloudXrSdk, cloudXrProfile);
    std::string error;
    m_rendererOk = m_renderer.init(error);
    if (¬m_rendererOk)
        m_error = "OptiX initialization failed: " + error;
```

This code is used in section 256.

261. The prim to select must exist on the stage, which we ask the stage itself, after checking that the string is a valid path at all (the constructor of *SdfPath* would complain loudly otherwise). Selecting with *reveal* set opens its ancestors in the tree, and *frameSelection* points the camera at it.

```
⟨Open the requested file and select the requested prim 261⟩ ≡
    if (¬file.empty())
        loadFile(file);
    if (¬selectPath.empty() ∧ m_scene.loaded() ∧ SdfPath::IsValidPathString(selectPath))
    {
        const SdfPath path(selectPath);
        if (m_scene.stage()→GetPrimAtPath(path))
        {
            select(path, true);
            frameSelection();
        }
    }
```

This code is used in section 256.

262. The camera comes last, because selecting a prim frames it with the orbit camera. A path that names no camera of the stage is reported in the error popup.

```
⟨Look through the requested camera 262⟩ ≡
    if (¬cameraPath.empty())
    {
        const std::vector<SceneCamera>& cameras = m_scene.cameras();
        const auto it = std::find_if(cameras.begin(), cameras.end(),
                                     [&](const SceneCamera& c) { return c.path ≡ cameraPath; });
        if (it ≠ cameras.end())
            lookThrough(int(it − cameras.begin()));
        else
            m_error = "No camera at " + cameraPath;
    }
```

This code is used in section 256.

263. Destruction undoes initialization in reverse. ImGui is shut down only if it was created, because *init* may have failed before reaching that point. The scene and the OptiX renderer are members and clean up after themselves when the object's lifetime ends.

```
⟨Destroy the application 263⟩ ≡
    App::~App()
    {
```

```
    if (ImGui::GetCurrentContext())
    {
        ImGui_ImplSDLRenderer3_Shutdown();
        ImGui_ImplSDL3_Shutdown();
        ImGui::DestroyContext();
    }
    if (m_texture)
        SDL_DestroyTexture(m_texture);
    if (m_mirrorTexture)
        SDL_DestroyTexture(m_mirrorTexture);
    if (m_sdlRenderer)
        SDL_DestroyRenderer(m_sdlRenderer);
    if (m_window)
        SDL_DestroyWindow(m_window);
    SDL_Quit();
}
```

This code is used in section 253.

264. Opening files. A file may be requested in four ways: on the command line, through the file dialog, by typing a path in the File menu, or by dropping it on the window. All but the first go through *requestOpen*, which merely records the path; the main loop loads it at a quiet moment, between two frames. This matters for the file dialog: SDL shows it asynchronously and may call us back on another thread, and loading a stage there would race with the drawing of the interface. A mutex makes the hand-over safe. If two requests arrive in the same frame, the later one wins, which is what a user who changes their mind expects.

⟨Request a file to be opened 264⟩ ≡

```
void App::requestOpen(const std::string& path)
{
    std::lock_guard<std::mutex> lock(m_pendingMutex);
    m_pendingOpen = path;
}
```

This code is used in section 253.

265. Loading forgets everything that belonged to the previous stage: the camera we were looking through, the selection, the prims to reveal and the filter of the tree. If the scene cannot open the file, its message is shown in the error popup. Otherwise the path is copied into the text field of the File menu (so that it can be edited and reloaded), the window title names the file, the scene's meshes, materials and textures are sent to the renderer, and the camera frames the whole stage.

⟨Load a USD file into the scene and the renderer 265⟩ ≡

```
void App::loadFile(const std::string& path)
{
    std::string error;
    m_camera = -1;   // camera indices refer to the previous stage
    m_selected = SdfPath();
    m_reveal.clear();
    m_appliedFilter.clear();
    m_filter[0] = '\0';
    if (¬m_scene.load(path, error))
    {
        m_error = error;
        return;
    }
    SDL_strlcpy(m_pathInput, path.c_str(), sizeof(m_pathInput));
    SDL_SetWindowTitle(m_window, ("USD OptiX Viewer - " + path).c_str());
    if (m_rendererOk)
    {
        m_renderer.setScene(m_scene);
        m_renderer.setSelectedMesh(kNoMesh);
    }
    frameBounds(m_scene.boundsMin(), m_scene.boundsMax());
}
```

This code is used in section 253.

266. The dialog is SDL's portable one; it uses the desktop's native dialog where there is one. Its callback receives a list of chosen files, an empty list if the user cancelled, or a null pointer if something went wrong. The filter list offers the four USD extensions first.

⟨Show the file dialog 266⟩ ≡
```
void App::openFileDialog()
{
    static const SDL_DialogFileFilter filters[] = {{"USD files", "usd;usda;usdc;usdz"}, {"All files", "*"}};
    SDL_ShowOpenFileDialog(
        [](void* userdata, const char* const* files, int) {
            if (files ∧ files[0])
                static_cast<App*>(userdata)→requestOpen(files[0]);
            else if (¬files)
                SDL_Log("File dialog error: %s", SDL_GetError());
        },
        this, m_window, filters, 2, nullptr, false);
}
```
This code is used in section 253.

267. The main loop. Each turn of the loop handles the pending events, loads a requested file if there is one, and then draws one frame. The pending path is taken out under the lock by swapping it with an empty string, and loaded after the lock has been released, so that the dialog's thread never waits for a stage to load. Then the XR output gets its turn (⟨Follow the XR session 326⟩), before the minimized test, so that the headset keeps receiving images while the window is minimized. While the window is minimized we draw nothing and sleep a little, rather than spin.

Drawing a frame is ImGui's usual ritual: start a new frame in both back ends and in ImGui, describe the whole interface (⟨Lay out the panels and handle shortcuts 269⟩), let ImGui turn the description into triangles, and have SDL draw them. On a high-density display, ImGui works in logical units and SDL must scale its drawing by the framebuffer scale to match.

⟨The main loop of the application 267⟩ ≡
```
void App::run()
{
    while (m_running)
    {
        SDL_Event event;
        while (SDL_PollEvent(&event))
            processEvent(event);

        std::string pending;
        {
            std::lock_guard<std::mutex> lock(m_pendingMutex);
            pending.swap(m_pendingOpen);
        }
        if (¬pending.empty())
            loadFile(pending);

        updateXr();  // keeps streaming while the window is minimized
        if (SDL_GetWindowFlags(m_window) & SDL_WINDOW_MINIMIZED)
        {
            SDL_Delay(10);
            continue;
        }

        ImGui_ImplSDLRenderer3_NewFrame();
        ImGui_ImplSDL3_NewFrame();
        ImGui::NewFrame();
        drawUi();
        ImGui::Render();

        const ImGuiIO& io = ImGui::GetIO();
        SDL_SetRenderScale(m_sdlRenderer, io.DisplayFramebufferScale.x, io.DisplayFramebufferScale.y);
        SDL_SetRenderDrawColorFloat(m_sdlRenderer, 0.08f, 0.08f, 0.09f, 1.0f);
```

```
        SDL_RenderClear(m_sdlRenderer);
        ImGui_ImplSDLRenderer3_RenderDrawData(ImGui::GetDrawData(), m_sdlRenderer);
        SDL_RenderPresent(m_sdlRenderer);
    }
}
```

This code is used in section 253.

268. Every event goes first to ImGui, which keeps its own record of the mouse and the keyboard. Only three events concern the application itself: quitting, closing the window, and a file dropped on it. A dropped file takes the same road as a file chosen in the dialog.

⟨Handle an SDL event 268⟩ ≡

```
void App::processEvent(const SDL_Event& event)
{
    ImGui_ImplSDL3_ProcessEvent(&event);
    switch (event.type)
    {
    case SDL_EVENT_QUIT:
        m_running = false;
        break;
    case SDL_EVENT_WINDOW_CLOSE_REQUESTED:
        if (event.window.windowID ≡ SDL_GetWindowID(m_window))
            m_running = false;
        break;
    case SDL_EVENT_DROP_FILE:
        if (event.drop.data)
            requestOpen(event.drop.data);
        break;
    default:
        break;
    }
}
```

This code is used in section 253.

269. The layout. The window is divided into fixed regions. The menu bar sits on top, and ImGui reports the rest of the window as the main viewport's *work area*, whose top edge *WorkPos.y* is just below the menu. A status bar one line high takes the bottom. Between them, the hierarchy takes 22% of the width (but at least 260 units), the inspector 28% (at least 320), and the viewport the rest. Each panel's position and size are set just before it is drawn.

⟨Lay out the panels and handle shortcuts 269⟩ ≡

```
void App::drawUi()
{
    drawMenuBar();
    ⟨Compute the regions of the window 270⟩
    ⟨Place and draw the four panels 271⟩
    drawPopups();
    ⟨Handle the keyboard shortcuts 272⟩
}
```

This code is used in section 253.

270.

⟨Compute the regions of the window 270⟩ ≡

```
const ImGuiViewport* vp = ImGui::GetMainViewport();
const float statusH = ImGui::GetFrameHeight() + ImGui::GetStyle().WindowPadding.y;
const float top = vp→WorkPos.y, bottom = vp→Pos.y + vp→Size.y − statusH;
const float leftW = std::max(260.0f, vp→Size.x * 0.22f), rightW = std::max(320.0f, vp→Size.x * 0.28f);
```

This code is used in section 269.

271.

⟨Place and draw the four panels 271⟩ ≡
```
ImGui::SetNextWindowPos(ImVec2(vp→Pos.x, top));
ImGui::SetNextWindowSize(ImVec2(leftW, bottom − top));
drawHierarchy();

ImGui::SetNextWindowPos(ImVec2(vp→Pos.x + vp→Size.x − rightW, top));
ImGui::SetNextWindowSize(ImVec2(rightW, bottom − top));
drawInspector();

ImGui::SetNextWindowPos(ImVec2(vp→Pos.x + leftW, top));
ImGui::SetNextWindowSize(ImVec2(vp→Size.x − leftW − rightW, bottom − top));
drawViewport();

ImGui::SetNextWindowPos(ImVec2(vp→Pos.x, bottom));
ImGui::SetNextWindowSize(ImVec2(vp→Size.x, statusH));
drawStatusBar();
```
This code is used in section 269.

272. Three keys act on the whole application: F frames the selection (or the whole stage when nothing is selected), Esc clears the selection, and Ctrl+O opens the file dialog. They must not fire while the user is typing in a text field—typing “foo” in the filter of the tree should not move the camera—so ImGui’s *WantTextInput* guards them.

⟨Handle the keyboard shortcuts 272⟩ ≡
```
if (¬ImGui::GetIO().WantTextInput)
{
    if (ImGui::IsKeyPressed(ImGuiKey_F))
        frameSelection();
    if (ImGui::IsKeyPressed(ImGuiKey_Escape))
        select(SdfPath(), false);
    if (ImGui::IsKeyDown(ImGuiMod_Ctrl) ∧ ImGui::IsKeyPressed(ImGuiKey_O))
        openFileDialog();
}
```
This code is used in section 269.

273. Menus. The menu bar has five menus.

⟨Draw the menu bar 273⟩ ≡
```
void App::drawMenuBar()
{
    if (¬ImGui::BeginMainMenuBar())
        return;
    ⟨The File menu 274⟩
    ⟨The View menu 275⟩
    ⟨The Camera menu 276⟩
    ⟨The XR menu 277⟩
    ⟨The Help menu 278⟩
    ImGui::EndMainMenuBar();
}
```
This code is used in section 253.

274. The File menu offers the dialog, a submenu with a text field in which a path can be typed or pasted (useful when no desktop dialog is available), reloading the current file, and quitting. The text field returns true when Enter is pressed, which counts as pressing Load. Reloading reads the file again from disk, which is handy while editing a `.usda` file in another window.

⟨The File menu 274⟩ ≡
```
if (ImGui::BeginMenu("File"))
{
    if (ImGui::MenuItem("Open...", "Ctrl+O"))
        openFileDialog();
    if (ImGui::BeginMenu("Open path"))
    {
        ImGui::SetNextItemWidth(420.0f);
        const bool enter = ImGui::InputText("##path", m_pathInput, sizeof(m_pathInput),
```

```
                                                    ImGuiInputTextFlags_EnterReturnsTrue);
            ImGui::SameLine();
            if ((ImGui::Button("Load") ∨ enter) ∧ m_pathInput[0])
                requestOpen(m_pathInput);
            ImGui::EndMenu();
        }
        if (ImGui::MenuItem("Reload", nullptr, false, m_scene.loaded()))
            requestOpen(m_scene.filePath());
        ImGui::Separator();
        if (ImGui::MenuItem("Quit"))
            m_running = false;
        ImGui::EndMenu();
    }
```

This code is used in section 273.

275. The View menu frames the stage or the selection and controls the image: shadows, texture mipmaps, normal maps, traced reflections, the direction of the light and the field of view. ImGui's check-mark items toggle the **bool** they are given and report the change, and every change sets *m_needsRender*, so that the progressive rendering starts over with the new settings. A USD camera has its own field of view, so the slider is disabled while we look through one.

⟨The View menu 275⟩ ≡

```
    if (ImGui::BeginMenu("View"))
    {
        if (ImGui::MenuItem("Frame all"))
            frameBounds(m_scene.boundsMin(), m_scene.boundsMax());
        if (ImGui::MenuItem("Frame selection", "F", false, ¬m_selected.IsEmpty()))
            frameSelection();
        if (ImGui::MenuItem("Shadows", nullptr, &m_shadows))
            m_needsRender = true;
        if (ImGui::MenuItem("Texture mipmaps", nullptr, &m_mipmaps))
            m_needsRender = true;
        if (ImGui::MenuItem("Normal maps", nullptr, &m_normalMaps))
            m_needsRender = true;
        if (ImGui::MenuItem("Reflections", nullptr, &m_reflections))
            m_needsRender = true;
        if (ImGui::SliderFloat3("Light direction", m_light, -1.0f, 1.0f))
            m_needsRender = true;
        ImGui::BeginDisabled(m_camera ≥ 0);   // a USD camera has its own field of view
        if (ImGui::SliderFloat("Field of view", &m_fovY, 15.0f, 100.0f))
            m_needsRender = true;
        ImGui::EndDisabled();
        ImGui::EndMenu();
    }
```

This code is used in section 273.

276. The Camera menu chooses the point of view: the orbit camera, or any of the cameras of the stage, listed by path in stage order. The current one carries a check mark. Choosing the orbit camera while looking through a USD camera keeps the view where it is, as described in ⟨Hand the view of a USD camera to the orbit camera 305⟩.

⟨The Camera menu 276⟩ ≡

```
    if (ImGui::BeginMenu("Camera"))
    {
        if (ImGui::MenuItem("Orbit camera", nullptr, m_camera < 0))
            orbitFromUsdCamera();
        const std::vector<SceneCamera>& cameras = m_scene.cameras();
        if (cameras.empty())
            ImGui::TextDisabled("This stage has no cameras.");
        else
            ImGui::SeparatorText("USD cameras");
        for (size_t i = 0; i < cameras.size(); ++i)
            if (ImGui::MenuItem(cameras[i].path.c_str(), nullptr, m_camera ≡ int(i)))
```

```
            lookThrough(int(i));
        ImGui::EndMenu();
    }
```

This code is used in section 273.

277. The XR menu reports on the headset output: the OpenXR runtime, whether a CloudXR client is connected when the runtime runs in this process, together with the addresses and port that the client on the headset must be given, the state of the session, and the headset, GPU and eye resolution once they are known. Its settings apply while the session runs: the world scale (how many meters one meter of head motion covers, relative to the stage's units), the samples per pixel and frame, and the mirror of the left eye in the viewport.

```
⟨The XR menu 277⟩ ≡
    if (ImGui::BeginMenu("XR"))
    {
        if (m_xr.state() ≡ XrOutput::State::Off)
            ImGui::TextDisabled("XR output is off. Start with --xr or --cloudxr.");
        else
        {
            ImGui::Text("Runtime: %s", m_xr.runtimeName().c_str());
            if (m_cloudXr.running())
            {
                ImGui::Text("CloudXR %s in this process, client %s", m_cloudXr.version().c_str(),
                            m_xrClient ? "connected" : "not connected");
                // what to type into the CloudXR client on the Vision Pro
                ImGui::SeparatorText("Connect a client to");
                if (m_cloudXr.addresses().empty())
                    ImGui::TextDisabled("No network interface with an IPv4 address");
                for (const std::string& address : m_cloudXr.addresses())
                    ImGui::Text("%s", address.c_str());
                if (m_cloudXr.serverPort())
                    ImGui::Text("Port: %lld", static_cast<long long>(m_cloudXr.serverPort()));
                else
                    ImGui::TextDisabled("Port: unknown, see the service's messages on the console");
            }
            ImGui::Text("State: %s", xrStatus().c_str());
            if (m_xr.eyeCount())
                ImGui::Text("%s on %s, %u eyes of %u x %u", m_xr.systemName().c_str(), m_xr.gpuName().c_str(),
                            m_xr.eyeCount(), m_xr.eyeWidth(), m_xr.eyeHeight());
        }
        ImGui::SeparatorText("Headset view");
        ImGui::SliderFloat("World scale", &m_xrWorldScale, 0.01f, 100.0f, "%.2f", ImGuiSliderFlags_Logarithmic);
        ImGui::SliderInt("Samples per frame", &m_xrSamples, 1, 8);
        ImGui::MenuItem("Mirror left eye in viewport", nullptr, &m_xrMirror);
        ImGui::EndMenu();
    }
```

This code is used in section 273.

278. The About box cannot be opened from inside the menu, because ImGui's popups are identified by the stack of windows that opens them; the menu only raises a flag, and ⟨Draw the popups 323⟩ opens the box at the top level.

```
⟨The Help menu 278⟩ ≡
    if (ImGui::BeginMenu("Help"))
    {
        if (ImGui::MenuItem("About"))
            m_showAbout = true;
        ImGui::EndMenu();
    }
```

This code is used in section 273.

279. The hierarchy. The left panel shows the prims of the stage as a tree. A USD stage already is a scene

graph: every prim knows its name, its type, its parent and its children, so the tree is read straight from the stage, with no copy of the hierarchy to build and keep in step. The tree is drawn by a recursive function, starting from the children of the pseudo-root, the invisible prim at the top of every USD stage.

Children are listed with the predicate *UsdTraverseInstanceProxies*. The prims below an instance live in a shared prototype and are not children of the instance in the ordinary sense; the predicate presents them as *instance proxies*, prims with paths of their own under the instance. That is how the scene sees them too, since it draws the meshes of instances, so a mesh picked in the image can always be found in the tree.

Above the tree, a text field filters the prims by name. The filter is recomputed only when its text changes (⟨Recompute the hierarchy filter 280⟩), not in every frame.

⟨Draw the hierarchy panel 279⟩ ≡

```
void App::drawHierarchy()
{
    ImGui::Begin("Hierarchy", nullptr, kPanelFlags);
    if (¬m_scene.loaded())
    {
        ImGui::TextWrapped("No stage loaded.\n\nUse File > Open, drop a .usd/.usda/.usdc/.usdz file
                on the "
                        "window, or pass a file on the command line.");
        ImGui::End();
        return;
    }
    ImGui::SetNextItemWidth(−FLT_MIN);
    ImGui::InputTextWithHint("##filter", "Filter by name", m_filter, sizeof(m_filter));
    updateFilter();
    ImGui::Separator();
    ImGui::BeginChild("tree");
    for (const UsdPrim& child : m_
            scene.stage()→GetPseudoRoot().GetFilteredChildren(UsdTraverseInstanceProxies()))
        drawHierarchyNode(child);
    m_revealSelection = false;
    ImGui::EndChild();
    ImGui::End();
}
```

This code is used in section 253.

280. A filtered tree must show not only the prims whose names match, but also every ancestor of each match; otherwise the matches would have no branch to hang from. Unlike the drawing of the tree, the filter must look at every prim, open or not, so it walks the whole stage with a *UsdPrimRange*, again including instance proxies. For each matching prim we walk up its path, inserting each prefix in the set; a path's parent is just the path with its last element removed, so no lookup on the stage is needed. The walk stops at the absolute root path, which stands for the pseudo-root and is never drawn, or as soon as an insertion fails: a path already in the set has had its own ancestors inserted before, by the same walk, so there is no point in climbing further. The total work is therefore proportional to the number of prims, however many of them match.

⟨Recompute the hierarchy filter 280⟩ ≡

```
void App::updateFilter()
{
    if (m_appliedFilter ≡ m_filter)
        return;
    m_appliedFilter = m_filter;
    m_visibleByFilter.clear();
    if (m_appliedFilter.empty())
        return;
    for (const UsdPrim& prim : UsdPrimRange::Stage(m_scene.stage(), UsdTraverseInstanceProxies()))
    {
        if (¬containsCaseInsensitive(prim.GetName().GetString(), m_appliedFilter))
            continue;
        for (SdfPath a = prim.GetPath(); a ≠ SdfPath::AbsoluteRootPath() ∧ m_visibleByFilter.insert(a).second;)
            a = a.GetParentPath();
    }
}
```

This code is used in section 253.

281. Names are compared without regard to case, which is what one expects of a search field. The standard library has no such function, but *std*::*search* with a predicate that compares lowered characters does the job. (The cast to **unsigned char** keeps *std*::*tolower* defined for bytes above 127.)

```
⟨Search a name ignoring case 281⟩ ≡
    bool containsCaseInsensitive(const std::string& text, const std::string& pattern)
    {
        return std::search(text.begin(), text.end(), pattern.begin(), pattern.end(), [](char a, char b) {
                    return std::tolower(static_cast<unsigned char>(a)) ≡ std::tolower(static_cast<unsigned
                        char>(b));
                }) ≠ text.end();
    }
```

This code is used in section 253.

282. Drawing a node. A node is skipped if a filter is active and its path is not in the filter's set. Otherwise we ask the stage for the prim's children; the answer is a lazy range, so finding out whether there are any costs one step. Then we compute the flags of ImGui's tree node: it opens only with its arrow or a double click (so that a single click on the label selects), it spans the width of the panel, it is a leaf if it has no children, and it is highlighted if it is the selection.

The recursion descends only into open nodes. A large stage may have hundreds of thousands of prims, but a person looks at a few dozen rows of the tree at a time; a closed node costs the same whatever lies beneath it, so drawing the tree takes time in proportion to what is shown, not to the size of the stage.

```
⟨Draw one node of the hierarchy 282⟩ ≡
    // The tree is drawn straight from the stage: USD already holds the hierarchy,
    // names and types. Only open nodes visit their children.
    void App::drawHierarchyNode(const UsdPrim& prim)
    {
        const SdfPath& path = prim.GetPath();
        if (¬m_appliedFilter.empty() ∧ ¬m_visibleByFilter.contains(path))
            return;
        const UsdPrimSiblingRange children = prim.GetFilteredChildren(UsdTraverseInstanceProxies());
        const bool leaf = children.empty();

        ⟨Choose the flags of the tree node 283⟩

        ⟨Draw the tree node and react to clicks 284⟩

        if (open ∧ ¬leaf)
        {
            for (const UsdPrim& child : children)
                drawHierarchyNode(child);
            ImGui::TreePop();
        }
    }
```

This code is used in section 253.

283. Leaves get *ImGuiTreeNodeFlags_NoTreePushOnOpen*, so that they need no matching *TreePop*; that is why the recursion above pops only nodes with children.

Two situations open a node from the program rather than by the user's hand. When the selection must be revealed, every ancestor of the selected prim is forced open, once. When a filter is active, every visible node is opened when it first appears, so that all matches can be seen; the condition *ImGuiCond_Appearing* still lets the user fold a branch afterwards.

```
⟨Choose the flags of the tree node 283⟩ ≡
    ImGuiTreeNodeFlags flags = ImGuiTreeNodeFlags_OpenOnArrow | ImGuiTreeNodeFlags_OpenOnDoubleClick |
                               ImGuiTreeNodeFlags_SpanAvailWidth;
    if (leaf)
        flags |= ImGuiTreeNodeFlags_Leaf | ImGuiTreeNodeFlags_NoTreePushOnOpen;
    if (path ≡ m_selected)
        flags |= ImGuiTreeNodeFlags_Selected;
    if (m_revealSelection ∧ m_reveal.contains(path))
        ImGui::SetNextItemOpen(true);
```

```
if (¬m_appliedFilter.empty())
    ImGui::SetNextItemOpen(true, ImGuiCond_Appearing);
```

This code is used in section 282.

284. Prims that produced triangles for the renderer, those for which the scene has a mesh, are drawn in light blue, so that the geometry stands out among transforms, scopes and materials. ImGui identifies tree nodes by an ID; prim names are not unique (many prims are called "Mesh"), but paths are, so we pass the path's text as the ID and the name only as the label.

A click on the label selects the prim, while a click on the arrow merely opens or closes the node; the function *IsItemToggledOpen* tells the two apart. When the selection is being revealed, the function *SetScrollHereY* scrolls the panel to the selected node. The type of the prim follows its name, in grey.

```
⟨Draw the tree node and react to clicks 284⟩ ≡
const bool hasMesh = m_scene.meshIndex(path) ≥ 0;
if (hasMesh)
    ImGui::PushStyleColor(ImGuiCol_Text, ImVec4(0.6f, 0.85f, 1.0f, 1.0f));
const bool open = ImGui::TreeNodeEx(path.GetText(), flags, "%s", prim.GetName().GetText());
if (hasMesh)
    ImGui::PopStyleColor();
if (ImGui::IsItemClicked() ∧ ¬ImGui::IsItemToggledOpen())
    select(path, false);
if (m_revealSelection ∧ path ≡ m_selected)
    ImGui::SetScrollHereY();
if (¬prim.GetTypeName().IsEmpty())
{
    ImGui::SameLine();
    ImGui::TextDisabled("%s", prim.GetTypeName().GetText());
}
```

This code is used in section 282.

285. Selecting a prim informs the renderer, which tints the selected mesh if the prim has one, and restarts the rendering. The renderer knows meshes by their mesh id, which is the mesh's index in the scene, and *meshIndex* finds it for a path. A selection made in the tree needs no revealing; one made by clicking on the image does, so in that case we collect the paths of the prim's ancestors, up to but excluding the absolute root, and raise *m_revealSelection*. The flag is lowered again at the end of the next drawing of the tree (⟨Draw the hierarchy panel 279⟩), so the forced opening and scrolling happen exactly once, and the user is free to fold the branches again.

```
⟨Select a prim 285⟩ ≡
void App::select(const SdfPath& path, bool reveal)
{
    m_selected = path;
    const int mesh = m_scene.meshIndex(path);
    m_renderer.setSelectedMesh(mesh ≥ 0 ? unsigned(mesh) : kNoMesh);
    m_needsRender = true;
    m_reveal.clear();
    if (reveal ∧ ¬path.IsEmpty())
    {
        for (SdfPath a = path.GetParentPath(); a ≠ SdfPath::AbsoluteRootPath() ∧ ¬a.IsEmpty(); a =
                a.GetParentPath())
            m_reveal.insert(a);
        m_revealSelection = true;
    }
}
```

This code is used in section 253.

286. The inspector. The right panel describes the selected prim; when nothing is selected, it describes the stage. For a prim it has three or four sections: the prim's identity, what the viewer made of it—a camera, or the geometry and material that the renderer draws—if anything, and a table of all its properties. The prim comes from the stage by its path; an empty path asks for no prim at all.

```
⟨Draw the inspector 286⟩ ≡
    void App::drawInspector()
    {
        ImGui::Begin("Inspector", nullptr, kPanelFlags);
        const UsdPrim prim = m_selected.IsEmpty() ? UsdPrim() : m_scene.stage()→GetPrimAtPath(m_selected);
        if (¬prim)
        {
            ⟨Describe the stage when nothing is selected 287⟩
            ImGui::End();
            return;
        }
        ⟨Describe the identity of the prim 288⟩
        ⟨Describe the camera and offer to look through it 289⟩
        ⟨Describe the rendered geometry and its material 290⟩
        ⟨List the properties of the prim 294⟩
        ImGui::End();
    }
```

This code is used in section 253.

287. The stage summary gives the file, the stage's up axis and units, its time range if it is animated, its default prim, and how many materials, textures and cameras the scene found. Any warnings the scene collected while reading materials and textures (an unsupported shader, a missing file, a translucent material drawn opaque) are listed in orange; they explain why an image may differ from what another renderer would show.

```
⟨Describe the stage when nothing is selected 287⟩ ≡
    if (m_scene.loaded())
    {
        const UsdStageRefPtr& stage = m_scene.stage();
        ImGui::SeparatorText("Stage");
        ImGui::TextWrapped("%s", m_scene.filePath().c_str());
        ImGui::Text("Up axis: %s", m_scene.zUp() ? "Z" : "Y");
        ImGui::Text("Meters per unit: %g", UsdGeomGetStageMetersPerUnit(stage));
        if (stage→HasAuthoredTimeCodeRange())
            ImGui::Text("Time codes: %g - %g", stage→GetStartTimeCode(), stage→GetEndTimeCode());
        if (SdfPrimSpecHandle def = stage→GetRootLayer()→GetPrimAtPath(stage→GetDefaultPrim().GetPath()))
            ImGui::Text("Default prim: %s", def→GetName().c_str());
        ImGui::Text("Materials: %zu   Textures: %zu   Cameras: %zu", m_scene.materials().size(),
                    m_scene.textures().size(), m_scene.cameras().size());
        if (¬m_scene.warnings().empty())
        {
            ImGui::SeparatorText("Warnings");
            for (const std::string& w : m_scene.warnings())
                ImGui::TextColored(ImVec4(1.0f, 0.75f, 0.3f, 1.0f), "%s", w.c_str());
        }
        ImGui::Spacing();
        ImGui::TextDisabled("Select a prim in the hierarchy or click on the viewport.");
    }
```

This code is used in section 286.

288. The identity of a prim is shown as a two-column table of key and value, filled by a small local function *row*. Some rows appear only when they say something: the *kind* of a model, instancing, references, applied API schemas. The specifier (def, over or class) is printed by OpenUSD as, for instance, "SdfSpecifierDef"; we strip the prefix.

```
⟨Describe the identity of the prim 288⟩ ≡
    ImGui::SeparatorText("Prim");
    ImGui::TextWrapped("%s", prim.GetPath().GetText());
    if (ImGui::BeginTable("prim", 2, ImGuiTableFlags_SizingStretchProp))
    {
        auto row = [](const char* key, const std::string& value) {
            ImGui::TableNextRow();
```

```
        ImGui::TableNextColumn();
        ImGui::TextDisabled("%s", key);
        ImGui::TableNextColumn();
        ImGui::TextWrapped("%s", value.c_str());
    };
    row("Type", prim.GetTypeName().IsEmpty() ? "(typeless)" : prim.GetTypeName().GetString());
    row("Specifier", TfStringReplace(TfStringify(prim.GetSpecifier()), "SdfSpecifier", ""));
    TfToken kind;
    if (UsdModelAPI(prim).GetKind(&kind) ∧ ¬kind.IsEmpty())
        row("Kind", kind.GetString());
    row("Active", prim.IsActive() ? "yes" : "no");
    if (prim.IsInstance() ∨ prim.IsInstanceProxy())
        row("Instancing", prim.IsInstance() ? "instance" : "instance proxy");
    if (prim.HasAuthoredReferences())
        row("Composition", "has references");
    const std::vector<std::string> schemas = [&] {
        std::vector<std::string> out;
        for (const TfToken& t : prim.GetAppliedSchemas())
            out.push_back(t.GetString());
        return out;
    }();
    if (¬schemas.empty())
        row("API schemas", TfStringJoin(schemas, ", "));
    ImGui::EndTable();
}
```

This code is used in section 286.

289. A camera prim shows what the viewer understood of it: the projection, the vertical field of view or the size of an orthographic view, and the clipping range. The raw lens attributes are in the property table below. The button looks through the camera; it is disabled, and says so, when we already do. Whether the prim is a camera the scene understood is answered by *cameraIndex*, which gives its index among the scene's cameras, or -1.

⟨Describe the camera and offer to look through it 289⟩ ≡

```
if (const int camera = m_scene.cameraIndex(m_selected); camera ≥ 0)
{
    const SceneCamera& c = m_scene.cameras()[camera];
    ImGui::SeparatorText("Camera");
    if (c.orthographic)
        ImGui::Text("Orthographic, %g x %g units", 2.0f * c.halfWidth, 2.0f * c.halfHeight);
    else
        ImGui::Text("Perspective, vertical field of view %.1f deg", c.fovYDeg);
    ImGui::Text("Clipping range: %g - %g", c.clipNear, c.clipFar);
    const bool active = m_camera ≡ camera;
    ImGui::BeginDisabled(active);
    if (ImGui::Button(active ? "Looking through this camera" : "Look through"))
        lookThrough(camera);
    ImGui::EndDisabled();
}
```

This code is used in section 286.

290. The next section exists only for prims that the scene turned into triangles, that is, prims for which *meshIndex* finds a *RenderMesh*. It shows what the *renderer* sees, after the scene has resolved the bound material, which is often more useful than the raw attributes when an image looks wrong: the size of the mesh, the material's path, and each of its inputs in turn. A button frames the prim.

The size is given in triangles and vertices, and with the number of triangle corners the vertices came from. The scene starts with one vertex per corner, since a corner may have a normal and texture coordinates of its own, and then welds corners that agree in everything; the two numbers show how much sharing the mesh allowed. A smooth, seamless mesh comes down to about one vertex per two triangles, while a mesh with flat shading or many texture seams keeps closer to three.

⟨Describe the rendered geometry and its material 290⟩ ≡

```
if (const int meshIndex = m_scene.meshIndex(m_selected); meshIndex ≥ 0)
```

```
{
    const RenderMesh* mesh = &m_scene.meshes()[meshIndex];
    ImGui::SeparatorText("Rendered geometry");
    ImGui::Text("Triangles: %zu   Vertices: %zu (from %zu corners)", mesh→triangleCount(),
            mesh→vertexCount(),
                mesh→corners);
    const std::vector<Texture>& textures = m_scene.textures();
    const Material& material = m_scene.materials()[mesh→material];
    ImGui::TextWrapped("Material: %s", material.name.c_str());
    ⟨Describe the diffuse color or texture 291⟩
    ⟨Describe the normal map 292⟩
    ⟨Describe roughness, metallic or specular color, and ior 293⟩
    if (mesh→subsetMaterials)
        ImGui::Text("GeomSubsets with their own material: %u", mesh→subsetMaterials);
    if (ImGui::Button("Frame (F)"))
        frameSelection();
}
```

This code is used in section 286.

291. A diffuse texture is described by its resolved path (which may point inside a `.usdz` package), its size, its color space and the primvar that supplies its texture coordinates. If the prim lacks that primvar the texture cannot be applied, and we say so. Without a texture, the diffuse color is shown as a small swatch; the *ColorEdit3* widget is used only for display, with its inputs and picker disabled.

⟨Describe the diffuse color or texture 291⟩ ≡

```
if (material.texture ≥ 0)
{
    const Texture& t = textures[material.texture];
    ImGui::TextWrapped("Diffuse texture: %s", t.path.c_str());
    ImGui::Text("  %d x %d, %s, uv set '%s'%s", t.width, t.height, t.srgb ? "sRGB" : "raw",
                material.uvSet.c_str(), mesh→uvs.empty() ? " (missing!)" : "");
}
else
{
    float color[3] = {material.color[0], material.color[1], material.color[2]};
    ImGui::ColorEdit3("Diffuse color", color, ImGuiColorEditFlags_NoInputs | ImGuiColorEditFlags_NoPicker);
}
```

This code is used in section 290.

292. For a normal map, the scale and bias matter as much as the image: they decide how the stored colors become tangent-space vectors, and an asset that flips the green channel (as many DirectX-style maps do) does it through them.

⟨Describe the normal map 292⟩ ≡

```
if (material.normalTexture ≥ 0)
{
    const Texture& t = textures[material.normalTexture];
    ImGui::TextWrapped("Normal map: %s", t.path.c_str());
    ImGui::Text("  %d x %d, %s, scale (%g, %g, %g), bias (%g, %g, %g)", t.width, t.height,
                t.srgb ? "sRGB" : "raw", material.normalScale[0], material.normalScale[1],
                material.normalScale[2], material.normalBias[0], material.normalBias[1],
                material.normalBias[2]);
}
```

This code is used in section 290.

293. Roughness and metallic are scalar inputs that may be constants or one channel of a texture; the local function *scalarText* prints either form, naming the channel by indexing the string `"rgba"`. Materials in the specular workflow have a specular color instead of a metallic value, so we show one or the other, followed by the index of refraction.

⟨Describe roughness, metallic or specular color, and ior 293⟩ ≡

```
auto scalarText = [&](const char* label, const ScalarInput& in) {
    if (in.texture ≥ 0)
```

```
            ImGui::TextWrapped("%s: texture channel %c of %s (x%g %+g)", label, "rgba"[in.channel],
                               textures[in.texture].path.c_str(), in.scale, in.bias);
        else
            ImGui::Text("%s: %g", label, in.value);
    };
    scalarText("Roughness", material.roughness);
    if (material.specularWorkflow)
    {
        if (material.specularTexture ≥ 0)
            ImGui::TextWrapped("Specular color: %s", textures[material.specularTexture].path.c_str());
        else
        {
            float color[3] = {material.specularColor[0], material.specularColor[1], material.specularColor[2]};
            ImGui::ColorEdit3("Specular color", color,
                              ImGuiColorEditFlags_NoInputs | ImGuiColorEditFlags_NoPicker);
        }
    }
    else
        scalarText("Metallic", material.metallic);
    ImGui::Text("IOR: %g", material.ior);
```

This code is used in section 290.

294. The last section lists every property of the prim, attributes first and then relationships, in a table of three columns: name, type and value. The table takes the rest of the panel's height and scrolls by itself, keeping its header row in view (*TableSetupScrollFreeze*).

```
⟨List the properties of the prim 294⟩ ≡
    ImGui::SeparatorText("Properties");
    if (ImGui::BeginTable("props", 3,
                          ImGuiTableFlags_RowBg | ImGuiTableFlags_BordersInnerV | ImGuiTableFlags_Resizable |
                              ImGuiTableFlags_ScrollY,
                          ImVec2(0, ImGui::GetContentRegionAvail().y)))
    {
        ImGui::TableSetupScrollFreeze(0, 1);
        ImGui::TableSetupColumn("Name", ImGuiTableColumnFlags_WidthStretch, 0.35f);
        ImGui::TableSetupColumn("Type", ImGuiTableColumnFlags_WidthStretch, 0.2f);
        ImGui::TableSetupColumn("Value", ImGuiTableColumnFlags_WidthStretch, 0.45f);
        ImGui::TableHeadersRow();
        ⟨List the attributes of the prim 295⟩
        ⟨List the relationships of the prim 297⟩
        ImGui::EndTable();
    }
```

This code is used in section 286.

295. An attribute's value is read at the default time. An attribute that has only time samples has no default value, so for it we show the value at the earliest sample, and note how many samples there are. The names of attributes whose value was authored in the file are drawn normally; those that merely report a fallback value defined by their schema are greyed out.

```
⟨List the attributes of the prim 295⟩ ≡
    for (const UsdAttribute& attr : prim.GetAttributes())
    {
        VtValue value;
        attr.Get(&value, UsdTimeCode::Default());
        if (value.IsEmpty() ∧ attr.GetNumTimeSamples() > 0)
            attr.Get(&value, UsdTimeCode::EarliestTime());
        ImGui::TableNextRow();
        ImGui::TableNextColumn();
        if (attr.HasAuthoredValue())
            ImGui::TextUnformatted(attr.GetName().GetText());
        else
            ImGui::TextDisabled("%s", attr.GetName().GetText());
        ImGui::TableNextColumn();
```

```
        ImGui::TextDisabled("%s", attr.GetTypeName().GetAsToken().GetText());
        ImGui::TableNextColumn();
        ImGui::TextWrapped("%s", valueString(value).c_str());
        if (attr.GetNumTimeSamples() > 0)
        {
            ImGui::SameLine();
            ImGui::TextDisabled("(%zu samples)", attr.GetNumTimeSamples());
        }
    }
```

This code is used in section 294.

296. Any USD value can be turned into text by *TfStringify*, but some values are long: the points of a mesh may run to megabytes. We cut the text at 160 characters, which is enough to recognize a value and keeps the table responsive.

⟨Format an attribute value for the inspector 296⟩ ≡

```
    std::string valueString(const VtValue& value, size_t maxLength = 160)
    {
        std::string s = value.IsEmpty() ? std::string("<no value>") : TfStringify(value);
        if (s.size() > maxLength)
            s = s.substr(0, maxLength) + " ...";
        return s;
    }
```

This code is used in section 253.

297. A relationship has no value but a list of target paths, such as the material bound to a prim.

⟨List the relationships of the prim 297⟩ ≡

```
    for (const UsdRelationship& rel : prim.GetRelationships())
    {
        SdfPathVector targets;
        rel.GetTargets(&targets);
        std::vector<std::string> names;
        for (const SdfPath& p : targets)
            names.push_back(p.GetString());
        ImGui::TableNextRow();
        ImGui::TableNextColumn();
        ImGui::TextUnformatted(rel.GetName().GetText());
        ImGui::TableNextColumn();
        ImGui::TextDisabled("rel");
        ImGui::TableNextColumn();
        ImGui::TextWrapped("%s", names.empty() ? "<none>" : TfStringJoin(names, ", ").c_str());
    }
```

This code is used in section 294.

298. The camera. The orbit camera is described by a target point $\mathbf{t}$, a distance d, a yaw angle θ and a pitch angle φ. The eye is at $\mathbf{e} = \mathbf{t} + d\,\mathbf{o}$, where the unit vector $\mathbf{o}$ points from the target towards the eye. USD stages declare which axis is "up", Y or Z, and the camera must follow suit, or a Z-up scene would lie on its side.

For a Y-up stage, θ turns around Y starting from the $+Z$ axis, and φ lifts the eye above the XZ plane:

$$\mathbf{o} = (\cos\varphi \sin\theta,\ \sin\varphi,\ \cos\varphi\cos\theta).$$

For a Z-up stage, θ turns around Z starting from the $+X$ axis, and φ lifts the eye above the XY plane:

$$\mathbf{o} = (\cos\varphi\cos\theta,\ \cos\varphi\sin\theta,\ \sin\varphi).$$

In both cases the up vector given to the renderer is simply the world's up axis; the renderer builds an orthonormal frame from it and the viewing direction.

A USD camera, when one is chosen, needs no such construction: the scene has already computed its eye, its directions and its window, and we only fit the window to the viewport. Either way, this is also the place where all the settings of the View menu reach the renderer, because it is called whenever *m_needsRender* is set.

⟨Send the camera and the options to the renderer 298⟩ ≡

```
void App::updateCamera()
{
    if (m_camera ≥ 0)
    {
        ⟨Send the USD camera to the renderer 299⟩
    }
    else
    {
        ⟨Send the orbit camera to the renderer 300⟩
    }
    m_renderer.setLightDir(m_light);
    m_renderer.setShadows(m_shadows);
    m_renderer.setMipmaps(m_mipmaps);
    m_renderer.setNormalMaps(m_normalMaps);
    m_renderer.setReflections(m_reflections);
}
```

This code is used in section 253.

299.

⟨Send the USD camera to the renderer 299⟩ ≡

```
const SceneCamera& c = m_scene.cameras()[m_camera];
float halfW, halfH;
fittedWindow(c, halfW, halfH);
m_renderer.setCameraView(c.eye.data(), c.forward.data(), c.up.data(), halfW, halfH, c.orthographic,
                         c.clipNear, c.clipFar);
```

This code is used in section 298.

300.

⟨Send the orbit camera to the renderer 300⟩ ≡

```
float offset[3], up[3], eye[3];
orbitOffset(offset, up);
for (int k = 0; k < 3; ++k)
    eye[k] = m_target[k] + offset[k] * m_distance;
m_renderer.setCamera(eye, m_target, up, m_fovY);
```

This code is used in section 298.

301. The formulas for **o** and the up axis live in a function of their own, because the XR output also needs the desktop view's eye and direction.

⟨Compute the orbit offset 301⟩ ≡

```
// Unit vector from the orbit target to the eye, and the world's up axis.
void App::orbitOffset(float offset[3], float up[3]) const
{
    const float cp = std::cos(m_pitch), sp = std::sin(m_pitch), cy = std::cos(m_yaw), sy = std::sin(m_yaw);
    if (m_scene.zUp())
    {
        offset[0] = cp * cy, offset[1] = cp * sy, offset[2] = sp;
        up[0] = 0, up[1] = 0, up[2] = 1;
    }
    else
    {
        offset[0] = cp * sy, offset[1] = sp, offset[2] = cp * cy;
        up[0] = 0, up[1] = 1, up[2] = 0;
    }
}
```

This code is used in section 253.

302. The XR output places the headset where the desktop view's eye is, looking the same way. For a USD camera these are its own eye and forward direction; for the orbit camera, the eye is $\mathbf{t} + d\,\mathbf{o}$ and the direction $-\mathbf{o}$.

⟨Find the eye and direction of the desktop view 302⟩ ≡

```
// Eye and viewing direction of the desktop view, orbit or USD camera.
void App::viewFrame(float eye[3], float forward[3]) const
{
    if (m_camera ≥ 0)
    {
        const SceneCamera& c = m_scene.cameras()[m_camera];
        std::copy(c.eye.begin(), c.eye.end(), eye);
        std::copy(c.forward.begin(), c.forward.end(), forward);
        return;
    }
    float offset[3], up[3];
    orbitOffset(offset, up);
    for (int k = 0; k < 3; ++k)
    {
        eye[k] = m_target[k] + offset[k] * m_distance;
        forward[k] = −offset[k];
    }
}
```

This code is used in section 253.

303. A camera's film back rarely has the shape of the viewport. Like usdview's default "fit" policy, we grow the camera's window along one axis until it has the viewport's aspect ratio a: if the viewport is relatively wider than the window, the window's height is kept and its width becomes a times the height; otherwise the width is kept. The whole film back thus stays visible, with extra scene around it, and nothing the camera's author framed is cut off.

⟨Fit a USD camera's window to the viewport 303⟩ ≡

```
// The camera's window grown along one axis to the viewport's aspect ratio, so
// the whole window stays visible (the "fit" conform policy of usdview).
void App::fittedWindow(const SceneCamera& camera, float& halfWidth, float& halfHeight) const
{
    const float aspect = float(m_renderer.width()) / float(std::max(m_renderer.height(), 1u));
    halfWidth = camera.halfWidth;
    halfHeight = camera.halfHeight;
    if (aspect > halfWidth / halfHeight)
        halfWidth = halfHeight * aspect;
    else
        halfHeight = halfWidth / aspect;
}
```

This code is used in section 253.

304. Looking through a camera is only a matter of remembering which one; the next frame sends it to the renderer.

⟨Look through a USD camera 304⟩ ≡

```
void App::lookThrough(int camera)
{
    m_camera = camera;
    m_needsRender = true;
}
```

This code is used in section 253.

305. A USD camera is fixed by its file, but the user will still want to look around from it. Any attempt to move the view therefore hands it over to the orbit camera, which should start exactly where the USD camera was, so that the image does not jump. The orbit camera needs a target in front of the eye, and the best one is what the user is looking at: the surface at the centre of the image, which the renderer recorded on frame 0. Its depth along the line of sight is $\delta = (\mathbf{p} - \mathbf{e}) \cdot \mathbf{f}$, where $\mathbf{f}$ is the viewing direction. The point is only trusted while no change is pending, because the image then still shows this camera. Without it (the centre shows the sky, or the mirror replaced the image) we use the depth of the scene's centre $\mathbf{c}$ instead. In a large scene, a city for instance, that centre is often behind a camera standing inside it; the target is then put 2% of the scene's diagonal ahead. Half the scene, a natural first choice, would put it kilometres away, and every drag

would swing the eye across the city instead of turning the view. The orbit offset **o** points from the target to the eye, so it is $-\mathbf{f}$. The field of view is that of the fitted window, $2\arctan h$.

Two things cannot be carried over. The orbit camera has no roll, so a tilted camera straightens up. And it is always a pinhole: an orthographic view becomes a perspective one whose distance makes the visible height at the target the same, $d = h/\tan(\phi/2)$.

⟨Hand the view of a USD camera to the orbit camera 305⟩ ≡

```
// Hands the view over to the orbit camera, which starts where the USD camera
// is, looking the same way. It orbits the surface at the center of the view,
// or, when the image shows none, a point at the depth of the scene's center.
// Roll is lost, and an orthographic camera becomes a perspective one that
// shows the same size at that point.
void App::orbitFromUsdCamera()
{
    if (m_camera < 0)
        return;
    const SceneCamera& c = m_scene.cameras()[m_camera];
    float halfW, halfH;
    fittedWindow(c, halfW, halfH);
    // the image still shows this camera unless a change is pending
    float hit[3];
    const bool onSurface =
        ¬m_needsRender ∧ m_renderer.hitPointAt(m_renderer.width() / 2, m_renderer.height() / 2, hit);
    m_camera = -1;
    m_needsRender = true;
    const float* lo = m_scene.boundsMin();
    const float* hi = m_scene.boundsMax();
    const float* f = c.forward.data();
    float depth = 0.0f, diagonal = 0.0f;
    for (int k = 0; k < 3; ++k)
    {
        depth += ((onSurface ? hit[k] : 0.5f * (lo[k] + hi[k])) − c.eye[k]) * f[k];
        diagonal += (hi[k] − lo[k]) * (hi[k] − lo[k]);
    }
    // With the scene's center behind the camera (inside a large scene), a
    // pivot that far away would swing the eye across the scene on every drag;
    // a near one turns the view almost in place.
    if (depth < 1e-3f)
        depth = std::max(0.02f * std::sqrt(diagonal), 1e-3f);
    float target[3], offset[3];
    for (int k = 0; k < 3; ++k)
    {
        target[k] = c.eye[k] + f[k] * depth;
        offset[k] = −f[k];
    }
    if (c.orthographic)
        orbitAround(target, offset, halfH / std::tan(0.5f * m_fovY * float(M_PI) / 180.0f));
    else
    {
        m_fovY = 2.0f * std::atan(halfH) * 180.0f / float(M_PI);
        orbitAround(target, offset, depth);
    }
}
```

This code is used in section 253.

306. Placing the orbit camera. Given a target, the unit offset **o** from the target to the eye and a distance, inverting the formulas of ⟨Send the orbit camera to the renderer 300⟩ gives the angles: for a Y-up stage $\varphi = \arcsin o_y$ and $\theta = \operatorname{atan2}(o_x, o_z)$; for a Z-up stage $\varphi = \arcsin o_z$ and $\theta = \operatorname{atan2}(o_y, o_x)$. The pitch is clamped like a drag's, short of the poles.

⟨Orbit a target from a direction and distance 306⟩ ≡

```
// Orbits `target` from `distance` away; `offset` is the unit vector from the
// target to the eye (the inverse of orbitOffset).
```

```
void App::orbitAround(const float target[3], const float offset[3], float distance)
{
    std::copy(target, target + 3, m_target);
    m_distance = distance;
    if (m_scene.zUp())
        m_pitch = std::asin(std::clamp(offset[2], -1.0f, 1.0f)), m_yaw = std::atan2(offset[1], offset[0]);
    else
        m_pitch = std::asin(std::clamp(offset[1], -1.0f, 1.0f)), m_yaw = std::atan2(offset[0], offset[2]);
    m_pitch = std::clamp(m_pitch, -1.55f, 1.55f);
    m_needsRender = true;
}
```

This code is used in section 253.

307. A double-click moves the pivot to the surface under the cursor: the orbit camera's target becomes that point, the eye stays where it is, and the view turns to face the point. This is what makes orbiting useful in a scene whose centre is far from what the user is looking at. A click on the sky does nothing. A USD camera first hands its view over, so the eye is the one on screen.

⟨Orbit the surface point under a pixel 307⟩ ≡

```
// Double-click: the surface point under the cursor becomes the orbit center.
// The eye stays where it is and turns to face the point.
void App::pivotAt(unsigned int x, unsigned int y)
{
    float hit[3];
    if (m_needsRender ∨ ¬m_renderer.hitPointAt(x, y, hit))
        return;
    orbitFromUsdCamera();
    float eye[3], forward[3], offset[3], distance = 0.0f;
    viewFrame(eye, forward);
    for (int k = 0; k < 3; ++k)
    {
        offset[k] = eye[k] − hit[k];
        distance += offset[k] * offset[k];
    }
    distance = std::sqrt(distance);
    if (distance < 1e-6f)
        return;
    for (float& o : offset)
        o /= distance;
    orbitAround(hit, offset, distance);
}
```

This code is used in section 253.

308. Framing. To frame an axis-aligned box, we aim at its center and enclose it in a sphere whose radius r is half the box's diagonal. A sphere of radius r seen from distance d subtends a half-angle α with $\sin\alpha = r/d$: the line of sight that grazes the sphere is tangent to it, and the radius to the point of tangency is perpendicular to that line. The sphere just fits the vertical field of view ϕ when $\alpha = \phi/2$, that is, when

$$d = \frac{r}{\sin(\phi/2)}.$$

We add 15% of margin. The viewport is wider than it is tall, so the horizontal direction then fits as well. A tiny lower bound on r keeps an empty or flat scene from putting the eye on the target. Framing is done with the orbit camera, so a USD camera first hands its view over; the framed view then keeps its direction.

⟨Frame a bounding box 308⟩ ≡

```
void App::frameBounds(const float lo[3], const float hi[3])
{
    orbitFromUsdCamera();
    float radius = 0.0f;
    for (int k = 0; k < 3; ++k)
    {
        m_target[k] = 0.5f * (lo[k] + hi[k]);
        radius += (hi[k] − lo[k]) * (hi[k] − lo[k]);
```

```
    }
    radius = std::max(0.5f * std::sqrt(radius), 1e-3f);
    m_distance = 1.15f * radius / std::sin(0.5f * m_fovY * float(M_PI) / 180.0f);
    m_needsRender = true;
}
```

This code is used in section 253.

309. The scene computes a world-space bounding box for every mesh, and one for the whole stage, when it reads the geometry. Framing the selection therefore needs only the selected prim's mesh, if it has one; without a selected mesh (nothing selected, or a prim such as a transform or a material) we frame the whole stage.

⟨Frame the selection 309⟩ ≡

```
void App::frameSelection()
{
    const int mesh = m_scene.meshIndex(m_selected);
    if (mesh < 0)
        frameBounds(m_scene.boundsMin(), m_scene.boundsMax());
    else
        frameBounds(m_scene.meshes()[mesh].boundsMin, m_scene.meshes()[mesh].boundsMax);
}
```

This code is used in section 253.

310. The viewport. The middle panel shows the ray-traced image and turns mouse actions into camera motion and selection. It is also where rendering happens: the viewport knows its own size, which is the size of the image to render.

⟨Draw the viewport 310⟩ ≡

```
void App::drawViewport()
{
    ⟨Open the borderless viewport window 311⟩
    ⟨Cover the viewport with an invisible button 312⟩
    ⟨Leave a USD camera when the view is moved 313⟩
    ⟨Orbit with the left mouse button 314⟩
    ⟨Pan with the right or middle mouse button 315⟩
    ⟨Zoom with the mouse wheel 316⟩
    ⟨Compute the resolution of the image 317⟩
    ⟨Pick the prim under a mouse click 318⟩
    ⟨Resize the image with the viewport 319⟩
    ⟨Restart or continue the progressive rendering 320⟩
    ⟨Show the image in the viewport 321⟩
    ImGui::End();
}
```

This code is used in section 253.

311. The viewport window has no title bar, no scroll bar and no padding, so that the image fills it to the edges. Without a working renderer, or while the window is too small to hold an image, it only shows a message (or nothing).

⟨Open the borderless viewport window 311⟩ ≡

```
ImGui::PushStyleVar(ImGuiStyleVar_WindowPadding, ImVec2(0, 0));
ImGui::Begin("Viewport", nullptr, kPanelFlags | ImGuiWindowFlags_NoScrollbar | ImGuiWindowFlags_
        NoTitleBar);
ImGui::PopStyleVar();

const ImVec2 size = ImGui::GetContentRegionAvail();
const ImVec2 origin = ImGui::GetCursorScreenPos();
if (¬m_rendererOk ∨ size.x < 4 ∨ size.y < 4)
{
    ImGui::TextWrapped("%s", m_rendererOk ? "" : "OptiX renderer unavailable.");
    ImGui::End();
    return;
```

```
}
```

This code is used in section 310.

312. An image drawn by ImGui does not react to the mouse, and dragging across a window's empty space would move the window. So we cover the whole viewport with an invisible button that accepts all three mouse buttons. While the button is *active*—a mouse button was pressed on it and is still held—the drag belongs to the viewport, even if the mouse leaves it.

```
⟨Cover the viewport with an invisible button 312⟩ ≡
    ImGui::InvisibleButton("viewport", size, ImGuiButtonFlags_MouseButtonLeft | ImGuiButtonFlags_
            MouseButtonRight |
                                                    ImGuiButtonFlags_MouseButtonMiddle);
    const ImGuiIO& io = ImGui::GetIO();
    const bool hovered = ImGui::IsItemHovered(), active = ImGui::IsItemActive();
```

This code is used in section 310.

313. The three mouse controls below move the orbit camera. If we are looking through a USD camera, the first drag or wheel step hands the view over to the orbit camera, before the motion is applied, so the motion continues smoothly from the USD camera's point of view.

```
⟨Leave a USD camera when the view is moved 313⟩ ≡
    // camera controls: left = orbit, right/middle = pan, wheel = zoom; they
    // leave a USD camera for the orbit camera
    const bool dragging = active ∧ (ImGui::IsMouseDragging(ImGuiMouseButton_Left, 2.0f) ∨
                                    ImGui::IsMouseDragging(ImGuiMouseButton_Right, 1.0f) ∨
                                    ImGui::IsMouseDragging(ImGuiMouseButton_Middle, 1.0f));
    if (dragging ∨ (hovered ∧ io.MouseWheel ≠ 0.0f))
        orbitFromUsdCamera();
```

This code is used in section 310.

314. Dragging with the left button orbits: horizontal motion changes the yaw and vertical motion the pitch, by 0.006 radians per unit, about a third of a degree. The pitch is kept within ± 1.55 radians, just short of $\pm\pi/2 \approx \pm 1.5708$: at exactly a right angle the viewing direction would be parallel to the up vector, and the camera frame built from their cross product would collapse. A drag must exceed two units before it counts, so that a click with a slightly unsteady hand still selects.

```
⟨Orbit with the left mouse button 314⟩ ≡
    if (active ∧ ImGui::IsMouseDragging(ImGuiMouseButton_Left, 2.0f))
    {
        m_yaw −= io.MouseDelta.x * 0.006f;
        m_pitch = std::clamp(m_pitch + io.MouseDelta.y * 0.006f, -1.55f, 1.55f);
        m_needsRender = true;
    }
```

This code is used in section 310.

315. Panning moves the target in the plane of the screen, so that the point under the mouse follows the mouse. We need the camera's right and up vectors in world space. With $\mathbf{f} = -\mathbf{o}$ the viewing direction and $\mathbf{u}_w$ the world's up axis, the right vector is $\mathbf{r} \propto \mathbf{f} \times \mathbf{u}_w$ and the camera's up vector is $\mathbf{u} = \mathbf{r} \times \mathbf{f}$. Working out the products with the two formulas for $\mathbf{o}$ gives, for a Y-up stage,

$$\mathbf{r} = (\cos\theta,\ 0,\ -\sin\theta), \qquad \mathbf{u} = (-\sin\varphi\sin\theta,\ \cos\varphi,\ -\sin\varphi\cos\theta),$$

and for a Z-up stage,

$$\mathbf{r} = (-\sin\theta,\ \cos\theta,\ 0), \qquad \mathbf{u} = (-\sin\varphi\cos\theta,\ -\sin\varphi\sin\theta,\ \cos\varphi).$$

How far should the target move for each unit the mouse moves? At distance d, a vertical field of view ϕ spans a height of $2d\tan(\phi/2)$ in the world, and this height fills the H units of the viewport's height; one unit of mouse motion is therefore worth $k = 2d\tan(\phi/2)/H$ in the world at the target's depth. Moving the mouse to the right should drag the scene to the right, which means moving the target to the left, along $-\mathbf{r}$; screen coordinates grow downwards, so moving the mouse down moves the target along $+\mathbf{u}$ and the scene down.

```
⟨Pan with the right or middle mouse button 315⟩ ≡
    if (active ∧ (ImGui::IsMouseDragging(ImGuiMouseButton_Right, 1.0f) ∨
```

```
            ImGui::IsMouseDragging(ImGuiMouseButton_Middle, 1.0f)))
{
    // move the target in the camera plane
    const float cp = std::cos(m_pitch), sp = std::sin(m_pitch), cy = std::cos(m_yaw), sy = std::sin(m_yaw);
    float right[3], up[3];
    if (m_scene.zUp())
        right[0] = -sy, right[1] = cy, right[2] = 0, up[0] = -sp * cy, up[1] = -sp * sy, up[2] = cp;
    else
        right[0] = cy, right[1] = 0, right[2] = -sy, up[0] = -sp * sy, up[1] = cp, up[2] = -sp * cy;
    const float k = m_distance * 2.0f * std::tan(0.5f * m_fovY * float(M_PI) / 180.0f) / size.y;
    for (int i = 0; i < 3; ++i)
        m_target[i] += (io.MouseDelta.y * up[i] - io.MouseDelta.x * right[i]) * k;
    m_needsRender = true;
}
```

This code is used in section 310.

316. The wheel zooms by multiplying the distance by 0.88 per notch forward and dividing by it per notch backward. A geometric step feels the same at every scale, from a whole city down to a screw, and the distance can approach the target but never reach or cross it.

```
⟨Zoom with the mouse wheel 316⟩ ≡
    if (hovered ∧ io.MouseWheel ≠ 0.0f)
    {
        m_distance *= std::pow(0.88f, io.MouseWheel);
        m_needsRender = true;
    }
```

This code is used in section 310.

317. ImGui measures in logical units, but the image should have one ray per physical pixel. The framebuffer scale converts one into the other. While the viewport mirrors the headset's left eye, the desktop image is neither rendered nor picked from: the mirror shows something else.

```
⟨Compute the resolution of the image 317⟩ ≡
    const float fbScale = io.DisplayFramebufferScale.x;
    const unsigned w = unsigned(size.x * fbScale), h = unsigned(size.y * fbScale);
    const bool mirroring = m_xrMirror ∧ m_xr.state() ≡ XrOutput::State::Running ∧ ¬m_mirrorPixels.empty();
```

This code is used in section 310.

318. Picking. Clicking on the image selects the prim under the mouse. The renderer already knows which mesh each pixel shows: on the first sample of every image, which goes through the center of each pixel, it records a mesh identifier per pixel and copies that buffer back to the CPU. Picking is therefore a lookup, with no extra ray to trace.

A click is a release of the left button that did not travel far: ImGui records the largest squared distance the mouse moved while the button was down, and we accept less than 16, that is, less than 4 units. Anything longer was an orbit, not a click. We also require that the renderer's buffer has the viewport's current width, so that pixel coordinates mean the same thing in both. A mesh id is the mesh's index in the scene, and each mesh records the path of the prim it came from, so the id leads straight to the prim to select. The selection is revealed in the tree; a click on the background, whose id is no mesh's index, clears it. A double-click also moves the orbit pivot to the point under the cursor (⟨Orbit the surface point under a pixel 307⟩). The pixel is computed only over the image, where the mouse position cannot be negative.

```
⟨Pick the prim under a mouse click 318⟩ ≡
    // click (left release without dragging): pick; double-click: orbit around the point
    const bool onImage = hovered ∧ m_renderer.width() ≡ w ∧ ¬mirroring;
    const unsigned px = onImage ? unsigned((io.MousePos.x - origin.x) * fbScale) : 0;
    const unsigned py = onImage ? unsigned((io.MousePos.y - origin.y) * fbScale) : 0;
    if (onImage ∧ ImGui::IsMouseReleased(ImGuiMouseButton_Left) ∧ io.MouseDragMaxDistanceSqr[0] < 16.0f)
    {
        const unsigned meshId = m_renderer.meshIdAt(px, py);
        select(meshId < m_scene.meshes().size() ? m_scene.meshes()[meshId].path : SdfPath(), true);
    }
    if (onImage ∧ ImGui::IsMouseDoubleClicked(ImGuiMouseButton_Left))
        pivotAt(px, py);
```

This code is used in section 310.

319. When the viewport changes size, the renderer reallocates its buffers and we replace the SDL texture by one of the new size. The texture is a *streaming* texture, meant to be rewritten often, in the pixel format *SDL_PIXELFORMAT_RGBA32*, whose bytes are red, green, blue and alpha in memory order—exactly the layout of the renderer's *uchar4* pixels.

```
⟨Resize the image with the viewport 319⟩ ≡
    if (w ≠ m_renderer.width() ∨ h ≠ m_renderer.height())
    {
        m_renderer.resize(w, h);
        if (m_texture)
            SDL_DestroyTexture(m_texture);
        m_texture = SDL_CreateTexture(m_sdlRenderer, SDL_PIXELFORMAT_RGBA32, SDL_TEXTUREACCESS_
                STREAMING, int(w), int(h));
        m_needsRender = true;
    }
```

This code is used in section 310.

320. Progressive rendering is driven from here. Any change—camera, selection, settings, a new file, a new size—has set *m_needsRender*. We then pass the new camera and options to the renderer and reset its accumulation, which throws away the samples of the old image. In every frame, as long as fewer than *kMaxSamples* samples have been taken, the renderer adds one more sample to each pixel, and the averaged image is copied into the texture. While the user drags the camera, each frame thus shows a fresh one-sample image; when the user lets go, the image refines itself in a few seconds and then the GPU rests.

A failure of the renderer, reported as an exception, disables it and shows the message in the error popup, rather than ending the program.

```
⟨Restart or continue the progressive rendering 320⟩ ≡
    // any change restarts the progressive accumulation; otherwise keep adding
    // samples until kMaxSamples (glossy reflections and antialiasing converge)
    if (m_needsRender)
    {
        updateCamera();
        m_renderer.resetAccumulation();
        m_needsRender = false;
    }
    if (m_texture ∧ m_rendererOk ∧ m_renderer.samples() < kMaxSamples ∧ ¬mirroring)
    {
        try
        {
            m_renderer.render();
            SDL_UpdateTexture(m_texture, nullptr, m_renderer.pixels().data(), int(w * 4));
        }
        catch (const std::exception& e)
        {
            m_error = std::string("Rendering failed: ") + e.what();
            m_rendererOk = false;
        }
    }
```

This code is used in section 310.

321. Finally the texture is drawn over the invisible button. ImGui's SDL renderer back end interprets a texture identifier as a pointer to an *SDL_Texture*. If the stage has nothing the renderer can draw, a note says so in the corner, so that an empty viewport is not mistaken for a broken one. The mirror of the headset, when on, takes the image's place.

```
⟨Show the image in the viewport 321⟩ ≡
    if (mirroring)
        drawXrMirror(origin.x, origin.y, size.x, size.y);
    else if (m_texture)
        ImGui::GetWindowDrawList()→AddImage(ImTextureRef(ImTextureID(reinterpret_cast<intptr_t>(m_
                texture))), origin,
```

```
                                        ImVec2(origin.x + size.x, origin.y + size.y));
    if (m_scene.loaded() ∧ m_scene.meshes().empty())
        ImGui::GetWindowDrawList()→AddText(ImVec2(origin.x + 12, origin.y + 12), IM_COL32(220, 220, 220, 255),
                                        "This stage has no renderable geometry (Mesh, Cube, Sphere).");
```

This code is used in section 310.

322. The status bar and the popups. The status bar is a one-line window at the bottom. It counts the prims (instance proxies included, the pseudo-root not), the meshes, the triangles and the textures, as the scene counted them when it loaded the stage; it names the camera in use, the OptiX version and the GPU, and shows the progress of the rendering with the time of the last sample, followed by the state of the XR output when there is one. *OPTIX_VERSION* encodes version 9.1.0 as 90100, so we take it apart with divisions.

```
⟨Draw the status bar 322⟩ ≡
    void App::drawStatusBar()
    {
        ImGui::Begin("Status", nullptr, kPanelFlags | ImGuiWindowFlags_NoTitleBar | ImGuiWindowFlags_
                NoScrollbar);
        ImGui::Text("%s  |  prims: %zu  meshes: %zu  triangles: %zu  textures: %zu  |  camera: %s  |  "
                    "OptiX %d.%d.%d on %s  |  samples %u/%u, %.2f ms%s",
                    m_scene.loaded() ? m_scene.filePath().c_str() : "no file",
                    m_scene.primCount(), m_scene.meshes().size(), m_scene.triangleCount(), m_scene.textures().size(),
                    m_camera ≥ 0 ? m_scene.cameras()[m_camera].path.c_str() : "orbit",
                    OPTIX_VERSION / 10000, (OPTIX_VERSION / 100) % 100, OPTIX_VERSION % 100,
                    m_renderer.deviceName().c_str(), m_renderer.samples(), kMaxSamples, m_renderer.lastRenderMs(),
                    m_xr.state() ≡ XrOutput::State::Off ? "" : ("  |  XR: " + xrStatus()).c_str());
        ImGui::End();
    }
```

This code is used in section 253.

323. Two modal popups complete the interface. The error popup opens whenever *m_error* holds a message and stays until the user acknowledges it, which clears the message. The About box is opened here, at the top level of the window stack, when the Help menu has raised *m_showAbout*; it summarizes the tools and the mouse and keyboard controls.

```
⟨Draw the popups 323⟩ ≡
    void App::drawPopups()
    {
        if (¬m_error.empty() ∧ ¬ImGui::IsPopupOpen("Error"))
            ImGui::OpenPopup("Error");
        if (ImGui::BeginPopupModal("Error", nullptr, ImGuiWindowFlags_AlwaysAutoResize))
        {
            ImGui::TextUnformatted(m_error.c_str());
            if (ImGui::Button("OK"))
            {
                m_error.clear();
                ImGui::CloseCurrentPopup();
            }
            ImGui::EndPopup();
        }
        if (m_showAbout)
        {
            ImGui::OpenPopup("About");
            m_showAbout = false;
        }
        if (ImGui::BeginPopupModal("About", nullptr, ImGuiWindowFlags_AlwaysAutoResize))
        {
            ImGui::Text("USD OptiX Viewer");
            ImGui::Separator();
            ImGui::Text("OpenUSD scene graph, meshoptimizer meshes, OptiX %d ray tracing,", OPTIX_
                    VERSION / 10000);
            ImGui::Text("SDL3 windowing and Dear ImGui %s user interface.", IMGUI_VERSION);
```

```
            ImGui::Spacing();
            ImGui::TextDisabled("Left drag: orbit   Right/middle drag: pan   Wheel: zoom");
            ImGui::TextDisabled("Double-click: orbit around the point under the cursor");
            ImGui::TextDisabled("Click: select   F: frame selection   Esc: clear selection");
            ImGui::TextDisabled("Camera menu: look through a USD camera (moving returns to the orbit
                camera)");
            ImGui::TextDisabled("XR menu: headset output through OpenXR / NVIDIA CloudXR (--xr,
                --cloudxr)");
            if (ImGui::Button("Close"))
                ImGui::CloseCurrentPopup();
            ImGui::EndPopup();
        }
    }
```

This code is used in section 253.

324. The XR output. The viewer can show its scene in a headset as well: in stereo, with the head's own motion, through any OpenXR runtime, and in particular through NVIDIA CloudXR, which streams the images over the network to an Apple Vision Pro, a Meta Quest or a similar device. The OpenXR side is the class *XrOutput* and the CloudXR side the class *CloudXrService*, both described in the next part. Here we see how the application drives them.

⟨The XR output of the application 324⟩ ≡

```
// ---- XR output ------------------------------------------------------------
⟨Start the XR output 325⟩
⟨Follow the XR session 326⟩
⟨Render the eyes of the headset 327⟩
⟨Move the renderer to the GPU of the headset 328⟩
⟨Draw the mirror of the left eye 329⟩
⟨Describe the state of the XR output 330⟩
```

This code is used in section 253.

325. There are two ways to reach a CloudXR headset. CloudXR may already run as a service of its own, started for example by NVIDIA's Isaac Teleop tools, which also set *XR_RUNTIME_JSON* so that OpenXR applications find it; then `--xr` is enough. Or the viewer runs the CloudXR runtime itself, from the libraries of a CloudXR SDK, with `--cloudxr`. The SDK's directory comes from `--cloudxr-sdk`, failing that from the environment variable *CLOUDXR_SDK_DIR*, and failing that from *CLOUDXR_SDK_DEFAULT*, a string literal that the build defines (⟨Choose the default CloudXR SDK 12⟩). By default it names the SDK installed for all users with the other tools, `/opt/viztools/cloudxr-6.2.1`, so that on this machine `--cloudxr` alone is enough. The most specific setting wins: a command line can point at another SDK for one run, and the environment for a whole shell session. Only a build configured without a default can end up with no directory at all. Either way the OpenXR instance is created next; a failure leaves the viewer working on the desktop, with the reason in the error popup.

⟨Start the XR output 325⟩ ≡

```
// --cloudxr runs the CloudXR runtime in this process; --xr alone uses the
// active OpenXR runtime (XR_RUNTIME_JSON), for instance a CloudXR runtime
// started by Isaac Teleop.
void App::startXr(bool cloudXr, std::string sdkDir, const std::string& profile)
{
    std::string error;
    if (cloudXr)
    {
        // the option, then the environment, then the SDK installed with the tools
        if (sdkDir.empty() ∧ std::getenv("CLOUDXR_SDK_DIR"))
            sdkDir = std::getenv("CLOUDXR_SDK_DIR");
        if (sdkDir.empty())
            sdkDir = CLOUDXR_SDK_DEFAULT;
        if (sdkDir.empty())
        {
            m_error = "--cloudxr needs the CloudXR SDK directory: --cloudxr-sdk DIR or CLOUDXR_SDK_
```

```
                DIR.";
            return;
        }
        if (¬m_cloudXr.start(sdkDir, profile, error))
        {
            m_error = "CloudXR: " + error;
            return;
        }
    }
    if (¬m_xr.init(error))
    {
        m_error = "XR output unavailable: " + error;
        return;
    }
    m_xrEnabled = true;
}
```

This code is used in section 324.

326. Once per turn of the main loop, the application lets the XR output handle the runtime's events and then, if a frame is due, renders the two eyes.

A headset is not always there. With CloudXR's *auto* device profiles the runtime reports no headset until a client connects, which may be minutes after the program started. When the headset appears, the output reports *SystemReady*: the runtime has chosen a GPU and made a Vulkan device on it. The eye images are shared between Vulkan and CUDA, which works only on one and the same GPU, so the renderer moves to that GPU if it is not already there, and then the session is created.

beginFrame waits until the headset wants the next frame. That wait paces the whole loop at the headset's refresh rate, so while the session runs the desktop must not wait for its own vertical blank as well, or the two clocks would beat against each other; vsync is switched off and back on as the session starts and stops. Any error ends the XR output, not the program.

⟨Follow the XR session 326⟩ ≡

```
// Called once per main-loop turn. When the headset appears, the renderer
// moves to the GPU the runtime chose, so that the eye buffers can be shared.
// While the session runs, xrWaitFrame paces the loop at the headset's rate,
// so the desktop stops waiting for vsync.
void App::updateXr()
{
    if (¬m_xrEnabled)
        return;
    m_xrClient = m_cloudXr.clientConnected();
    std::string error;
    if (m_xr.update(error) ≡ XrOutput::State::SystemReady ∧ useRendererOn(m_xr.deviceUuid(), error))
        m_xr.createSession(m_xrResolution, error);
    if (m_xr.beginFrame(error))
    {
        const auto start = std::chrono::steady_clock::now();
        if (m_xr.shouldRender() ∧ m_rendererOk)
            renderXrEyes(error);
        m_xr.endFrame(error);
        m_xrFrameMs = std::chrono::duration<float, std::milli>(std::chrono::steady_clock::now() − start).count();
    }
    if (¬error.empty())
    {
        m_error = "XR output stopped: " + error;
        m_xr.stop();
        m_xrEnabled = false;
    }
    const bool paced = m_xr.state() ≡ XrOutput::State::Running;
    if (paced ≠ m_xrPaced)
    {
        SDL_SetRenderVSync(m_sdlRenderer, paced ? 0 : 1);
        m_xrPaced = paced;
    }
```

```
}
```

This code is used in section 324.

327. Where is the user in the scene? The headset reports the head's pose in its own space, in meters, relative to where the head was when the session started. We put that origin at the eye of the desktop view, facing the horizontal part of its direction (tilting the whole world with the desktop camera's pitch would be uncomfortable), so the headset starts where the desktop view is and follows it when the desktop user orbits or picks another camera. A meter of head motion spans $1/m$ scene units, where m is the stage's *metersPerUnit*, times the world scale of the XR menu.

Each eye is then rendered by the OptiX renderer straight into its shared buffer. For the mirror, the left eye is also copied to main memory; this costs a transfer per frame, which is why the mirror is optional.

⟨Render the eyes of the headset 327⟩ ≡

```
// The XR origin sits at the desktop view's eye, facing its horizontal
// direction, so the headset starts where the desktop camera is and follows it.
void App::renderXrEyes(std::string& error)
{
    XrOutput::Anchor anchor{};
    viewFrame(anchor.eye, anchor.forward);
    anchor.zUp = m_scene.zUp();
    const double metersPerUnit = m_scene.loaded() ? UsdGeomGetStageMetersPerUnit(m_scene.stage()) : 1.0;
    anchor.unitsPerMeter = m_xrWorldScale / float(metersPerUnit);
    try
    {
        for (unsigned int i = 0; i < m_xr.eyeCount(); ++i)
            m_renderer.renderView(m_xr.eyeCamera(i, anchor), m_xr.eyeWidth(), m_xr.eyeHeight(), m_
                    xr.eyeBuffer(i),
                                  unsigned(m_xrSamples));
        if (m_xrMirror)
        {
            m_mirrorPixels.resize(size_t(m_xr.eyeWidth()) * m_xr.eyeHeight());
            if (cudaMemcpy(m_mirrorPixels.data(), m_xr.eyeBuffer(0), m_mirrorPixels.size() * sizeof(uchar4),
                           cudaMemcpyDeviceToHost) ≠ cudaSuccess)
                throw std::runtime_error("copying the mirror image failed");
        }
    }
    catch (const std::exception& e)
    {
        error = std::string("rendering failed: ") + e.what();
    }
}
```

This code is used in section 324.

328. Moving the renderer to another GPU means starting it again there: release everything, initialize on the GPU with the given UUID, and upload the scene and the selection anew. The desktop image follows by itself: the renderer's image size is now zero, so the viewport resizes it on the next frame.

⟨Move the renderer to the GPU of the headset 328⟩ ≡

```
// Moves the renderer to the GPU with the given UUID and uploads the scene again.
bool App::useRendererOn(const std::array<unsigned char, 16>& uuid, std::string& error)
{
    if (m_rendererOk ∧ m_renderer.deviceUuid() ≡ uuid)
        return true;
    m_renderer.release();
    m_rendererOk = m_renderer.init(error, uuid.data());
    if (¬m_rendererOk)
        return false;
    if (m_scene.loaded())
        m_renderer.setScene(m_scene);
    const int mesh = m_scene.meshIndex(m_selected);
    m_renderer.setSelectedMesh(mesh ≥ 0 ? unsigned(mesh) : kNoMesh);
    m_needsRender = true;  // the viewport also resizes the new image
    return true;
```

```
}
```

This code is used in section 324.

329. The mirror is an SDL texture of the eye's size, recreated when that size changes, and drawn as large as fits in the viewport without distortion, on black.

```
⟨Draw the mirror of the left eye 329⟩ ≡
    // The left eye's latest image, fitted inside the viewport.
    void App::drawXrMirror(float x, float y, float width, float height)
    {
        const int w = int(m_xr.eyeWidth()), h = int(m_xr.eyeHeight());
        if (¬m_mirrorTexture ∨ w ≠ m_mirrorWidth ∨ h ≠ m_mirrorHeight)
        {
            if (m_mirrorTexture)
                SDL_DestroyTexture(m_mirrorTexture);
            m_mirrorTexture = SDL_CreateTexture(m_sdlRenderer, SDL_PIXELFORMAT_RGBA32, SDL_
                    TEXTUREACCESS_STREAMING, w, h);
            m_mirrorWidth = w;
            m_mirrorHeight = h;
        }
        if (¬m_mirrorTexture ∨ m_mirrorPixels.size() ≠ size_t(w) * h)
            return;
        SDL_UpdateTexture(m_mirrorTexture, nullptr, m_mirrorPixels.data(), w * 4);
        const float scale = std::min(width / float(w), height / float(h));
        const ImVec2 lo(x + 0.5f * (width − w * scale), y + 0.5f * (height − h * scale));
        ImDrawList* list = ImGui::GetWindowDrawList();
        list→AddRectFilled(ImVec2(x, y), ImVec2(x + width, y + height), IM_COL32(0, 0, 0, 255));
        list→AddImage(ImTextureRef(ImTextureID(reinterpret_cast<intptr_t>(m_mirrorTexture))), lo,
                ImVec2(lo.x + w * scale, lo.y + h * scale));
    }
```

This code is used in section 324.

330. Finally, the state of the XR output in words, for the XR menu and the status bar. While the session runs, it includes the time the last frame took to render and hand over, not counting the wait for the headset.

```
⟨Describe the state of the XR output 330⟩ ≡
    std::string App::xrStatus() const
    {
        switch (m_xr.state())
        {
        case XrOutput::State::Off:
            return "off";
        case XrOutput::State::WaitingForSystem:
            return m_cloudXr.running() ? "waiting for a CloudXR client" : "waiting for a headset";
        case XrOutput::State::SystemReady:
        case XrOutput::State::Idle:
            return "headset idle";
        case XrOutput::State::Running:
            break;
        }
        char text[64];
        std::snprintf(text, sizeof(text), "running, %.1f ms per frame", m_xrFrameMs);
        return text;
    }
```

This code is used in section 324.

331. Decoding images. Our build of OpenUSD leaves out its imaging libraries, and with them OpenUSD's image readers, so the scene decodes textures with Sean Barrett's `stb_image.h`, a single-header library in the public domain. Such a header contains only declarations, unless a certain macro, *STB_IMAGE_IMPLEMENTATION*, is defined before the header is included; then it contains the implementation as well. Exactly one source file of the program must do so, and this is it.

⟨`UsdOptixViewer/src/StbImage.cpp` 331⟩ ≡

```
// Image decoding for textures (PNG, JPEG, TGA, BMP, HDR, ...).
#define STB_IMAGE_IMPLEMENTATION
#include "third_party/stb_image.h"
```

This code is written to file `UsdOptixViewer/src/StbImage.cpp`.

XR output

332. Everything so far draws on a monitor. This part sends the same ray-traced scene to a head-mounted display, one image per eye, through *OpenXR*, the Khronos standard that sits between an application and the *runtime* of a headset. The runtime we have in mind is NVIDIA CloudXR, which does not drive a headset cable at all: it encodes the eye images as video and streams them over the network to an Apple Vision Pro or a Meta Quest, which send their head poses back. To the application CloudXR is an ordinary OpenXR runtime, so nothing below is specific to it except the small class that can start it inside our own process.

Two classes do the work. *CloudXrService* (in `CloudXrService.h` and `CloudXrService.cpp`) loads the CloudXR runtime from a CloudXR SDK and starts it, so that a single command brings up both the viewer and the streaming server. *XrOutput* (in `XrOutput.h` and `XrOutput.cpp`) is the OpenXR client proper: it finds the headset, creates the session, and every frame hands the application the eye cameras and two buffers to render into, then passes the finished images to the runtime. The application, in the previous part, decides when to call it.

The guiding constraint is that the eye images must not cross the PCI bus. The renderer produces them in GPU memory with CUDA; the runtime wants them in Vulkan images. Rather than downloading the pixels to the host and uploading them again—some 35 MB per frame for two eyes of 2400 by 1920 pixels, ninety times a second—the eye buffers are allocated by Vulkan, *exported* to CUDA, and written by OptiX directly. A copy on the GPU then moves each buffer into the image the runtime gave us.

333. The CloudXR service. CloudXR comes as a set of shared libraries. The OpenXR runtime, which the OpenXR loader opens when an application creates an instance, is one of them: `libopenxr_cloudxr.so`. It does not do the streaming itself: it talks, through a local pipe, to the CloudXR *service*, which owns the network connection to the headset and the video encoder. The service lives in `libcloudxr.so` and is driven by a small C API, `cxrServiceAPI.h`, that the SDK ships under a permissive license; we keep a copy of the header among the third-party sources.

The service can run in a process of its own (NVIDIA's Isaac Teleop starts it that way, and then any OpenXR application on the machine can use it), or in the application's process, which is what *CloudXrService* does. Either way it must be running *before* the OpenXR instance is created, because creating the instance is when the runtime library connects to it. The class therefore sits apart from *XrOutput*, and the application starts it first.

⟨`UsdOptixViewer/src/CloudXrService.h` 333⟩ ≡

```
⟨Opening of the CloudXR service header 334⟩
class CloudXrService
{
public:
    ⟨Public interface of the CloudXR service 335⟩
private:
    ⟨State and entry points of the CloudXR service 336⟩
};
```

This code is written to file `UsdOptixViewer/src/CloudXrService.h`.

334. The header names the class's purpose and includes the SDK's API header.

⟨Opening of the CloudXR service header 334⟩ ≡

```
// Runs the NVIDIA CloudXR runtime inside this process, from the libraries of
// a CloudXR SDK, and points the OpenXR loader at it. The service must be
// started before the OpenXR instance is created.
#pragma once

#include <cstdint>
#include <string>
#include <vector>

#include "third_party/cxrServiceAPI.h"
```

This code is used in section 333.

335. The SDK is found at run time, from a directory that the application chooses (the command line, the

environment, or the default recorded by the build; see ⟨Start the XR output 325⟩), so the program neither links against CloudXR nor needs it to build. The *device profile* tells the service what kind of client to expect, and with it the eye resolution and frame rate: *apple−vision−pro* and *quest3* are static profiles, whose OpenXR system exists as soon as the service starts, while the **auto**−* profiles wait for a client to connect and then adopt its parameters.

⟨Public interface of the CloudXR service 335⟩ ≡

```
CloudXrService() = default;
~CloudXrService();
CloudXrService(const CloudXrService&) = delete;
CloudXrService& operator=(const CloudXrService&) = delete;

// sdkDir holds libcloudxr.so and openxr_cloudxr.json. deviceProfile is a
// CloudXR device profile ("apple-vision-pro", "auto-webrtc", ...), or empty
// for the runtime's default.
bool start(const std::string& sdkDir, const std::string& deviceProfile, std::string& error);
void stop();
bool running() const { return m_service ≠ nullptr; }
const std::string& version() const { return m_version; }

// Where a client such as the Vision Pro app connects: the IPv4 address of
// each network interface ("eth0 192.168.1.20") and the service's port
// (0 if the service did not say).
const std::vector<std::string>& addresses() const { return m_addresses; }
int64_t serverPort() const { return m_serverPort; }

// Drains the service's events; true while a CloudXR client is connected.
bool clientConnected();
```

This code is used in section 333.

336. The service is a handle into the loaded library. The library's functions are reached through pointers that *start* resolves with *dlsym*; their types come from the SDK header. The addresses and the port are found once, when the service has started, by the private *findAddresses* and by asking the service.

⟨State and entry points of the CloudXR service 336⟩ ≡

```
void findAddresses();

void* m_library = nullptr;
nv_cxr_service* m_service = nullptr;
std::string m_version;
bool m_clientConnected = false;
std::vector<std::string> m_addresses;
int64_t m_serverPort = 0;

PFN_nv_cxr_get_runtime_version m_getRuntimeVersion = nullptr;
PFN_nv_cxr_service_create m_create = nullptr;
PFN_nv_cxr_service_set_string_property m_setStringProperty = nullptr;
PFN_nv_cxr_service_get_int64_property m_getInt64Property = nullptr;
PFN_nv_cxr_service_start m_start = nullptr;
PFN_nv_cxr_service_stop m_stop = nullptr;
PFN_nv_cxr_service_join m_join = nullptr;
PFN_nv_cxr_service_destroy m_destroy = nullptr;
PFN_nv_cxr_service_poll_event m_pollEvent = nullptr;
```

This code is used in section 333.

337. The implementation file holds the six member functions, in the order of the service's life.

⟨`UsdOptixViewer/src/CloudXrService.cpp` 337⟩ ≡

```
⟨Headers of the CloudXR service 338⟩

⟨Destroy the CloudXR service 339⟩

⟨Start the CloudXR service 340⟩

⟨Find the network addresses of this machine 346⟩

⟨Stop the CloudXR service 347⟩

⟨Poll the events of the CloudXR service 348⟩
```

This code is written to file `UsdOptixViewer/src/CloudXrService.cpp`.

338. It needs the file-system library to build paths, the signal functions for a reason explained below, *dlopen*, and the socket headers that list the network interfaces.

⟨Headers of the CloudXR service 338⟩ ≡
```
#include "CloudXrService.h"
#include <cstdlib>
#include <filesystem>
#include <type_traits>
#include <csignal>
#include <arpa/inet.h>
#include <dlfcn.h>
#include <ifaddrs.h>
#include <net/if.h>
```
This code is used in section 337.

339. A running service must be stopped before the library that contains it is unloaded, and the destructor sees to it.

⟨Destroy the CloudXR service 339⟩ ≡
```
CloudXrService::~CloudXrService()
{
    stop();
}
```
This code is used in section 337.

340. Starting the service is a sequence of steps, any of which can fail; each failure leaves an explanation in *error* and everything released.

⟨Start the CloudXR service 340⟩ ≡
```
bool CloudXrService::start(const std::string& sdkDir, const std::string& deviceProfile, std::string& error)
{
    ⟨Find and load the CloudXR library 341⟩
    ⟨Resolve the entry points of the CloudXR service 342⟩
    ⟨Create the CloudXR service and choose its device profile 343⟩
    ⟨Start the CloudXR service, keeping our signal handlers 344⟩
    ⟨Report a failed CloudXR start, or point OpenXR at the runtime 345⟩
}
```
This code is used in section 337.

341. An SDK directory is recognized by its OpenXR manifest, `openxr_cloudxr.json`, which we shall later hand to the loader. The service library depends on several others in the same directory (the streaming engine, a private OpenSSL, a CUDA runtime); its *run path* is *$ORIGIN*, so loading it by its full path is enough for the dynamic linker to find them all. *RTLD_LOCAL* keeps its symbols out of the global namespace: the SDK's CUDA runtime, version 12, must not be confused with the version 13 runtime that the renderer is linked against.

⟨Find and load the CloudXR library 341⟩ ≡
```
const std::filesystem::path dir(sdkDir);
const std::filesystem::path manifest = dir / "openxr_cloudxr.json";
if (¬std::filesystem::exists(manifest))
{
    error = "No CloudXR SDK in " + sdkDir + " (openxr_cloudxr.json not found)";
    return false;
}
// the library finds its own dependencies next to it (RUNPATH $ORIGIN)
m_library = dlopen((dir / "libcloudxr.so").c_str(), RTLD_NOW | RTLD_LOCAL);
if (¬m_library)
{
    error = std::string("Could not load libcloudxr.so: ") + dlerror();
    return false;
}
```
This code is used in section 340.

342. The generic lambda *resolve* looks a symbol up and casts it to the type of the pointer it is given, which saves repeating every function type. Once the functions are known we ask for the runtime's version; the XR menu shows it, and its major number is the streaming protocol a client must speak.

⟨Resolve the entry points of the CloudXR service 342⟩ ≡

```
auto resolve = [&](auto& fn, const char* name) {
    fn = reinterpret_cast<std::remove_reference_t<decltype(fn)>>(dlsym(m_library, name));
    if (¬fn)
        error = std::string("libcloudxr.so lacks ") + name;
    return fn ≠ nullptr;
};
if (¬resolve(m_getRuntimeVersion, "nv_cxr_get_runtime_version") ∨ ¬resolve(m_create, "nv_cxr_service_
        create") ∨
    ¬resolve(m_setStringProperty, "nv_cxr_service_set_string_property") ∨
    ¬resolve(m_getInt64Property, "nv_cxr_service_get_int64_property") ∨
    ¬resolve(m_start, "nv_cxr_service_start") ∨ ¬resolve(m_stop, "nv_cxr_service_stop") ∨
    ¬resolve(m_join, "nv_cxr_service_join") ∨ ¬resolve(m_destroy, "nv_cxr_service_destroy") ∨
    ¬resolve(m_pollEvent, "nv_cxr_service_poll_event"))
{
    stop();
    return false;
}
uint32_t major = 0, minor = 0, patch = 0;
m_getRuntimeVersion(&major, &minor, &patch);
m_version = std::to_string(major) + "." + std::to_string(minor) + "." + std::to_string(patch);
```

This code is used in section 340.

343. Properties can only be set between creating and starting the service. The only one we set is the device profile, and only when the user named one.

⟨Create the CloudXR service and choose its device profile 343⟩ ≡

```
nv_cxr_result_t result = m_create(&m_service);
if (result ≡ NV_CXR_SUCCESS ∧ ¬deviceProfile.empty())
{
    const std::string name = "device-profile";
    result = m_setStringProperty(m_service, name.data(), name.size(), deviceProfile.data(), deviceProfile.size());
}
```

This code is used in section 340.

344. Starting the service has a side effect that is easy to miss. The service, like the Monado runtime it is built on, installs handlers for *SIGINT* and *SIGTERM* that stop it. In a process of its own that is right; in ours it is not, because a signal would pull the runtime away while our session is still submitting frames, the next OpenXR call would fail, and SDL, whose own handlers were replaced, would never learn that it should quit. So we remember the handlers that were in place—SDL's, which turn the signals into a quit event—and put them back as soon as the service has started. A *SIGTERM* then takes the normal way out: the main loop ends, the session is destroyed, and only then the service stops.

⟨Start the CloudXR service, keeping our signal handlers 344⟩ ≡

```
if (result ≡ NV_CXR_SUCCESS)
{
    // The service installs SIGINT/SIGTERM handlers that stop it while the
    // app still uses it. Give the signals back to SDL, whose quit event
    // ends the session first and then stops the service.
    struct sigaction interrupt{}, terminate{};
    sigaction(SIGINT, nullptr, &interrupt);
    sigaction(SIGTERM, nullptr, &terminate);
    result = m_start(m_service);
    sigaction(SIGINT, &interrupt, nullptr);
    sigaction(SIGTERM, &terminate, nullptr);
}
```

This code is used in section 340.

345. The usual reason for a failed start is another CloudXR runtime already running on the machine, since

only one service can hold the network ports. The embedded service writes its log to the console, which is where the message sends the user. On success, *XR_RUNTIME_JSON* is pointed at the SDK's manifest; the OpenXR loader reads this variable when the instance is created, and so the instance will be served by the CloudXR runtime library, which in turn connects to the service we have just started.

Last, we collect what the user must type into the client on the headset: the machine's address and the service's port. The port is not a constant: it depends on the device profile (48010 for *apple−vision−pro*, 49100 for the WebRTC profiles) and can be changed with the environment variable *NV_CXR_SERVER_PORT*, so we ask the service for its *server−port* property rather than guess.

⟨Report a failed CloudXR start, or point OpenXR at the runtime 345⟩ ≡

```
if (result ≠ NV_CXR_SUCCESS)
{
    error = "The CloudXR service did not start (error " + std::to_string(int(result)) +
            "). Is another CloudXR runtime running? The service's messages are on the console.";
    if (m_service)
    {
        m_destroy(m_service);
        m_service = nullptr;
    }
    stop();
    return false;
}
// the OpenXR loader reads this when the instance is created
setenv("XR_RUNTIME_JSON", manifest.c_str(), 1);

// The port depends on the device profile (48010 for apple-vision-pro)
// and on NV_CXR_SERVER_PORT, so ask the service.
const std::string portName = "server-port";
if (m_getInt64Property(m_service, portName.data(), portName.size(), &m_serverPort) ≠ NV_CXR_SUCCESS)
    m_serverPort = 0;
findAddresses();
return true;
```

This code is used in section 340.

346. The service listens on every interface, and a machine often has several (wired, wireless, a VPN), of which only the user knows which one the headset can reach. So we list the IPv4 address of every interface that is up, except the loopback, each with its interface's name; *getifaddrs* returns them as a linked list, which we must free.

⟨Find the network addresses of this machine 346⟩ ≡

```
void CloudXrService::findAddresses()
{
    m_addresses.clear();
    ifaddrs* list = nullptr;
    if (getifaddrs(&list) ≠ 0)
        return;
    for (const ifaddrs* entry = list; entry; entry = entry→ifa_next)
    {
        if (¬entry→ifa_addr ∨ entry→ifa_addr→sa_family ≠ AF_INET ∨ (entry→ifa_flags & IFF_LOOPBACK)
                ∨
            ¬(entry→ifa_flags & IFF_UP))
            continue;
        char text[INET_ADDRSTRLEN] = {};
        inet_ntop(AF_INET, &reinterpret_cast<const sockaddr_in*>(entry→ifa_addr)→sin_addr, text,
                sizeof(text));
        m_addresses.push_back(std::string(entry→ifa_name) + "  " + text);
    }
    freeifaddrs(list);
}
```

This code is used in section 337.

347. Stopping is *stop* followed by *join*, which waits until the service has really ended, and then destroying it. *stop* is safe to call at any time, also on a half-started service.

⟨Stop the CloudXR service 347⟩ ≡
```
void CloudXrService::stop()
{
    if (m_service)
    {
        m_stop(m_service);
        m_join(m_service);
        m_destroy(m_service);
        m_service = nullptr;
    }
    if (m_library)
    {
        dlclose(m_library);
        m_library = nullptr;
    }
    m_clientConnected = false;
    m_addresses.clear();
    m_serverPort = 0;
}
```
This code is used in section 337.

348. The service queues events: a client (the headset) connected or disconnected, an OpenXR application connected or disconnected. Only the first pair interests us, for the status shown in the XR menu. One oddity of the API is that "no event" is reported as a successful call that returns an event of type *NV_CXR_EVENT_NONE*, so the loop must test the type as well as the result.

⟨Poll the events of the CloudXR service 348⟩ ≡
```
bool CloudXrService::clientConnected()
{
    if (¬m_service)
        return false;
    // poll_event reports "no event" as a success with type NV_CXR_EVENT_NONE
    nv_cxr_event_t event{};
    while (m_pollEvent(m_service, &event) ≡ NV_CXR_SUCCESS ∧ event.type ≠ NV_CXR_EVENT_NONE)
    {
        if (event.type ≡ NV_CXR_EVENT_CLOUDXR_CLIENT_CONNECTED)
            m_clientConnected = true;
        else if (event.type ≡ NV_CXR_EVENT_CLOUDXR_CLIENT_DISCONNECTED)
            m_clientConnected = false;
        event = {};
    }
    return m_clientConnected;
}
```
This code is used in section 337.

349. The XR output: interface. *XrOutput* has to cope with a headset that may appear late, disappear, and come back. Its life is therefore a small state machine, and the application asks for the current state every frame.

⟨`UsdOptixViewer/src/XrOutput.h` 349⟩ ≡
```
⟨Opening of the XR output header 350⟩
class XrOutput
{
public:
    ⟨States of the XR output 351⟩
    ⟨Placing the XR user in the scene 352⟩
    ⟨Construction and session control of the XR output 353⟩
    ⟨Frames and eyes of the XR output 354⟩
private:
    ⟨An eye of the XR output 355⟩
    ⟨Private steps of the XR output 356⟩
```

⟨OpenXR state of the XR output 357⟩

⟨Vulkan state and eye images of the XR output 358⟩

};

This code is written to file `UsdOptixViewer/src/XrOutput.h`.

350. The header pulls in the CUDA runtime (for the external-memory handle of each eye), the Vulkan and OpenXR headers, and the shared launch parameters, whose *Camera* the eye cameras are expressed in. The build defines *VK_NO_PROTOTYPES*, so the Vulkan header declares types but no functions, and *XR_USE_GRAPHICS_API_VULKAN*, which makes `openxr_platform.h` declare the Vulkan binding of OpenXR.

⟨Opening of the XR output header 350⟩ ≡

```
// Stereo output through OpenXR, for example to a headset streamed by NVIDIA
// CloudXR. The runtime chooses the GPU and creates the Vulkan device. Each
// eye has a buffer in Vulkan memory that is shared with CUDA, so OptiX renders
// straight into it; a copy on the Vulkan queue moves it into the eye's
// swapchain image.
#pragma once

#include <array>
#include <chrono>
#include <string>
#include <vector>

#include <cuda_runtime.h>
#include <vulkan/vulkan.h>
#include <openxr/openxr.h>
#include <openxr/openxr_platform.h>

#include "LaunchParams.h"
```

This code is used in section 349.

351. The states, in the order they are normally visited. Without an instance the output is *Off*. With one, it looks for an OpenXR *system*—a headset—and stays *WaitingForSystem* until there is one. A static CloudXR profile has a system at once; an **auto**−* profile only after a client has connected, which may take minutes, and the desktop must remain usable meanwhile. Once the system is there and the Vulkan device has been made, the output is *SystemReady* and waits for the application to create the session; the application has a job to do first, as we shall see. The session then sits *Idle* until the runtime declares it ready, and is *Running* while frames flow.

⟨States of the XR output 351⟩ ≡

```
enum class State
{
    Off,                  // no OpenXR instance
    WaitingForSystem,     // no headset yet (a CloudXR "auto" profile waits for a client)
    SystemReady,          // Vulkan device created; the app must call createSession()
    Idle,                 // session created, not running
    Running,              // frames are being submitted
};
```

This code is used in section 349.

352. The headset reports the eyes in its own coordinates, in meters, relative to where the head was when the session began. Something must say where that origin is in the scene. The *Anchor* does: an eye point, a direction to face, which axis is up, and a scale. The application fills it from the desktop view, so that the user starts where the desktop camera is.

⟨Placing the XR user in the scene 352⟩ ≡

```
// Places the user in the scene: the origin of the XR space (the head's
// position when the session started) goes to `eye`, facing the
// horizontal part of `forward`, and one meter spans unitsPerMeter units.
struct Anchor
{
    float eye[3];
    float forward[3];
    bool zUp;
```

```
    float unitsPerMeter;
};
```

This code is used in section 349.

353. The life cycle in calls: *init* creates the instance, *update* is called every main-loop turn, and when the state is *SystemReady* the application makes CUDA current on the GPU identified by *deviceUuid* before calling *createSession*. The UUID is the only reliable way to name a GPU across APIs: CUDA and Vulkan number the two GPUs of this machine independently.

⟨Construction and session control of the XR output 353⟩ ≡

```
XrOutput() = default;
~XrOutput();
XrOutput(const XrOutput&) = delete;
XrOutput& operator=(const XrOutput&) = delete;

// Creates the OpenXR instance with the active runtime (XR_RUNTIME_JSON).
bool init(std::string& error);
// Ends the session and releases everything (State::Off).
void stop();
// Handles runtime events and looks for the headset. On an error the XR
// output is shut down (State::Off) and `error` is set.
State update(std::string& error);
State state() const { return m_state; }
// In State::SystemReady: CUDA must be current on the GPU with this UUID
// before createSession(), which shares memory between Vulkan and CUDA.
const std::array<unsigned char, 16>& deviceUuid() const { return m_deviceUuid; }
bool createSession(float resolutionScale, std::string& error);
```

This code is used in section 349.

354. Each frame is bracketed by *beginFrame* and *endFrame*. In between, when *shouldRender* is true, the application renders every eye with *eyeCamera* into *eyeBuffer*. The names of the runtime, the headset and the GPU are kept for the XR menu.

⟨Frames and eyes of the XR output 354⟩ ≡

```
// One frame: when beginFrame() returns true, render each eye into
// eyeBuffer() if shouldRender(), then call endFrame().
bool beginFrame(std::string& error);
bool shouldRender() const { return m_frameState.shouldRender ∧ m_viewsValid; }
void endFrame(std::string& error);

unsigned int eyeCount() const { return unsigned(m_eyes.size()); }
unsigned int eyeWidth() const { return m_width; }
unsigned int eyeHeight() const { return m_height; }
uchar4* eyeBuffer(unsigned int eye) const { return m_eyes[eye].pixels; }
Camera eyeCamera(unsigned int eye, const Anchor& anchor) const;

const std::string& runtimeName() const { return m_runtimeName; }
const std::string& systemName() const { return m_systemName; }
const std::string& gpuName() const { return m_gpuName; }
```

This code is used in section 349.

355. An eye owns a swapchain—the ring of images the runtime gives us to fill, usually three—and the shared buffer: a Vulkan buffer, its memory, the same memory as CUDA sees it, and the device pointer through which OptiX writes it.

⟨An eye of the XR output 355⟩ ≡

```
struct Eye
{
    XrSwapchain swapchain = XR_NULL_HANDLE;
    std::vector<XrSwapchainImageVulkan2KHR> images;
    VkBuffer buffer = VK_NULL_HANDLE;
    VkDeviceMemory memory = VK_NULL_HANDLE;
    cudaExternalMemory_t cudaMemory = nullptr;
    uchar4* pixels = nullptr;   // the buffer, mapped into CUDA
};
```

This code is used in section 349.

356. The private steps follow the order of the life cycle; each is presented in its own section of the implementation.

⟨Private steps of the XR output 356⟩ ≡
```
void findSystem();
void createVulkanDevice();
void createSharedBuffer(Eye& eye);
void pollEvents();
void copyToSwapchains();
void destroySession();
void destroyVulkan();
```
This code is used in section 349.

357. The OpenXR side of the state: instance, system, session, the reference space in which views are located, the blend mode and frame state of the current frame, the located views, and the time of the next look for a system.

⟨OpenXR state of the XR output 357⟩ ≡
```
State m_state = State::Off;
XrInstance m_instance = XR_NULL_HANDLE;
XrSystemId m_systemId = XR_NULL_SYSTEM_ID;
XrSession m_session = XR_NULL_HANDLE;
XrSpace m_space = XR_NULL_HANDLE;
XrEnvironmentBlendMode m_blendMode = XR_ENVIRONMENT_BLEND_MODE_OPAQUE;
XrFrameState m_frameState{XR_TYPE_FRAME_STATE};
std::vector<XrView> m_views;
bool m_viewsValid = false;
std::chrono::steady_clock::time_point m_nextSystemPoll;
std::string m_runtimeName, m_systemName, m_gpuName;
```
This code is used in section 349.

358. The Vulkan side: the loaded library, the instance and device that the runtime created for us, one queue, one command buffer and one fence (a frame records one batch of copies and waits for it), and the UUID of the GPU. Last come the eyes and their common size.

⟨Vulkan state and eye images of the XR output 358⟩ ≡
```
void* m_vulkanLibrary = nullptr;
VkInstance m_vkInstance = VK_NULL_HANDLE;
VkPhysicalDevice m_physicalDevice = VK_NULL_HANDLE;
VkDevice m_device = VK_NULL_HANDLE;
uint32_t m_queueFamily = 0;
VkQueue m_queue = VK_NULL_HANDLE;
VkCommandPool m_commandPool = VK_NULL_HANDLE;
VkCommandBuffer m_commandBuffer = VK_NULL_HANDLE;
VkFence m_fence = VK_NULL_HANDLE;
std::array<unsigned char, 16> m_deviceUuid{};

std::vector<Eye> m_eyes;
unsigned int m_width = 0, m_height = 0;
```
This code is used in section 349.

359. The XR output: setting up. The implementation file has a few helpers in an anonymous namespace, then the member functions in the order of the life cycle: instance and system, Vulkan, the shared buffers, the session, the frames, and the eye cameras.

⟨`UsdOptixViewer/src/XrOutput.cpp` 359⟩ ≡
```
⟨Headers of the XR output 360⟩

namespace {

⟨Vulkan functions loaded at run time for XR 361⟩

⟨OpenXR functions of the Vulkan binding 362⟩
```

⟨Error checks of the XR output 363⟩

⟨Rotate a vector by a quaternion for XR 364⟩

} // namespace

⟨Destroy or stop the XR output 365⟩

⟨Create the OpenXR instance 366⟩

⟨Update the XR output 371⟩

⟨Look for the XR system 372⟩

⟨Create the XR Vulkan device 373⟩

⟨Destroy the XR Vulkan device 381⟩

⟨Share an eye buffer between Vulkan and CUDA 382⟩

⟨Create the XR session 388⟩

⟨Destroy the XR session 395⟩

⟨Follow the XR session state 396⟩

⟨Begin an XR frame 397⟩

⟨End an XR frame 399⟩

⟨Copy the eye buffers into the swapchain images 400⟩

⟨Compute the camera of an XR eye 405⟩

This code is written to file `UsdOptixViewer/src/XrOutput.cpp`.

360.

⟨Headers of the XR output 360⟩ ≡

```
#include "XrOutput.h"

#include <algorithm>
#include <cmath>
#include <cstring>
#include <stdexcept>

#include <dlfcn.h>
```

This code is used in section 359.

361. The program does not link against Vulkan. Its driver-provided loader, `libvulkan.so.1`, is opened at run time, and every function we call is looked up through *vkGetInstanceProcAddr*. This keeps the build free of a Vulkan SDK (only the headers are needed) and the binary free of a hard dependency that only the XR output uses. The list of functions is written once, as an "X macro": $VULKAN_FUNCTIONS(X)$ applies X to every name, so the same list both declares the pointers and, later, loads them.

⟨Vulkan functions loaded at run time for XR 361⟩ ≡

```
// Vulkan is loaded at run time from the driver, so the program needs no
// Vulkan SDK. There is one Vulkan device per process, so the function table
// can be global.
#define VULKAN_FUNCTIONS(X) \
    X(vkDestroyInstance) \
    X(vkGetPhysicalDeviceProperties2) \
    X(vkGetPhysicalDeviceQueueFamilyProperties) \
    X(vkGetPhysicalDeviceMemoryProperties) \
    X(vkDestroyDevice) \
    X(vkDeviceWaitIdle) \
    X(vkGetDeviceQueue) \
    X(vkCreateCommandPool) \
    X(vkDestroyCommandPool) \
    X(vkAllocateCommandBuffers) \
    X(vkBeginCommandBuffer) \
    X(vkEndCommandBuffer) \
    X(vkCmdPipelineBarrier) \
    X(vkCmdCopyBufferToImage) \
    X(vkQueueSubmit) \
    X(vkCreateFence) \
```

```
    X(vkDestroyFence) \
    X(vkWaitForFences) \
    X(vkResetFences) \
    X(vkCreateBuffer) \
    X(vkDestroyBuffer) \
    X(vkGetBufferMemoryRequirements) \
    X(vkAllocateMemory) \
    X(vkFreeMemory) \
    X(vkBindBufferMemory) \
    X(vkGetMemoryFdKHR)
struct Vulkan
{
    PFN_vkGetInstanceProcAddr vkGetInstanceProcAddr = nullptr;
#define DECLARE(name) PFN_##name name = nullptr;
    VULKAN_FUNCTIONS(DECLARE)
#undef DECLARE
} vk;
```

This code is used in section 359.

362. The OpenXR functions of the Vulkan binding are extension functions; the loader does not export them, and they too must be fetched, from the instance.

```
⟨OpenXR functions of the Vulkan binding 362⟩ ≡
// The functions of the Vulkan binding are extensions, found through the instance.
struct
{
    PFN_xrGetVulkanGraphicsRequirements2KHR getGraphicsRequirements = nullptr;
    PFN_xrCreateVulkanInstanceKHR createInstance = nullptr;
    PFN_xrGetVulkanGraphicsDevice2KHR getGraphicsDevice = nullptr;
    PFN_xrCreateVulkanDeviceKHR createDevice = nullptr;
} xrVulkan;
```

This code is used in section 359.

363. Errors become exceptions, caught at the entry points, which then shut the output down and report the message. OpenXR can name its result codes, but only through an instance; the most likely error, no runtime at all, happens before there is one, so it gets a message of its own that also says what to do about it.

```
⟨Error checks of the XR output 363⟩ ≡
void xrCheck(XrInstance instance, XrResult result, const char* what)
{
    if (XR_SUCCEEDED(result))
        return;
    if (result ≡ XR_ERROR_RUNTIME_UNAVAILABLE)  // before any instance exists, so not in
            xrResultToString
        throw std::runtime_error("no OpenXR runtime is available. Set XR_RUNTIME_JSON to a runtime's
                manifest, "
                                 "or use --cloudxr");
    char name[XR_MAX_RESULT_STRING_SIZE] = "";
    if (instance ≡ XR_NULL_HANDLE ∨ XR_FAILED(xrResultToString(instance, result, name)))
        snprintf(name, sizeof(name), "%d", int(result));
    throw std::runtime_error(std::string(what) + " failed: " + name);
}
void vkCheck(VkResult result, const char* what)
{
    if (result ≠ VK_SUCCESS)
        throw std::runtime_error(std::string(what) + " failed: VkResult " + std::to_string(int(result)));
}
void cudaCheck(cudaError_t result, const char* what)
{
    if (result ≠ cudaSuccess)
        throw std::runtime_error(std::string(what) + " failed: " + cudaGetErrorString(result));
}
```

This code is used in section 359.

364. Head poses arrive as unit quaternions $q = (\mathbf{u}, w)$. Rotating a vector $\mathbf{v}$ by q is $q\mathbf{v}q^{-1}$, which expands to the cheaper form

$$\mathbf{v}' = \mathbf{v} + 2\,\mathbf{u} \times (\mathbf{u} \times \mathbf{v} + w\,\mathbf{v}).$$

⟨Rotate a vector by a quaternion for XR 364⟩ ≡

```
// q * v * q^-1 for a unit quaternion
XrVector3f rotate(const XrQuaternionf& q, XrVector3f v)
{
    const XrVector3f u{q.x, q.y, q.z};
    auto cross = [](XrVector3f a, XrVector3f b) {
        return XrVector3f{a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
    };
    const XrVector3f c = cross(u, v);
    const XrVector3f t{c.x + q.w * v.x, c.y + q.w * v.y, c.z + q.w * v.z};
    const XrVector3f d = cross(u, t);
    return {v.x + 2.0f * d.x, v.y + 2.0f * d.y, v.z + 2.0f * d.z};
}
```

This code is used in section 359.

365. *stop* tears down in the reverse order of creation and is safe in any state, so it serves both as the destructor and as the common error path.

⟨Destroy or stop the XR output 365⟩ ≡

```
XrOutput::~XrOutput()
{
    stop();
}
void XrOutput::stop()
{
    destroySession();
    destroyVulkan();
    if (m_instance ≠ XR_NULL_HANDLE)
        xrDestroyInstance(m_instance);
    m_instance = XR_NULL_HANDLE;
    m_systemId = XR_NULL_SYSTEM_ID;
    m_state = State::Off;
}
```

This code is used in section 359.

366. Creating the instance takes four steps: check that the runtime can work with Vulkan, create the instance, record the runtime's name, and fetch the functions of the Vulkan binding.

⟨Create the OpenXR instance 366⟩ ≡

```
// ---- instance and system ------------------------------------------------------
bool XrOutput::init(std::string& error)
{
    try
    {
        ⟨Make sure the OpenXR runtime supports Vulkan 367⟩
        ⟨Create the OpenXR instance with the Vulkan extension 368⟩
        ⟨Name the OpenXR runtime 369⟩
        ⟨Load the OpenXR Vulkan binding functions 370⟩
    }
    catch (const std::exception& e)
    {
        error = e.what();
        stop();
        return false;
    }
    return true;
}
```

This code is used in section 359.

367. Enumerating the instance extensions is the first call that needs a runtime, so it doubles as the test that there is one. The renderer's images can only reach the runtime through Vulkan, which this program speaks through the second version of the Vulkan binding, *XR_KHR_vulkan_enable2*. The CloudXR runtime offers exactly that (and no desktop OpenGL binding).

⟨Make sure the OpenXR runtime supports Vulkan 367⟩ ≡
```
uint32_t count = 0;
xrCheck(XR_NULL_HANDLE, xrEnumerateInstanceExtensionProperties(nullptr, 0, &count, nullptr),
        "Looking for an OpenXR runtime");
std::vector<XrExtensionProperties> extensions(count, {XR_TYPE_EXTENSION_PROPERTIES});
xrEnumerateInstanceExtensionProperties(nullptr, count, &count, extensions.data());
if (std::none_of(extensions.begin(), extensions.end(), [](const XrExtensionProperties& e) {
        return std::strcmp(e.extensionName, XR_KHR_VULKAN_ENABLE2_EXTENSION_NAME) == 0;
    }))
    throw std::runtime_error("the OpenXR runtime does not support Vulkan (XR_KHR_vulkan_enable2)");
```
This code is used in section 366.

368. We ask for OpenXR 1.0, which every current runtime supports.

⟨Create the OpenXR instance with the Vulkan extension 368⟩ ≡
```
const char* enabled[] = {XR_KHR_VULKAN_ENABLE2_EXTENSION_NAME};
XrInstanceCreateInfo info{XR_TYPE_INSTANCE_CREATE_INFO};
std::strncpy(info.applicationInfo.applicationName, "UsdOptixViewer", XR_MAX_APPLICATION_NAME_SIZE -
        1);
std::strncpy(info.applicationInfo.engineName, "UsdOptixViewer", XR_MAX_ENGINE_NAME_SIZE - 1);
info.applicationInfo.applicationVersion = 1;
info.applicationInfo.apiVersion = XR_API_VERSION_1_0;
info.enabledExtensionCount = 1;
info.enabledExtensionNames = enabled;
xrCheck(XR_NULL_HANDLE, xrCreateInstance(&info, &m_instance), "xrCreateInstance");
```
This code is used in section 366.

369.

⟨Name the OpenXR runtime 369⟩ ≡
```
XrInstanceProperties properties{XR_TYPE_INSTANCE_PROPERTIES};
xrCheck(m_instance, xrGetInstanceProperties(m_instance, &properties), "xrGetInstanceProperties");
m_runtimeName = std::string(properties.runtimeName) + " " +
                std::to_string(XR_VERSION_MAJOR(properties.runtimeVersion)) + "." +
                std::to_string(XR_VERSION_MINOR(properties.runtimeVersion)) + "." +
                std::to_string(XR_VERSION_PATCH(properties.runtimeVersion));
```
This code is used in section 366.

370.

⟨Load the OpenXR Vulkan binding functions 370⟩ ≡
```
auto load = [&](auto& fn, const char* name) {
    xrCheck(m_instance, xrGetInstanceProcAddr(m_instance, name, reinterpret_cast<PFN_
            xrVoidFunction*>(&fn)),
            name);
};
load(xrVulkan.getGraphicsRequirements, "xrGetVulkanGraphicsRequirements2KHR");
load(xrVulkan.createInstance, "xrCreateVulkanInstanceKHR");
load(xrVulkan.getGraphicsDevice, "xrGetVulkanGraphicsDevice2KHR");
load(xrVulkan.createDevice, "xrCreateVulkanDeviceKHR");
m_state = State::WaitingForSystem;
```
This code is used in section 366.

371. The main loop calls *update* every turn. Waiting for a system and following the session's events are the two things that can happen between frames.

⟨Update the XR output 371⟩ ≡
```
XrOutput::State XrOutput::update(std::string& error)
```

```
{
    try
    {
        if (m_state ≡ State::WaitingForSystem)
            findSystem();
        else if (m_state ≡ State::Idle ∨ m_state ≡ State::Running)
            pollEvents();
    }
    catch (const std::exception& e)
    {
        error = e.what();
        stop();
    }
    return m_state;
}
```

This code is used in section 359.

372. Looking for a system must not block, and must not hammer the runtime either; twice a second is plenty. A runtime without a headset answers *XR_ERROR_FORM_FACTOR_UNAVAILABLE*, which is not an error here, just "not yet". When the system appears, the Vulkan device is created at once, because its UUID is what the application needs next.

```
⟨Look for the XR system 372⟩ ≡
// A CloudXR "auto" device profile has no system until a client connects, so
// the system is looked for twice a second until it appears.
void XrOutput::findSystem()
{
    const auto now = std::chrono::steady_clock::now();
    if (now < m_nextSystemPoll)
        return;
    m_nextSystemPoll = now + std::chrono::milliseconds(500);

    XrSystemGetInfo info{XR_TYPE_SYSTEM_GET_INFO};
    info.formFactor = XR_FORM_FACTOR_HEAD_MOUNTED_DISPLAY;
    const XrResult result = xrGetSystem(m_instance, &info, &m_systemId);
    if (result ≡ XR_ERROR_FORM_FACTOR_UNAVAILABLE)
        return;
    xrCheck(m_instance, result, "xrGetSystem");
    XrSystemProperties properties{XR_TYPE_SYSTEM_PROPERTIES};
    xrCheck(m_instance, xrGetSystemProperties(m_instance, m_systemId, &properties),
            "xrGetSystemProperties");
    m_systemName = properties.systemName;
    createVulkanDevice();
    m_state = State::SystemReady;
}
```

This code is used in section 359.

373. Vulkan for the runtime. With *XR_KHR_vulkan_enable2* the application does not create its Vulkan instance and device by itself. It fills in the usual creation structures and hands them to the runtime, which adds the extensions it needs for its own work (sharing images with its compositor process, in CloudXR's case) and creates the objects with our loader function. The runtime also chooses the physical device: the GPU the headset is driven from, which on a machine with two GPUs need not be the first one.

```
⟨Create the XR Vulkan device 373⟩ ≡
// ---- Vulkan ------------------------------------------------------------
// The runtime creates the Vulkan instance and device itself (vulkan_enable2),
// adding the extensions it needs to ours, and tells us which GPU to use.
void XrOutput::createVulkanDevice()
{
    ⟨Ask the XR runtime for the Vulkan version 374⟩

    ⟨Load the Vulkan library for XR 375⟩

⟨Have the XR runtime create the Vulkan instance 376⟩
```

⟨Find the GPU the XR runtime chose 377⟩

⟨Pick a graphics queue for XR 378⟩

⟨Have the XR runtime create the Vulkan device 379⟩

⟨Create the XR command buffer and fence 380⟩

}

This code is used in section 359.

374. The runtime states the oldest Vulkan version it can work with; we ask for that or 1.1, whichever is newer. Version 1.1 is the first in which external memory and dedicated allocations are part of the core.

```
⟨Ask the XR runtime for the Vulkan version 374⟩ ≡
    XrGraphicsRequirementsVulkan2KHR requirements{XR_TYPE_GRAPHICS_REQUIREMENTS_VULKAN2_
            KHR};
    xrCheck(m_instance, xrVulkan.getGraphicsRequirements(m_instance, m_systemId, &requirements),
            "xrGetVulkanGraphicsRequirements2KHR");
    const uint32_t apiVersion =
        std::max(VK_API_VERSION_1_1, VK_MAKE_API_VERSION(0, XR_VERSION_
                MAJOR(requirements.minApiVersionSupported),
                                                         XR_VERSION_
                                                             MINOR(requirements.minApiVersionSupported),
                                                             0));
```

This code is used in section 373.

375.

```
⟨Load the Vulkan library for XR 375⟩ ≡
    m_vulkanLibrary = dlopen("libvulkan.so.1", RTLD_NOW | RTLD_LOCAL);
    if (¬m_vulkanLibrary)
        throw std::runtime_error("libvulkan.so.1 not found");
    vk.vkGetInstanceProcAddr =
        reinterpret_cast<PFN_vkGetInstanceProcAddr>(dlsym(m_vulkanLibrary, "vkGetInstanceProcAddr"));
```

This code is used in section 373.

376. The instance needs no extensions of ours. Once it exists, all the functions in our list can be looked up through it, device functions included: the loader returns trampolines that dispatch to the right device. The loading macro sits in the left margin, as a preprocessor definition should.

```
⟨Have the XR runtime create the Vulkan instance 376⟩ ≡
        VkApplicationInfo app{VK_STRUCTURE_TYPE_APPLICATION_INFO};
        app.pApplicationName = "UsdOptixViewer";
        app.apiVersion = apiVersion;
        VkInstanceCreateInfo instanceInfo{VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO};
        instanceInfo.pApplicationInfo = &app;
        XrVulkanInstanceCreateInfoKHR xrInstanceInfo{XR_TYPE_VULKAN_INSTANCE_CREATE_INFO_KHR};
        xrInstanceInfo.systemId = m_systemId;
        xrInstanceInfo.pfnGetInstanceProcAddr = vk.vkGetInstanceProcAddr;
        xrInstanceInfo.vulkanCreateInfo = &instanceInfo;
        VkResult vkResult = VK_SUCCESS;
        xrCheck(m_instance, xrVulkan.createInstance(m_instance, &xrInstanceInfo, &m_vkInstance, &vkResult),
                "xrCreateVulkanInstanceKHR");
        vkCheck(vkResult, "vkCreateInstance");
    #define LOAD(name) \
        vk.name = reinterpret_cast<PFN_##name>(vk.vkGetInstanceProcAddr(m_vkInstance,
                #name)); \
        if (!vk.name) \
            throw std::runtime_error("Vulkan function " #name " not found");
        VULKAN_FUNCTIONS(LOAD)
    #undef LOAD
```

This code is used in section 373.

377. The UUID of the chosen GPU comes from the device's ID properties; the application compares it with the UUIDs that CUDA reports.

⟨Find the GPU the XR runtime chose 377⟩ ≡
```
XrVulkanGraphicsDeviceGetInfoKHR deviceGetInfo{XR_TYPE_VULKAN_GRAPHICS_DEVICE_GET_INFO_
        KHR};
deviceGetInfo.systemId = m_systemId;
deviceGetInfo.vulkanInstance = m_vkInstance;
xrCheck(m_instance, xrVulkan.getGraphicsDevice(m_instance, &deviceGetInfo, &m_physicalDevice),
        "xrGetVulkanGraphicsDevice2KHR");
VkPhysicalDeviceIDProperties ids{VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_ID_PROPERTIES};
VkPhysicalDeviceProperties2 properties{VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROPERTIES_2, &ids};
vk.vkGetPhysicalDeviceProperties2(m_physicalDevice, &properties);
std::copy(std::begin(ids.deviceUUID), std::end(ids.deviceUUID), m_deviceUuid.begin());
m_gpuName = properties.properties.deviceName;
```
This code is used in section 373.

378. Any queue family with graphics capability will do: we only record copies, and the binding requires the queue the runtime may synchronize with to be one it can also use for its own work.

⟨Pick a graphics queue for XR 378⟩ ≡
```
uint32_t familyCount = 0;
vk.vkGetPhysicalDeviceQueueFamilyProperties(m_physicalDevice, &familyCount, nullptr);
std::vector<VkQueueFamilyProperties> families(familyCount);
vk.vkGetPhysicalDeviceQueueFamilyProperties(m_physicalDevice, &familyCount, families.data());
const auto graphics = std::find_if(families.begin(), families.end(), [](const VkQueueFamilyProperties& f) {
    return f.queueFlags & VK_QUEUE_GRAPHICS_BIT;
});
if (graphics ≡ families.end())
    throw std::runtime_error("the XR GPU has no graphics queue");
m_queueFamily = uint32_t(graphics − families.begin());
```
This code is used in section 373.

379. The one device extension we need is *VK_KHR_external_memory_fd*, which exports device memory as a POSIX file descriptor that CUDA can import.

⟨Have the XR runtime create the Vulkan device 379⟩ ≡
```
const float priority = 1.0f;
VkDeviceQueueCreateInfo queueInfo{VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO};
queueInfo.queueFamilyIndex = m_queueFamily;
queueInfo.queueCount = 1;
queueInfo.pQueuePriorities = &priority;
const char* deviceExtensions[] = {VK_KHR_EXTERNAL_MEMORY_FD_EXTENSION_NAME};   // memory
        shared with CUDA
VkDeviceCreateInfo deviceInfo{VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO};
deviceInfo.queueCreateInfoCount = 1;
deviceInfo.pQueueCreateInfos = &queueInfo;
deviceInfo.enabledExtensionCount = 1;
deviceInfo.ppEnabledExtensionNames = deviceExtensions;
XrVulkanDeviceCreateInfoKHR xrDeviceInfo{XR_TYPE_VULKAN_DEVICE_CREATE_INFO_KHR};
xrDeviceInfo.systemId = m_systemId;
xrDeviceInfo.pfnGetInstanceProcAddr = vk.vkGetInstanceProcAddr;
xrDeviceInfo.vulkanPhysicalDevice = m_physicalDevice;
xrDeviceInfo.vulkanCreateInfo = &deviceInfo;
xrCheck(m_instance, xrVulkan.createDevice(m_instance, &xrDeviceInfo, &m_device, &vkResult),
        "xrCreateVulkanDeviceKHR");
vkCheck(vkResult, "vkCreateDevice");
vk.vkGetDeviceQueue(m_device, m_queueFamily, 0, &m_queue);
```
This code is used in section 373.

380. A single command buffer is enough, because each frame records its copies, submits them, and waits on the fence before the next frame begins.

⟨Create the XR command buffer and fence 380⟩ ≡
```
VkCommandPoolCreateInfo poolInfo{VK_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO};
poolInfo.flags = VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT;
poolInfo.queueFamilyIndex = m_queueFamily;
```

```
vkCheck(vk.vkCreateCommandPool(m_device, &poolInfo, nullptr, &m_commandPool), "vkCreateCommandPool");
VkCommandBufferAllocateInfo bufferInfo{VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO};
bufferInfo.commandPool = m_commandPool;
bufferInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;
bufferInfo.commandBufferCount = 1;
vkCheck(vk.vkAllocateCommandBuffers(m_device, &bufferInfo, &m_commandBuffer),
        "vkAllocateCommandBuffers");
VkFenceCreateInfo fenceInfo{VK_STRUCTURE_TYPE_FENCE_CREATE_INFO};
vkCheck(vk.vkCreateFence(m_device, &fenceInfo, nullptr, &m_fence), "vkCreateFence");
```

This code is used in section 373.

381. Destroying the device waits for it to be idle first.

```
⟨Destroy the XR Vulkan device 381⟩ ≡
    void XrOutput::destroyVulkan()
    {
        if (m_device)
        {
            vk.vkDeviceWaitIdle(m_device);
            vk.vkDestroyFence(m_device, m_fence, nullptr);
            vk.vkDestroyCommandPool(m_device, m_commandPool, nullptr);
            vk.vkDestroyDevice(m_device, nullptr);
        }
        if (m_vkInstance)
            vk.vkDestroyInstance(m_vkInstance, nullptr);
        if (m_vulkanLibrary)
            dlclose(m_vulkanLibrary);
        m_device = VK_NULL_HANDLE;
        m_fence = VK_NULL_HANDLE;
        m_commandPool = VK_NULL_HANDLE;
        m_commandBuffer = VK_NULL_HANDLE;
        m_queue = VK_NULL_HANDLE;
        m_physicalDevice = VK_NULL_HANDLE;
        m_vkInstance = VK_NULL_HANDLE;
        m_vulkanLibrary = nullptr;
    }
```

This code is used in section 359.

382. Memory shared between Vulkan and CUDA. Each eye buffer is created by Vulkan, exported, and imported into CUDA, where it becomes an ordinary device pointer. It could not be the other way round: CUDA can import Vulkan memory, but Vulkan cannot import CUDA allocations. The buffer holds the eye image in the renderer's own format, four bytes per pixel, row after row.

```
⟨Share an eye buffer between Vulkan and CUDA 382⟩ ≡
    // A device-local Vulkan buffer exported as a file descriptor and imported
    // into CUDA: OptiX writes the eye's pixels into it, Vulkan copies them out.
    void XrOutput::createSharedBuffer(Eye& eye)
    {
        ⟨Create the exportable eye buffer 383⟩
        ⟨Choose device-local memory for the eye buffer 384⟩
        ⟨Allocate and bind the exportable eye memory 385⟩
        ⟨Export the eye memory as a file descriptor 386⟩
        ⟨Import the eye memory into CUDA 387⟩
    }
```

This code is used in section 359.

383. The buffer must announce at creation that its memory will be exported as an opaque file descriptor; its only use is as the source of copies.

```
⟨Create the exportable eye buffer 383⟩ ≡
    const VkDeviceSize size = VkDeviceSize(m_width) * m_height * sizeof(uchar4);
    VkExternalMemoryBufferCreateInfo external{VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_BUFFER_
```

```
        CREATE_INFO};
    external.handleTypes = VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT;
    VkBufferCreateInfo bufferInfo{VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO, &external};
    bufferInfo.size = size;
    bufferInfo.usage = VK_BUFFER_USAGE_TRANSFER_SRC_BIT;
    bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
    vkCheck(vk.vkCreateBuffer(m_device, &bufferInfo, nullptr, &eye.buffer), "vkCreateBuffer");
```

This code is used in section 382.

384. The memory type must be one the buffer accepts and device-local: the whole point is that the pixels never leave the GPU.

```
⟨Choose device-local memory for the eye buffer 384⟩ ≡
    VkMemoryRequirements requirements{};
    vk.vkGetBufferMemoryRequirements(m_device, eye.buffer, &requirements);
    VkPhysicalDeviceMemoryProperties memory{};
    vk.vkGetPhysicalDeviceMemoryProperties(m_physicalDevice, &memory);
    uint32_t type = 0;
    while (type < memory.memoryTypeCount ∧
            ¬((requirements.memoryTypeBits & (1u << type)) ∧
              (memory.memoryTypes[type].propertyFlags & VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT)))
        ++type;
    if (type ≡ memory.memoryTypeCount)
        throw std::runtime_error("no device-local Vulkan memory for the eye buffers");
```

This code is used in section 382.

385. The allocation is *dedicated* to the buffer, and exportable. CUDA must be told the former, below, since a dedicated allocation is imported with a different flag.

```
⟨Allocate and bind the exportable eye memory 385⟩ ≡
    VkMemoryDedicatedAllocateInfo dedicated{VK_STRUCTURE_TYPE_MEMORY_DEDICATED_ALLOCATE_
            INFO};
    dedicated.buffer = eye.buffer;
    VkExportMemoryAllocateInfo exportInfo{VK_STRUCTURE_TYPE_EXPORT_MEMORY_ALLOCATE_INFO,
            &dedicated};
    exportInfo.handleTypes = VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT;
    VkMemoryAllocateInfo allocInfo{VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO, &exportInfo};
    allocInfo.allocationSize = requirements.size;
    allocInfo.memoryTypeIndex = type;
    vkCheck(vk.vkAllocateMemory(m_device, &allocInfo, nullptr, &eye.memory), "vkAllocateMemory");
    vkCheck(vk.vkBindBufferMemory(m_device, eye.buffer, eye.memory, 0), "vkBindBufferMemory");
```

This code is used in section 382.

386.

```
⟨Export the eye memory as a file descriptor 386⟩ ≡
    VkMemoryGetFdInfoKHR fdInfo{VK_STRUCTURE_TYPE_MEMORY_GET_FD_INFO_KHR};
    fdInfo.memory = eye.memory;
    fdInfo.handleType = VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT;
    int fd = -1;
    vkCheck(vk.vkGetMemoryFdKHR(m_device, &fdInfo, &fd), "vkGetMemoryFdKHR");
```

This code is used in section 382.

387. *cudaImportExternalMemory* takes ownership of the file descriptor and closes it when the external memory is destroyed. Mapping a buffer from the imported memory yields a device pointer, which is what the renderer writes to.

```
⟨Import the eye memory into CUDA 387⟩ ≡
    cudaExternalMemoryHandleDesc handle{};
    handle.type = cudaExternalMemoryHandleTypeOpaqueFd;
    handle.handle.fd = fd;   // owned by CUDA once imported
    handle.size = requirements.size;
    handle.flags = cudaExternalMemoryDedicated;
    cudaCheck(cudaImportExternalMemory(&eye.cudaMemory, &handle), "cudaImportExternalMemory");
```

```
cudaExternalMemoryBufferDesc mapping{};
mapping.size = size;
void* pointer = nullptr;
cudaCheck(cudaExternalMemoryGetMappedBuffer(&pointer, eye.cudaMemory, &mapping),
          "cudaExternalMemoryGetMappedBuffer");
eye.pixels = static_cast<uchar4*>(pointer);
```

This code is used in section 382.

388. The session. A session binds the instance to our Vulkan device and is the object through which frames are submitted. Creating it also fixes the size and format of the eye images.

⟨Create the XR session 388⟩ ≡

```
// ---- session ------------------------------------------------------------
bool XrOutput::createSession(float resolutionScale, std::string& error)
{
    try
    {
        ⟨Create the OpenXR session on our Vulkan device 389⟩
        ⟨Create the LOCAL reference space 390⟩
        ⟨Choose the XR environment blend mode 391⟩
        ⟨Size the XR eye images 392⟩
        ⟨Choose the XR swapchain format 393⟩
        ⟨Create the XR swapchains and eye buffers 394⟩
    }
    catch (const std::exception& e)
    {
        error = e.what();
        stop();
        return false;
    }
    return true;
}
```

This code is used in section 359.

389.

⟨Create the OpenXR session on our Vulkan device 389⟩ ≡

```
XrGraphicsBindingVulkan2KHR binding{XR_TYPE_GRAPHICS_BINDING_VULKAN2_KHR};
binding.instance = m_vkInstance;
binding.physicalDevice = m_physicalDevice;
binding.device = m_device;
binding.queueFamilyIndex = m_queueFamily;
binding.queueIndex = 0;
XrSessionCreateInfo sessionInfo{XR_TYPE_SESSION_CREATE_INFO, &binding};
sessionInfo.systemId = m_systemId;
xrCheck(m_instance, xrCreateSession(m_instance, &sessionInfo, &m_session), "xrCreateSession");
```

This code is used in section 388.

390. Views are located in the *LOCAL* reference space, whose origin is the head's position and heading when the session started. Every runtime supports it, and it is what the *Anchor* attaches to the scene.

⟨Create the LOCAL reference space 390⟩ ≡

```
// LOCAL space: the origin is where the head was when the session started
XrReferenceSpaceCreateInfo spaceInfo{XR_TYPE_REFERENCE_SPACE_CREATE_INFO};
spaceInfo.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL;
spaceInfo.poseInReferenceSpace.orientation.w = 1.0f;
xrCheck(m_instance, xrCreateReferenceSpace(m_session, &spaceInfo, &m_space), "xrCreateReferenceSpace");
```

This code is used in section 388.

391. A see-through headset may offer to blend our images with the real world; we want them opaque whenever the runtime allows.

⟨Choose the XR environment blend mode 391⟩ ≡
```
uint32_t count = 0;
xrCheck(m_instance,
        xrEnumerateEnvironmentBlendModes(m_instance, m_systemId, XR_VIEW_CONFIGURATION_TYPE_
                PRIMARY_STEREO, 0,
                                         &count, nullptr),
        "xrEnumerateEnvironmentBlendModes");
std::vector<XrEnvironmentBlendMode> blendModes(count);
xrEnumerateEnvironmentBlendModes(m_instance, m_systemId, XR_VIEW_CONFIGURATION_TYPE_
        PRIMARY_STEREO, count,
                                 &count, blendModes.data());
const bool opaque = blendModes.empty() ∨
                    std::find(blendModes.begin(), blendModes.end(), XR_ENVIRONMENT_BLEND_MODE_
                            OPAQUE) ≠
                        blendModes.end();
m_blendMode = opaque ? XR_ENVIRONMENT_BLEND_MODE_OPAQUE : blendModes.front();
```
This code is used in section 388.

392. The runtime recommends an image size per eye, typically somewhat larger than the display so that its final distortion step keeps detail. The user may scale it down to save rendering time; the result is kept within the runtime's limits. Both eyes of a stereo headset share one size.

⟨Size the XR eye images 392⟩ ≡
```
xrCheck(m_instance,
        xrEnumerateViewConfigurationViews(m_instance, m_systemId, XR_VIEW_CONFIGURATION_TYPE_
                PRIMARY_STEREO,
                                          0, &count, nullptr),
        "xrEnumerateViewConfigurationViews");
std::vector<XrViewConfigurationView> configViews(count, {XR_TYPE_VIEW_CONFIGURATION_VIEW});
xrEnumerateViewConfigurationViews(m_instance, m_systemId, XR_VIEW_CONFIGURATION_TYPE_PRIMARY_
        STEREO, count,
                                  &count, configViews.data());
if (configViews.empty())
    throw std::runtime_error("the headset has no stereo views");
const XrViewConfigurationView& view = configViews[0];
m_width = std::clamp(unsigned(view.recommendedImageRectWidth * resolutionScale), 16u,
                     view.maxImageRectWidth);
m_height = std::clamp(unsigned(view.recommendedImageRectHeight * resolutionScale), 16u,
                      view.maxImageRectHeight);
```
This code is used in section 388.

393. The renderer writes 8-bit RGBA that is already gamma encoded (with the square-root curve described with the ray generation program, close to sRGB). An sRGB swapchain format shows those values as intended, because the runtime decodes them; a linear UNORM format is the fallback.

⟨Choose the XR swapchain format 393⟩ ≡
```
// The renderer writes 8-bit gamma-encoded RGBA, which an sRGB format
// shows as intended.
xrCheck(m_instance, xrEnumerateSwapchainFormats(m_session, 0, &count, nullptr),
        "xrEnumerateSwapchainFormats");
std::vector<int64_t> formats(count);
xrEnumerateSwapchainFormats(m_session, count, &count, formats.data());
int64_t format = 0;
for (int64_t wanted : {int64_t(VK_FORMAT_R8G8B8A8_SRGB), int64_t(VK_FORMAT_R8G8B8A8_
        UNORM)})
    if (¬format ∧ std::find(formats.begin(), formats.end(), wanted) ≠ formats.end())
        format = wanted;
if (¬format)
    throw std::runtime_error("the runtime offers no RGBA8 swapchain format");
```
This code is used in section 388.

394. Each eye gets a swapchain whose images we only ever write by transfer, so *XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT* is the flag that matters; the colour attachment flag is there because the runtime

expects colour images in that layout. The images themselves are enumerated as Vulkan images, and the shared buffer is made next to them.

⟨Create the XR swapchains and eye buffers 394⟩ ≡
```
m_eyes.resize(configViews.size());
for (Eye& eye : m_eyes)
{
    XrSwapchainCreateInfo swapchainInfo{XR_TYPE_SWAPCHAIN_CREATE_INFO};
    swapchainInfo.usageFlags = XR_SWAPCHAIN_USAGE_TRANSFER_DST_BIT | XR_SWAPCHAIN_USAGE_
        COLOR_ATTACHMENT_BIT;
    swapchainInfo.format = format;
    swapchainInfo.sampleCount = 1;
    swapchainInfo.width = m_width;
    swapchainInfo.height = m_height;
    swapchainInfo.faceCount = 1;
    swapchainInfo.arraySize = 1;
    swapchainInfo.mipCount = 1;
    xrCheck(m_instance, xrCreateSwapchain(m_session, &swapchainInfo, &eye.swapchain), "xrCreateSwapchain");
    xrCheck(m_instance, xrEnumerateSwapchainImages(eye.swapchain, 0, &count, nullptr),
        "xrEnumerateSwapchainImages");
    eye.images.assign(count, {XR_TYPE_SWAPCHAIN_IMAGE_VULKAN2_KHR});
    xrEnumerateSwapchainImages(eye.swapchain, count, &count,
                               reinterpret_cast<XrSwapchainImageBaseHeader*>(eye.images.data()));
    createSharedBuffer(eye);
}
m_views.assign(m_eyes.size(), {XR_TYPE_VIEW});
m_state = State::Idle;
```
This code is used in section 388.

395. Destroying the session releases the CUDA view of each buffer before the Vulkan memory behind it.

⟨Destroy the XR session 395⟩ ≡
```
void XrOutput::destroySession()
{
    if (m_device)
        vk.vkDeviceWaitIdle(m_device);
    for (Eye& eye : m_eyes)
    {
        if (eye.pixels)
            cudaFree(eye.pixels);
        if (eye.cudaMemory)
            cudaDestroyExternalMemory(eye.cudaMemory);
        if (eye.buffer)
            vk.vkDestroyBuffer(m_device, eye.buffer, nullptr);
        if (eye.memory)
            vk.vkFreeMemory(m_device, eye.memory, nullptr);
        if (eye.swapchain ≠ XR_NULL_HANDLE)
            xrDestroySwapchain(eye.swapchain);
    }
    m_eyes.clear();
    if (m_space ≠ XR_NULL_HANDLE)
        xrDestroySpace(m_space);
    if (m_session ≠ XR_NULL_HANDLE)
        xrDestroySession(m_session);
    m_space = XR_NULL_HANDLE;
    m_session = XR_NULL_HANDLE;
    m_frameState = {XR_TYPE_FRAME_STATE};
    m_viewsValid = false;
}
```
This code is used in section 359.

396. The runtime drives the session through state changes delivered as events. *READY* means "begin the session", *STOPPING* means "end it". *EXITING* and *LOSS_PENDING* mean the session is over: with CloudXR, the client has gone. We then release the session and the Vulkan device (which belonged to that

system) and go back to waiting for a system, so that a headset reconnecting resumes the stream without restarting the viewer. The loss of the instance itself is final.

⟨Follow the XR session state 396⟩ ≡

```
// The runtime drives the session through state changes: READY means "begin",
// STOPPING means "end". When the headset goes away (CloudXR client
// disconnected) the session is lost and we wait for a system again.
void XrOutput::pollEvents()
{
    XrEventDataBuffer event{XR_TYPE_EVENT_DATA_BUFFER};
    while (xrPollEvent(m_instance, &event) ≡ XR_SUCCESS)
    {
        if (event.type ≡ XR_TYPE_EVENT_DATA_INSTANCE_LOSS_PENDING)
            throw std::runtime_error("the OpenXR runtime is shutting down");
        if (event.type ≡ XR_TYPE_EVENT_DATA_SESSION_STATE_CHANGED)
        {
            const auto& change = reinterpret_cast<const XrEventDataSessionStateChanged&>(event);
            if (change.state ≡ XR_SESSION_STATE_READY)
            {
                XrSessionBeginInfo beginInfo{XR_TYPE_SESSION_BEGIN_INFO};
                beginInfo.primaryViewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_
                        STEREO;
                xrCheck(m_instance, xrBeginSession(m_session, &beginInfo), "xrBeginSession");
                m_state = State::Running;
            }
            else if (change.state ≡ XR_SESSION_STATE_STOPPING)
            {
                xrCheck(m_instance, xrEndSession(m_session), "xrEndSession");
                m_state = State::Idle;
            }
            else if (change.state ≡ XR_SESSION_STATE_EXITING ∨ change.state ≡ XR_SESSION_STATE_
                    LOSS_PENDING)
            {
                destroySession();
                destroyVulkan();
                m_systemId = XR_NULL_SYSTEM_ID;
                m_state = State::WaitingForSystem;
                return;
            }
        }
        event = {XR_TYPE_EVENT_DATA_BUFFER};
    }
}
```

This code is used in section 359.

397. Frames. A frame starts with *xrWaitFrame*, which blocks until the runtime wants the next frame; this is what paces the application's main loop at the headset's rate while the session runs. *xrBeginFrame* then opens the frame. The runtime may say the frame should not be rendered (the headset is off, or the app not visible); the frame must still be begun and ended, only without images.

⟨Begin an XR frame 397⟩ ≡

```
// ---- frames ------------------------------------------------------------
bool XrOutput::beginFrame(std::string& error)
{
    if (m_state ≠ State::Running)
        return false;
    try
    {
        XrFrameWaitInfo waitInfo{XR_TYPE_FRAME_WAIT_INFO};
        m_frameState = {XR_TYPE_FRAME_STATE};
        xrCheck(m_instance, xrWaitFrame(m_session, &waitInfo, &m_frameState), "xrWaitFrame");  // paces the
                loop
        XrFrameBeginInfo beginInfo{XR_TYPE_FRAME_BEGIN_INFO};
```

```
        xrCheck(m_instance, xrBeginFrame(m_session, &beginInfo), "xrBeginFrame");
        m_viewsValid = false;
        ⟨Locate the eyes for the predicted display time 398⟩
    }
    catch (const std::exception& e)
    {
        error = e.what();
        stop();
        return false;
    }
    return true;
}
```

This code is used in section 359.

398. The eyes are located for the time at which the frame will be displayed, not for now. The runtime predicts the head's pose for that moment, which hides most of the latency of rendering and streaming. Without a valid orientation there is nothing sensible to draw.

```
⟨Locate the eyes for the predicted display time 398⟩ ≡
  if (m_frameState.shouldRender)
  {
      XrViewLocateInfo locateInfo{XR_TYPE_VIEW_LOCATE_INFO};
      locateInfo.viewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO;
      locateInfo.displayTime = m_frameState.predictedDisplayTime;
      locateInfo.space = m_space;
      XrViewState viewState{XR_TYPE_VIEW_STATE};
      uint32_t count = 0;
      xrCheck(m_instance,
              xrLocateViews(m_session, &locateInfo, &viewState, uint32_t(m_views.size()), &count,
                            m_views.data()),
              "xrLocateViews");
      m_viewsValid = count ≡ m_views.size() ∧ (viewState.viewStateFlags & XR_VIEW_STATE_ORIENTATION_
              VALID_BIT);
  }
```

This code is used in section 397.

399. Ending the frame submits one projection layer, with one view per eye: the pose and field of view the eye was rendered with, and the swapchain image holding it. A frame that was not rendered ends with no layers.

```
⟨End an XR frame 399⟩ ≡
  void XrOutput::endFrame(std::string& error)
  {
      try
      {
          std::vector<XrCompositionLayerProjectionView> views;
          if (shouldRender())
          {
              copyToSwapchains();
              for (size_t i = 0; i < m_eyes.size(); ++i)
              {
                  XrCompositionLayerProjectionView view{XR_TYPE_COMPOSITION_LAYER_PROJECTION_
                          VIEW};
                  view.pose = m_views[i].pose;
                  view.fov = m_views[i].fov;
                  view.subImage.swapchain = m_eyes[i].swapchain;
                  view.subImage.imageRect.extent = {int32_t(m_width), int32_t(m_height)};
                  views.push_back(view);
              }
          }
          XrCompositionLayerProjection layer{XR_TYPE_COMPOSITION_LAYER_PROJECTION};
          layer.space = m_space;
          layer.viewCount = uint32_t(views.size());
```

```
        layer.views = views.data();
        const XrCompositionLayerBaseHeader* layers[] = {reinterpret_
                cast<XrCompositionLayerBaseHeader*>(&layer)};
        XrFrameEndInfo endInfo{XR_TYPE_FRAME_END_INFO};
        endInfo.displayTime = m_frameState.predictedDisplayTime;
        endInfo.environmentBlendMode = m_blendMode;
        endInfo.layerCount = views.empty() ? 0 : 1;
        endInfo.layers = layers;
        xrCheck(m_instance, xrEndFrame(m_session, &endInfo), "xrEndFrame");
    }
    catch (const std::exception& e)
    {
        error = e.what();
        stop();
    }
}
```

This code is used in section 359.

400. Moving the eye buffers into the swapchains. Each image must first be acquired and waited for (the runtime may still be reading it), and released afterwards. OpenXR's Vulkan binding fixes the layout of colour images at these moments: they arrive in *COLOR_ATTACHMENT_OPTIMAL* and must be handed back in it.

⟨Copy the eye buffers into the swapchain images 400⟩ ≡

```
// The eye buffers were written by CUDA (the renderer synchronizes its stream
// before returning). Swapchain images arrive in COLOR_ATTACHMENT_OPTIMAL
// layout and must be returned in it.
void XrOutput::copyToSwapchains()
{
    ⟨Acquire the XR swapchain images 401⟩
    ⟨Record the copies into the swapchain images 402⟩
    ⟨Submit the XR copies and wait for them 403⟩
    ⟨Release the XR swapchain images 404⟩
}
```

This code is used in section 359.

401.

⟨Acquire the XR swapchain images 401⟩ ≡

```
std::vector<uint32_t> indices(m_eyes.size());
for (size_t i = 0; i < m_eyes.size(); ++i)
{
    XrSwapchainImageAcquireInfo acquireInfo{XR_TYPE_SWAPCHAIN_IMAGE_ACQUIRE_INFO};
    xrCheck(m_instance, xrAcquireSwapchainImage(m_eyes[i].swapchain, &acquireInfo, &indices[i]),
            "xrAcquireSwapchainImage");
    XrSwapchainImageWaitInfo waitInfo{XR_TYPE_SWAPCHAIN_IMAGE_WAIT_INFO};
    waitInfo.timeout = XR_INFINITE_DURATION;
    xrCheck(m_instance, xrWaitSwapchainImage(m_eyes[i].swapchain, &waitInfo), "xrWaitSwapchainImage");
}
```

This code is used in section 400.

402. For every eye, a barrier moves the image into the layout for transfer writes, a buffer-to-image copy fills it, and a second barrier returns it to the colour-attachment layout. The first barrier may discard the old contents, since the copy overwrites every pixel. The buffer needs no barrier of its own: CUDA finished writing it before we got here, because the renderer synchronizes its stream before it returns.

⟨Record the copies into the swapchain images 402⟩ ≡

```
VkCommandBufferBeginInfo beginInfo{VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO};
beginInfo.flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT;
vkCheck(vk.vkBeginCommandBuffer(m_commandBuffer, &beginInfo), "vkBeginCommandBuffer");
for (size_t i = 0; i < m_eyes.size(); ++i)
{
    VkImageMemoryBarrier barrier{VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER};
```

```
        barrier.srcQueueFamilyIndex = barrier.dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
        barrier.image = m_eyes[i].images[indices[i]].image;
        barrier.subresourceRange = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1};
        barrier.oldLayout = VK_IMAGE_LAYOUT_UNDEFINED;   // the whole image is overwritten
        barrier.newLayout = VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL;
        barrier.dstAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
        vk.vkCmdPipelineBarrier(m_commandBuffer, VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT, VK_PIPELINE_
                STAGE_TRANSFER_BIT, 0,
                                0, nullptr, 0, nullptr, 1, &barrier);
        VkBufferImageCopy region{};
        region.imageSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1};
        region.imageExtent = {m_width, m_height, 1};
        vk.vkCmdCopyBufferToImage(m_commandBuffer, m_eyes[i].buffer, barrier.image,
                                  VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, 1, &region);
        barrier.oldLayout = VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL;
        barrier.newLayout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;
        barrier.srcAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
        barrier.dstAccessMask = 0;
        vk.vkCmdPipelineBarrier(m_commandBuffer, VK_PIPELINE_STAGE_TRANSFER_BIT,
                                VK_PIPELINE_STAGE_ALL_COMMANDS_BIT, 0, 0, nullptr, 0, nullptr, 1,
                                    &barrier);
    }
```

This code is used in section 400.

403. We wait for the copies to complete before releasing the images. That costs a little latency but keeps the synchronization trivial: the runtime is handed finished images, and the eye buffers are free to be rendered into again next frame.

⟨Submit the XR copies and wait for them 403⟩ ≡

```
    vkCheck(vk.vkEndCommandBuffer(m_commandBuffer), "vkEndCommandBuffer");
    VkSubmitInfo submit{VK_STRUCTURE_TYPE_SUBMIT_INFO};
    submit.commandBufferCount = 1;
    submit.pCommandBuffers = &m_commandBuffer;
    vkCheck(vk.vkQueueSubmit(m_queue, 1, &submit, m_fence), "vkQueueSubmit");
    // the eye buffers are rewritten next frame, so wait for the copy to finish
    vkCheck(vk.vkWaitForFences(m_device, 1, &m_fence, VK_TRUE, UINT64_MAX), "vkWaitForFences");
    vkCheck(vk.vkResetFences(m_device, 1, &m_fence), "vkResetFences");
```

This code is used in section 400.

404.

⟨Release the XR swapchain images 404⟩ ≡

```
    for (Eye& eye : m_eyes)
    {
        XrSwapchainImageReleaseInfo releaseInfo{XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO};
        xrCheck(m_instance, xrReleaseSwapchainImage(eye.swapchain, &releaseInfo), "xrReleaseSwapchainImage");
    }
```

This code is used in section 400.

405. Eye cameras. The renderer's *Camera* wants an eye point and three vectors in scene coordinates; OpenXR gives each eye a pose and four field of view angles in the *LOCAL* space. Two conversions are needed: from the XR space to the scene, and from the angles to the camera's vectors.

⟨Compute the camera of an XR eye 405⟩ ≡

```
    // ---- eye cameras ------------------------------------------------------------
    // The XR space is Y-up and looks along -Z, in meters. Its axes are mapped to
    // the scene: +X to the anchor's right, +Y to the scene's up axis and -Z to the
    // anchor's horizontal forward direction. Eye frusta are asymmetric, so w
    // points at the centre of the image plane at distance 1 instead of straight
    // ahead (the renderer accepts this because XR views use no clipping range).
    Camera XrOutput::eyeCamera(unsigned int index, const Anchor& anchor) const
    {
        ⟨Build the XR anchor frame 406⟩
```

⟨Turn the XR eye pose into the scene 407⟩

⟨Fill the camera of an XR eye 408⟩

}

This code is used in section 359.

406. The XR space is right-handed with $+y$ up and the viewer looking along $-z$. We map it to the scene with an orthonormal frame built from the anchor: $\mathbf{a}_u$ is the scene's up axis, $\mathbf{a}_f$ the anchor's forward direction with its vertical part removed and normalized, and $\mathbf{a}_r = \mathbf{a}_f \times \mathbf{a}_u$ points right. Keeping only the horizontal part matters: the desktop camera usually looks down at the scene, but a headset user's horizon must stay level, or the world would appear tilted. If the anchor looks straight up or down there is no horizontal part, and a fixed direction is used instead. An XR vector (x, y, z) then becomes $x\,\mathbf{a}_r + y\,\mathbf{a}_u - z\,\mathbf{a}_f$.

⟨Build the XR anchor frame 406⟩ ≡

```
const XrView& view = m_views[index];
const float up[3] = {0.0f, anchor.zUp ? 0.0f : 1.0f, anchor.zUp ? 1.0f : 0.0f};
float forward[3];
const float along = anchor.forward[0] * up[0] + anchor.forward[1] * up[1] + anchor.forward[2] * up[2];
for (int k = 0; k < 3; ++k)
    forward[k] = anchor.forward[k] - along * up[k];
float length = std::sqrt(forward[0] * forward[0] + forward[1] * forward[1] + forward[2] * forward[2]);
if (length < 1e-6f)  // looking straight up or down: face the scene's -Y (Z-up) or -Z (Y-up)
{
    forward[0] = 0.0f, forward[1] = anchor.zUp ? -1.0f : 0.0f, forward[2] = anchor.zUp ? 0.0f : -1.0f;
    length = 1.0f;
}
for (float& f : forward)
    f /= length;
const float right[3] = {forward[1] * up[2] - forward[2] * up[1], forward[2] * up[0] - forward[0] * up[2],
                        forward[0] * up[1] - forward[1] * up[0]};
auto toScene = [&](XrVector3f v) {
    return make_float3(right[0] * v.x + up[0] * v.y - forward[0] * v.z,
                       right[1] * v.x + up[1] * v.y - forward[1] * v.z,
                       right[2] * v.x + up[2] * v.y - forward[2] * v.z);
};
```

This code is used in section 405.

407. The eye's own axes are the XR axes rotated by its orientation, then taken to the scene. Its position is an offset in meters from the XR origin, which the anchor's scale converts to scene units.

⟨Turn the XR eye pose into the scene 407⟩ ≡

```
const XrQuaternionf& q = view.pose.orientation;
const float3 eyeRight = toScene(rotate(q, {1.0f, 0.0f, 0.0f}));
const float3 eyeUp = toScene(rotate(q, {0.0f, 1.0f, 0.0f}));
const float3 eyeForward = toScene(rotate(q, {0.0f, 0.0f, -1.0f}));
const float3 offset = toScene(view.pose.position);
const float s = anchor.unitsPerMeter;
```

This code is used in section 405.

408. Headset views are not symmetric: each eye sees farther towards the temple than towards the nose. OpenXR gives the four half-angles of the view, and the image plane at distance 1 spans $[\tan\alpha_l, \tan\alpha_r]$ horizontally and $[\tan\alpha_d, \tan\alpha_u]$ vertically (α_l and α_d are negative). The ray generation program shoots through $\mathbf{w} + x\,\mathbf{u} + y\,\mathbf{v}$ for $x, y \in [-1, 1]$, so we aim $\mathbf{w}$ at the *centre* of that window rather than straight ahead,

$$\mathbf{w} = \mathbf{f} + c_x\,\mathbf{r} + c_y\,\mathbf{u}, \qquad c_x = \frac{\tan\alpha_r + \tan\alpha_l}{2}, \quad c_y = \frac{\tan\alpha_u + \tan\alpha_d}{2},$$

and give $\mathbf{u}$ and $\mathbf{v}$ the half extents $h_x = (\tan\alpha_r - \tan\alpha_l)/2$ and $h_y = (\tan\alpha_u - \tan\alpha_d)/2$. This $\mathbf{w}$ is not a unit vector, which the renderer tolerates only because an eye camera has no clipping range: the depth correction of the clipping planes assumes a unit $\mathbf{w}$, and with $[0, 10^{16}]$ it has nothing to correct.

⟨Fill the camera of an XR eye 408⟩ ≡

```
const float tanLeft = std::tan(view.fov.angleLeft), tanRight = std::tan(view.fov.angleRight);
const float tanDown = std::tan(view.fov.angleDown), tanUp = std::tan(view.fov.angleUp);
const float cx = 0.5f * (tanRight + tanLeft), cy = 0.5f * (tanUp + tanDown);
```

```
const float hx = 0.5f * (tanRight − tanLeft), hy = 0.5f * (tanUp − tanDown);
Camera camera{};
camera.eye = make_float3(anchor.eye[0] + offset.x * s, anchor.eye[1] + offset.y * s, anchor.eye[2] + offset.z * s);
camera.u = make_float3(eyeRight.x * hx, eyeRight.y * hx, eyeRight.z * hx);
camera.v = make_float3(eyeUp.x * hy, eyeUp.y * hy, eyeUp.z * hy);
camera.w = make_float3(eyeForward.x + eyeRight.x * cx + eyeUp.x * cy, eyeForward.y + eyeRight.y * cx + eyeUp.y
        * cy,
                               eyeForward.z + eyeRight.z * cx + eyeUp.z * cy);
camera.orthographic = 0;
camera.clipNear = 0.0f;
camera.clipFar = 1e16f;
return camera;
```

This code is used in section 405.

Names of the sections

⟨`UsdOptixViewer/cmake/EmbedFile.cmake` 18⟩ Root (written to a file).
⟨`UsdOptixViewer/CMakeLists.txt` 6⟩ Root (written to a file).
⟨`UsdOptixViewer/src/App.cpp` 253⟩ Root (written to a file).
⟨`UsdOptixViewer/src/App.h` 243⟩ Root (written to a file).
⟨`UsdOptixViewer/src/CloudXrService.cpp` 337⟩ Root (written to a file).
⟨`UsdOptixViewer/src/CloudXrService.h` 333⟩ Root (written to a file).
⟨`UsdOptixViewer/src/devicePrograms.cu` 186⟩ Root (written to a file).
⟨`UsdOptixViewer/src/LaunchParams.h` 19⟩ Root (written to a file).
⟨`UsdOptixViewer/src/main.cpp` 242⟩ Root (written to a file).
⟨`UsdOptixViewer/src/OptixRenderer.cpp` 121⟩ Root (written to a file).
⟨`UsdOptixViewer/src/OptixRenderer.h` 110⟩ Root (written to a file).
⟨`UsdOptixViewer/src/Scene.cpp` 39⟩ Root (written to a file).
⟨`UsdOptixViewer/src/Scene.h` 32⟩ Root (written to a file).
⟨`UsdOptixViewer/src/StbImage.cpp` 331⟩ Root (written to a file).
⟨`UsdOptixViewer/src/XrOutput.cpp` 359⟩ Root (written to a file).
⟨`UsdOptixViewer/src/XrOutput.h` 349⟩ Root (written to a file).
⟨A camera of the stage 106⟩ Used in section 32.
⟨A linear congruential generator 191⟩ Used in section 189.
⟨A renderable mesh 34⟩ Used in section 32.
⟨Accumulate the sample and write the pixel 216⟩ Used in section 213.
⟨Acquire the XR swapchain images 401⟩ Used in section 400.
⟨Add a glossy reflection 234⟩ Used in section 219.
⟨Add the ambient fill 229⟩ Used in section 219.
⟨Add the diffuse and specular reflection of the light 233⟩ Used in section 230.
⟨Add the directional light 230⟩ Used in section 219.
⟨Allocate a mipmapped array for the texture 170⟩ Used in section 169.
⟨Allocate and bind the exportable eye memory 385⟩ Used in section 382.
⟨An eye of the XR output 355⟩ Used in section 349.
⟨Ask the XR runtime for the Vulkan version 374⟩ Used in section 373.
⟨Assign a material to every face of a prim 90⟩ Used in section 46.
⟨Average one channel of the block 130⟩ Used in section 127.
⟨Begin an XR frame 397⟩ Used in section 359.
⟨Build an acceleration structure 165⟩ Used in section 121.
⟨Build Dear ImGui and the viewer executable 16⟩ Used in section 6.
⟨Build the geometry acceleration structure of a mesh 161⟩ Used in section 159.
⟨Build the indexed mesh of a prim 93⟩ Used in section 46.
⟨Build the instance acceleration structure 164⟩ Used in section 153.
⟨Build the pipeline and set the default parameters 139⟩ Used in section 136.
⟨Build the rays through the neighbouring pixels 203⟩ Used in section 201.
⟨Build the shader binding table 148⟩ Used in section 121.
⟨Build the tangent frame from positions and texture coordinates 207⟩ Used in section 206.
⟨Build the XR anchor frame 406⟩ Used in section 405.
⟨Camera and display options of the application 248⟩ Used in section 243.
⟨Camera, light and rendering switches 30⟩ Used in section 28.
⟨Choose device-local memory for the eye buffer 384⟩ Used in section 382.
⟨Choose the acceleration structure build options 166⟩ Used in section 165.
⟨Choose the default CloudXR SDK 12⟩ Used in section 8.
⟨Choose the flags of the tree node 283⟩ Used in section 282.
⟨Choose the root of the installed tools and of CUDA 9⟩ Used in section 8.
⟨Choose the XR environment blend mode 391⟩ Used in section 388.

⟨Choose the XR swapchain format 393⟩ Used in section 388.
⟨Close the bookkeeping of a generated primitive 59⟩ Used in section 39.
⟨Collect a warning once 43⟩ Used in section 39.
⟨Collect the cameras of the stage 107⟩ Used in section 39.
⟨Colors, textures and inputs of a material 26⟩ Used in section 24.
⟨Compile the device programs to OptiX-IR and embed them 15⟩ Used in section 6.
⟨Compile the embedded OptiX-IR into a module 143⟩ Used in section 140.
⟨Compute smooth normals for a subdivision surface 57⟩ Used in section 55.
⟨Compute the camera of an XR eye 405⟩ Used in section 359.
⟨Compute the geometric and shading normals 221⟩ Used in section 219.
⟨Compute the next mip level of a texture 127⟩ Used in section 121.
⟨Compute the orbit offset 301⟩ Used in section 253.
⟨Compute the regions of the window 270⟩ Used in section 269.
⟨Compute the resolution of the image 317⟩ Used in section 310.
⟨Compute the shading normals of each triangle corner 55⟩ Used in section 51.
⟨Compute the st gradients of the pixel footprint 201⟩ Used in section 199.
⟨Compute the world bounds of a mesh 102⟩ Used in section 46.
⟨Constants of the application 255⟩ Used in section 253.
⟨Construction and session control of the XR output 353⟩ Used in section 349.
⟨Conversions from host material fields 156⟩ Used in section 155.
⟨Convert a UsdGeomMesh into a local triangle soup 51⟩ Used in section 39.
⟨Convert a UsdShade material into a Material record 72⟩ Used in section 39.
⟨Copy the eye buffers into the swapchain images 400⟩ Used in section 359.
⟨Count the triangles of the scene 44⟩ Used in section 39.
⟨Cover the viewport with an invisible button 312⟩ Used in section 310.
⟨Create the CloudXR service and choose its device profile 343⟩ Used in section 340.
⟨Create the exportable eye buffer 383⟩ Used in section 382.
⟨Create the hit group 146⟩ Used in section 140.
⟨Create the LOCAL reference space 390⟩ Used in section 388.
⟨Create the miss program groups 145⟩ Used in section 140.
⟨Create the OpenXR instance 366⟩ Used in section 359.
⟨Create the OpenXR instance with the Vulkan extension 368⟩ Used in section 366.
⟨Create the OpenXR session on our Vulkan device 389⟩ Used in section 388.
⟨Create the OptiX device context 138⟩ Used in section 136.
⟨Create the OptiX pipeline 140⟩ Used in section 121.
⟨Create the ray generation program group 144⟩ Used in section 140.
⟨Create the texture object 173⟩ Used in section 169.
⟨Create the window and the SDL renderer 257⟩ Used in section 256.
⟨Create the XR command buffer and fence 380⟩ Used in section 373.
⟨Create the XR session 388⟩ Used in section 359.
⟨Create the XR swapchains and eye buffers 394⟩ Used in section 388.
⟨Create the XR Vulkan device 373⟩ Used in section 359.
⟨CUDA and OptiX handles of the renderer 118⟩ Used in section 110.
⟨Data members of the scene 37⟩ Used in section 35.
⟨Decide how to decode the normal map 83⟩ Used in section 82.
⟨Decide the color space and wrap modes, and consult the texture cache 88⟩ Used in section 86.
⟨Declare the embedded OptiX-IR 123⟩ Used in section 121.
⟨Declare the project and the language standard 7⟩ Used in section 6.
⟨Decode a texture through the asset resolver 89⟩ Used in section 86.
⟨Decode the normal map and leave tangent space 208⟩ Used in section 206.
⟨Derive the roughness and the reflectance at normal incidence 227⟩ Used in section 225.
⟨Describe how the texture is sampled 172⟩ Used in section 169.

⟨Describe roughness, metallic or specular color, and ior 293⟩ Used in section 290.
⟨Describe the camera and offer to look through it 289⟩ Used in section 286.
⟨Describe the diffuse color or texture 291⟩ Used in section 290.
⟨Describe the identity of the prim 288⟩ Used in section 286.
⟨Describe the normal map 292⟩ Used in section 290.
⟨Describe the rendered geometry and its material 290⟩ Used in section 286.
⟨Describe the stage when nothing is selected 287⟩ Used in section 286.
⟨Describe the state of the XR output 330⟩ Used in section 324.
⟨Destroy or stop the XR output 365⟩ Used in section 359.
⟨Destroy the application 263⟩ Used in section 253.
⟨Destroy the CloudXR service 339⟩ Used in section 337.
⟨Destroy the renderer 135⟩ Used in section 121.
⟨Destroy the XR session 395⟩ Used in section 359.
⟨Destroy the XR Vulkan device 381⟩ Used in section 359.
⟨Diffuse, cutout and normal-map fields of a material 65⟩ Used in section 64.
⟨Download the image and the picking ids 182⟩ Used in section 179.
⟨Draw one node of the hierarchy 282⟩ Used in section 253.
⟨Draw the hierarchy panel 279⟩ Used in section 253.
⟨Draw the inspector 286⟩ Used in section 253.
⟨Draw the menu bar 273⟩ Used in section 253.
⟨Draw the mirror of the left eye 329⟩ Used in section 324.
⟨Draw the popups 323⟩ Used in section 253.
⟨Draw the status bar 322⟩ Used in section 253.
⟨Draw the tree node and react to clicks 284⟩ Used in section 282.
⟨Draw the viewport 310⟩ Used in section 253.
⟨End an XR frame 399⟩ Used in section 359.
⟨Error checking of CUDA and OptiX calls 124⟩ Used in section 121.
⟨Error checks of the XR output 363⟩ Used in section 359.
⟨Evaluate the material at the hit 225⟩ Used in section 219.
⟨Export the eye memory as a file descriptor 386⟩ Used in section 382.
⟨Express the view direction in a frame around the shading normal 235⟩ Used in section 234.
⟨Extract the renderable meshes of the stage 46⟩ Used in section 39.
⟨Fan-triangulate the faces of a mesh 53⟩ Used in section 51.
⟨Fetch the hit triangle and its material 220⟩ Used in section 219.
⟨File requests from other threads 251⟩ Used in section 243.
⟨Fill in one device material 157⟩ Used in section 155.
⟨Fill the camera of an XR eye 408⟩ Used in section 405.
⟨Find and load the CloudXR library 341⟩ Used in section 340.
⟨Find SDL3, OpenUSD, meshoptimizer and CUDA 13⟩ Used in section 8.
⟨Find the authored normals of a mesh 56⟩ Used in section 55.
⟨Find the Dear ImGui sources 11⟩ Used in section 8.
⟨Find the distinct corners of a mesh 95⟩ Used in section 94.
⟨Find the eye and direction of the desktop view 302⟩ Used in section 253.
⟨Find the GPU the XR runtime chose 377⟩ Used in section 373.
⟨Find the hit point and the offset for secondary rays 231⟩ Used in section 230.
⟨Find the mesh or the camera of a prim 45⟩ Used in section 39.
⟨Find the network addresses of this machine 346⟩ Used in section 337.
⟨Find the OpenXR loader and the Vulkan headers 14⟩ Used in section 8.
⟨Find the OptiX headers 10⟩ Used in section 8.
⟨Find the primvar that holds the diffuse texture coordinates 79⟩ Used in section 77.
⟨Find the primvar that holds the normal-map coordinates 84⟩ Used in section 82.
⟨Find the shader that drives an input 69⟩ Used in section 39.

⟨Fit a USD camera's window to the viewport 303⟩ Used in section 253.
⟨Follow the XR session 326⟩ Used in section 324.
⟨Follow the XR session state 396⟩ Used in section 359.
⟨Forget the current stage 42⟩ Used in section 39.
⟨Format an attribute value for the inspector 296⟩ Used in section 253.
⟨Frame a bounding box 308⟩ Used in section 253.
⟨Frame the selection 309⟩ Used in section 253.
⟨Frames and eyes of the XR output 354⟩ Used in section 349.
⟨Free a device buffer 133⟩ Used in section 121.
⟨Free the buffers of a mesh 134⟩ Used in section 121.
⟨Free the old scene and restart the accumulation 154⟩ Used in section 153.
⟨Free the scene 168⟩ Used in section 121.
⟨Free the textures 174⟩ Used in section 121.
⟨Gather the 2x2 block of source texels 129⟩ Used in section 127.
⟨Gather the corners of a prim's triangles 92⟩ Used in section 46.
⟨Gather the hit triangle in world space 197⟩ Used in section 195.
⟨Gather the triangle and the hit point 202⟩ Used in section 201.
⟨Generate the triangles of a cube 60⟩ Used in section 39.
⟨Generate the triangles of a sphere 61⟩ Used in section 39.
⟨Group the triangles by material 98⟩ Used in section 94.
⟨Hand the view of a USD camera to the orbit camera 305⟩ Used in section 253.
⟨Handle an empty image or scene 180⟩ Used in section 179.
⟨Handle an SDL event 268⟩ Used in section 253.
⟨Handle the keyboard shortcuts 272⟩ Used in section 269.
⟨Have the XR runtime create the Vulkan device 379⟩ Used in section 373.
⟨Have the XR runtime create the Vulkan instance 376⟩ Used in section 373.
⟨Headers and declarations of the renderer interface 111⟩ Used in section 110.
⟨Headers needed by the scene declarations 33⟩ Used in section 32.
⟨Headers needed by the scene implementation 40⟩ Used in section 39.
⟨Headers of the application 254⟩ Used in section 253.
⟨Headers of the CloudXR service 338⟩ Used in section 337.
⟨Headers of the renderer implementation 122⟩ Used in section 121.
⟨Headers of the XR output 360⟩ Used in section 359.
⟨Host-side results of the renderer 120⟩ Used in section 110.
⟨Identify a shader and its wrap modes 70⟩ Used in section 39.
⟨Import the eye memory into CUDA 387⟩ Used in section 382.
⟨Initialize CUDA and OptiX 136⟩ Used in section 121.
⟨Initialize the application 256⟩ Used in section 253.
⟨Install the viewer 17⟩ Used in section 6.
⟨Interpolate the shading normal from the corner normals 222⟩ Used in section 221.
⟨Interpolate the texture coordinates at the hit 200⟩ Used in section 199.
⟨Launch one sample per pixel 181⟩ Used in section 179.
⟨Lay out the panels and handle shortcuts 269⟩ Used in section 253.
⟨Leave a USD camera when the view is moved 313⟩ Used in section 310.
⟨Link the pipeline and set its stack size 147⟩ Used in section 140.
⟨List the attributes of the prim 295⟩ Used in section 294.
⟨List the properties of the prim 294⟩ Used in section 286.
⟨List the relationships of the prim 297⟩ Used in section 294.
⟨Load a texture once 86⟩ Used in section 39.
⟨Load a USD file into the scene and the renderer 265⟩ Used in section 253.
⟨Load the OpenXR Vulkan binding functions 370⟩ Used in section 366.
⟨Load the Vulkan library for XR 375⟩ Used in section 373.

⟨Local triangle soup of one prim 50⟩ Used in section 39.
⟨Locate the eyes for the predicted display time 398⟩ Used in section 397.
⟨Locate the visualization tools 8⟩ Used in section 6.
⟨Look for the XR system 372⟩ Used in section 359.
⟨Look through a USD camera 304⟩ Used in section 253.
⟨Look through the requested camera 262⟩ Used in section 256.
⟨Look up the base color 226⟩ Used in section 225.
⟨Look up the mesh under a pixel 184⟩ Used in section 121.
⟨Make sure the OpenXR runtime supports Vulkan 367⟩ Used in section 366.
⟨Material data on the device 24⟩ Used in section 19.
⟨Material for a prim without a bound material 71⟩ Used in section 39.
⟨Mesh data on the device 27⟩ Used in section 19.
⟨Module compile options 141⟩ Used in section 140.
⟨Move the renderer to the GPU of the headset 328⟩ Used in section 324.
⟨Name the OpenXR runtime 369⟩ Used in section 366.
⟨Normal mapping in the tangent frame 206⟩ Used in section 186.
⟨Open a stage and build the scene 41⟩ Used in section 39.
⟨Open the borderless viewport window 311⟩ Used in section 310.
⟨Open the requested file and select the requested prim 261⟩ Used in section 256.
⟨Opening of the CloudXR service header 334⟩ Used in section 333.
⟨Opening of the device programs file 187⟩ Used in section 186.
⟨Opening of the shared parameters header 20⟩ Used in section 19.
⟨Opening of the XR output header 350⟩ Used in section 349.
⟨OpenXR functions of the Vulkan binding 362⟩ Used in section 359.
⟨OpenXR state of the XR output 357⟩ Used in section 349.
⟨Orbit a target from a direction and distance 306⟩ Used in section 253.
⟨Orbit the surface point under a pixel 307⟩ Used in section 253.
⟨Orbit with the left mouse button 314⟩ Used in section 310.
⟨Output buffers of a launch 29⟩ Used in section 28.
⟨Pack the hit records 151⟩ Used in section 148.
⟨Pack the materials for the device 155⟩ Used in section 153.
⟨Pack the miss records 150⟩ Used in section 148.
⟨Pack the ray generation record 149⟩ Used in section 148.
⟨Pan with the right or middle mouse button 315⟩ Used in section 310.
⟨Parse the command line 259⟩ Used in section 256.
⟨Pass the transform of a mesh to the renderer 101⟩ Used in section 46.
⟨Payload helpers and the sky 192⟩ Used in section 186.
⟨Perturb the shading normal with the normal map 223⟩ Used in section 221.
⟨Pick a graphics queue for XR 378⟩ Used in section 373.
⟨Pick the prim under a mouse click 318⟩ Used in section 310.
⟨Pick the shading normal of each triangle corner 58⟩ Used in section 55.
⟨Pipeline compile options 142⟩ Used in section 140.
⟨Place and draw the four panels 271⟩ Used in section 269.
⟨Place the mesh in the world with an instance 162⟩ Used in section 159.
⟨Placing the XR user in the scene 352⟩ Used in section 349.
⟨Point the shader binding table at the records 152⟩ Used in section 148.
⟨Poll the events of the CloudXR service 348⟩ Used in section 337.
⟨Prepare the extraction of the geometry 47⟩ Used in section 46.
⟨Private helpers of the scene 38⟩ Used in section 35.
⟨Private member functions of the application 245⟩ Used in section 243.
⟨Private steps of the renderer 117⟩ Used in section 110.
⟨Private steps of the XR output 356⟩ Used in section 349.

⟨Public interface of the application 244⟩ Used in section 243.
⟨Public interface of the CloudXR service 335⟩ Used in section 333.
⟨Public interface of the scene 36⟩ Used in section 35.
⟨Publish the mesh records 163⟩ Used in section 153.
⟨Random numbers on the device 189⟩ Used in section 186.
⟨Read a float input that may come from a texture 85⟩ Used in section 39.
⟨Read an alpha cutout of a surface 80⟩ Used in section 72.
⟨Read back the surface point under a pixel 185⟩ Used in section 121.
⟨Read the diffuse color of a surface 76⟩ Used in section 72.
⟨Read the diffuse texture of a surface 77⟩ Used in section 72.
⟨Read the normal map of a surface 82⟩ Used in section 72.
⟨Read the points and the face topology of a mesh 52⟩ Used in section 51.
⟨Read the scale, bias and fallback of the diffuse texture 78⟩ Used in section 77.
⟨Read the specular parameters of a surface 81⟩ Used in section 72.
⟨Read the texture coordinates of a prim 91⟩ Used in section 46.
⟨Read the texture coordinates of the corners 198⟩ Used in section 195.
⟨Read the value of a shader input 68⟩ Used in section 39.
⟨Read the window and the clipping range of a camera 108⟩ Used in section 107.
⟨Reading scalar material inputs 205⟩ Used in section 186.
⟨Recognize shadow rays 193⟩ Used in section 192.
⟨Recompute the hierarchy filter 280⟩ Used in section 253.
⟨Record the bounds of the scene 105⟩ Used in section 46.
⟨Record the copies into the swapchain images 402⟩ Used in section 400.
⟨Reflect the view direction about the sampled normal 237⟩ Used in section 234.
⟨Register the mesh of a prim 104⟩ Used in section 46.
⟨Release the XR swapchain images 404⟩ Used in section 400.
⟨Render one progressive sample 179⟩ Used in section 121.
⟨Render one view into a device buffer 183⟩ Used in section 121.
⟨Render the eyes of the headset 327⟩ Used in section 324.
⟨Renderer construction and initialization 112⟩ Used in section 110.
⟨Renderer frame production and results 114⟩ Used in section 110.
⟨Renderer scene and view settings 113⟩ Used in section 110.
⟨Reorder one attribute stream through a remap table 96⟩ Used in section 94.
⟨Replace the corners by the distinct vertices 97⟩ Used in section 94.
⟨Report a failed CloudXR start, or point OpenXR at the runtime 345⟩ Used in section 340.
⟨Request a file to be opened 264⟩ Used in section 253.
⟨Resize a device buffer 131⟩ Used in section 121.
⟨Resize the image with the viewport 319⟩ Used in section 310.
⟨Resize the output image 175⟩ Used in section 121.
⟨Resolve the entry points of the CloudXR service 342⟩ Used in section 340.
⟨Resolve the file of a texture 87⟩ Used in section 86.
⟨Restart or continue the progressive rendering 320⟩ Used in section 310.
⟨Return the cached Material of a UsdShade material 73⟩ Used in section 72.
⟨Return the color and the seed to the caller 240⟩ Used in section 219.
⟨Rewind the triangles of a left-handed mesh 54⟩ Used in section 51.
⟨Rotate a vector by a quaternion for XR 364⟩ Used in section 359.
⟨Run the build and wait for it 167⟩ Used in section 165.
⟨Sample a visible normal of the GGX distribution 236⟩ Used in section 234.
⟨Scalar material inputs 23⟩ Used in section 19.
⟨Scene and output buffers of the renderer 119⟩ Used in section 110.
⟨Scene and renderer of the application 247⟩ Used in section 243.
⟨Schlick's Fresnel approximation 212⟩ Used in section 209.

⟨Search a name ignoring case 281⟩ Used in section 253.
⟨Seed the random generator for this pixel and frame 214⟩ Used in section 213.
⟨Select a prim 285⟩ Used in section 253.
⟨Select the GPU and its primary context 137⟩ Used in section 136.
⟨Selection and hierarchy filter state 249⟩ Used in section 243.
⟨Send the camera and the options to the renderer 298⟩ Used in section 253.
⟨Send the orbit camera to the renderer 300⟩ Used in section 298.
⟨Send the USD camera to the renderer 299⟩ Used in section 298.
⟨Set a general camera view 177⟩ Used in section 121.
⟨Set the camera 176⟩ Used in section 121.
⟨Set the light direction 178⟩ Used in section 121.
⟨Set the scene 153⟩ Used in section 121.
⟨Set up Dear ImGui 258⟩ Used in section 256.
⟨Share an eye buffer between Vulkan and CUDA 382⟩ Used in section 359.
⟨Shoot the camera ray through a jittered point of the pixel 215⟩ Used in section 213.
⟨Show the file dialog 266⟩ Used in section 253.
⟨Show the image in the viewport 321⟩ Used in section 310.
⟨Size the XR eye images 392⟩ Used in section 388.
⟨Skip prims that are not visible renderable geometry 48⟩ Used in section 46.
⟨Small vector algebra on the device 188⟩ Used in section 186.
⟨Small vector helpers of the host 126⟩ Used in section 121.
⟨Sort the triangles of each material spatially 99⟩ Used in section 94.
⟨Specular fields of a material 66⟩ Used in section 64.
⟨Split the energy between specular and diffuse reflection 228⟩ Used in section 225.
⟨Start a Material with the UsdPreviewSurface defaults 74⟩ Used in section 72.
⟨Start the CloudXR service 340⟩ Used in section 337.
⟨Start the CloudXR service, keeping our signal handlers 344⟩ Used in section 340.
⟨Start the XR output 325⟩ Used in section 324.
⟨Start the XR output and the OptiX renderer 260⟩ Used in section 256.
⟨State and entry points of the CloudXR service 336⟩ Used in section 333.
⟨States of the XR output 351⟩ Used in section 349.
⟨Stop the CloudXR service 347⟩ Used in section 337.
⟨Store a new Material in the cache 75⟩ Used in section 72.
⟨Store the vertices in the order the triangles first use them 100⟩ Used in section 94.
⟨Submit the XR copies and wait for them 403⟩ Used in section 400.
⟨Test whether a material reads any texture 67⟩ Used in section 64.
⟨Text input and dialog state 250⟩ Used in section 243.
⟨Texture remapping coefficients of a material 25⟩ Used in section 24.
⟨Texture sampling with ray differentials 199⟩ Used in section 186.
⟨Texture wrap modes and decoded textures 62⟩ Used in section 32.
⟨The any-hit program for alpha cutouts 218⟩ Used in section 186.
⟨The background mesh id and the ray types 21⟩ Used in section 19.
⟨The camera frame 22⟩ Used in section 19.
⟨The Camera menu 276⟩ Used in section 273.
⟨The closest-hit program 219⟩ Used in section 186.
⟨The device buffer helper 115⟩ Used in section 110.
⟨The device buffers of one mesh 116⟩ Used in section 110.
⟨The File menu 274⟩ Used in section 273.
⟨The GGX microfacet model 209⟩ Used in section 186.
⟨The GGX normal distribution 210⟩ Used in section 209.
⟨The Help menu 278⟩ Used in section 273.
⟨The hit triangle record 196⟩ Used in section 195.

⟨The launch parameters 28⟩ Used in section 19.
⟨The main loop of the application 267⟩ Used in section 253.
⟨The Material record 64⟩ Used in section 32.
⟨The miss programs 217⟩ Used in section 186.
⟨The ray generation program 213⟩ Used in section 186.
⟨The ScalarInput material parameter 63⟩ Used in section 32.
⟨The scene as seen by the device 31⟩ Used in section 28.
⟨The Scene class 35⟩ Used in section 32.
⟨The shader binding table record type 125⟩ Used in section 121.
⟨The sky 194⟩ Used in section 192.
⟨The Smith masking function 211⟩ Used in section 209.
⟨The sRGB to linear lookup table 128⟩ Used in section 127.
⟨The TEA hash 190⟩ Used in section 189.
⟨The triangle under the hit 195⟩ Used in section 186.
⟨The View menu 275⟩ Used in section 273.
⟨The XR menu 277⟩ Used in section 273.
⟨The XR output of the application 324⟩ Used in section 253.
⟨Trace a shadow ray towards the light 232⟩ Used in section 230.
⟨Trace the reflection ray and clamp fireflies 239⟩ Used in section 238.
⟨Triangulate a prim in its local space 49⟩ Used in section 46.
⟨Turn a neighbouring ray into an st offset 204⟩ Used in section 201.
⟨Turn both normals towards the viewer 224⟩ Used in section 221.
⟨Turn the XR eye pose into the scene 407⟩ Used in section 405.
⟨Update the XR output 371⟩ Used in section 359.
⟨Upload every mesh and build its acceleration structure 159⟩ Used in section 153.
⟨Upload every mip level 171⟩ Used in section 169.
⟨Upload host data to a device buffer 132⟩ Used in section 121.
⟨Upload the textures 169⟩ Used in section 121.
⟨Upload the textures and publish the material tables 158⟩ Used in section 153.
⟨Upload the vertex data of a mesh 160⟩ Used in section 159.
⟨Vulkan functions loaded at run time for XR 361⟩ Used in section 359.
⟨Vulkan state and eye images of the XR output 358⟩ Used in section 349.
⟨Warn about textured prims without texture coordinates 103⟩ Used in section 46.
⟨Weight the reflected light and fetch it 238⟩ Used in section 234.
⟨Weld the corners of a prim into an indexed mesh 94⟩ Used in section 39.
⟨Window and SDL objects of the application 246⟩ Used in section 243.
⟨XR output state of the application 252⟩ Used in section 243.
⟨Zoom with the mouse wheel 316⟩ Used in section 310.